NLIAA | 长空御风

南京航空航天大学 / 长空御风战队

ROBOMASTER

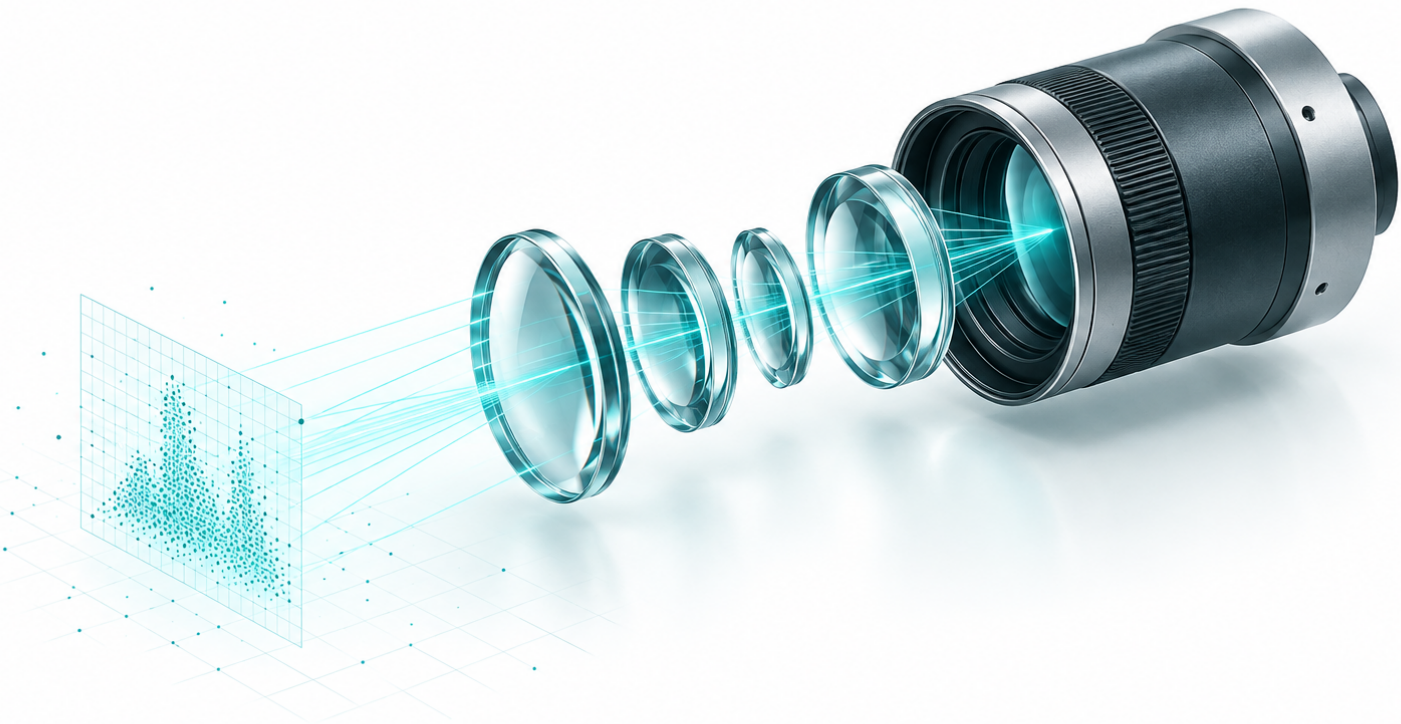
VISION TEAM

视觉组培训教材

VOLUME / 01

卷一 / 基础通识

FOUNDATIONS

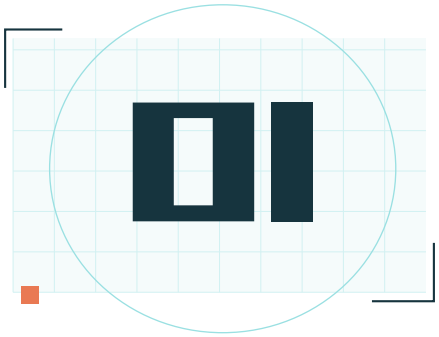


Linux / 现代 C++ / CMake / Git

卷一 FOUNDATIONS

学习导览

基础通识



VOLUME / 01

从开发环境到第一个可以复现的工程。
为视觉组专项学习建立工具、语言与协作基础。

学习路径 / 3 个阶段

01 建立工作环境

01 / 03

安装与维护 Linux，掌握命令行、Shell 和 Git，
让每次修改都有记录，让开发过程可以重复。

02 用 C++ 组织程序

02 / 03

从类与对象到容器、资源管理和并发，
写清数据怎样流动、对象由谁持有。

03 构建并验证工程

03 / 03

理解编译与链接，用 CMake 管理多文件工程，
按阶段定位错误，交付能够运行的结果。

编写 长空御风视觉组
版本 2026.09

视觉组新人培训系列
南京航空航天大学·长空御风战队

标 * 的小节为选修，不计入通识考核。

编者说明

欢迎来到视觉组。

作为比你们早走几步的学长，我们最想说的不是“这里有多难”，而是：别急。刚开始看不懂代码、配不好环境、被一行报错困住一下午，都是再正常不过的事。基础打得慢一点没关系，真正危险的是似懂非懂地往下冲，最后连问题出在哪里都说不清。

遇到问题，先读报错、查资料、动手试，再带着现象和尝试过的方法来问。不要害怕求助，也不要把“求助”等同于把问题丢给别人。能把环境、操作、预期和结果讲清楚，是对自己负责，也是对队友时间的尊重。

写下的每一行代码，迟早都会由自己或队友来维护。动手之前先备份，修改之前先确认分支，提交之前再检查一遍；不确定会造成什么后果的命令，不要靠胆量执行。赛场不会因为一句“在我电脑上能跑”就网开一面，可靠往往来自那些看起来不起眼的习惯。

也别把一时的进度当成能力，更别拿自己的起点和别人的积累比较。承认不会，耐心补课，遇到问题不甩锅，答应的事情有回音——这些品质比偶尔写出一段漂亮代码更难得。

希望你们保留好奇心，也保留敬畏心。路还很长，慢慢来，但别停下。

认识技术组与视觉组

技术组如何协作

一台机器人从图纸走到赛场，需要结构能够承载、执行机构能够受控、传感器能够提供信息，软件还要把这些能力组织起来。技术组通常分为机械、电控与视觉等组别，分工围绕各自负责的系统展开。

- **机械组：让结构满足运动与安装需求。**负责机构设计、零件加工与装配，兼顾强度、重量、空间和维护便利性。例如，相机装在哪里、视野是否被遮挡、机器人运动时支架是否明显振动，都需要机械组参与解决。SolidWorks 等建模工具用于表达和验证设计。
- **电控组：让执行机构按指令稳定工作。**负责嵌入式软件、运动控制与设备通信，电路设计、焊接和供电调试也常由该组承担。例如，读取电机反馈、控制底盘速度、反馈云台姿态，以及接收上层软件的指令。常用工具包括 STM32 开发工具与电子设计自动化（EDA）软件。
- **视觉组：把传感器数据变成可用的状态与任务信息。**参与工业相机、镜头、激光雷达和计算平台的选型与调试，开发目标识别、跟踪、定位、导航及决策相关软件。软件以 C++ 为主，Python 常用于数据处理、训练与实验。视觉组既要关心算法结果，也要关心设备能否稳定采集、程序能否在机器人上持续运行。

以相机接入为例，视觉组提出成像需求，机械组落实安装位置和支架，电控组协助确认供电、通信及姿态反馈；三组共同验证运动时的数据是否可用。接口应约定单位、坐标系、时间戳与更新频率，否则各模块接在一起仍可能出错。本教材也将帮助视觉组新生逐步建立系统协作意识。

视觉组负责哪些任务

理解视觉组，可以先看它需要回答哪些问题。下面按自瞄、导航、决策和雷达四类业务任务介绍；它们彼此协作，具体人员与模块的分工可以交叉。

- **自瞄：目标在哪里，运动状态如何？**从相机图像中识别装甲板等目标，估计其空间位置，并通过连续观测维持跟踪、估计运动状态，向控制系统提供目标信息。检测解决“这一帧看到了什么”，跟踪解决“前后帧是否为同一目标、状态如何变化”。交付结果应包含有效状态；目标丢失时，也要让下游知道当前信息已经不可用。
- **导航：机器人在哪里，怎样到达目的地？**利用激光雷达、惯性测量单元（IMU）等传感器进行定位，结合地图、路径规划和局部避障，让机器人向给定目标移动。同步定位与建图（SLAM）用于同时估计自身位置与构建地图，导航系统则还要处理行进过程中的障碍和失败情况。ROS2 与 Nav2 是常见的软件基础。导航需要反馈到达、受阻或失败等状态，供上层决定下一步。
- **决策：何时去哪里，执行什么任务？**结合比赛状态、自身状态、感知结果和任务执行反馈，选择或切换机器人当前的行为。以哨兵为例，决策可以安排巡逻、前往指定区域或等待条件满足，再把目标交给导航等模块执行。状态机与行为树（Behavior Tree）是组织行为的常见方法。决策负责选择任务，导航负责完成移动；遇到目标不可达、信息过期等情况，决策还要决定重试、改选任务还是等待。
- **雷达：场上有哪些目标，它们分布在哪里？**借助较开阔的观测视角，识别与跟踪场上机器人，把观测结果转换到场地坐标中，并按通信约定共享目标位置等信息，为队伍提供全局态势参考。这

里的“雷达”指比赛中的雷达站任务，不等同于导航所用的激光雷达传感器。其重点包括覆盖范围、目标身份关联、遮挡后的重新识别，以及共享信息的准确性与时效性。

这些任务共享一些方法与工具：传统图像处理和深度学习用于提取信息；OpenCV 是计算机视觉库，PyTorch 常用于模型训练；ROS2 提供节点通信等基础，Nav2 组织导航功能。学习时先明确问题与输入输出，再选择工具。

日常工作：从采集数据到交付系统

按以下环节梳理一个功能，可以帮助你判断问题出在数据、算法，还是系统集成上。

1. **明确需求与评价方式。**先描述功能在什么场景下工作、接收什么数据、输出给谁。例如，识别任务需要统计漏检和误检，跟踪任务还要观察目标切换与丢失后的恢复，导航任务需要记录是否到达及失败原因。评价方式应在实验前确定，才能比较改动是否有效。
2. **接入设备并建立可靠输入。**根据视野、距离、接口带宽、算力与安装条件选择传感器和计算平台；完成驱动配置、曝光调节、相机标定及安装关系测量。保存能代表实际场景的原始数据，并检查时间戳、丢帧和设备断连情况。后续算法的效果首先受输入质量约束。
3. **实现算法并验证中间结果。**根据任务选择图像处理、学习模型或状态估计等方法，先用记录的数据验证。除了最终输出，还应检查检测框、坐标变换、跟踪状态或规划路径等中间结果。使用学习模型时，还需要整理和标注数据、划分训练与评估数据，避免用同一批样本同时训练和证明效果。
4. **接入机器人并完成联调。**把算法封装为可调用的模块或 ROS2 节点，接入设备、通信与控制链路，核对字段、单位、坐标和频率。补全启动配置、参数文件与异常处理，在真实计算平台上检查运行速度、资源占用和连续运行情况。
5. **复盘问题并留下可复现的交付物。**保留代码版本、参数、设备配置、测试数据和结论，提供启动与排错说明。发现问题后，尽量形成可以回放或重复执行的案例，再验证修改。交付的目标是让队友能够运行、理解和继续维护这项功能。

实际研发会在这些环节之间反复迭代。例如，远处目标总是漏检，既可能需要补充训练样本，也可能是镜头视野过大、目标像素太少。先查看原始图像和中间结果，才能判断应该改哪一层。

能力如何逐步建立

入组时无需全部掌握以下能力：先学会运行与修改小程序，再进入专项，在项目中补齐知识。

- **第一步：能够独立完成基础开发。**熟悉 Linux 下的文件、终端与进程操作，理解 C++ 的基本语法、类、容器和资源管理，能够使用 CMake 构建程序、借助 Git 管理改动。进一步学习智能指针与多线程，理解对象生命周期和共享数据问题。这些是卷一的主要训练内容。
- **第二步：能够解释数据与算法的含义。**逐步补充向量、矩阵、坐标变换与概率统计基础，理解图像、相机模型和常见视觉处理流程，再根据任务学习 OpenCV、深度学习、状态估计等知识。卷二提供相关基础与专项内容；学习时应能说清一个量表示什么、属于哪个坐标系、结果怎样验证。
- **第三步：能够负责一段完整功能。**结合自瞄、导航、决策或雷达任务，学习所需的 ROS2、规划、行为组织等技术，把输入、处理、输出和异常反馈接起来。卷三帮助理解赛场任务与工程协作，再将所学用于联调和复盘。

- **贯穿始终：能够把问题讲清楚。**记录复现步骤，阅读日志，使用调试器和可视化工具，并在求助时给出环境、现象、预期和已尝试的方法。对队友说明接口变化，对实验保留证据。这些习惯从第一份练习就可以开始培养。

主要挑战：为什么演示成功还不够

单张图片上识别正确、一次路线顺利跑通，只验证了有限条件下的功能。机器人在实际运行中还会受到环境变化、运动、通信与资源限制的共同影响。下面列出几类需要持续验证的问题。

1. **环境变化会破坏原本有效的观测。**光照改变、反光、运动模糊、远距离小目标和遮挡，都可能造成漏检、误检或跟踪中断。问题会继续传到下游，使目标状态跳变或失效。调试时应先检查原始图像，再按场景收集失败案例，分别判断是成像质量、检测还是跟踪环节的问题。修改后要回放多类场景，检查改善某一场景时是否损害了其他场景。
2. **数据“看起来合理”，仍可能没有对齐。**相机、激光雷达与机器人本体各有坐标系，安装关系和标定参数会影响空间转换；不同设备的数据还可能对应不同时间。静止时误差不明显，运动后却可能表现为目标位置漂移或地图重影。需要核对单位、坐标轴方向、变换关系与时间戳，并在静止和运动条件下分别验证。安装位置变化后，也应重新检查有关参数。
3. **单个算法运行快，不代表整条链路响应及时。**采集、传输、处理、排队和发送都会消耗时间。程序即使持续输出，也可能正在处理积压的旧数据；偶发卡顿还会使输出间隔忽长忽短，影响下游使用。应记录各环节耗时、数据采集到输出的时间差，以及延迟波动，定位瓶颈后再优化。队列如何处理过期数据、下游如何识别过期结果，都需要明确约定。
4. **离线评估与实际场景可能存在差距。**训练或调试数据可能只覆盖固定场地、视角与光照，实机却会遇到新的距离、背景和遮挡组合。评估数据若与训练数据过于相似，也容易高估效果。应将独立采集的场景用于评估，并持续补充实机失败样本，分别统计不同条件下的表现。每次调整都要在保留的测试数据上比较，避免只凭少数成功画面判断效果。
5. **计算平台与设备需要经受持续运行。**开发电脑上的程序迁移到机器人后，会受到算力、内存、功耗和散热条件的限制；多个模块同时运行，还可能争用资源。设备重连、日志持续写入和长时间运行也会暴露短时测试没有发现的问题。应在目标平台上同时运行实际所需模块，观察资源占用、温度、错误日志与数据连续性，并验证重新启动后的设备连接和配置是否正确。
6. **任务受阻时，系统仍要有明确行为。**路径被挡住、定位失效、目标消失或状态更新中断，都可能让正常流程无法继续。如果只设计成功路径，机器人可能反复切换任务、持续等待，或继续使用过期信息。应为任务定义开始、完成、失败和超时条件，规定重试、取消或等待的处理方式。联调时主动构造受阻与数据中断场景，检查各模块是否给出一致的状态反馈。
7. **跨组问题需要共同的证据来定位。**相似现象可能来自不同原因：画面抖动可能与支架振动有关，输出异常可能来自参数、通信或软件版本不一致。应记录测试时的代码版本、硬件状态、配置与日志，沿数据流逐段检查。接口变更要同步更新文档并通知相关成员；修复后保留复现案例，方便验证同类问题是否再次出现，也让后续成员能够接手。

通识阶段先练好一件事：让程序稳定运行，能够解释输入输出，遇到错误时留下可复现的记录。再逐步扩大负责的范围，把所学与机器人上的实际问题连接起来。

目录

编者说明 **i**

认识技术组与视觉组 **ii**

第一章 Linux 工作环境 **1**

1.1 安装 Ubuntu 22.04 双系统 1
1.2 系统数据备份 * 13
1.3 卸载重装 Ubuntu 系统 * 17
1.4 文件系统与常用命令 18
1.5 Shell 脚本与环境变量 31
1.6 Git 工具的使用 37
1.7 Conda 的简单使用 * 43
1.8 CLion 与 VS Code 开发环境 44
1.9 Linux 串口通信基础 56
1.10 本章小结 62
第一章 练习 63
1.11 本章验收 63

第二章 现代 C++ 基础 **65**

2.1 面向对象编程：类 65
2.2 Lambda 表达式、引用与移动 74
2.3 智能指针 81
2.4 标准模版库（STL） 85
2.5 C++ 多线程 97
2.6 本章小结 110
第二章 练习 111

第三章 编译过程与 CMake 基础 **112**

3.1 C++ 的编译过程 112
3.2 CMake 115
3.3 本章小结 127
第三章 练习 127

卷一小结 **128**

参考题解 **135**

第一章题解：Linux 工作环境 135
第二章题解：现代 C++ 基础 145

第三章题解：编译过程与 CMake 基础 163

第一章 Linux 工作环境

内容提要

- 学习目标：独立完成 Ubuntu 22.04 双系统的分区与安装；使用 timeshift 制作并恢复系统快照；运用常用命令完成文件管理、查找与权限修改；编写并运行简单的 shell 脚本，正确配置环境变量；使用 git 完成克隆、修改、提交与推送；安装 CLion 与 VS Code，识别界面按钮并完成 C++ 工程的构建、运行和断点调试；识别队内 USB CDC ACM 设备、配置权限与固定别名、完成收发实验，记录收发字节并定位常见连接故障。
- 建议学时：约 1 周，另留半天完成 CLion 与 VS Code 安装、运行和调试练习，另用 2–3 小时完成串口识别、权限配置与回环实验。
- 本卷边界：不涉及 ROS2 工作空间细节，相关内容见卷二；串口只讲 Linux 设备访问与基础收发，帧协议、CRC 和 ROS2 桥接见卷二“多机通信方法”；标 * 的小节（备份、重装、美化、conda 等）为选修，不计入考核。

1.1 安装 Ubuntu 22.04 双系统

本部分以 windows11 系统为例进行安装说明，如有使用特殊体质 mac 系统还请寻找其他方法（也许后续会补充进来）。安装双系统主要分为以下四个流程：系统分区，制作启动盘，从 U 盘进行安装，系统设置相关。下面将进行一一说明。

1.1.1 系统分区

笔记 制作双系统的第一步是需要获得一个空闲的系统盘分区，这一步我们一般需要在 Windows 系统下完成，但是如果你本来就有一个空闲的分区，比如说我的 D 盘里面没有什么数据，我想直接把整个 D 盘都作为下面的 Ubuntu 系统盘，你也可以跳过这一步，直接在 Ubuntu 安装时选择整个 D 盘作为对应分区即可（不过 Ubuntu 里面看不到具体的”D 盘” 字样，你得按照容量选择）。

右键桌面上的” 此电脑” 图标，点击” 管理” 选项，进入” 磁盘管理” 界面，在这里你一般会看到电脑上存在若干个磁盘分区如下图所示。

笔记 你也可以换用一些比较高级的分区工具，例如 DiskGenius 来完成这一步。

笔记 分区操作直接作用于磁盘，动手前请确认重要文件已有备份。” 压缩卷” 只是划出一块空闲区域，不会删除数据，但后续安装环节的格式化操作会。

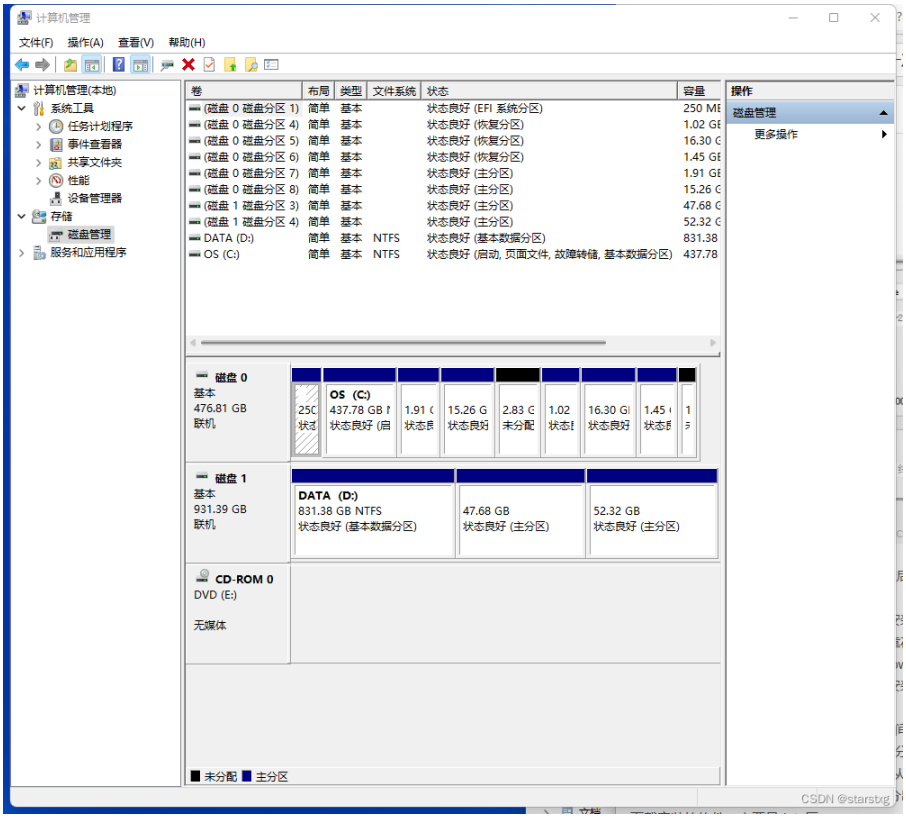


图 1.1: 磁盘分区

选择一个剩余空间较大的磁盘，右击，点击” 压缩卷”，等待计算完成后会显示当前磁盘中可用的压缩空间大小。输入你要留给 ubuntu 系统的空间量（注意是以 MB 为单位的），推荐是预留 200G，后续内存不够虽然可以扩容但操作起来有点麻烦。

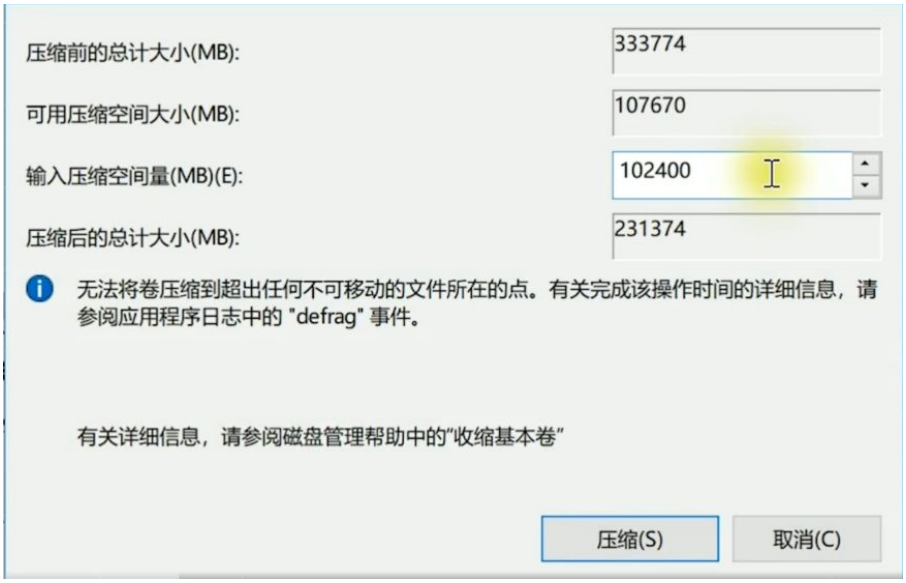


图 1.2: 分配 linux 空间

输入好后点击” 压缩”，压缩完成后出现一个” 未分配” 的区域则说明分区完成。

✍ **练习 1.1** 打开 Windows 的磁盘管理，记录你电脑各分区的容量与剩余空间，规划留给 Ubuntu 的空间大小，并写出你打算采用的分区方案（各挂载点及容量）。

1.1.2 制作启动盘

首先需要一个内存大于 16GB 的 U 盘，请先对其中的文件进行备份，后续操作需要格式化这个 U 盘。

然后进入 ubuntu 官网下载 ubuntu22.04 版本的镜像，下载链接为

https://ubuntu.com/download/alternative-downloads

注意是下载 22.04.5 Desktop！

下载好后还需要下载一个启动盘制作器来将刚刚的 ISO 写入 U 盘中，这里介绍 UI 最简单的 balenaEtcher，你也可以自行下载其他烧录器，比如软碟通（UltraISO）或是 Rufus，balenaEtcher 的下载链接

https://etcher.balena.io/

安装后点击”Select image” 选择刚刚下载的 ubuntu 镜像，随后在”Select target” 中选择你的启动盘，最后点击”Flash!” 即可。

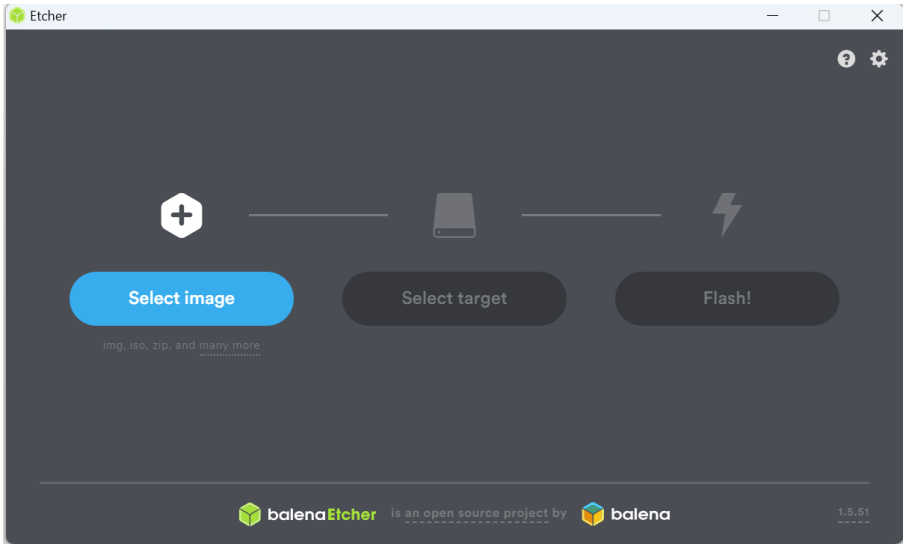


图 1.3: 制作启动盘

U 盘被制作成启动盘后仍可以存放其它的文件，插到 windows 中会提示该 U 盘有问题建议修复之类的，U 盘没坏的话这时候不用管它就好。启动盘被称为” 数字生命”，所以请好好保存。

1.1.3 从 U 盘进行安装

插入之前做好的启动盘，重启电脑，重启时快速重复按下启动选项快捷键，各个主板的启动选项快捷键有所不同，可以参考下表进行操作，如果没有的话可以观察启动时 BIOS 页面上的信息，或者是直接启动时脸滚键盘。

组装机主板		品牌笔记本		品牌台式机	
主板品牌	启动按键	笔记本品牌	启动按键	台式机品牌	启动按键
华硕主板	F8	联想笔记本	F12	联想台式机	F12
技嘉主板	F12	宏基笔记本	F12	惠普台式机	F12
微星主板	F11	华硕笔记本	ESC	宏基台式机	F12
映泰主板	F9	惠普笔记本	F9	戴尔台式机	ESC
梅捷主板	ESC或F12	联想Thinkpad	F12	神舟台式机	F12
七彩虹主板	ESC或F11	戴尔笔记本	F12	华硕台式机	F8
华擎主板	F11	神舟笔记本	F12	方正台式机	F12
斯巴达克主板	ESC	东芝笔记本	F12	清华同方台式机	F12
昂达主板	F11	三星笔记本	F12	海尔台式机	F12
双敏主板	ESC	IBM笔记本	F12	明基台式机	F8
翔升主板	F10	富士通笔记本	F12		
精英主板	ESC或F11	海尔笔记本	F12		
冠盟主板	F11或F12	方正笔记本	F12		
富士康主板	ESC或F12	清华同方笔记本	F12		
顶星主板	F11或F12	微星笔记本	F11		
铭瑄主板	ESC	明基笔记本	F9		
盈通主板	F8	技嘉笔记本	F12		
捷波主板	ESC	Gateway笔记本	F12		
Intel主板	F12	eMachines笔记本	F12		
杰微主板	ESC或F8	索尼笔记本	ESC		
致铭主板	F12	苹果笔记本	长按“option”键		
磐英主板	ESC				
磐正主板	ESC				

图 1.4: 启动选项快捷键一览

进入启动选项界面后通过上下键选择 Linpus lite 选项，回车，进入系统安装程序，选择 Try or Install Ubuntu，再回车，现在你离成功安装 ubuntu 双系统只有一步之遥啦。

笔记 ”Linpus lite” 只是启动盘在某些主板上的显示名称，不同机器上可能显示为 U 盘的品牌名（如 Kingston、SanDisk）或其他字样。认准你的 U 盘对应的那个选项即可，不必纠结名字是否一致。



图 1.5: 开始安装 ubuntu

进入安装环节，首先选择语言为简体中文，点击” 安装 Ubuntu”，” 键盘布局” 确认是中文就好，下一步无线连接现在可以不用连。” 更新和其他软件” 界面选择” 正常安装”，勾选” 为图形或无线硬件，以及其它媒体格式安装第三方软件” 并设置自己用户的密码，注意这个密码后面会经常用到，所以不建议设的过长。

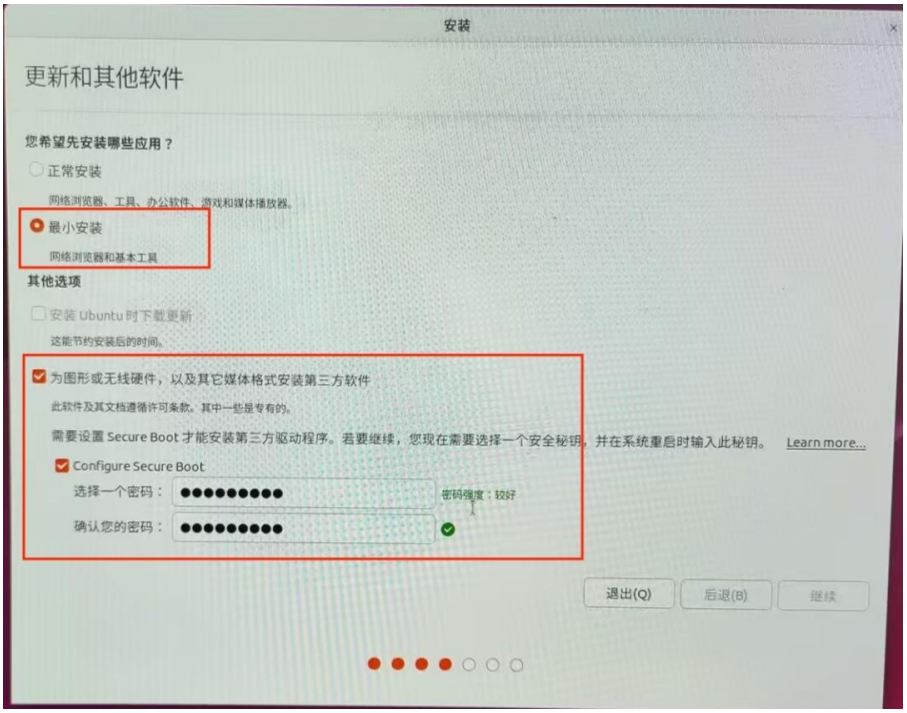


图 1.6: 设置密码

在下一步” 安装类型” 中一定一定要选择” 其它选项”!!!

然后来到” 安装类型”，在这里我们需要创建 ubuntu 的分区（注意内存大小依然是以 MB 为单位）

并挂载到相应的挂载点上。这里有两种方法：

1. **全部挂载到根目录下**

   找到之前压缩出来的空闲区域，双击选项，在弹出窗口的挂载点一栏输入”/” 即可。这种方式最简洁明了，适合啥也不明白的新手操作，相当于一个只有 C 盘的 Windows 系统，所以在备份系统镜像的时候无法按照不同分区分开备份，重装系统会导致所有数据丢失，因此建议还是按照第二种方法进行分区。

2. **挂载到不同目录下**

   先分逻辑分区，再分主分区。

   (a). ”/boot” 启动分区: 逻辑分区，一般选择 200M-2G，放置 Ubuntu 的启动引导文件。

   (b). ”/swap” 交换分区（虚拟内存）: 逻辑分区，一般分 16GB 就够了，也可以设置为物理内存大小，或是其倍数关系。如果你觉得你的物理内存够大（32G，64G 等），也可以不设置。

   (c). ”/” 根分区（root 分区）: 逻辑分区，80GB 左右差不多。

   (d). ”/home” 分区: 主分区，剩下的容量一起分给/home 就好，平时文件也都放这边，要的空间最大，不过也可以不设置，和根分区合并。

   点击压缩的空闲区域，点击左下角的加号：

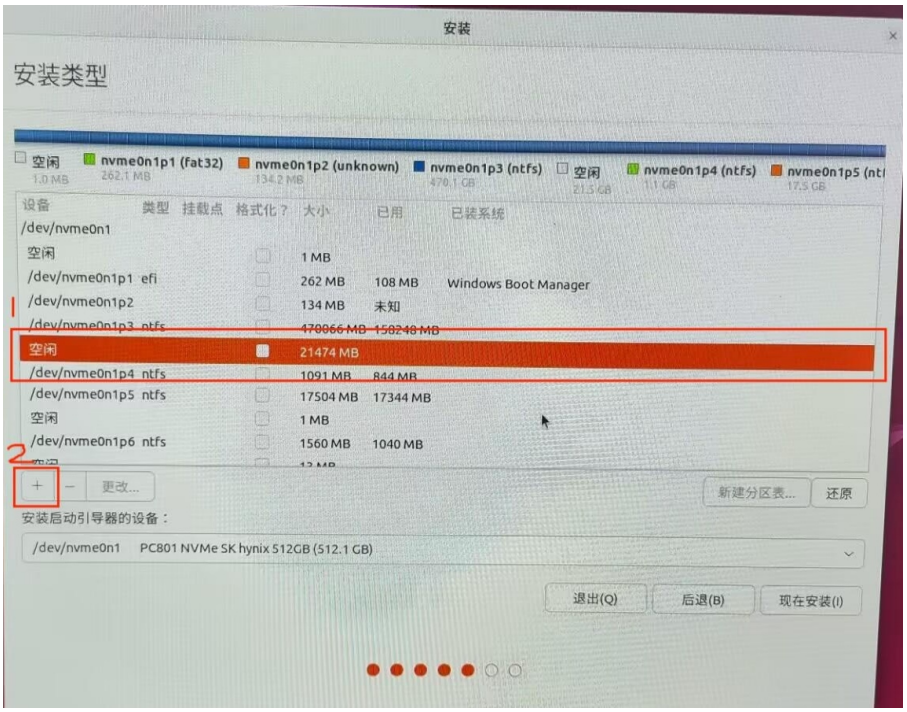


图 1.7: 选择分区

在弹出的窗口选择大小，新分区类型，新分区位置，文件系统类型，挂载点，依次对”/boot”，”/swap”，”/”，”/home” 进行分区

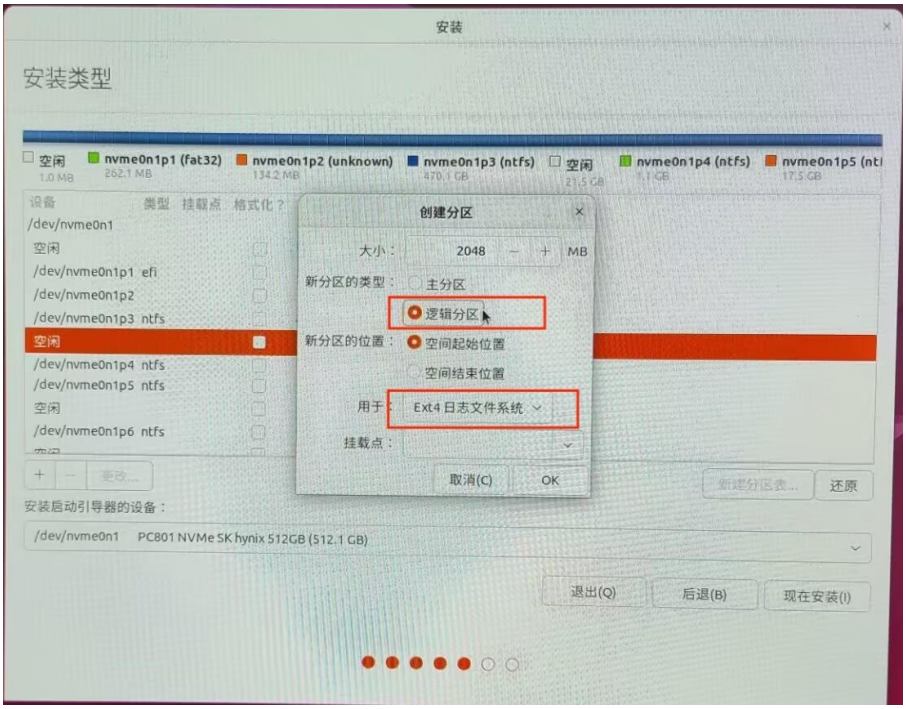


图 1.8: 分区

再次确定安装启动引导盘的设备选择”/boot” 分区那个路径

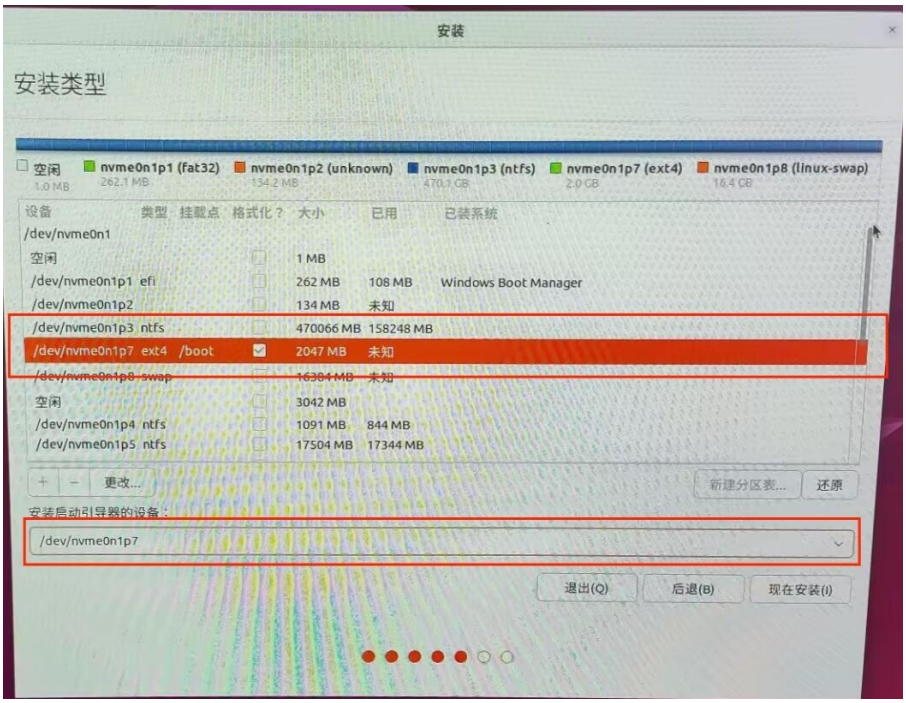


图 1.9: 安装启动引导盘设备路径

结论两种分区方案对比：

方案	优点	缺点
全部挂载到 /	操作简单，不易出错	重装系统会清空所有数据
/boot、/swap、/、/home 分开	重装时可保留 /home 中的数据	步骤较多，需提前规划容量

建议按第二种方案分区：日后重装系统时能保留 /home 下的工作数据——不然就只能抱着移动硬盘含泪抢救数据了。

分区完成后点击” 现在安装”，选择时区”Asia/Shanghai” 就行了。然后填写用户名和计算机密码，如果在上一步中填过了就不需要再填了。

点击” 继续”，系统就会对 ubuntu 系统开始安装了，出现安装完成的提示后点击现在重启”，等待一会，出现提示”Please remove the installation medium,then press ENTER” 后拔掉启动盘并按下 ENTER 键，进入 ubuntu 系统的启动引导。在启动页面选择 Ubuntu，电脑正常启动后就会进入 Ubuntu 系统，至此安装就全部完成啦。

ubuntu 系统初始是一个什么都没有的状态，但是大家配好了也是可以和 windows 一样的，接下来各位就按照自己的习惯和喜好进行配置吧。

笔记 安装完毕后，可能会出现电脑直接黑屏的情况，这也不用担心。这多半是由于缺少显卡驱动导致的。如果你的电脑中有独立显卡，那在使用 ubuntu 之前请在 BIOS 中将显卡选择为” 混合模式” 而非” 独显直连”，你需要先通过” 混合模式” 进入系统，然后安装完毕显卡驱动后，再切回” 独显直连”。

1.1.4 系统设置相关

ubuntu 系统安装完成后你的默认启动系统会变成 ubuntu，而有的电脑无法通过 ubuntu 的启动引导进入 windows 系统，这里我们有一种简单的办法，默认启动 Windows 选项，只在你需要 ubuntu 时主动去选择启动 ubuntu。

笔记 如果你觉得每次都需要手动调整这些比较繁琐，我们也有稍微复杂一些的方法。但是配置完毕后可以轻松切换系统，而且再也不会有丑丑的黑框框，详见下面的 grub 美化章节。

首先再次进入 BIOS，具体方法在图 1.4 已经写明。

进入 BIOS 后找到 Boot 相关设置，根据电脑提示将 Windows Boot Manager 移到 ubuntu 的启动引导上方，保存退出即可。

练习 1.2 重启电脑进入 BIOS，找到 Boot 启动顺序设置并记录当前的启动顺序（不需要修改），确认自己知道如何将 Windows Boot Manager 调整到首位。

1.1.5 系统扩容 *

如果系统中的文件已经多到空间不足了，那么此时我们就需要给 ubuntu 系统进行扩容。

扩容可以使用 ubuntu 自带的 gparted 分区管理工具，也可以类似第一步中使用 Windows 下的分区工具进行。不过上面既然介绍过了 Windows 下的工具，这里我们就换换口味，使用 ubuntu 下的 gparted 分区管理工具吧。

安装并启动 gparted 分区工具：

```
sudo apt-get install gparted
sudo gparted
```

进入 gparted 图形化界面，可以看到这里显示了电脑中的 ubuntu 和 windows 的所有分区，选中你要扩容的 ubuntu 分区，可以看到上方有一个箭头形状的图标亮起

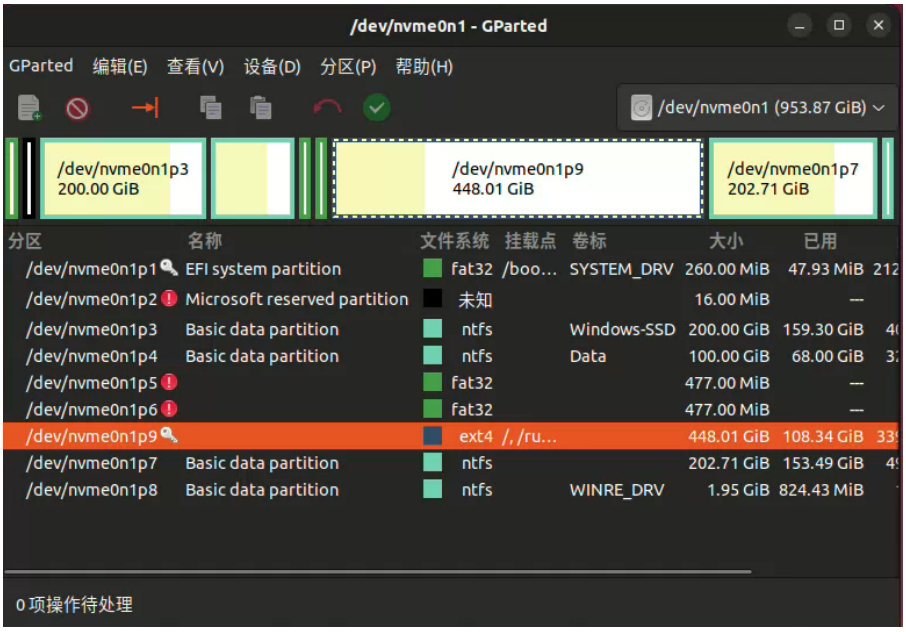


图 1.10: gparted 图形化界面

但是我们如果直接点击箭头图标却会发现以下报错：

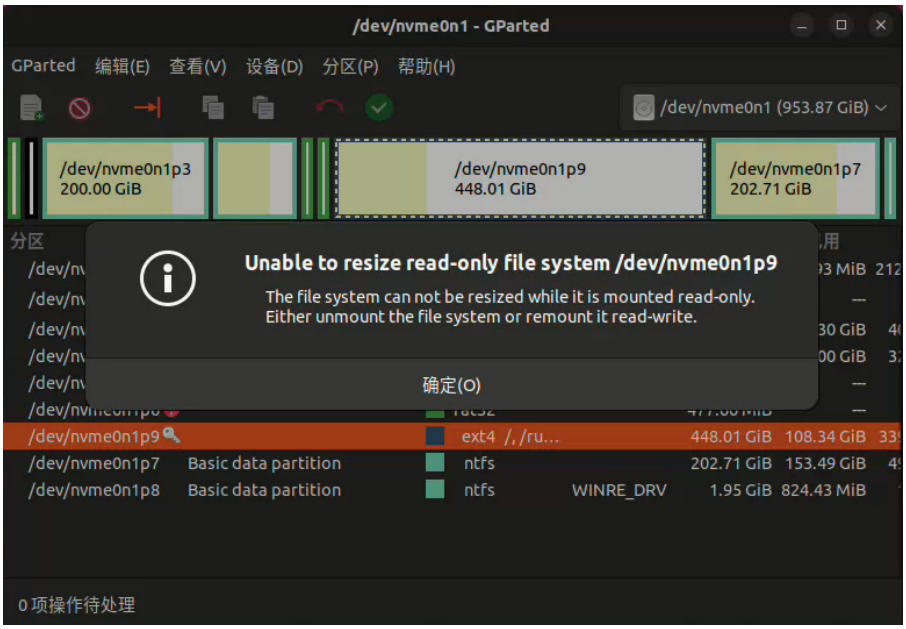


图 1.11: gparted 扩容失败

这是因为当你处于 ubuntu 系统时，自己系统的分区是被使用而被上锁的，所以此时你无法对分区进行修改。但是其实解决方法也很简单，我们可以在启动盘中的 ubuntu 系统中对自己的 ubuntu 分区进行修改。

在切换之前还有一件事要做：调节 linux-swap 和 extend 分区位置（如果存在的话），这是因为使用 gparted 扩容时只能对相邻的分区进行操作，如果 swap 和 extend 分区挡在 ubuntu 分区和 windows 分区之间，那么就不能从这个 windows 分区中分出空间给 ubuntu 分区了。

调节完毕之后插上安装 ubuntu 系统时制作的启动盘，重启电脑，重启时快速重复按下启动选项快捷键。进入启动选项界面后通过上下键选择 Linpus lite 选项，回车，进入系统安装程序，选择 Try or

Install Ubuntu，再回车，这些步骤和安装 ubuntu 时是一样的，只是进入安装页面后，这次我们要选择试用 Ubuntu



图 1.12: 进入试用 ubuntu 界面

选择试用后我们会进入到 ubuntu 的主页面，打开终端，再次打开 gparted

```
sudo apt-get install gparted
sudo gparted
```

此时就可以对原来 ubuntu 系统的分区进行修改了，修改完点击确认等待扩容操作完成即可。

1.1.6 grub 美化 *

也许你对那个系统启动时进入 ubuntu 的黑框框觉得厌烦了，而且每次启动系统都得手动点一下对应选项，或是觉得每次启动 ubuntu 都需要 BIOS 特意去设置一下真是太繁琐了，其实这只是因为你默认的这个 grub 没有美化而已，我们下面来解决这个问题。

在这个网页中

https://www.gnome-look.org/browse?cat=109&ord=latest

我们可以找到很多 grub 的主题。挑一个你喜欢的。笔者自己使用的是

https://www.gnome-look.org/p/1307852

这款主题。具体的安装教程在这个主题的 git 仓库中

https://github.com/vinceliuice/grub2-themes

如果你选择其他主题的话，教程位置可能变化，你需要自己找找。

你可以下载这个仓库到你的 ubuntu 系统中（github 的很多功能都和 git 有关，但是在介绍 git 之前，你可以通过点击网页中的 code 按钮点击 download zip 下载）。然后按照这个主题中的 README.md 文件的教程安装。

```
sudo ./install.sh -t tela -s 2k
```

完成了这一步后，如果你用的是笔者的这个主题，启动时就可以看到这样的界面了。而且 10s 没有操作可以自动进入 ubuntu 系统。

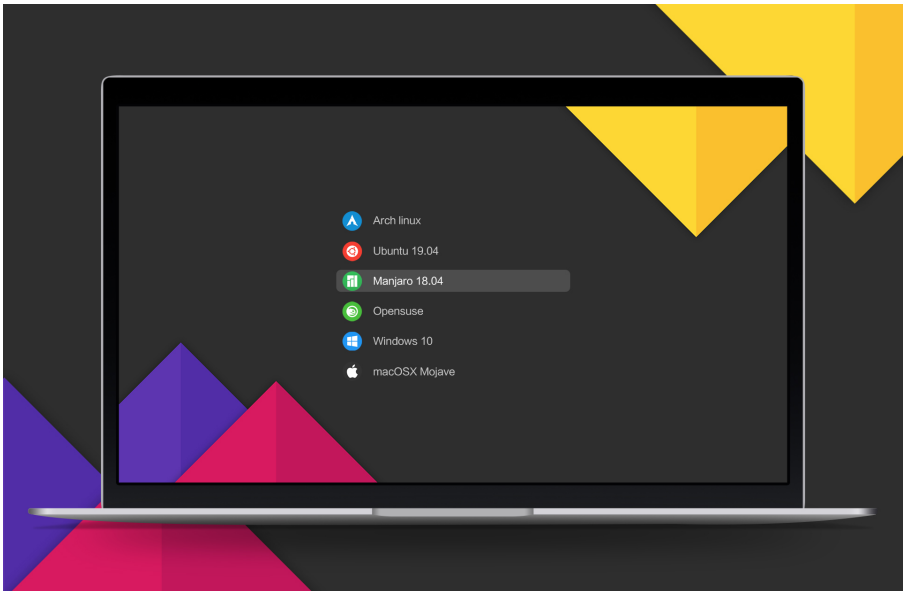


图 1.13: Tera 主题

不过也许你还需要在没有操作的情况下进入 Windows。那么我们需要小小的配置一下 grub 的配置文件。我们需要修改/etc/default/grub 这个文件，找到 GRUB_DEFAULT=0 这一行，看一下你的 windows 系统在启动时在哪一行（一般是第 3 行），我们将这个 0 修改为行数减一（一般是 2）即可。

1.1.7 桌面美化 *

grub 好看了，桌面也不能输。Ubuntu 22.04 默认的 GNOME 桌面虽然朴素，但换个主题、装几个插件之后，颜值完全可以反超 Windows。本节为选修内容。

主题与图标

GNOME 的主题和图标可以在 https://www.gnome-look.org/ 上挑选：左侧分类里 GTK3/GTK4 Themes 是窗口主题，Full Icon Themes 是图标包。下载得到的一般是压缩包，解压到对应目录即可被系统识别：

示例 1.1 把下载好的主题和图标解压到用户主题目录（目录不存在就先创建）：

```
mkdir -p ~/.themes ~/.icons
tar -xf 主题包.tar.gz -C ~/.themes
tar -xf 图标包.tar.gz -C ~/.icons
```

装好后需要 GNOME Tweaks（GNOME 优化）来切换主题和图标，顺便装上后面要用的插件管理器：

示例 1.2 安装 GNOME Tweaks 与 Extension Manager（插件管理器）：

```
sudo apt install gnome-tweaks gnome-shell-extension-manager
```

打开 Tweaks，在” 外观” 一栏即可选择刚装好的主题和图标。Extension Manager 则是本节的主角：它能直接在桌面上浏览、安装、开关各种 GNOME 插件，不用再去网页上折腾。

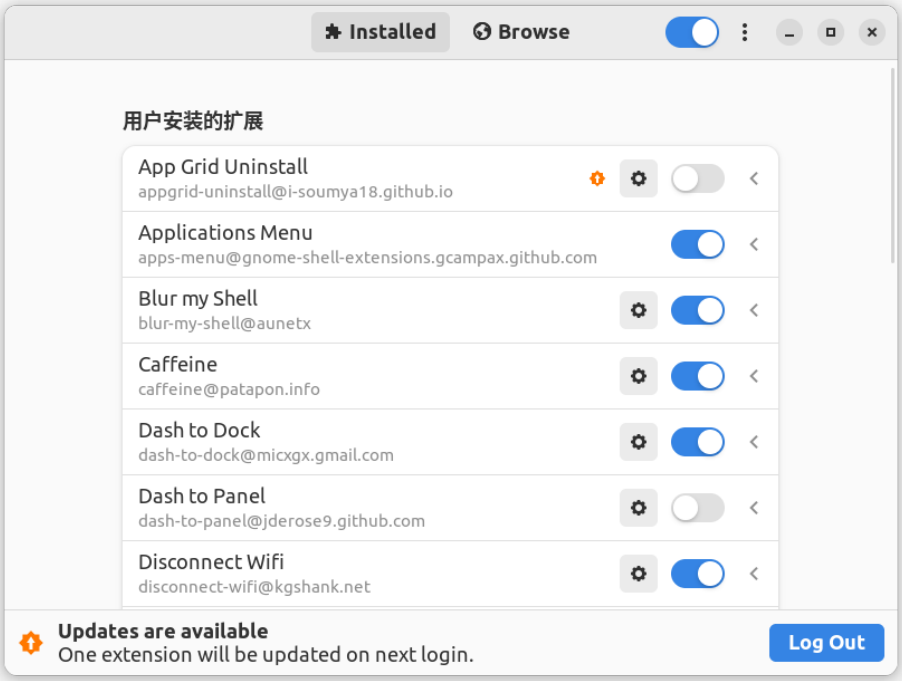


图 1.14: Extension Manager 插件管理界面

常用插件

打开 Extension Manager，在” 浏览” 页搜索插件名，点安装即可。以下是视觉组往届同学常用的几款：

插件名	作用
Caffeine	保持屏幕常亮，编译、跑数据集时防止自动锁屏
TopHat / Vitals	在顶栏实时显示 CPU、内存、温度等信息
Dark Mode Toggle / Night Theme Switcher	一键或按计划自动切换深色/浅色模式
Dash to Dock	把左侧 Dock 栏变成可自定义的程序坞
Dash to Panel + ArcMenu	合成类似 Windows 的底部任务栏与” 开始菜单”
Rounded Window Corners Reborn	给窗口加上圆角
Blur my Shell	给顶栏、Dock 等加上毛玻璃半透明效果
Sound Input & Output Device Chooser	在右上角菜单直接切换声音输出/输入设备
Clipboard Indicator	剪贴板历史记录，复制过的内容随时找回来

建议先挑两三个顺手的装上，别一次全装——插件之间偶尔互相打架，装多了排查起来很痛苦。

笔记 所有插件的官方来源是 GNOME 扩展商店 https://extensions.gnome.org/，Extension Manager 的” 浏览” 页就是它的目录。上表插件的主页链接：

- Caffeine：https://extensions.gnome.org/extension/517/caffeine/

- TopHat：
  https://extensions.gnome.org/extension/5219/tophat/
- Vitals：https://extensions.gnome.org/extension/1460/vitals/
- Night Theme Switcher：
  https://extensions.gnome.org/extension/2236/night-theme-switcher/
- Dash to Dock：https://extensions.gnome.org/extension/307/dash-to-dock/
- Dash to Panel：https://extensions.gnome.org/extension/1160/dash-to-panel/；ArcMenu：
  https://extensions.gnome.org/extension/3628/arcmenu/
- Rounded Window Corners Reborn：
  https://extensions.gnome.org/extension/7048/rounded-window-corners-reborn/
- Blur my Shell：https://extensions.gnome.org/extension/3193/blur-my-shell/
- Sound Input & Output Device Chooser：
  https://extensions.gnome.org/extension/906/sound-output-device-chooser/
- Clipboard Indicator：https://extensions.gnome.org/extension/779/clipboard-indicator/

笔记 插件由社区维护，系统大版本升级（尤其是 GNOME 版本升级）后部分插件可能失效，届时在 Extension Manager 中更新或暂时禁用即可。Blur my Shell 这类特效插件会占用一定的显卡资源，笔记本续航会略有下降。

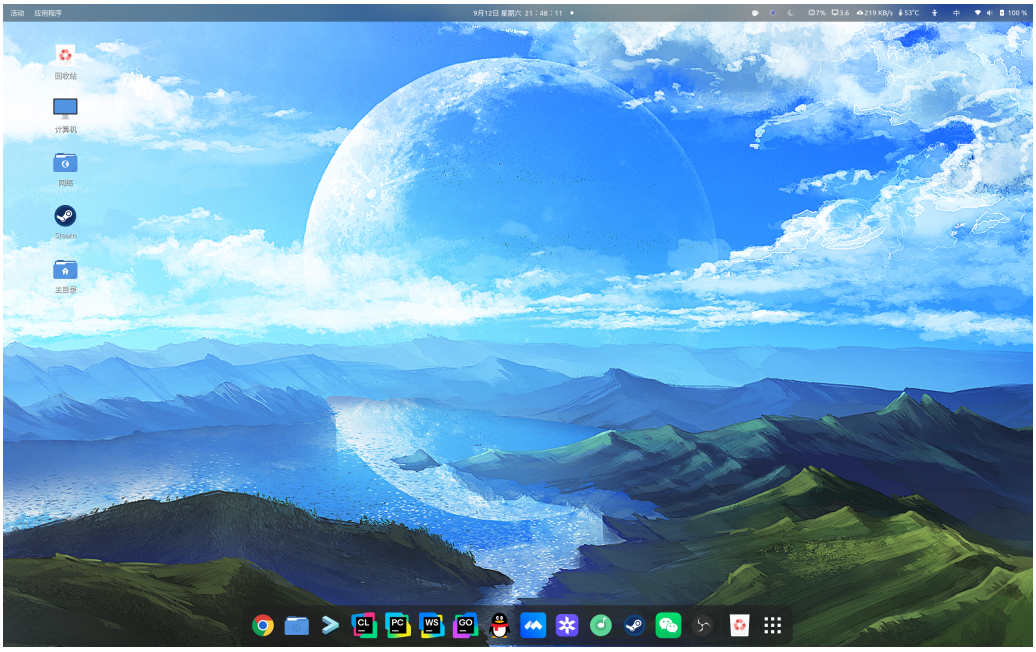


图 1.15: 笔者美化后的桌面效果

练习 1.3 安装 Extension Manager，启用 Caffeine 和 Clipboard Indicator 两个插件，并验证：点亮 Caffeine 后屏幕不再自动熄灭；复制几段文字后能从剪贴板历史中找回。

1.2 系统数据备份 *

配置环境前须完成本节。系统异常时，快照恢复通常优于重装。恭喜你已经成功安装了 ubuntu 双系统，作为新手的你想来在刚刚的安装过程中一定是非常小心谨慎的，毕竟你也不想系统出现什么差

错。在后续的工作中我们经常需要安装各种各样的安装包并对系统的环境进行配置，如果不小心配错了东西，重则系统可能无法正常工作（简称寄了）。这时候你就需要在启动页面进入“Ubuntu 高级选项”对系统进行修复，或者最惨的情况是需要直接重装系统。当然我们肯定不愿看到这一点，因此我们很有必要学习对系统的环境进行备份，这样即使环境配崩了也能恢复到配环境之前的状态，保留之前的工作。当然如果你比较自信这一章不看也不是不行，只是到时候别后悔。

1.2.1 timeshift 系统快照

timeshift 是 linux 上一款用于数据备份和恢复的软件，使用时会像相机一样给系统照个相。它的使用流程三分钟就能学会，比事后后悔便宜得多。

安装 timeshift

示例 1.3 使用 apt 安装 timeshift：

```
sudo apt update
sudo apt install timeshift
```

ubuntu 也可以在软件中心直接搜索下载。

制作快照

首先最好准备一个 U 盘，然后点击 timeshift 的图标启动，或者在终端输入

```
sudo timeshift-launcher
```

进入 timeshift 图形化操作界面，首先选择快照类型。RSYNC 是一种常见的文件同步协议，在 Timeshift 中用于增量式系统备份。备份时仅复制有改动的部分，因此通常比完整备份要小，创建速度更快。Btrfs 是一种支持快照等高级特性的文件系统。Timeshift 利用 Btrfs 的快照功能，可以创建系统的只读快照。这种类型的快照创建速度比 RSYNC 更快，占用的存储空间更小。但前提是，你的系统必须使用 Btrfs 文件系统。一般来说我们选择“RSYNC”。

备份位置最好选择在非系统硬盘或移动 U 盘，但要保证磁盘类型是 ext4 等 linux 类型。否则暂时选系统盘也行。选好盘后备份数据默认会保存在该硬盘下根目录下的 /timeshift 目录下。

备份时间有每月备份一次，每周备份一次、每日备份一次、每小时备份一次、每次开机备份一次，数字表示最多存几个快照，多了的话会删除最旧的那次快照。

“用户主目录”那栏根据自己需要的备份情况来选择，“筛选”那栏也是，可以选择添加所需目录或将其过滤。

快照恢复

如果系统损坏不大还可以正常使用，那么可以进入 timeshift 图形化界面，找到先前已经制作好的快照



图 1.16: timeshift 图形化界面

点击“恢复”，确认要恢复的目录后点击“下一步”即可进行恢复操作

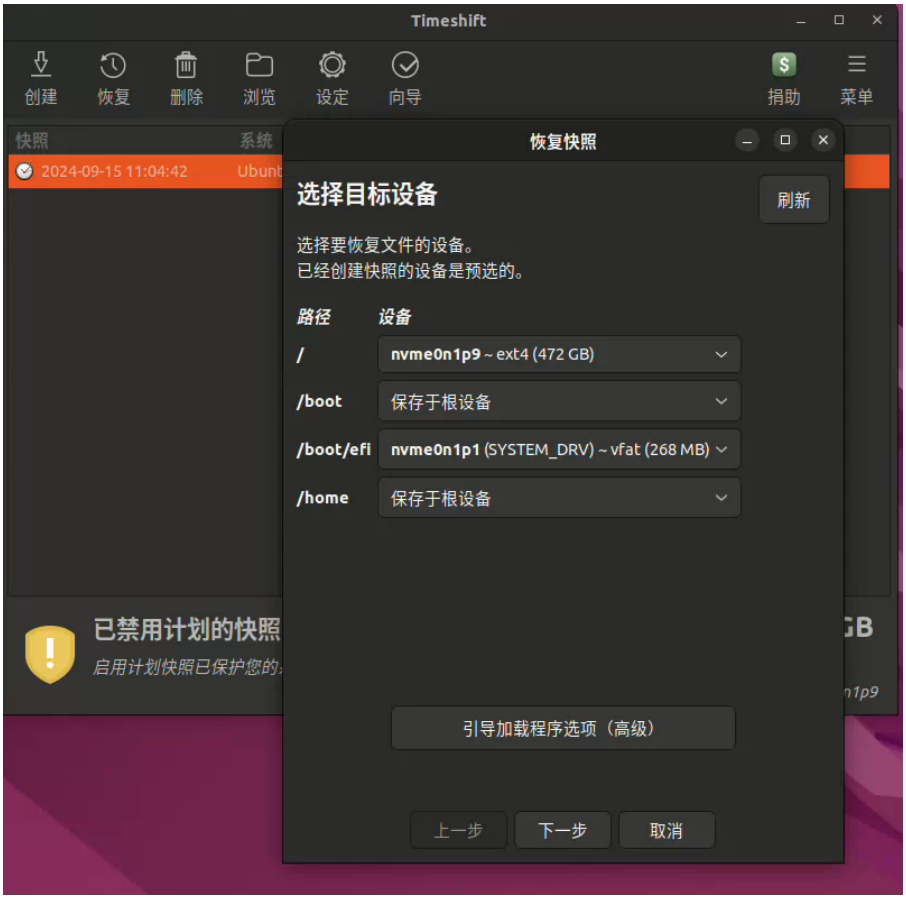


图 1.17: timeshift 快照恢复

当然这个过程还可以通过命令行来完成：

示例 1.4 使用命令行制作一个快照（磁盘名可以通过 `fdisk -l` 或 `df -TH` 查看）：

```
sudo timeshift --create --comments "快照名" --backup-device /dev/磁盘名
```

示例 1.5 查看已存在的快照，并选择其中一个进行恢复（如快照'2024-07-01_18-00-00'）：

```
sudo timeshift --list
sudo timeshift --restore --snapshot '2024-07-01_18-00-00' --skip-grub
```

如果已经无法加载图形化桌面，可以在开机时按“ctrl+alt+F1”（一般 F1 到 F6 都可以）进入 tty 终端，可以输入命令进行恢复。

如果连命令终端都进不去，只能通过启动盘再次进入安装 ubuntu 的页面，选择试用 ubuntu，然后安装 timeshift 再进行恢复。

练习 1.4 使用 timeshift 图形界面或命令行为当前系统创建一个快照，并用 `sudo timeshift --list` 确认快照已生成。

1.2.2 系统镜像备份

linux 秉承着一切皆文件的思想，系统备份就相当于把整个根目录所有文件打包压缩保存。

镜像制作

备份前先切换到 root 用户，避免权限问题，然后切换到根目录。

示例 1.6 将整个根目录打包压缩，备份到挂载在 `/media` 下的移动硬盘中：

```
tar -cvpzf /media/Disk/myDisk/ubuntu_backup@$(date +%Y-%m-%d).tar.gz --exclude=/proc
    --exclude=/tmp --exclude=/boot --exclude=/home --exclude=/lost+found --exclude=/media
    --exclude=/mnt --exclude=/run /
```

下面解释一下上面这条命令：tar 就是一个打包命令，各参数的含义见第 1.4 节的 tar 参数表。

`/media/Disk/myDisk/` 是备份文件的存放路径。这里，`myDisk` 表示你的移动硬盘名字；移动硬盘挂载在 `/media/Disk` 目录下。文件名中的 `$(date +%Y-%m-%d)` 是一条 shell 命令，会把执行时的日期写进文件名，方便日后辨认不同时间的备份。接下来看看 –exclude 参数中的内容。

假设现在 ubuntu 中有四个分区：/、/home、/boot、swap，其中/home、/boot 两个分区出问题的概率很小，而且变动也不会太频繁，因此在定期备份时可以减小对这两个分区的备份频率。一般系统坏了都是/分区的问题，/home 没什么关系。再来看看其他排除的目录：

/proc：一个虚拟文件系统，系统运行的每一个进程都会自动在这个目录下面创建一个进程目录。既然是系统自动创建，也就没必要备份的必要了。

/tmp：一个临时文件夹，系统的一些临时文件会放在这里。

/lost+found：系统发生错误时（比如非法关机），可以在这里找回一些丢失文件。

/media：多媒体挂载点，像 u 盘、移动硬盘、windons 分区等都会自动挂载到这个目录下。

/mnt：临时挂载点，你可以自己挂载一些文件系统到这里。

/run：系统从启动以来产生的一些信息文件。

另外，有可能备份到最后系统会提示”tar: 由于前次错误, 将以上次的错误状态退出”，这个警告可以忽略，没什么影响的。

镜像恢复

这里有两种还原方式：

1. 直接操作

   如果你系统出问题了，但是还可以进入终端，那就可以直接解压备份文件进行还原。切换到 root，并且换到/根目录，打开终端，输入

```
tar -xvpzf /media/Disk/myDisk/ubuntu_backup@2016-6-6.tar.gz -C /

```

2. 使用启动盘还原

   如果你连系统都不能登录了，就要使用 U 盘启动盘进行还原了。依旧是先进入试用 ubuntu 界面，切换到 root，打开终端，输入

```
mkdir /mnt/sys
mount /dev/sdaX /mnt/sys
tar -xvpzf /media/Disk/myDisk/ubuntu_backup@2016-6-6.tar.gz -C /mnt/sys

```

   注意先创建一个临时目录用于挂载你的/根目录分区，sdaX 代表你的/根目录分区，如果不知道就用 fdisk -l 查看一下，另外如果你的移动硬盘没有被自动挂载，你也需要手动创建一个临时目录进行挂载。

   因为 tar 还原是只会覆盖相同的文件，但是这种方法只是恢复备份时的文件，就是说如果某些文件丢失或损坏了，这样可以恢复修复这些文件，但不能删除自备份到恢复前这期间所生成的其它文件，说白了就是假如你备份系统时有 1234 这四个文件，如果三天后，由于某些原因变成了 1234'5（4 改变了），你恢复后，就会变成 12345，其中 4' 恢复成备份时的文件，5 保留。

笔记 tar 恢复是直接向目标目录解压覆盖，-C 后面的路径写错会把系统文件释放到错误的位置。恢复前请确认解压路径和备份文件都没有选错。

结论两种备份方式对比：

方式	适用场景	特点
timeshift 快照	日常配置环境之前	增量备份，速度快，可一键恢复
tar 系统镜像	系统迁移或长期归档	完整打包，体积较大，恢复需手动解压

日常防护用 timeshift 就够了：配环境前拍一张，环境炸了就回档。tar 镜像则相当于通关前的手动存档位，适合在系统大改之前留一份完整存档。

练习 1.5 说出镜像制作命令中 tar 的 -c、-v、-p、-z、-f 五个参数各自的含义（可查阅 man 手册或第 1.4 节的参数表）。

1.3 卸载重装 Ubuntu 系统 *

如果你真的碰巧忘记备份镜像了，而你修改的环境变量又真的无法修复回去，或者有修复它的时间还不如重装一遍，那还是老老实实卸载重装吧。当然，希望各位自己永远不要用到这里的方法，在每次配环境前一定养成备份镜像的好习惯。

笔记重装会格式化原有的 Ubuntu 分区，分区上的所有数据都会消失，且没有撤回键。动手之前，请先把原系统中的工作数据备份到 Windows 分区或 U 盘。

重装过程其实很简单，就是比较耗时。我们直接按照原先的安装 Ubuntu 方式，一直运行到选择分区的那一步，然后将原先的 Ubuntu 分区格式化后重新分配，随后继续按照原先一样的正常安装过程就好。

1.4 文件系统与常用命令

ubuntu 系统提供了完整的图形化界面，日常操作点点鼠标都能完成。但命令行才是真正的基本功：装环境、连开发板、远程调试，样样绕不开它。下面我们就把最常用的操作过一遍。

1.4.1 linux 文件系统基本概念

linux 目录结构

首先我们来看一下 linux 系统的树状目录结构：

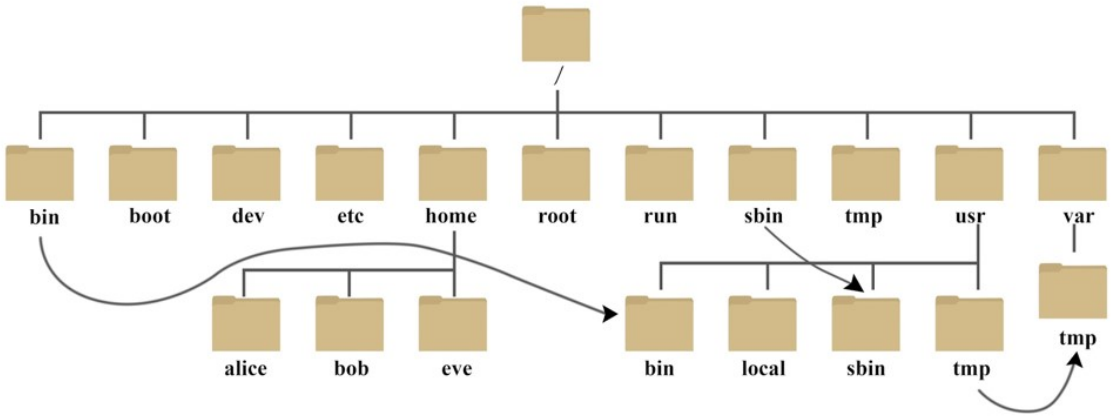


图 1.18: linux 系统树状目录结构

可以看到整个 linux 系统是由若干个文件夹反复嵌套组成的，目录结构如同一棵倒置的树，因此最上面的文件夹被称为“根目录”。

“一切皆文件”

在 linux 系统中有一个非常重要的、区别于 windows 系统的概念是“一切皆文件”，也就是把一切资源都看作是文件，包括硬件设备（通常称为设备文件），这样好处是用户可以用读写文件的方式实现对硬件的访问。举个简单的例子：在 linux 中则可以在/dev 目录下找到新出现的文件，注意这里的文件表示的是 U 盘这个对象，你可以像操纵文件一样操纵这个对象，选择把他挂载到一个特定目录，然后读取其中的文件。简单来说：你以为是 U 盘设备？其实是我文件哒！

笔记如果你非要较真的话，网卡相关的部分并没有文件，不过这里不是重点，详情大家可以自己研究。

挂载

在上述例子中，U 盘与 linux 系统建立连接的过程称为“挂载”。Linux 系统中没有磁盘分区的逻辑概念，任何一个种类的文件系统被创建后，都需要挂载到某个特定的目录才能使用，这个过程相当于

激活一个文件系统，使它能够使用。简单来说“挂载”就是将 linux 本身文件目录与硬件设备的文件目录合二为一，下图中展示了一个简单的挂载实例。

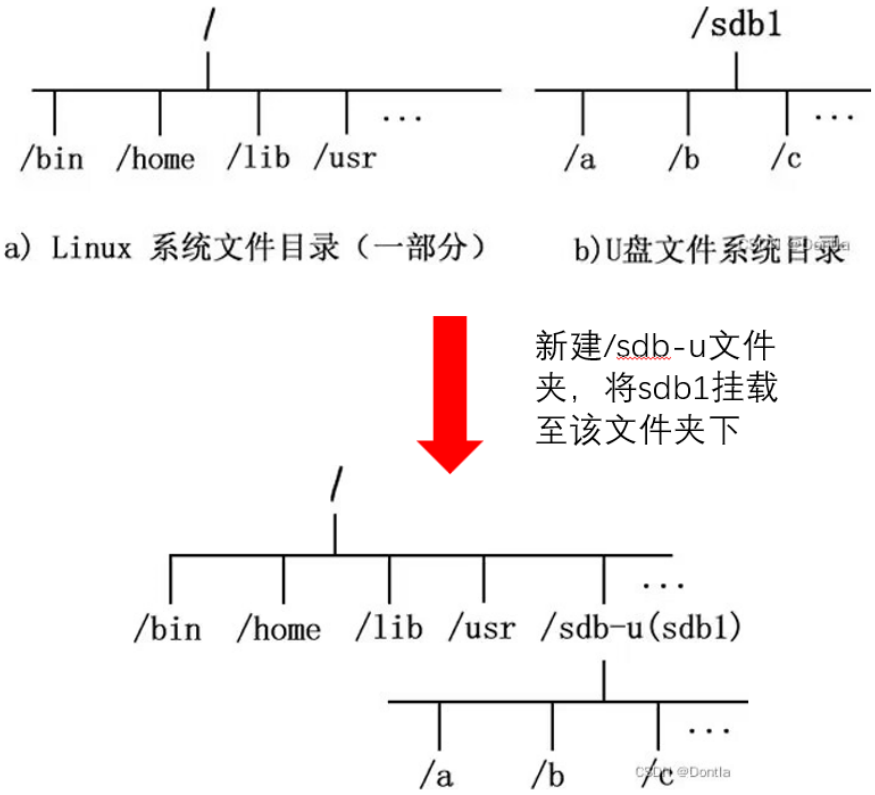


图 1.19: 挂载实例

Windows 的文件系统挂载使用其内部机制完成，用户基本无法探知其过程。而 Linux 使用 mount 工具对文件系统进行挂载。挂载文件系统时，需要明确挂载点，在创建文件系统后，操作系统会提示将此文件系统挂载至哪个位置，这个位置就是挂载点，通常 U 盘等设备系统会自动挂载到“/media”目录下。挂载操作在备份系统镜像时经常用到。

主要文件夹及其作用

/bin：bin 是 Binaries (二进制文件) 的缩写, 这个目录存放着最经常使用的命令。

/boot：这里存放的是启动 Linux 时使用的一些核心文件，包括一些连接文件以及镜像文件。

/dev：dev 是 Device(设备) 的缩写, 该目录下存放的是 Linux 的外部设备，在 Linux 中访问设备的方式和访问文件的方式是相同的。

/etc：etc 是 Etcetera(等等) 的缩写, 这个目录用来存放所有的系统管理所需要的配置文件和子目录。

/home：用户的主目录，在 Linux 中，每个用户都有一个自己的目录，一般该目录名是以用户的账号命名的，如上图中的 alice、bob 和 eve。

/lib：lib 是 Library(库) 的缩写这个目录里存放着系统最基本的动态连接共享库，其作用类似于 Windows 里的 DLL 文件。几乎所有的应用程序都需要用到这些共享库。

/opt：opt 是 optional(可选) 的缩写，这是给主机额外安装软件所摆放的目录。比如你安装一个 ORACLE 数据库则就可以放到这个目录下。默认是空的。

/root：该目录为系统管理员，也称作超级权限者的用户主目录。

/usr：usr 是 unix system resources(unix 系统资源) 的缩写，这是一个非常重要的目录，用户的很多

应用程序和文件都放在这个目录下，类似于 windows 下的 program files 目录。

/var：var 是 variable(变量) 的缩写，这个目录中存放着在不断扩充着的东西，我们习惯将那些经常被修改的目录放在这个目录下。包括各种日志文件。

在这些文件夹中，绝大多数目录都包含重要的系统文件，如果随意删除可能会导致系统出错，因此在平时操作时要多加注意。

✍ **练习 1.6** 在终端执行 `ls /` 查看根目录下的文件夹，对照本节内容说出 `/etc`、`/home`、`/dev` 各自的作用；再执行 `ls /dev`，找出其中的设备文件。

1.4.2 linux 系统基本操作

由于篇幅有限，本部分仅能列出平时操作中最常用的部分，其它内容如需用到请读者自行查阅资料。

01 用户管理

命令	作用
id [name]	查看用户 UID 和 GID 信息
who	查看当前所有登录的用户列表
whoami	查看当前登录用户的账户名
su - [name]	切换用户，并且切换目录
exit	退出当前登录用户
passwd [name]	设置用户密码

说明：

1. - 可以切换到用户家目录，否则保持位置不变
2. su 不接用户名，可以切换到 root 用户
3. 命令前加 sudo：以超级用户权限运行命令

示例 1.7 查看当前登录用户的账户名与 UID/GID 信息：

```
whoami
id
```

02 系统信息

语法	作用
free [options]	显示系统内存使用情况
uname [options]	显示电脑以及操作系统的相关信息
df [options] [file]	显示文件系统的磁盘空间使用情况
du [options] [directory]	显示目录或文件的磁盘占用量
top	动态显示系统中运行的进程及资源占用情况
hostname	查看当前主机名

free 选项和参数

参数	作用
$-h$	以易读的单位（GB、MB 等）显示
$-m$	以 MB 为单位显示
$-g$	以 GB 为单位显示
$-s$ [秒数]	每隔指定秒数持续刷新显示

uname 选项和参数

参数	作用
$-a$ $--all$	显示全部的信息
$-m$ $--machine$	显示处理器类型
$-n$ $--nodename$	显示主机名
$-r$ $--release$	显示内核版本号
$-s$ $--sysname$	显示操作系统名称
$-v$	显示操作系统的版本
$--help$	显示帮助
$--version$	显示版本信息
$-p$	显示处理器类型（与 -m 选项相同）

df 选项和参数

参数	作用
$-h$	以易读的单位（GB、MB 等）显示
$-T$	同时显示文件系统类型

du 选项和参数

参数	作用
$-h$	以易读的单位（GB、MB 等）显示
$-s$	只显示总计，不列出每个子目录
$--max-depth=1$	只统计指定层级的子目录

你可以用 man 命令查阅完全的参数详情（例如 `man du` 查询 du 命令的全体参数）。

示例 1.8 统计当前目录下各子目录的磁盘占用，并按易读单位显示：

```
du -h --max-depth=1 .
```

练习 1.7 执行 `df -hT` 查看各文件系统的磁盘使用情况，找到你的根分区（挂载点为 `/` 的一行），记录其文件系统类型、容量与已用空间。

03 软件安装

软件安装命令有 apt，snap，yum 等等，在我们 ubuntu 系统中用的是 apt，不过有时候也得用到 snap。安装和卸载软件需要 root 权限。

options：可选，选项包括 -h（帮助），-y（当安装过程提示选择全部为”yes”），-q（不显示安装的过程）等等

语法	作用
$apt\ [options]\ [command]\ [package...]$	查找、安装、升级、删除软件包

command：要进行的操作

package：安装的包名

apt 常用命令如下

命令	作用
$apt\ update$	列出所有可更新的软件清单
$apt\ upgrade$	升级软件包
$apt\ list\ --upgradable$	列出可更新的软件包及版本信息
$apt\ full-upgrade$	升级软件包前先删除需要更新软件包
$apt\ install\ <package1><package2>$	安装指定的软件
$apt\ install\ --only-upgrade\ <package>$	更新指定的软件
$apt\ show\ <package>$	显示软件包具体信息
$apt\ remove\ <package>$	删除软件包
$apt\ autoremove$	清理不再使用的依赖和库文件
$apt\ purge<package>$	移除软件包及配置文件
$apt\ search\ <keyword>$	找软件包命令
$apt\ list\ --installed$	列出所有已安装的包
$apt\ list\ --all-versions$	列出所有已安装的包的版本信息

与上文类似，其余命令与参数请查阅 man 手册（`man apt`）。

示例 1.9 更新软件清单后安装 `tree`（一个以树状图显示目录结构的小工具）：

```
sudo apt update
sudo apt install tree
```

练习 1.8 使用 `apt search` 搜索并安装一个你感兴趣的小软件（如 `cowsay`），运行验证后用 `sudo apt remove` 将其卸载。

04 文件与目录管理

先认识几个常用概念，后面的命令都要用到它们：

1. 绝对路径：路径的写法，由根目录 / 写起，例如：/usr/share/doc 这个目录。
2. 相对路径：路径的写法，相对于当前所处的文件夹位置，不是由 / 写起，例如由 /usr/share/doc 要到 /usr/share/man 底下时，可以写成：cd man
3. . ：表示当前目录
4. .. ：表示上一级目录
5. 文件类型：分为目录（d），文件（-），链接文件（l），装置文件里可供储存的接口设备（b），装置文件里面的串行端口设备（c）

6. 文件基本属性：分为可读（r），可写（w），可执行（x）。Linux 文件属性有两种设置方法，一种是数字，一种是符号。使用数字来代表各个权限时，各权限的分数对照为：r:4，w:2，x:1
7. 文件所有者（user）：表示该文件或目录的拥有者，也就是文件权限中的第一个权限对象（u）。
8. 用户组（group）：表示该文件或目录所属的用户组，也就是文件权限中的第二个权限对象（g）。文件权限通常依次针对文件所有者、所属用户组以及其他用户（others）进行设置。
9. 其他用户（others）：表示既不是文件所有者，也不属于该文件所属用户组的其他用户，是文件权限中的第三个权限对象（o）。
10. 路径名展开（pathname expansion）：属于 Shell（如 Bash）的语法，用于使用通配符匹配文件名。常见的通配符包括 * 、 ? 和 [ ] 。例如： *.txt 可以匹配当前目录下所有以.txt 结尾的文件；? 表示匹配一个字符； [abc] 表示匹配 a、b 或 c 中的一个字符。

处理文件和目录的命令有很多，这里分表整理了一些常用命令的语法及其选项和参数仅供大家参考

常用处理文件和目录的命令

语法	作用
ls	列出当前目录中的文件和子目录
pwd	显示当前工作目录的路径
$cd\ /path/to/directory$	切换工作目录
$mkdir\ [directory]$	创建新目录
$cp\ [source]\ [destination]$	复制文件或目录
$rm\ [filename]$	删除文件或目录
$mv\ [old]\ [new]$	移动或重命名文件或目录
$ln\ [options]\ [source]\ [linkname]$	创建链接文件
$chmod\ [permissions]\ [filename]$	修改文件或目录的权限
$tar\ [options]\ -f\ archive.tar\ [files...]$	解压或压缩文件或目录
$zip\ [options]\ output.zip\ [files...]$	压缩文件或目录
$unzip\ archive.zip$	解压文件或目录

ls 选项和参数

参数	作用
$-a$	列出当前目录中全部的文件，包括隐藏文件
$-d$	仅列出目录本身，而不是列出目录内的文件数据
$-l$	长数据串列出，包含文件的属性与权限等

mkdir 选项和参数

参数	作用
$-m$	配置文件的权限
$-p$	递归创建所需目录

cp 选项和参数

参数	作用
$-a$	相当于 -pdr 的意思
$-d$	若来源档为链接档的属性 (link file)，则复制链接档属性而非文件本身
$-f$	强制，若目标文件已经存在且无法开启，则移除后再尝试一次
$-i$	若目标文件或目录已经存在，在覆盖时会先询问用户
$-l$	进行硬式链接的链接档创建，而非复制文件本身
$-p$	连同文件的属性一起复制过去，而非使用默认属性 (备份常用)
$-r$	递归持续复制，用于目录的复制 (常用)
$-s$	复制成为符号链接档，即“捷径”文件
$-u$	若 destination 比 source 旧才升级 destination

rm 选项和参数

参数	作用
$-f$	强制的意思，删除时不询问且忽略不存在的文件
$-i$	互动模式，在删除前会询问使用者是否动作
$-r$	递归删除, 常用在目录的删除



笔记 `rm -rf` 会不做任何确认地递归删除目标及其全部内容，且命令行下的删除不进回收站，无法恢复。这条命令删库跑路一气呵成，使用前请三思——你的存档没有读档功能。脚本中使用 `rm -rf $dir/` 这类写法时，务必先检查变量非空：变量一旦为空，路径可能直接变成根目录。

mv 选项和参数

参数	作用
$-f$	强制的意思，如果目标文件已经存在，不会询问而直接覆盖
$-i$	若目标文件已经存在时，会询问是否覆盖
$-u$	若目标文件已经存在，且 source 比较新，才会升级

ln 选项和参数

参数	作用
$-s$	创建符号链接（软链接），不加该选项时默认创建硬链接
$-f$	强制创建，若链接文件已存在则直接覆盖

结论硬链接、软链接与复制的区别：

- 硬链接：与源文件指向同一份数据，删除源文件不影响链接；但不能跨文件系统，也不能给目录创建；
- 软链接（符号链接）：类似 Windows 的快捷方式，只记录源文件的路径；源文件被删除后链接失效，但可以跨文件系统、可以链接目录；
- 复制（cp）：得到一份完全独立的新文件，与源文件互不影响。

chmod 选项和参数

参数	作用
$-R$	进行递归的持续变更，连同次目录下的所有文件都会变更
u	user，用户身份
g	group，组
o	others，其他身份
a	all，全部的身份
$+\ -\ =$	对权限加入，除去，设定

使用 chmod 命令修改文件或目录权限有两种方式。一是用数字类型的权限属性更改，通用形式为：

chmod [-R] xyz 文件或目录

其中 xyz 是三位数字，分别对应所有者、用户组、其他用户的权限，每位数字为该身份 rwx 属性数值（r:4，w:2，x:1）的和。二是用符号类型改变文件权限，通用形式为：

chmod [身份] +/-/= [permissions] [file name]

示例 1.10 为 `ex1.py` 的文件拥有者增加可执行权限；再将 `run.sh` 递归地改为所有人可读可写可执行：

```
chmod u+x ex1.py
chmod -R 777 run.sh
```



笔记 不要图省事对所有文件都 `chmod 777`——这相当于把家门钥匙挂在门外，谁路过都能进来逛一圈。权限放得越宽，误操作和恶意篡改的风险越大；脚本只需要给所有者加上执行权限（`chmod u+x`）就够了。



练习 1.9 在家目录下创建 `practice` 目录，在其中新建文件 `hello.txt`，将它复制为 `hello_bak.txt`，再把原文件移动到 `practice/sub` 目录（该目录需用 `mkdir` 一并创建），最后删除整个 `practice` 目录。

tar 选项和参数

参数	作用
$-c$	创建新的归档文件
$-f$	指定归档文件的名称
$-v$	显示详细输出，列出被添加/解压到归档中的文件
$-x$	解压归档文件
$-z$	使用 gzip 压缩归档文件
$-r$	向已存在的归档中追加文件

tar 命令的参数繁多，此处只列举了以上最最常见的参数。

zip 选项和参数

参数	作用
$-r$	递归压缩目录及其子目录中的所有文件
$-e$	为压缩文件设置密码保护
$-q$	静默模式，不显示压缩过程
$-v$	显示详细的压缩过程
$-x$	排除某些文件或目录，不进行压缩
$-m$	压缩后删除原始文件
-0 ~ -9	指定压缩级别，-0 表示存储不压缩，-9 表示最高压缩率，默认是 -6

05 查找、管道与重定向

find 的语法

语法	作用
$find\ [path]\ [option]\ [expression]$	在指定目录下查找符合条件的文件或目录

path：要查找的起始目录，省略时默认从当前目录开始查找
expression：查找条件，例如文件名、文件类型等

find 的参数

参数	作用
$-name\ pattern$	按文件名查找，支持通配符
$-iname\ pattern$	按文件名查找，忽略大小写
$-type\ f/d/l$	按类型查找，f 为普通文件，d 为目录，l 为链接文件
$-size\ [+/-]n$	按文件大小查找，+ 表示大于，- 表示小于
$-mtime\ [+/-]n$	按修改时间查找，n 表示 n 天前

其余参数请查阅 man 手册（`man find`）。

示例 1.11 在 `/home` 目录下查找所有以`.txt` 结尾的文件，再在当前目录下查找名为 `test` 的目录：

```
find /home -name "*.txt"
find . -type d -name "test"
```

grep 的语法

语法	作用
$grep\ [option]\ pattern\ [files]$	查找文件里符合条件的字符串或正则表达式

pattern：表示要查找的字符串或正则表达式
files：要查找的文件名，可以同时查找多个文件，如果省略 files 参数，则默认从标准输入中读取数据

重定向是一种强大的功能，它允许用户改变命令的输入或输出方向，从而更灵活地控制数据流。重定向主要包括三种类型：标准输入重定向 (stdin)、标准输出重定向（stdout）和标准错误输出重定向（stderr）。

重定向

输入输出	重定向	意义
stdin	< / «	< 从文件读取内容作为命令的输入，« 以指定字符串作为输入的结束标志
stdout	> / »	> 会覆盖文件原有内容，» 则会在文件末尾追加内容
stderr	2> / 2»	作用与标准输出重定向类似，但专门用于处理错误信息

管道符 | 主要用于多重命令处理，前面命令的打印结果作为后面命令的输入, 不过其只能重定向 stdout 即成功执行的命令

管道符常和 grep 结合使用用于过滤信息，列出所查找关键字所在的位置

grep 的参数

参数	作用
$-i$	忽略大小写进行匹配
$-v$	反向查找，只打印不匹配的行
$-n$	显示匹配行的行号
$-r$	递归查找子目录中的文件
$-l$	只打印匹配的文件名
$-c$	只打印匹配的行数

其余参数请查阅 man 手册（`man grep`）。

示例 1.12 在 `dmesg`（内核日志）的输出中筛选包含 `error` 的行并显示行号：

```
dmesg | grep -n error
```

✍ **练习 1.10** 使用 find 在你的家目录下查找所有 7 天内修改过的 `.py` 文件。

✍ **练习 1.11** 使用管道和 grep，从 `apt list --installed` 的输出中筛选出名字包含 `python` 的软件包；再结合 `wc -l` 统计数量。

06 文件内容

文件内容操作

语法	作用
$cat[filename]$	由第一行开始显示文件内容（只读）
$head[filename]$	显示文件开头的内容（默认前 10 行）
$tail[filename]$	显示文件末尾的内容（默认后 10 行）
$less[filename]$	分页查看文件内容，可上下翻页，按 q 退出
$touch[filename]$	创建空文件或更新文件的时间戳
$gedit[filename]$	创建空文件或更改文件内容
$vim/vi[filename]$	创建空文件或编辑文件内容

head/tail 选项和参数

参数	作用
$-n$ [行数]	指定显示的行数（head 与 tail 均适用）
$-f$	（仅 tail）持续跟踪文件末尾的新增内容，常用于查看日志

在所有关于文件内容的操作中，这里着重介绍一下 vim。vim 是从 vi 发展出来的一个文本编辑器。代码补全、编译及错误跳转等方便编程的功能特别丰富。

vi/vim 共分为三种模式，命令模式、输入模式和命令行模式

命令模式：用户刚刚启动 vi/vim，便进入了命令模式。此状态下敲击键盘动作会被 vim 识别为命令，而非输入字符，比如我们此时按下 i，并不会输入一个字符，i 被当作了一个命令。

命令模式常用操作

命令	作用
i	切换到输入模式，在光标当前位置开始输入文本
x	删除当前光标所在处的字符
$:$	切换到底线命令模式，以在最底一行输入命令
a	进入插入模式，在光标下一个位置开始输入文本
o	在当前行的下方插入一个新行，并进入插入模式
O	在当前行的上方插入一个新行，并进入插入模式
dd	剪切当前行
yy	复制当前行
p	粘贴剪贴板内容到光标下方
P	粘贴剪贴板内容到光标上方
u	撤销上一次操作
$Ctrl + r$	重做上一次撤销的操作

输入模式：在此状态下敲击键盘会被 vim 识别为输入内容，同时也有一些按键命令。

输入模式常用操作

命令	作用
$keyboard + Shift$	输入字符
$ENTER$	回车键，换行
$BACKSPACE$	退格键，删除光标前一个字符
DEL	删除键，删除光标后一个字符
$Arrowkeys$	在文本中移动光标
$HOME/END$	移动光标到行首/行尾
$PageUp/PageDown$	上/下翻页
$Insert$	切换光标为输入/替换模式，光标将变成竖线/下划线
ESC	退出输入模式，切换到命令模式

底线命令模式：在命令模式下按下 ：（英文冒号）就进入了底线命令模式。

底线模式基本命令

命令	作用
$:w$	保存文件
$:q$	退出 vim 编辑器
$:wq$	保存文件并退出 vim 编辑器
$:q!$	强制退出 vim 编辑器，并不保存修改
$:qa!$	不保存修改，强制退出所有打开的文件

结论 vim 三种模式的切换关系：启动后默认处于命令模式；按 i 等键进入输入模式，按 ESC 返回命令模式；命令模式下按：进入底线命令模式，执行完命令（如:wq）后自动回到命令模式。江湖传言，”让新手自己尝试退出 vim” 是一种可靠的随机字符串生成方式——卡在任何地方不知所措时，连按 ESC 再输入:q! 即可强制退出。

练习 1.12 使用 vim 创建文件 note.txt，输入三行任意内容，保存退出后用 cat 验证内容；再次用 vim 打开并删除第二行，这次不保存直接强制退出，用 cat 确认文件没有变化。

笔记 vim 是一个完全可以在终端中运行的文本编辑器，功能强大，如果是远程控制等情况没有图形化界面，vim 的优点就会比较突出，但是新手可能难以适应。如果你对 vim 感到无从下手，可以试试更简单的终端文本编辑器 nano，这个操作比较符合直觉。或者是使用在 vim 基础上发展的 nvim，而且加入一些美化，比如 AstroNvim https://astronvim.com/（开箱即用的 Neovim 配置，安装方法见本节末尾的”AstroNvim” 小节）。如果你想进行 vim 的命令学习，推荐 vim-adventurehttps://vim-adventures.com/，这是一个图形化的使用 vim 命令的小游戏，可以轻松入门 vim 内容，但是完整版需要付费，大家自行寻找学习版。

以上就是 linux 系统中关于文件和目录的基本操作，俗话说熟能生巧，请读者利用空余时间多加练习，尽快适应 linux 系统的命令行操作。

1.4.3 终端里的 IDE：AstroNvim*

裸装的 vim 对新手不太友好：没有文件树、没有补全、没有语法高亮，突出一个毛坯房，新手进去三秒就想退房。Neovim（nvim）是 vim 的现代化分支，而 AstroNvim 是一套开箱即用的 Neovim 配置——装完就有文件树、模糊搜索、代码补全和漂亮的界面，毛坯房直接变精装，终端里也能有接近 IDE 的体验。

安装新版 Neovim

AstroNvim 需要较新版本的 Neovim（0.10 及以上），而 Ubuntu 22.04 仓库自带的版本太老，因此从官网下载预编译包安装：

示例 1.13 下载并解压 Neovim 到 /opt，顺手装上两个依赖（ripgrep 用于全局搜索，gcc 用于编译语法高亮组件）：

```
curl -LO https://github.com/neovim/neovim/releases/latest/download/nvim-linux-x86_64.tar.gz
sudo tar -C /opt -xzf nvim-linux-x86_64.tar.gz
sudo apt install ripgrep gcc
```

然后把 `/opt/nvim-linux-x86_64/bin` 追加到 PATH（写入 ~/.bashrc 或 ~/.zshrc，方法见 1.5 节）：

```
export PATH=$PATH:/opt/nvim-linux-x86_64/bin
```

生效后执行 `nvim --version` 能看到版本号即安装成功。

安装 AstroNvim

示例 1.14 备份可能存在的旧配置，然后克隆 AstroNvim 模板并启动：

```
mv ~/.config/nvim ~/.config/nvim.bak
mv ~/.local/share/nvim ~/.local/share/nvim.bak
git clone --depth 1 https://github.com/AstroNvim/template ~/.config/nvim
rm -rf ~/.config/nvim/.git
nvim
```

首次启动会自动下载安装全部插件，泡杯茶等它跑完，看到欢迎界面就大功告成了。如果之前没有安装过 nvim，前两行备份命令报"文件不存在"属正常现象，忽略即可。

笔记 第四步的 `rm -rf` 是为了去掉模板自带的 git 仓库信息，让这个配置目录变成你自己的仓库（方便日后提交和同步个人配置）。路径请照抄，不要自行发挥。

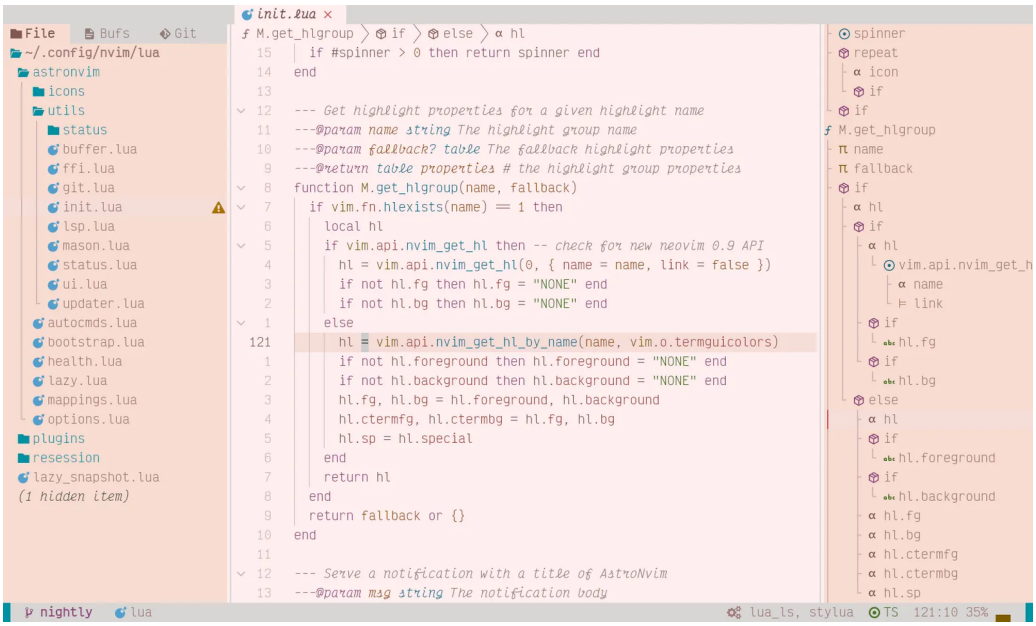


图 1.20: AstroNvim 界面

上手要点

- Leader 键是空格：按下 `<Space>` 稍等片刻会弹出快捷键菜单，忘了按键组合也不用背，跟着菜单按就行；
- `<Space>ff` 模糊搜索并打开文件，`<Space>fw` 在整个项目中搜索文字（依赖 ripgrep）；
- 使用 `:LspInstall clangd` 安装 C++ 的语言服务器后，即可获得代码补全和跳转。

笔记 界面中的图标和 zsh 的 p10k 主题一样依赖 Nerd Font，装过 MesloLGS NF 的话直接通用（见 1.5 节）。更多用法见官方文档：https://docs.astronvim.com/

练习 1.13 完成 AstroNvim 的安装，用 `<Space>ff` 打开任意一个文本文件，再用 `<Space>fw` 在某个项目目录中搜索一个关键词，体验模糊搜索。

1.5 Shell 脚本与环境变量

1.5.1 shell 简介

在 windows 系统中，我们使用电脑主要是通过图形化界面点击某个图标来启动某个程序；在 linux 系统中我们更频繁的使用终端和命令行。然而这些都只是我们控制计算机的表象，真正能够控制计算机硬件（CPU、内存、显示器等）的只有操作系统内核（Kernel），图形界面和命令行只是架设在用户和内核之间的一座桥梁。

考虑到安全等原因，用户不能直接接触内核，因此需要另外再开发一个程序，让用户通过使用这个程序来间接地与内核进行交互。该程序的作用就是接收用户的操作（点击图标、输入命令），并进行简单的处理，然后再传递给内核。用户界面和命令行就是这个另外开发的程序，在 Linux 下，这个命令行程序叫做 shell 。

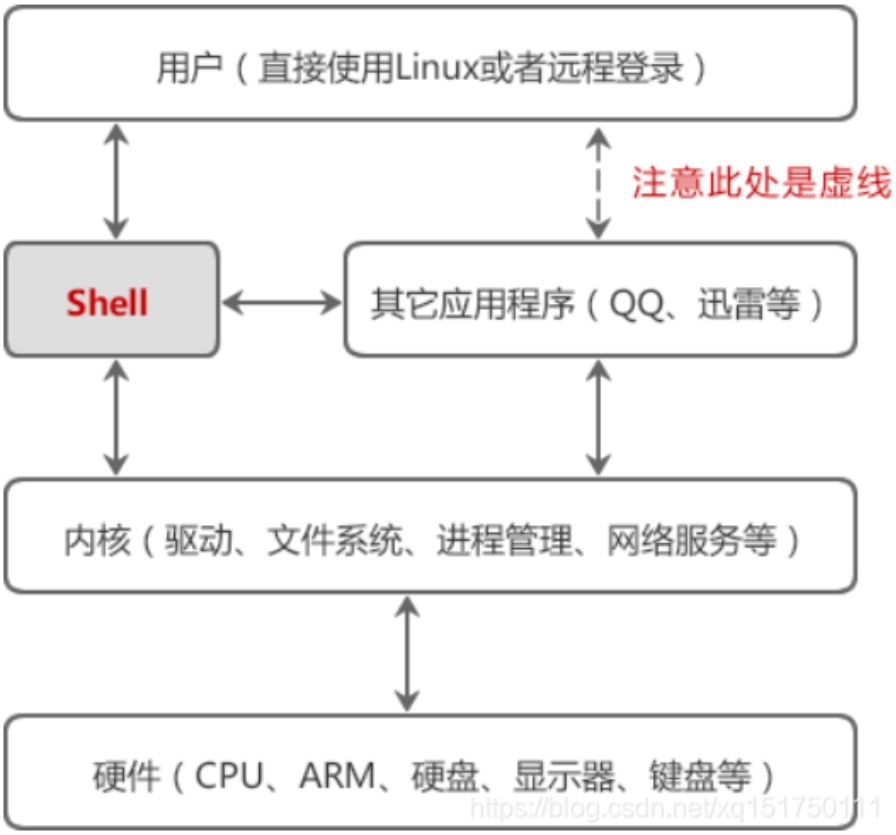


图 1.21: shell 工作流程示意图

shell 的一大特性就是它是一个应用程序，它连接了用户和 Linux 内核，让用户能够更加高效、安全、低成本地使用 Linux 内核。另一大特性是它是一个命令语言解释器，shell 把我们在计算机上的操作或我们的命令，翻译为计算机可识别的二进制命令，传递给内核，以便调用计算机硬件执行相关的操作；计算机执行完命令后，再通过 Shell 翻译成自然语言，呈现在我们面前。同时，Shell 程序设计语言支持在高级语言里所能见到的绝大多数程序控制结构，比如循环，函数，变量和数组等等。

常见的 shell 有 Bourne shell (sh), C shell (csh), Korn shell (ksh), Bourne Again shell(bash, sh 的扩展), Z shell (zsh), friendly interactive shell (fish) 等等。

启动 shell 的方法有两种，一是进入 linux 控制台，二是使用终端（快捷键为 ctrl+alt+t）

1.5.2 简单的 shell 脚本

新建一个 shell 脚本

打开文本编辑器，新建一个文件 test.sh

```
vim test.sh
```

然后在 shell 脚本的第一行输入

```
#!/bin/bash
```

#! 是一个约定的标记，它告诉系统这个脚本需要什么解释器来执行，即使用哪一种 shell。在第一行下面就可以用 shell 编程语言输入脚本内容了。

示例 1.15 一个最简单的 shell 脚本，运行后打印一行文字：

```
#!/bin/bash
echo "Hello World !"
```

shell 脚本可用于 ROS 程序自启，即编写一系列命令行操作，启动程序时无需打开终端敲命令行，只需启动一个 shell 脚本即可。

运行 shell 脚本

运行 shell 脚本也有两种方式：

1. 作为可执行程序

   先使脚本具有执行权限

   ```
   chmod +x ./test.sh
   ```

   再执行脚本

   ```
   ./test.sh
   ```

2. 作为解释器参数

   直接运行解释器，其参数就是 shell 脚本的文件名。eg:

   ```
   /bin/sh test.sh
   /bin/php test.php
   ```

   这种方式运行的脚本，不需要在第一行指定解释器信息，写了也没用

练习 1.14 编写脚本 `prepare.sh`，依次执行：创建目录 `mydir`、在其中创建空文件 `a.txt`、列出 `mydir` 的内容。分别以” 可执行程序” 和” 解释器参数” 两种方式运行该脚本，并观察结果。

1.5.3 linux 中的 bash

根据刚刚对 shell 的介绍可以知道 bash 也是一种 shell，事实上在大多数的 linux 系统中，bash 是默认的缺省 shell。当然如果你不想用 bash 也可以换成其他的 shell（比如 zsh 也很好用）。

bash 和 shell 一样有许多强大的功能，经常用到的有命令补全，通配符，命令历史记录，重定向，管道等等。bash 的配置文件分为全局配置文件和个人配置文件：

bash 主要文件

文件	作用
/etc/profile	系统的主配置文件，用于设置系统范围内的环境变量和其他系统的设置
~/.bashrc	用于定义用户的命令别名和本地变量
~/.bash_profile	用于定义用户的环境变量，运行命令或脚本

其中我们会经常对.bashrc 文件进行操作。

.bashrc 是终端里面指令运行的配置脚本，是系统的隐藏文件，存放在 ~ 目录下。.bashrc 的功能主要有两个：

1. 个性化指令 alias：alias [指令别名]=' 原指令'
   注意 alias 和 = 之间不能有空格
2. 设定环境路径：export PATH=$PATH: 路径

对.bashrc 中的内容进行修改后，在下一次打开新终端时修改内容才会生效，或执行 source ~/.bashrc 立刻生效。

示例 1.16 在 `.bashrc` 中添加一条别名和一个环境路径，让 `ll` 等价于 `ls -l`：

```
alias ll='ls -l'
export PATH=$PATH:/opt/myapp/bin
```

保存后执行 `source ~/.bashrc`，修改立即在当前终端生效。

练习 1.15 在你的 `.bashrc` 中添加别名 `cls`（等价于 `clear`），用 `source` 使其生效并验证；确认无误后将其删除，恢复原状。

1.5.4 linux 系统环境变量简介

linux 系统中有非常多的环境变量在控制系统的运行，常用的有：

1. PATH：决定了 shell 将到哪些目录中寻找命令或程序
2. HOME：当前用户主目录
3. SHELL：当前用户 Shell 类型

这里我们主要介绍 PATH 环境变量

PATH 说简单点就是一个字符串变量，当输入命令的时候 linux 会去查找 PATH 里面记录的路径，path 配置的路径下的文件可以在任何位置执行，并且可以通过 which 可执行文件命令来找到该文件的位置。注意 PATH 是大写，linux 中区分大小写。

我们先来查看一下 PATH：

示例 1.17 查看当前的 PATH 环境变量：

```
echo $PATH
```

终端中会输出

/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:
/usr/local/games:/usr/lib/wsl/lib:/snap/bin

每个路径中间用: 隔开，输入命令时系统根据这些路径来寻找对应的可执行文件，先找到谁就用谁

设置环境变量的方法有两种：

1. export PATH=/bin:/sbin:/usr/bin:/usr/sbin
2. PATH="/bin:/sbin:/usr/bin:/usr/sbin"

这两种方法给 PATH 赋的值完全相同，区别在于：不用 export 设置的环境变量仅对当前 shell 起作用，不会传导到子进程；export 设置的环境变量在子进程也有效，因此我们常用 export 命令来设置 PATH 环境变量。

笔记 修改 PATH 时千万不要写成 `export PATH=/usr/local/mongodb/bin` 这种不带 `$PATH` 的形式——新路径会覆盖掉所有原有路径，`ls`、`sudo` 等系统命令会集体罢工，终端里打出什么都是"找不到命令"，场面一度十分尴尬。万一写错了：终端内直接修改的，重开一个终端即可恢复；写进配置文件的，把写错的行注释掉再重开终端。

修改环境变量的方法有三种：

1. 直接在终端输入

```
export PATH=/usr/local/mongodb/bin:$PATH
```

$PATH 表示在原有 PATH 的基础上继续在前面或后面添加相应路径，这样可以避免修改原有路径。由于是在终端中输入命令，因此这种修改方式仅对当前用户当前终端有效。

2. 修改.bashrc 文件：在.bashrc 文件中添加 export PATH=/usr/local/mongodb/bin:$PATH 一行，保存退出后 source 或打开新终端即可生效。该方法对当前用户永久有效。
3. 修改/etc/profile 文件：找到设置 PATH 的行，添加 export PATH=/usr/local/mongodb/bin:$PATH，系统重启或运行 source /etc/profile 后生效。由于是修改了整个系统的配置文件，因此该方法对所有用户永久有效。

结论 三种修改环境变量方式的对比：

方式	生效范围	持久性
终端中直接输入 export	当前终端	关闭终端即失效
写入 ~/.bashrc	当前用户的所有终端	永久
写入 /etc/profile	所有用户	永久

日常配置环境优先改 ~/.bashrc：影响面小，改错了也好修。

练习 **1.16** 在终端中直接用 export 把一个自定义目录（如 `~/mybin`）追加到 PATH 末尾，用 `echo $PATH` 验证；再新开一个终端，确认该修改已失效，并说明原因。

环境变量这里介绍的很浅，主要目的是让读者先对环境变量这个概念有所了解，在后续自己配环境的过程中逐步加深对环境变量的理解，避免对系统环境变量的误操作。

1.5.5 zsh 与终端美化 *

bash 够用，但用久了难免嫌弃：命令没有建议、语法没有高亮、提示符万年一张脸。zsh（Z shell）配合 Oh My Zsh 框架和 Powerlevel10k 主题，可以把终端武装到牙齿——命令自动建议、语法高亮、git 状态提示全都有，属于从功能机换到智能机的体验。本节为选修内容，不影响后续学习，但装完大概率就回不去了。

安装 zsh

示例 **1.18** 安装 zsh 并将它设为默认 shell：

```
sudo apt install zsh
chsh -s $(which zsh)
```

chsh 修改的是登录 shell，需要**注销并重新登录**（或重启）后才会生效。重新登录后打开终端，会看到 zsh 的首次配置提示，按 q 跳过即可——反正接下来的配置都交给 Oh My Zsh 统一管理。

安装 Oh My Zsh

Oh My Zsh 是 zsh 的配置管理框架，主题和插件都通过它安装。

示例 1.19 安装 Oh My Zsh：

```
sh -c "$(curl␣-fsSL␣
    https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

安装脚本做的事情很简单：把框架克隆到 ~/.oh-my-zsh 目录，并生成一份默认的 ~/.zshrc 配置文件。

笔记 如果访问 GitHub 缓慢，可以先配置代理，或使用国内镜像源安装。

安装 Powerlevel10k 主题

Powerlevel10k（简称 p10k）是目前最流行的 zsh 主题，能在提示符中直接显示目录、git 分支与提交状态、命令执行耗时等信息。

示例 1.20 安装 p10k 主题：

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git
    ~/.oh-my-zsh/custom/themes/powerlevel10k
```

然后打开 ~/.zshrc，把主题行改为：

```
ZSH_THEME="powerlevel10k/powerlevel10k"
```

重开终端后会自动进入 p10k 的配置向导（也可以随时用 `p10k configure` 手动启动），按提示逐项选择喜欢的样式即可。

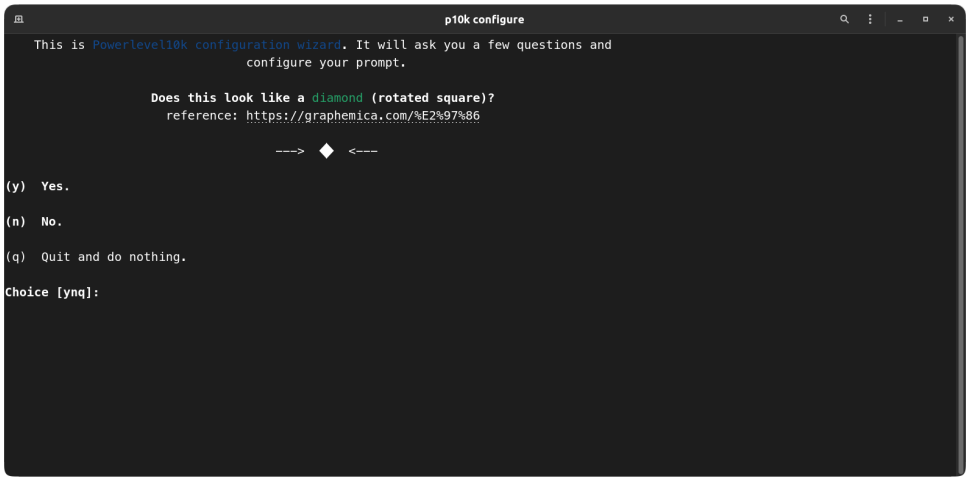


图 1.22: p10k 配置向导

笔记 p10k 提示符里的图标（文件夹、git 分支符号等）依赖 Nerd Font 字体，推荐按 p10k 文档安装 MesloLGS NF，并在终端设置中把字体改为它，否则图标会显示为方块或问号。字体下载地址：https:

//github.com/romkatv/powerlevel10k#fonts

推荐插件

zsh 的插件生态是它相比 bash 的一大优势，最常用的两个：

- zsh-autosuggestions：根据历史命令给出灰色补全建议，按方向键右键采纳；
- zsh-syntax-highlighting：命令输入时实时高亮，命令拼错时显示为红色。

示例 1.21 安装这两个插件并在 ~/.zshrc 中启用：

```
1 git clone https://github.com/zsh-users/zsh-autosuggestions
    ~/.oh-my-zsh/custom/plugins/zsh-autosuggestions
2 git clone https://github.com/zsh-users/zsh-syntax-highlighting
    ~/.oh-my-zsh/custom/plugins/zsh-syntax-highlighting
```

然后修改 ~/.zshrc 中的插件行：

```
1 plugins=(git zsh-autosuggestions zsh-syntax-highlighting)
```

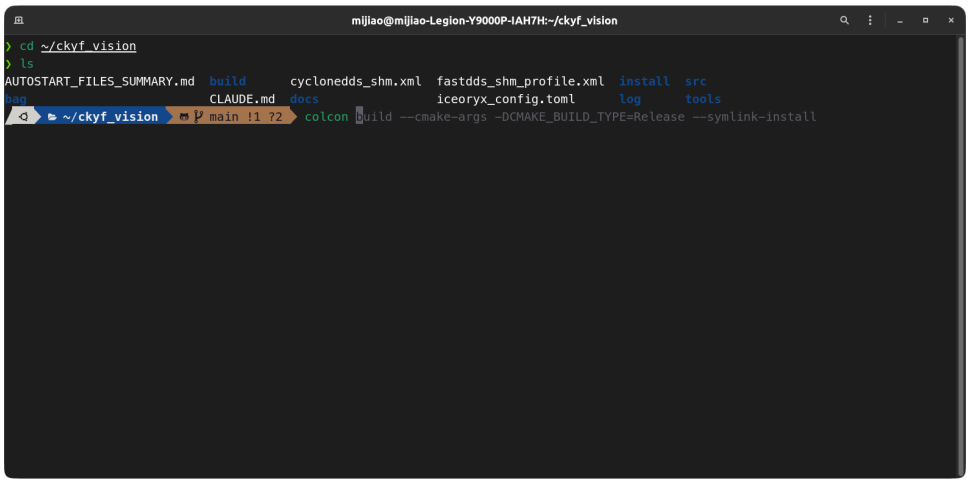


图 1.23: 配置完成后的终端效果

与 bash 的不同

换用 zsh 后，日常使用的大部分命令完全一样，但有一些区别需要知道：

结论 zsh 与 bash 的主要区别：

对比项	bash	zsh
配置文件	~/.bashrc	~/.zshrc
通配符无匹配时	原样保留（如 `*.txt`）	报错 no matches found
数组下标	从 0 开始	从 1 开始
Tab 补全	基础补全	支持命令参数、选项的智能补全

两者脚本语法大部分兼容；带 `#!/bin/bash` 行首的脚本仍由 bash 执行，更换交互式 shell 不影响已有脚本的运行。

笔记 ~/.bashrc 中的内容不会自动迁移到 ~/.zshrc——尤其是 source 类的环境配置行（后续卷二安装 ROS2 时会向 ~/.bashrc 写入 source 命令）。换用 zsh 后记得手动把这些行加到 ~/.zshrc 里，否则会出现"昨天还能编译，今天找不到命令"的灵异事件。

练习 1.17 完成 zsh、Oh My Zsh、p10k 的安装与主题配置，启用 autosuggestions 插件，并把原来 ~/.bashrc 中自己添加的别名迁移到 ~/.zshrc。

1.6 Git 工具的使用

1.6.1 git 简介

git 是一个开源的分布式版本控制系统，用于敏捷高效地处理任何或小或大的项目，它可以在任何时间点，将文档的状态作为更新记录保存起来，也可以在任何时间点，将更新记录恢复回来。Git 可以帮我们做到很多事情，比如代码的版本控制，分支管理，也提供了便于多人合作的平台。

示例 1.22 在 Ubuntu 上安装 git 并验证安装成功：

```
sudo apt install git
git --version
```

1.6.2 git 基本概念

git 的基本概念比较多，这一屏密度有点高，先混个脸熟就行——不用背，后面的工作流程里会全部用到：

本地仓库：对本地代码进行管理的仓库，会包含代码的所有历史版本。

远程仓库：在开发者可以访问的网络内的某个服务器上有一个包含所有版本的仓库。

开发者可以把本地的新版本推送到远程仓库上，也可以把远程仓库上的新版本拉取到本地仓库上。由此可见，远程仓库为不同开发者之间的协作提供了一个渠道。

工作区：在本地计算机上的项目目录，你在这里进行文件的创建、修改和删除操作。工作区包含了当前项目的所有文件和子目录。

暂存区：临时存储区域，它包含了即将被提交到版本库中的文件快照。

版本库：包含项目的所有版本历史记录，每次提交都会在版本库中创建一个新的快照，这些快照是不可变的，确保了项目的完整历史记录。

分支：简单来说就是同一棵树上的两个分叉，实际上是指向更改快照的指针。

暂存：添加文件到暂存区准备提交。

提交：提交暂存区中的内容到本地仓库。

推送：推送本地仓库中的内容到远程仓库。

克隆：从远程仓库下载代码到本地的过程。

合并：假设分支 A 和分支 B 是从节点 P 分叉出来的，合并之后分支 A 和分支 B 会在一个新的节点后汇合。分支 A 和分支 B 表示：我们联合！

变基：从节点 P 开始，把分支 A 中各个节点按提交的顺序加在分支 B 的上面，因为一个节点对应一次内容改变，所以分支 A 中不想要的改变可以在变基的过程中跳过相应的节点。变基其实也可以理解成是另一种合并，只不过他的合并会让提交历史更为干净清晰。

子模块：将一个 git 仓库作为另一个 git 仓库的子目录。它能让你将另一个仓库克隆到自己的项目中，同时还保持提交的独立。

1.6.3 git 基本配置

git 提供了一个叫做 git config 的命令，用来配置或读取相应的工作环境变量。这些环境变量，决定了 Git 在各个环节的具体工作方式和行为。这些变量可以存放在以下三个不同的地方：

1. /etc/gitconfig 文件：系统中对所有用户都普遍适用的配置。若使用 git config 时用 –system 选项，读写的就是这个文件。
2. ~/.gitconfig 文件：用户目录下的配置文件只适用于该用户。若使用 git config 时用 –global 选项，读写的就是这个文件。
3. 当前项目的 Git 目录中的配置文件（也就是工作目录中的.git/config 文件）：这里的配置仅仅针对当前项目有效。每一个级别的配置都会覆盖上层的相同配置，所以.git/config 里的配置会覆盖 /etc/gitconfig 中的同名变量。

配置提交者信息

每次提交时 git 都会记录提交者的名字和邮箱，首次使用前先配置好（只需配置一次）：

```
git config --global user.name "你的名字"
git config --global user.email "你的邮箱"
```

配置个人访问令牌（https 方式）

github 已经弃用了原先直接用账号密码验证身份的方案，改用个人访问令牌。配置个人访问令牌的方法如下：

1. 在 github 设置界面点击进入“Developer settings”，访问 Personal Access Tokens 中的 Tokens(classic) 页面
2. 点击“Generate new token”，选择“Generate new token(classic)”
3. 为令牌选择一个描述性名称，选择 repo 权限，点击“Generate token”生成令牌，复制生成的令牌
4. 在提交代码时，当提示输入用户名和密码时，输入你的 GitHub 用户名和新生成的个人访问令牌即可

配置 ssh 密钥

克隆 github 上的私人仓库时，https 和 SSH 两种协议都可以：https 方式需要把个人访问令牌当作密码使用，而 SSH 配好密钥后就再也不用输令牌，一劳永逸，因此组内推荐配置 SSH 密钥，具体方法如下：

1. 生成 SSH 密钥

```
ssh-keygen
```

   生成后可在 ~ 目录下找到.ssh 文件夹
2. 进入.ssh 文件夹，复制其中 id_rsa.pub 文件中的内容
3. 在 github 中进入个人设置界面，选择“SSH and GPG keys”，点击“New SSH key”
4. 给新的 SSH 起个标题，并将刚刚复制的内容粘贴到下方的文本框中，点击“Add SSH key”

配置好后就可以克隆私人仓库中的代码了。

1.6.4 git 基本操作

git 的学问很多，各种操作也是五花八门，这里只列举出 git 工具的基本操作供读者参考。

git 基本操作

命令	作用
$git\ config\ --list$	检查已有的配置信息
$git\ --version$	查看 git 版本（验证安装）
$git\ clone\ <url>$	将远程仓库克隆到本地
$git\ remote\ add\ origin\ <url>$	本地仓库链接远程仓库
$git\ add$	添加文件到暂存区
$git\ status$	查看仓库当前状态，显示变更
$git\ commit\ -m\ "description"$	提交到本地仓库并添加描述
$git\ reset\ [--soft\|--mixed\|--hard]\ [HEAD]$	回退某一次提交的版本
$git\ rm$	将文件从暂存区和工作区中删除
$git\ checkout\ <branchname>$	分支切换
$git\ log$	查看提交历史
$git\ diff$	查看工作区和暂存区之间的差异
$git\ diff\ --cached$	查看暂存区和最后一次提交的差异
$git\ remote$	列出当前仓库中已配置的远程仓库
$git\ remote\ rename\ <old>\ <new>$	将已配置的远程仓库重命名
$git\ remote\ show\ <remote>$	显示指定远程仓库的详细信息
$git\ fetch\ [remote]$	提取远程仓库中更新的数据
$git\ merge\ [remote]/[branch]$	将远程仓库的更新合并到当前分支
$git\ pull\ [remote]\ [branch]$	从远程获取代码并合并本地的版本
$git\ push\ [remote]\ [branch]$	将本地版本上传到远程并合并
$git\ submodule\ init$	初始化子模块
$git\ submodule\ update$	更新子模块
$git\ submodule\ update\ --recursive\ --remote$	递归更新子模块并拉取最新更改
$git\ submodule\ add\ <url>\ [<path>]$	将 Git 仓库作为子模块加到当前仓库中

git 基本操作

内容	作用
$git\ submodule\ deinit\ [<path>]$	.git/config 中移除子模块并删除文件
$git\ rm\ [<path>]$	从仓库中删除子模块引用（需再 commit 生效）
$git\ submodule$	列出当前仓库中的所有子模块
$git\ branch$	查看所有分支
$git\ branch\ -r$	查看远程分支
$git\ branch\ -a$	查看所有本地和远程分支
$git\ merge\ <branch>$	将其他分支合并到当前分支
$git\ add\ <conflict-file>$	标记合并冲突解决完成
$git\ checkout\ -b\ <branch>$	创建分支并切换到该分支
$git\ branch\ -d\ <branch>$	删除本地分支
$git\ branch\ -D\ <branch>$	强制删除未合并的分支
$git\ push\ origin\ --delete\ <branch>$	删除远程分支



笔记 `git reset --hard` 会把工作区和暂存区强制回退到指定提交，未提交的修改全部丢失且无法恢复——git 不拦你，但时间也不倒流。执行前请确认这些修改真的不要了。

除了这些命令操作外，git 中还有一个非常重要的工具 —— .gitignore 忽略文件。尽管远程仓库在云端服务器上不会受到内存的限制，但是如果随便将大文件上传到远程仓库的话在下拉代码的时候会耗时非常久甚至拉代码失败。因此，上传至远程仓库中的内容一定是最核心的部分。

但是，在本地工作区往往会需要那些不需要被上传的“无用文件”，为了能在不上传“无用文件”的同时保证本地工作区的正常工作，我们就需要在本地工作区新建一个.gitignore 文件，里面记录着不需要被版本库跟踪的文件，这样就能拥有一个干净简洁的版本库了。

需要注意的是，.gitignore 只能忽略尚未提交到仓库的未被追踪的文件。如果一个文件已经被提交过了，此时再在.gitignore 文件中添加忽略此文件时会发现这个文件依旧存在于版本库中。要想清除这个文件首先要将其从索引中删除：

```
git rm --cached .env
```

这条命令会从版本库中删除文件，但不删除实际的文件。这意味着该文件仍然在你的本地系统和工作目录中作为一个被忽略的文件存在。此时可以重新提交修改过的.gitignore 文件来使新的忽略生效。

所以最好的方法还是在创建一个新的仓库的时候就把.gitignore 文件写好，在首次提交前忽略不需要上传的文件。

.gitignore 文件也有自己的语法，这里同样也列举一些用法：

.gitignore 语法

内容	作用
$/text.txt$	忽略位于项目根目录下的 text.txt 文件
$/test/text.txt$	忽略位于项目根目录下 test 目录中的 text.txt 文件
$text.txt$	忽略任何 text.txt 文件
$test/$	忽略整个目录及其所有内容
$text$	忽略任何名字带有 test 的文件和目录
$img*$	忽略所有名称以 img 开头的文件和目录
$*.md$	忽略所有以.md 文件扩展名结尾的文件
$!README.md$	不忽略 README.md 文件

1.6.5 git 工作流程

git 的基本工作流程就是从创建新仓库（或克隆别人的仓库），链接本地仓库和远程仓库或者建立新的分支（可选），修改完成后添加到暂存区，然后提交到本地，最后再推送回远程仓库的过程，可以参考下面这幅图。

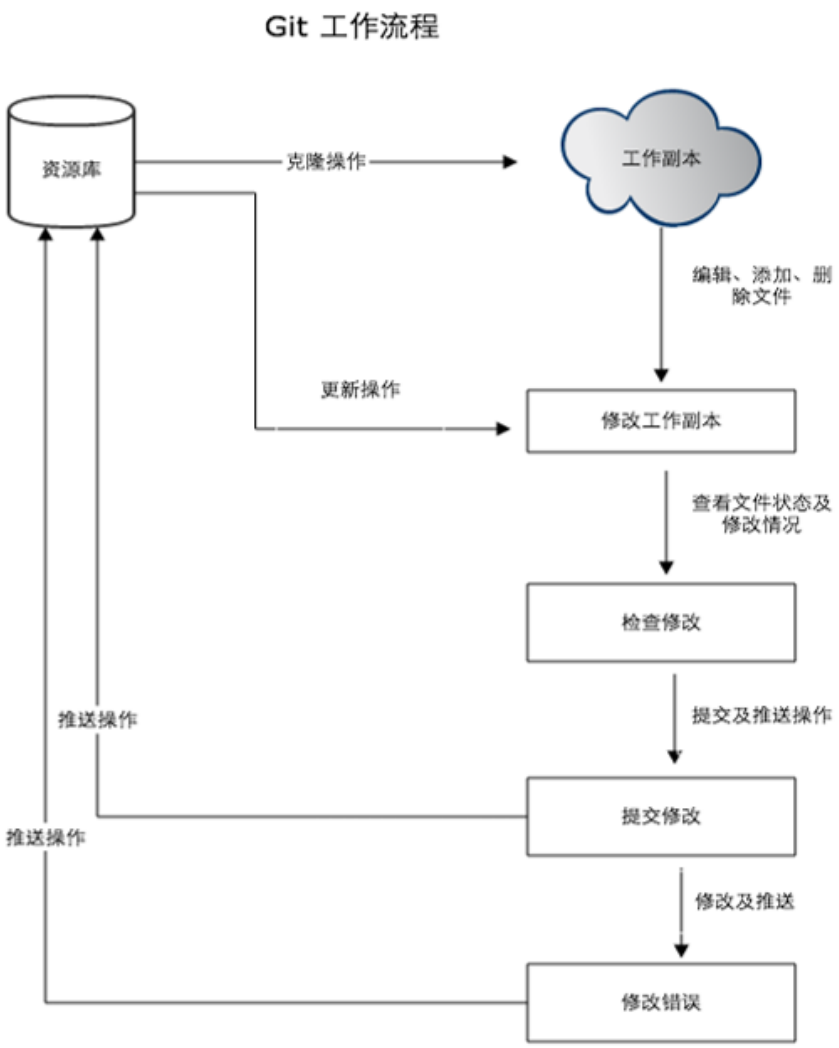


图 1.24: git 工作流程示意图

结论 git 的四个工作区域与流转命令：

- 工作区：你实际编辑文件的目录；
- 暂存区：git add 后，变更先进入这里等待提交；
- 本地版本库：git commit 把暂存区固化为一次版本记录；
- 远程仓库：git push 把本地版本推送上去，git pull / git fetch 把远程更新取回来。

这套流程一开始总要忘两步，很正常。忘了文件进行到哪个阶段了，就 git status 看一眼——它相当于游戏里的任务日志，随时告诉你现在该干嘛。

示例 1.23 一次完整的日常提交流程（以克隆好的仓库为例）——add、commit、push 这套连招以后每天要搓十几遍，出招表如下，建议早日练成肌肉记忆：

```
git status
git add main.cpp
git commit -m "fix armor detection offset"
git push origin main
```

练习 1.18 在任意 git 仓库中连续做两次提交，用 git log 查看提交历史；再修改一个文件，用 git diff 查看工作区与暂存区的差异。

练习 1.19 在仓库中新建.gitignore，实现：忽略所有.log 文件；忽略 build/ 目录下的所有内容，但保留其中的 keep.txt（提示：先写 build/*，再用！取消对单个文件的忽略）。

对自己的仓库进行版本管理主要用到以下这些命令：

git 工作流程基本命令

git init
git clone [url]
git checkout -b <branch>
git remote add origin [url]
git add
git status
git commit -m ”description”
git push [remote] [branch]
git pull [remote] [branch]

笔记 对于 git 的学习，这里也推荐一个网站 https://learngitbranching.js.org/?locale=zh_CN，从图形化角度直白展示了整个 git 树，并且可以互动。

使用 git 时我们可以选择直接在终端中通过命令行的方式提交代码，这种方式有时不够直观，因此除了命令行我们还可以使用一些 git 的图形化工具，比如 GitKraken 等等，当然也可以直接在 clion 或 vscode 中使用 git 相关插件，这些插件也提供了直观的版本库界面来辅助我们管理代码。

1.7 Conda 的简单使用 *

1.7.1 conda 简介



笔记如果你接触的 py 项目不多，你也可以跳过 conda，用 python 自带的 venv 管理环境，更轻量化。

conda 是一个包，依赖和环境管理工具，可以用于在同一个机器上安装不同 Python 版本的软件包及其依赖，并能够在不同的 Python 环境之间切换。一个 conda 环境就是一个目录包含所有的安装包和依赖，不同环境之间独立且不相互影响，跟 python 的虚拟环境 virtualenv 一样。通过 conda 的创建的虚拟环境，我们可以管理不同的 python 版本及其安装包而无需担心因不同版本导致的冲突；同时，在配置一个新环境时我们还可以利用虚拟环境来试错而不用担心对电脑原生环境造成无法挽回的伤害。想给每个项目都配一套独立环境？和 conda 签订契约，成为虚拟环境管理大师吧。

conda 适用于多种语言，如: Python,R,Scala, Java,Javascript, C/ C++, FORTRAN 等。conda 默认随 miniconda 或 anaconda 发行。

安装 anacoda 后，在用户 HOME 目录下，会有一个 anaconda 目录，其中 bin,include,lib,share 里面是 conda 默认环境的文件，envs 存放 conda 管理的环境信息，pkgs 里面是解压的软件安装包。

1.7.2 conda 配置

conda 默认的配置文件为 ~/.condarc，通过修改配置文件我们可以配置如下内容：

1. conda 从哪里获取安装包
2. conda 是否使用代理服务器
3. conda 从哪里获取环境信息
4. 当 conda 环境激活的时候是否更新 bash 提示

查看所有配置信息

```
conda config --show --json
```

配置使用清华的源

示例 1.24 将 conda 的软件源切换为清华镜像，并显示下载渠道：

```
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/free
conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkgs/main
conda config --set show_channel_urls yes
```

配置安装软件时不要提示输入 yes

```
conda config --set always_yes true
```

取消每次启动自动激活 conda 的基础环境

```
conda config --set auto_activate_base false
```

建议安装后关闭自动激活。后续学习 ROS2 时，conda 基环境自动激活易与 ROS2 工作空间冲突，导致功能包无法正常编译。通识阶段应养成按需激活环境的习惯，细则见卷二 ROS2 章。

1.7.3 conda 基础操作

这里介绍一些 conda 命令行的基础操作：

conda 基础操作

命令	作用
$conda\ \ --version$	查看当前版本
$conda\ activate\ [env]$	激活环境
$conda\ deactivate$	退出环境
$conda\ update\ conda$	更新 conda
$conda\ create\ \ --name\ [env]\ python = 3.9\ [packages...]$	创建 python 3.9 的环境并安装包
$conda\ info\ \ --envs$	查看所有的环境
$conda\ list$	查看当前环境下的包
$conda\ install\ \ --name\ [env]\ [package]$	安装包到指定环境中
$conda\ remove\ \ --name\ [env]\ [package]$	删除指定环境里的包

示例 1.25 创建一个名为 py39 的 Python 3.9 环境，激活后查看其中已安装的包，最后退出环境：

```
conda create --name py39 python=3.9
conda activate py39
conda list
conda deactivate
```

练习 1.20 创建一个 Python 3.10 的环境并激活，在其中安装 `numpy`，用 `conda list` 确认安装成功后退出环境，并删除该环境（提示：`conda remove --name 环境名 --all`）。

1.8 CLion 与 VS Code 开发环境

1.8.1 先分清编辑器、工具链和工程

写代码需要编辑器，把代码变成程序需要编译器，暂停程序并查看变量需要调试器。集成开发环境（Integrated Development Environment，IDE）把这些操作放进同一个窗口。CLion 面向 C/C++ 工程，集成了 CMake 和调试入口；Visual Studio Code（简称 VS Code）以编辑器为基础，通过扩展接入 C++ 和 CMake。**VS Code 与 Visual Studio 是两个软件**，下载时看清名字。

工具链（Toolchain）包括编译器、构建工具和调试器等。CMake 读取工程描述，生成构建规则；编译器把源码编译成程序。本节只要求把现成的小工程跑起来，C++ 语法见第 2 章，CMake 原理见第 3 章。

项目	CLion	VS Code
C++ 支持	内置代码分析、CMake 与调试界面	安装 C/C++、CMake Tools 扩展
Windows 工具链	可使用随安装包提供的 MinGW	另装工具链，本节采用 MSYS2 UCRT64

项目	CLion	VS Code
Ubuntu 工具链	使用系统 GCC、GDB 等	使用系统 GCC、GDB 等
打开工程	打开含 CMakeLists.txt 的文件夹	File（文件）→ Open Folder（打开文件夹）

两套软件都要会认界面，日常使用可以任选一套。**先完成所用系统的一条安装路线**，不要把 Windows、Ubuntu、WSL 的路径混着填。Windows 截图用于认按钮；在 Ubuntu 中，同名工具窗口和工作流程基本相同。图标位置会随版本、语言包、窗口宽度和扩展变化，找不到时先悬停看提示，再用菜单或动作搜索按名称查找。

1.8.2 安装 CLion：从安装包到工具链检查

Windows 安装步骤

1. 从 JetBrains 官网下载与系统架构匹配的 CLion 安装包，运行安装程序，选择安装位置。
2. 在 Installation Options（安装选项）中按需勾选桌面快捷方式；Add launchers dir to the PATH 让终端能找到启动器，**不等于安装了编译器**；Open Folder as Project 增加资源管理器右键入口；文件关联决定双击 `.cpp` 等文件时用什么打开。这些选项不影响本节核心流程。
3. 完成安装后启动 CLion。首次启动可跳过导入旧设置，选择主题；若出现许可证页面，按实际用途选择适用的授权或试用方式，具体条件以官网为准。
4. 欢迎页的 New Project（新建项目）用于生成工程；Open（打开）用于打开已有工程；Get from VCS（从版本控制获取）用于克隆仓库；Customize（自定义）调整外观；Plugins（插件）管理扩展。中文界面可在插件页搜索 Chinese Language Pack，安装后按提示重启。
5. 打开工程后进入 File → Settings → Build, Execution, Deployment → Toolchains（文件 → 设置 → 构建、执行、部署 → 工具链）。选择 MinGW，Toolset 选 Bundled（捆绑），等待 C/C++ Compiler、Debugger 和 Build Tool 检测完成。若安装版本未附带工具链，先按官方 Windows 工具链教程安装，不能把编辑器目录当作编译器目录。

Ubuntu 安装步骤　先准备本机编译工具，然后从 JetBrains 官网下载 Linux 版 CLion，或通过 JetBrains Toolbox 安装。以下命令在 **Ubuntu 终端**执行，不能粘贴到 Windows PowerShell。

示例 1.26 安装并检查 Ubuntu 工具链：

```
sudo apt update
sudo apt install build-essential gdb cmake ninja-build
g++ --version
gdb --version
cmake --version
```

三个版本命令都应打印版本信息。若下载的是压缩包，在文件管理器中解压到自己的软件目录，进入解压目录的 `bin`，在终端执行 `./clion.sh` 启动。然后在 Toolchains 中选择 System（系统），检查编译器和 GDB 路径；CMake 可用捆绑版本或已安装的系统版本。

笔记安装与工具链参考：

https://www.jetbrains.com/help/clion/installation-guide.html

https://www.jetbrains.com/help/clion/quick-tutorial-on-configuring-clion-on-windows.html

Windows 的 MinGW 不能直接构建依赖 Ubuntu/ROS2 的工程；本节的跨平台小程序可以，后续 ROS2 实验应使用相应 Linux 环境。

1.8.3 准备两套软件共用的练习工程

教材附带 `examples/ide-demo` 目录。先复制一份作为自己的练习工程。Windows 可放在 `C:/dev/ide-demo`，Ubuntu 可放在 `~/dev/ide-demo`。也可以手动建立文件夹并创建下面两个文件。打开时选这一层目录，文件树顶层应直接出现 `CMakeLists.txt` 和 `main.cpp`。

示例 1.27 计分器：预期输出 total = 75 `main.cpp` 的完整内容如下：

```cpp
#include <iostream>

int add_score(int current, int bonus) {
    int result = current + bonus;
    return result;
}

int main() {
    int score = 60;
    int bonus = 15;
    int total = add_score(score, bonus);
    std::cout << "total = " << total << '\n';
    return 0;
}
```

`CMakeLists.txt` 的完整内容如下：

```
cmake_minimum_required(VERSION 3.16)
project(ide_demo LANGUAGES CXX)
add_executable(ide_demo main.cpp)
target_compile_features(ide_demo PRIVATE cxx_std_17)
```

先记住：`ide_demo` 是可执行目标名；`main.cpp` 是它的源文件；工程使用 C++17。两套软件操作同一份源码时，分别使用自己的构建目录，避免不同编译器共用 CMake 缓存。

1.8.4 CLion 主界面：每一块负责什么

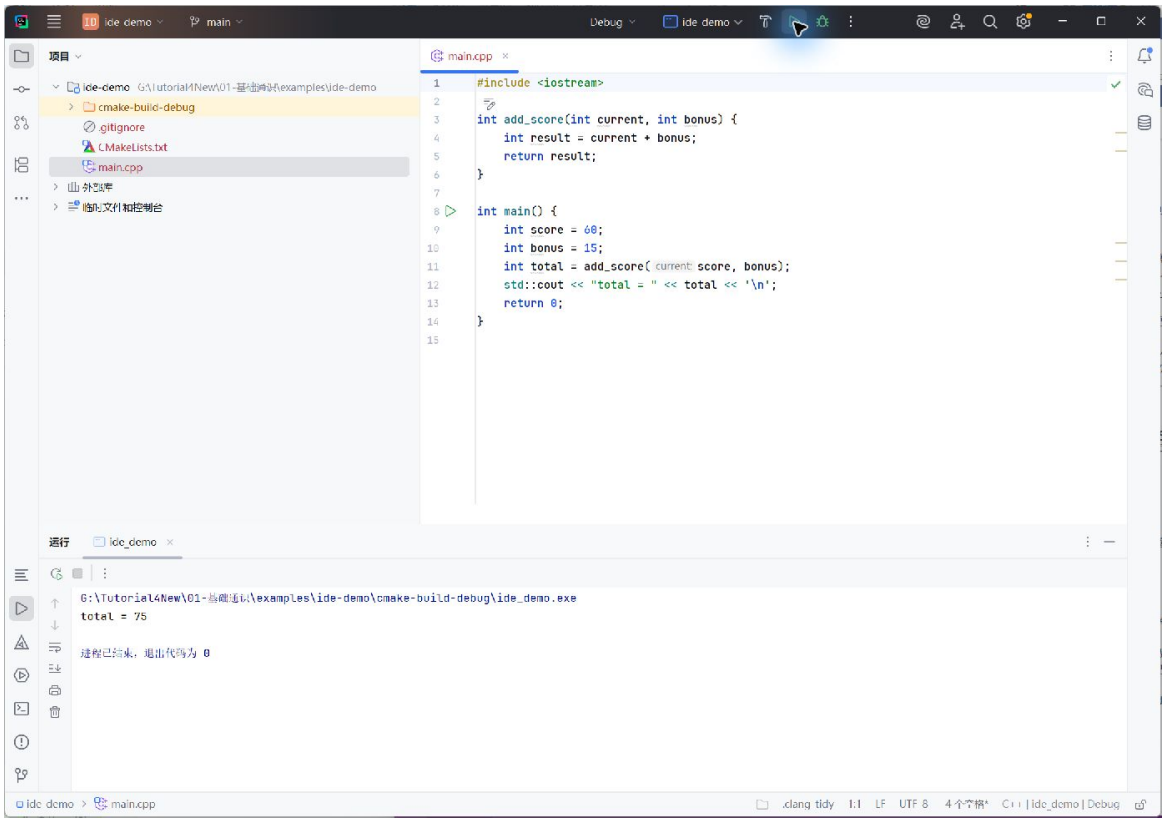


图 1.25: CLion 2025.1.3 Windows 中文界面：教学工程与运行结果（本机截图）

先沿窗口边缘找入口，再看中央编辑区。下表按位置说明常用按钮；被折叠的工具窗口可从 View → Tool Windows（视图 → 工具窗口）重新打开。

位置 / 按钮	点击后的功能与使用时机
左上角三横线主菜单	展开 File、Edit、View、Navigate、Code、Refactor、Build、Run、Tools、Git、Window、Help；打开项目、设置与退出从这里找。
顶部项目名下拉	切换或打开项目；先看名称，避免在别的工程中运行。
顶部 Git 分支名	查看、切换和新建分支；提交前核对分支。未启用 Git 时可能不显示。
顶部 Debug / Release 下拉	选择 CMake 配置；初学调试选 Debug。配置决定构建参数和构建目录。
顶部目标名下拉	选择运行目标；Edit Configurations（编辑配置）设置参数、环境变量、工作目录。
锤子 Build（构建）	编译并链接所选目标，成功后生成程序；只构建不会执行程序。
三角 Run（运行）	通常先构建再运行当前目标，在 Run 窗口看输出；普通运行不在断点暂停。
小虫 Debug（调试）	带调试器启动目标，可在断点暂停并观察变量。
方块 Stop（停止）	终止当前运行或调试进程；仅有活动进程时可用。
放大镜 Search Everywhere	查找文件、符号和动作；连按两次 Shift 也能打开。
齿轮 Settings（设置）	配置字体、快捷键、工具链等；注意区分全局设置与工程设置。

位置 / 按钮	点击后的功能与使用时机
右侧铃铛 Notifications（通知）	查看后台任务、更新和错误提示；读清内容再处理，通知不是编译结果的替代品。
顶部 AI / 协作图标（若启用）	AI 入口提供代码辅助，Code With Me 等协作入口用于共享会话；本节的本机构建和调试不依赖它们。
窗口最小化 / 最大化 / 关闭	控制整个应用窗口；不同于关闭一个编辑标签或隐藏一个工具面板。

位置 / 按钮	点击后的功能与使用时机
左侧文件夹 Project（项目）	展开目录；双击文件打开；右键目录 →New 新建文件，右键文件 →Refactor → Rename 重命名并更新引用。
左侧 Structure（结构）	列出当前文件的函数、类等符号，点击可跳转。
编辑区顶部文件标签及叉号	切换 / 关闭文件；关闭标签不会删除磁盘文件。CLion 有自动保存机制，运行前仍可按 Ctrl+S 明确保存。
行号旁空白槽	单击添加 / 取消红色断点；右键断点可加条件。主函数旁的绿色三角可为相应目标运行或调试。
代码旁灯泡 / 波浪线	查看检查结果与建议修复；Alt+Enter 展开动作，先读说明再应用。
右侧滚动条标记	定位错误、警告和搜索命中；悬停查看提示，点击跳到相应位置。
CMake 工具窗口	查看配置与生成日志；Reload CMake Project（重新加载）重新读取构建描述，不等于编译。
Build（构建）窗口	查编译器、链接器输出；点击报错中的文件与行号跳转。
Run（运行）窗口	看程序输出与退出码；程序等待标准输入时，点击控制台输入。
Debug（调试）窗口	看调用栈、局部变量、监视表达式与单步控制，详见后面的调试表。
Terminal（终端）	打开当前机器的 shell；Windows 默认终端不因此变成 Ubuntu。
Git / Commit 工具窗口	查看差异、选择提交文件、写提交说明；提交到本地与推送到远程是两步。
工具窗口的省略号 / 隐藏按钮	省略号管理该面板的选项与布局；隐藏只收起面板，不代表停止进程。
底部状态栏	行列号用于定位，UTF-8 是编码，LF/CRLF 是换行格式，缩进项控制空格 / Tab。点击可调整，勿为消除提示随意改全文件。

1.8.5 在 CLion 中构建、运行和调试

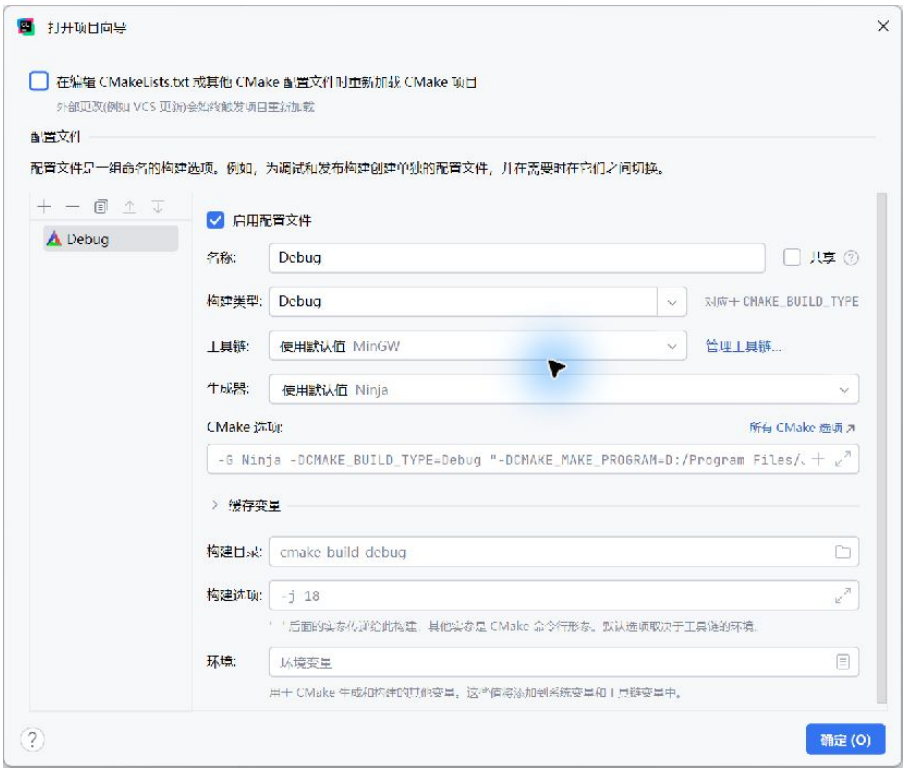


图 1.26: CLion 打开项目向导：构建类型、工具链、生成器与构建目录

示例 1.28

1. 用 Open 打开练习目录。若出现 CMake 向导，选 Debug、已检测成功的工具链和默认构建目录；等待配置完成。
2. 在顶部选择 `ide_demo`，点锤子。Build 窗口应显示成功；再点绿色三角，Run 窗口应出现 `total = 75` 和退出码 0。
3. 打开 `main.cpp`，在第 11 行（调用 `add_score`）左侧放断点，点小虫。程序暂停时该行**还没有执行**，所以此时不要读取尚未完成初始化的 `total`。
4. 在变量区确认 `score=60`、`bonus=15`。点 Step Into（步入）进入函数，查看形参 `current` 和 `bonus`；单步执行加法，再看 `result=75`。
5. 点 Step Out（步出）返回调用者，必要时再单步到下一条语句，确认 `total=75`；点 Resume（继续）直到输出并正常结束。

运行参数位于目标下拉 →Edit Configurations。本例不用参数，工作目录设为工程目录即可；以后程序使用相对路径读图片时，这个目录决定从哪里找文件。设置里的 Toolchains 选择“用哪些工具”，CMake 配置选择“怎样构建”，运行配置选择“启动哪个程序、传什么参数”。三个入口各管一层。

✍ **练习 1.21** 在 CLion 中将 `bonus` 改成 20，预测输出，再构建运行验证。在函数内部设置断点，记录执行加法前后的变量值，最后恢复为 15。

1.8.6 安装 VS Code 与 C++ 扩展

Windows 安装步骤

1. 从 VS Code 官网下载对应架构的 Windows User Installer。运行后选择安装目录，按需勾选桌面快捷方式、右键“通过 Code 打开”与“添加到 PATH”。安装结束后重新打开终端，PATH 才会在新终端中生效。
2. 启动 VS Code，左侧点四个方块的 Extensions（扩展），搜索并核对发布者 Microsoft，安装 **C/C++**（`ms-vscode.cpptools`）和 **CMake Tools**（`ms-vscode.cmake-tools`）。二者分别提供 C++ 语言 / 调试支持和 CMake 工程操作。
3. 如需中文，安装 Microsoft 的 Chinese (Simplified) Language Pack，在命令面板中执行 Configure Display Language，选简体中文并重启。主题和语言不会改变工程的编译方式。
4. 安装 C++ 编译工具。Windows 本节采用 MSYS2：从其官网安装到默认 `C:/msys64`，打开 **MSYS2 UCRT64** 终端执行下面命令；遇到包组选择可回车使用默认项，再按提示确认安装。

```
pacman -S --needed mingw-w64-ucrt-x86_64-toolchain
pacman -S --needed mingw-w64-ucrt-x86_64-cmake mingw-w64-ucrt-x86_64-ninja
```

在 Windows“编辑账户的环境变量”中编辑用户 Path，新增 `C:\msys64\ucrt64\bin`（自定义安装则换成实际路径）。完全退出并重新打开 VS Code，在新 PowerShell 终端运行 `g++ --version`、`gdb --version`、`cmake --version`、`ninja --version`，应分别得到版本号。若有多套工具链，用 `where.exe g++` 检查搜索结果，随后在 CMake Tools 中选择同一套 UCRT64 编译器和调试器。

Ubuntu 安装步骤 从官网选择 Linux 的 `.deb` 安装包，架构与机器一致。在下载目录执行 `sudo apt install ./实际下载的文件名.deb`；这里要替换为真实文件名，不要原样照抄。编译工具使用前面 Ubuntu 示例的安装命令。启动 VS Code 后同样安装 C/C++ 和 CMake Tools，工具链选系统 GCC。

笔记 VS Code 本体及 C/C++ 扩展不包含 **C++** 编译器。先确认版本命令能运行，再处理编辑器配置。安装参考：

https://code.visualstudio.com/docs/setup/windows

https://code.visualstudio.com/docs/setup/linux

Windows GCC 配置：

https://code.visualstudio.com/docs/cpp/config-mingw

MSYS2 下载：https://www.msys2.org/。

1.8.7 VS Code 主界面：活动栏、编辑器和底部面板

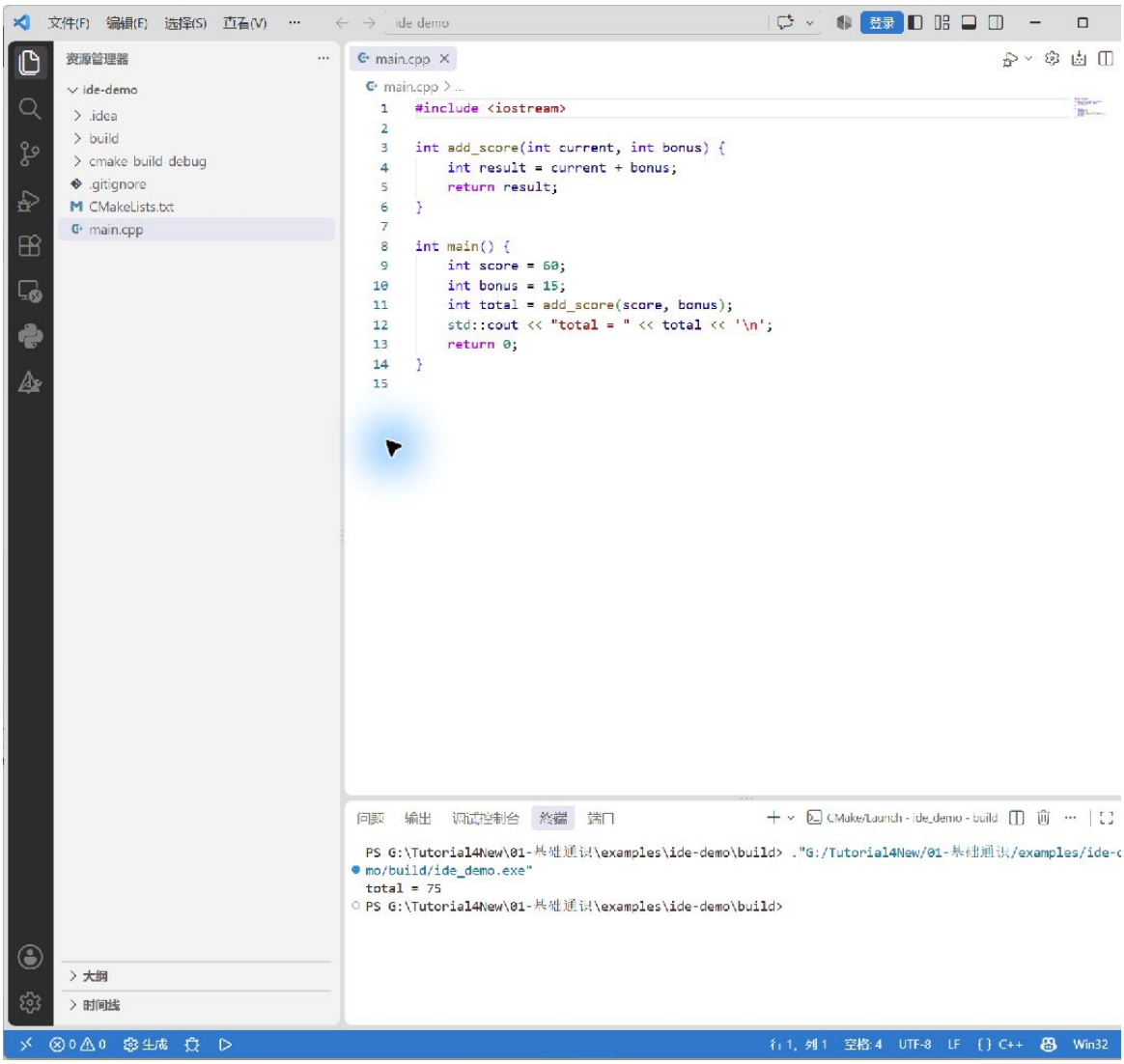


图 1.27: VS Code 1.137.0 Windows 中文界面：文件树、编辑区和面板入口（本机截图）

位置 / 按钮	点击后的功能与使用时机
顶部 File / Edit / Selection	文件菜单管理打开、保存；编辑菜单提供撤销、查找与替换；选择菜单提供多光标等选择操作。
顶部 View / Go / Run	视图菜单显示面板与命令面板；转到菜单定位文件、行和定义；运行菜单管理调试。窄窗口可能折进省略号。
顶部 Terminal / Help	终端菜单新建终端、运行任务；帮助菜单查看版本、文档和快捷键。
顶部搜索框 / 命令中心	快速打开文件、查找命令；Ctrl+P 搜索文件，Ctrl+Shift+P 打开命令面板。
左侧纸张 Explorer（资源管理器）	显示文件树。悬停目录标题出现新建文件、新建文件夹、刷新和折叠全部按钮；双击文件固定标签，单击常为预览。
左侧放大镜 Search（搜索）	跨文件搜索；展开替换框可批量替换。Aa 区分大小写，整词按钮限制完整单词，.* 启用正则表达式。

位置 / 按钮	点击后的功能与使用时机
左侧分支 Source Control（源代码管理）	查看差异；文件旁加号暂存，暂存区减号取消暂存，提交按钮创建本地提交。同步 / 推送会访问远程。
左侧三角加小虫 Run and Debug	选择调试配置，查看变量、监视、调用堆栈和断点；单击大运行按钮可能要求先配置调试器。
左侧四方块 Extensions（扩展）	搜索、安装、禁用或卸载扩展；齿轮管理某个扩展，核对发布者后再安装。
左侧 CMake 图标（安装扩展后）	展开项目状态与项目大纲，选择工具链、配置、构建目标、启动目标并运行 / 调试。
左下角头像 / 齿轮 Manage	头像用于账户相关功能；齿轮打开设置、快捷键、主题等。设置里区分 User（用户）和 Workspace（工作区）作用范围。
标题栏布局按钮	显示或隐藏主侧栏、底部面板、辅助侧栏等；隐藏侧栏不会关闭工程。
额外的活动栏图标	Python、远程开发、AI 等扩展会增加入口；悬停看名称，用活动栏右键菜单管理显示项。

位置 / 按钮	点击后的功能与使用时机
编辑标签、圆点和叉号	标签切换文件；圆点表示有未保存修改，Ctrl+S 保存；叉号关闭标签，不删除文件。
编辑区右上分栏 / 省略号	分栏可并排查看两个文件；省略号包含当前编辑器的更多动作。
编辑区右上三角及下拉	含义由语言与扩展决定，可能是运行当前文件；CMake 工程按后文的 CMake 命令运行整个目标。
面包屑 / 缩略图	面包屑显示文件路径与函数层级，点击定位；右侧缩略图帮助快速滚动长文件。
行号左侧与折叠箭头	点空白槽设断点；点折叠箭头收起代码块，不会删除代码。
Problems（问题）	汇总诊断；点某条跳转到对应行，错误计数不等于本次构建一定失败，须结合构建日志判断。
Output（输出）	看扩展日志，右侧下拉选择 CMake 等通道；通常不能在这里给程序输入。
Debug Console（调试控制台）	看调试信息、求表达式值；C++ 程序的标准输入一般使用终端或外部控制台。
Terminal（终端）	运行 shell 命令或与程序交互;加号新建终端,下拉选 PowerShell/Bash 等，分栏并排显示，垃圾桶结束该终端及其会话。
面板最大化 / 叉号	最大化扩大面板，叉号隐藏面板；隐藏终端面板不会自动结束里面的程序。
底部行列号、Spaces、UTF-8、CRLF/LF	分别控制定位、缩进、编码和换行格式；C++ 表示当前语言模式，若误为 Plain Text，应改为 C++。
底部错误 / 警告、分支、远程标记	点计数打开问题；分支项处理版本控制；远程标记用于连接 WSL/SSH 等环境。先确认当前到底连在哪台机器。

1.8.8 在 VS Code 中运行同一个 CMake 工程

示例 1.29

1. File → Open Folder 打开练习目录。检查文件树顶层的两个文件。自己创建或已检查的教材工程可按提示信任；未知工程先在受限模式下检查内容。
2. 按 Ctrl+Shift+P 打开命令面板，输入 `CMake: Select a Kit`，选实际安装的 GCC：Windows 用 UCRT64，Ubuntu 用系统 GCC。若找不到，先运行 `CMake: Scan for Kits`。
3. 执行 `CMake: Select Variant`，选择 Debug；再执行 `CMake: Configure`。在 Output 中选 CMake 通道，确认配置、生成成功。
4. 执行 `CMake: Build`，等待成功，再执行 `CMake: Run Without Debugging`。若询问启动目标，选择 `ide_demo`；终端应输出 `total = 75`。
5. 在第 11 行左侧点出红色断点，执行 `CMake: Debug`，观察程序停在该行。按后面表格单步并检查变量，再继续到退出。

本例没有 CMake Presets，因而使用 Kit（工具链组合）与 Variant（构建类型）路线。如果工作区启用了 Presets，命令可能变成 Select Configure Preset；先在工作区设置中将 `cmake.useCMakePresets` 设为 `never` 再做本例。以后接手带 `CMakePresets.json` 的队内工程时，应使用工程提供的预设。CMake Tools 的按钮可能在侧栏 Project Status，也可能在底部；**上述命令面板是布局变化时的备用入口**。若需要显式指定构建目录，可在工作区设置搜索 `cmake.buildDirectory`，使用 `${workspaceFolder}/build-vscode`。

笔记 本例用 CMake Tools 直接启动调试，不要求手写 `launch.json`。以后需要自定义时，`tasks.json` 描述任务，`launch.json` 描述如何启动调试，`c_cpp_properties.json` 配置代码分析；它们都不代替 `CMakeLists.txt`。代码补全无红线只能说明分析器接受配置，不能证明程序已经编译成功。

练习 1.22 用 VS Code 打开同一练习工程，搜索 `bonus`，跳转到定义，再将它重命名为 `reward`（使用 Rename Symbol / 重命名符号，而非无差别全文替换）。保存、构建并运行，确认输出仍为 75；最后撤销并保存。

1.8.9 调试工具栏：不要只记小虫和三角

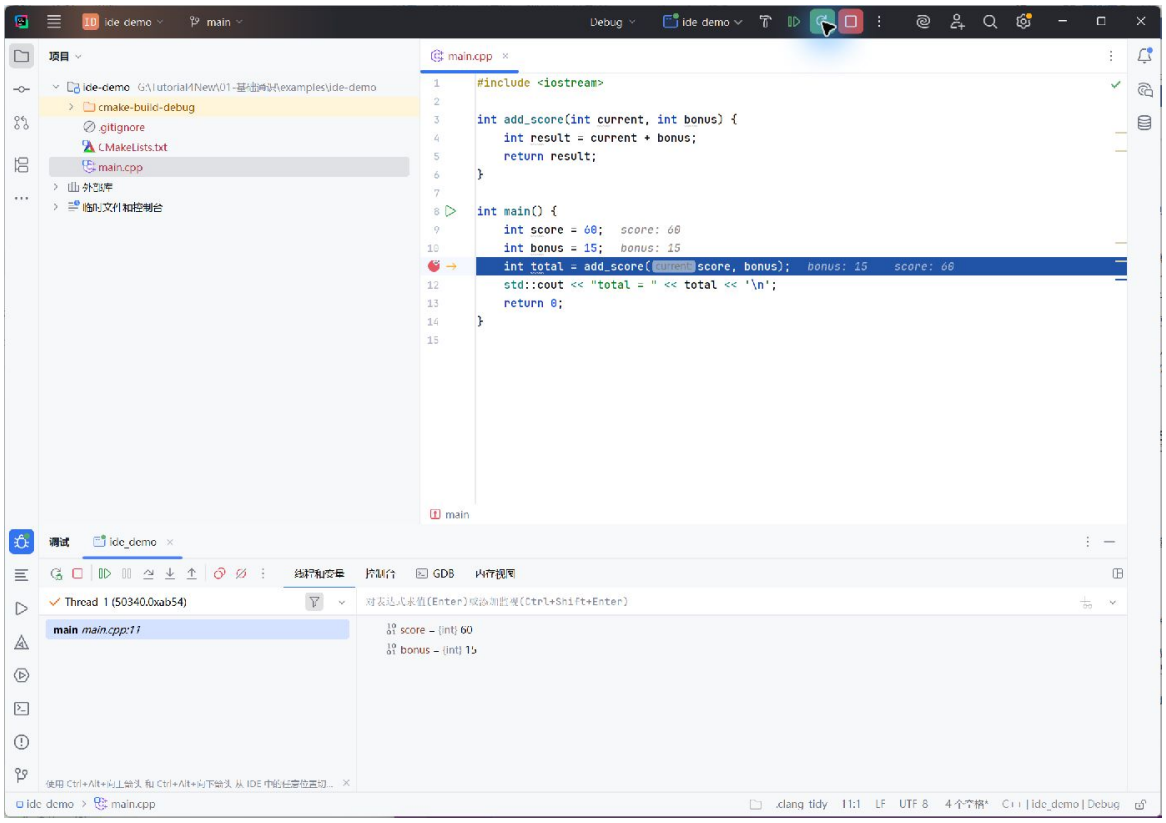


图 1.28: CLion 在调用前暂停：高亮执行行、底部单步按钮、调用栈与局部变量

调试（Debugging）的目标是观察程序如何执行。断点让程序在某行执行前暂停，单步决定下一次停在哪里。以下快捷键为 Windows/Linux 常见默认键位；若换过 Keymap，以按钮悬停提示为准。

按钮 / 功能	CLion	VS Code	使用时机
Continue / Resume 继续（三角）	F9	F5	跑到下一断点或程序结束。
Pause 暂停（双竖线）	看工具栏	F6	运行中请求暂停，排查卡在哪里。
Step Over 步过（弯箭头）	F8	F10	执行当前行，函数调用整体完成。
Step Into 步入（向下箭头）	F7	F11	进入当前行调用的函数内部。
Step Out 步出（向上箭头）	Shift+F8	Shift+F11	执行到当前函数返回调用者。
Restart / Rerun 重新开始（回转箭头）	看工具栏	Ctrl+Shift+F5	重新启动进程，不是把时间倒回上一行。
Stop 停止（方块）	Ctrl+F2	Shift+F5	结束当前调试会话 / 启动的程序。
Toggle Breakpoint 切换断点	Ctrl+F8	F9	在当前行添加或取消断点。

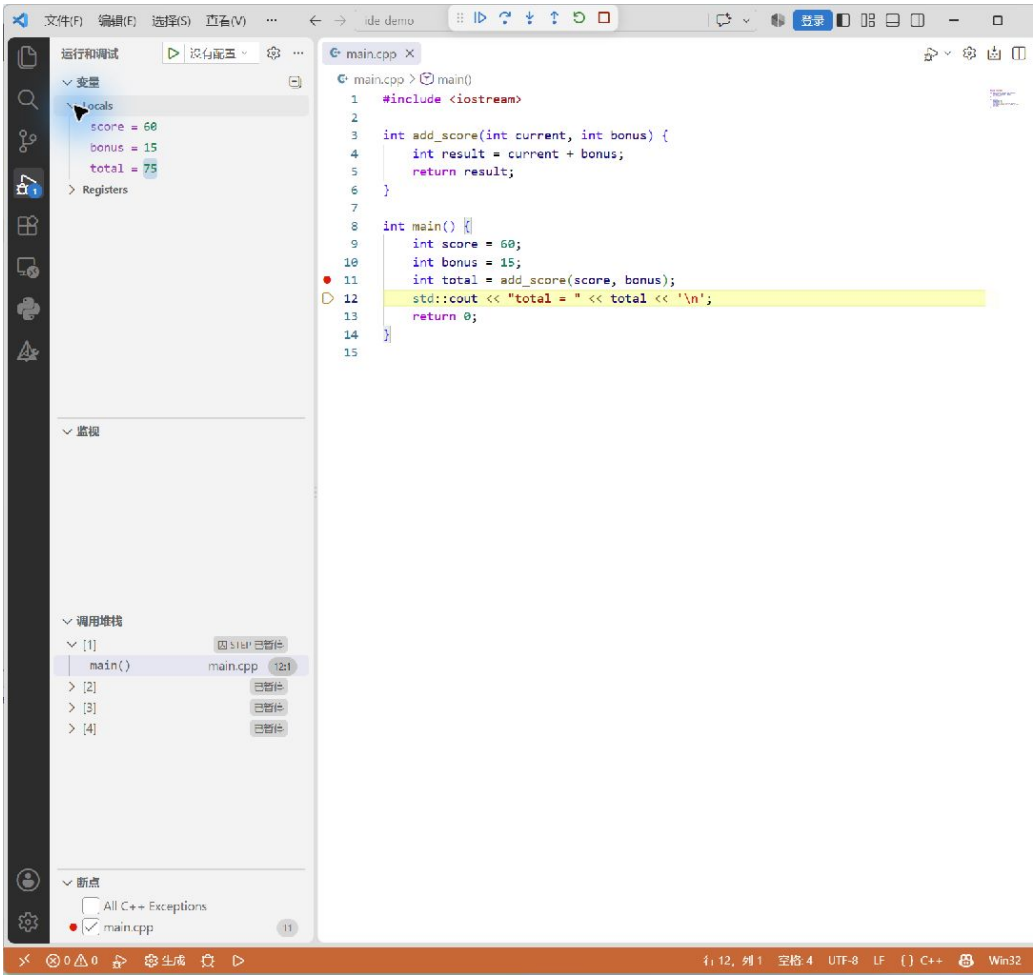


图 1.29: VS Code 单步执行调用后暂停：顶部调试工具栏与左侧变量 total = 75

变量（Variables）显示当前栈帧的局部变量；**监视（Watches/Watch）**可以添加 `score + bonus` 等表达式；**调用栈（Frames/Call Stack）**显示函数调用关系，点击不同帧会切换变量作用域；**断点列表（Breakpoints）**集中启用、禁用或移除断点。表达式必须在当前作用域有效：进入 `add_score` 后看形参 `current`，想查看 `main` 的 `score` 则切回它的栈帧。在暂停状态下查看简单表达式即可；会修改变量或调用函数的表达式可能改变被调试程序。

结论

- 保存只把修改写到磁盘；配置读取 CMake 规则；构建生成程序；运行启动程序；调试带着调试器启动程序。
- Run 不停在断点，Debug 才会；断点高亮行尚未执行，变量也必须已初始化才有可解释的值。
- 工具链、构建配置、运行配置分开检查；界面布局变了就按动作名搜索，不凭图标颜色猜功能。

1.8.10 常见问题与排查顺序

现象	先检查什么
找不到编译器或 CMake	在当前环境的终端查版本和路径；安装后重启 IDE。CLion 检查 Toolchains，VS Code 检查 Kit，不能只安装扩展。

现象	先检查什么
运行按钮灰色 / 没有目标	是否打开了含 CMakeLists.txt 的目录？CMake 配置是否成功？该文件中是否创建了可执行目标？
修改了代码但输出不变	保存文件，检查当前工程、构建目录和启动目标，再构建运行，避免运行另一份旧程序。
断点空心 / 不停 / 变量被优化	使用 Debug 构建并启动调试；确认正在运行刚构建的程序，断点所在代码确实会执行。
更换工具链后 CMake 报缓存冲突	为新工具链换一个构建目录并重新配置；不要让 MSVC、MinGW、Linux 共用缓存。
终端有输出却不能输入	确认是 Terminal/Run 控制台，且程序还活着并在读输入；Output 只是日志。本例输出后立即退出。
编辑器红线与构建结果不一致	检查 C++ 扩展的编译器路径、标准与 CMake 配置提供者，再查看真实构建日志。
Windows 能跑，Ubuntu 找不到文件	检查路径大小写、斜杠、工作目录和系统依赖；不要复制 Windows 可执行文件到 Ubuntu 当作本机程序。

笔记 界面与操作核对资料（2026 年 9 月查阅；本机截图版本以图注为准）：

https://www.jetbrains.com/help/clion/guided-tour-around-the-user-interface.html
https://www.jetbrains.com/help/clion/debugging-code.html
https://code.visualstudio.com/docs/getstarted/userinterface
https://code.visualstudio.com/docs/cpp/cmake-linux
https://code.visualstudio.com/docs/debugtest/debugging

1.9 Linux 串口通信基础

1.9.1 串口连接了什么

视觉程序算出了目标角度，还需要把结果发给电控板。串口通信（Serial Communication）是常见的传输方式：发送端把数据逐位送出，接收端按约定还原成字节。本节先学会在 Ubuntu 中找到设备、取得权限并验证收发；帧头、校验和重同步等工程协议见卷二“多机通信方法”。

我们队使用的是 USB CDC ACM 设备。CDC 是通信设备类（Communication Device Class），ACM 是抽象控制模型（Abstract Control Model）。电控板通过 USB 数据线连接电脑，Linux 通常由 `cdc_acm` 驱动将其呈现为 `/dev/ttyACM0` 等串口设备，程序仍可按串口接口读写字节。

这条链路是“视觉程序 → Linux 串口接口 → USB → 电控板固件”。它与“USB 转串口模块 → TX/RX → UART”不同：队内这种 USB 连接不需要在电脑和板子之间另接 TX/RX，也不能通过短接 USB 的 D+/D- 来做回环。板端是否回传数据、是否要求 DTR 控制信号，应查看电控固件约定。

下面先补充 UART 参数作为通用知识。

UART 的全称是 Universal Asynchronous Receiver/Transmitter，中文为通用异步收发器。它没有单独的时钟线，因此两端必须约定相同的通信参数。USB 转串口模块连接板上 UART 时，这些参数直接影响线路收发。

参数	作用
波特率（Baud rate）	每秒传输的符号数；常见 UART 中一个符号承载一位，例如 115200。两端须匹配。
数据位（Data bits）	每个字符承载的数据位数，常用 8 位。
校验位（Parity）	可选的奇偶校验，N 表示无校验，E 为偶校验，O 为奇校验。
停止位（Stop bits）	标记字符结束的位，常用 1 位。
流控（Flow control）	防止接收方来不及处理；可使用 RTS/CTS 硬件流控或 XON/XOFF 软件流控。本节均关闭。

115200、8N1 表示波特率为 115200、8 个数据位、无校验、1 个停止位。对于 UART，加上 1 个起始位，每传一个字节需要 10 位，因此理论数据吞吐上限约为 115200/10 = 11520 字节每秒；实际有效载荷还要扣除协议开销。

笔记 不要把 **UART** 的波特率公式套到原生 **USB CDC ACM** 上。电脑设置的波特率、数据位等会作为线路编码配置交给设备，固件可能使用，也可能忽略；它们不是 USB 总线速率。队内工程应采用电控固件约定的配置，不能推断“设置 115200 就把 USB 限速为 11520 字节每秒”，也不能断言所有 CDC 设备都不关心这些配置。

示例 1.30 先核对接线，再打开软件 连接两个兼容电平的 UART 接口时，将电脑侧模块的 TX（发送）接到板端 RX（接收），模块 RX 接板端 TX，GND 接 GND。记录板端要求的电平、波特率和 8N1 等参数，再进入后面的设备识别步骤。

笔记 TTL/CMOS 电平 UART、RS-232 和 RS-485 的电气接口不同，不能直接互接。先按硬件说明确认 3.3 V/5 V 电平兼容性，并在断电时接线；开发板已有供电时，不要随手再接模块的 VCC。以下物理回环实验只对脱离电控板的 **USB** 转 **TTL** 串口模块进行，不向连接电机的控制板发送测试字符。

1.9.2 找到设备并取得访问权限

Linux 把串口暴露为 `/dev` 下的字符设备。USB 转串口模块常见名称为 `/dev/ttyUSB0`，USB CDC ACM 设备常见名称为 `/dev/ttyACM0`，板载串口也可能叫 `/dev/ttyS0`。名称取决于设备与驱动，不能仅凭名字判断它连接了哪块板。

示例 1.31 通过插拔确认设备 在终端运行下面命令观察内核日志，然后通过 USB 数据线连接电控板；记下新出现的 `ttyACM` 设备名，按 Ctrl+C 结束观察。

```
sudo dmesg -w
```

接着列出设备及其稳定链接：

```
ls -l /dev/ttyUSB* /dev/ttyACM*
ls -l /dev/serial/by-id/
```

未匹配的类型或不存在的目录可能提示“没有那个文件或目录”，不代表所有串口都不可用。`ttyACM0` 和 `ttyUSB0` 的编号都可能随插拔顺序变化；若设备提供可用的唯一标识，优先使用实际列出的 `/dev/serial/by-id/` 链接。下一小节将为队内 ACM 设备配置更易读的固定别名。

访问串口需要读写权限。先用 `ls -l` 查看设备所属组，用 `id -nG` 查看当前会话的组。Ubuntu 中串口设备通常属于 `dialout` 组。`usermod -aG 组名用户名`可以把用户追加到组中；`-a` 必须保留，避免替换用户已有的附加组。

示例 1.32 为普通用户配置串口权限 下面假设刚才确认的设备是 `/dev/ttyACM0`，且所属组为 `dialout`。在自己的普通用户终端执行：

```
PORT=/dev/ttyACM0
ls -l "$PORT"
id -nG
sudo usermod -aG dialout "$USER"
```

保存工作，**完整注销桌面会话并重新登录**，再执行 `id -nG` 确认含有 `dialout`。仅新开一个终端可能仍继承旧会话的组信息。后续示例都需要在当前终端重新设置 `PORT`；若实际设备名不同，请替换它。

笔记 不要把 `chmod 777 /dev/ttyACM0` 当成长期方案：它把设备开放给所有用户，重新插拔后权限还可能被重建。也不要长期用 sudo 启动整个开发工具来绕过权限问题。若设备所属组不是 `dialout`，应先核对本机设备规则。

练习 1.23 连接队内 USB CDC ACM 板，记录设备名、所属组和可用的稳定链接，解释为什么不能在工程中永远假定它叫 `ttyACM0`。没有硬件时，解释“设备不存在”与“权限不足”分别应先检查什么。

1.9.3 用 udev 绑定固定名称

udev 是 Linux 用户空间的设备管理机制，设备出现时会根据规则设置权限、创建符号链接。我们的目标是让程序始终打开 `/dev/rm_control`，由 udev 将它指向当前的 `ttyACM0` 或 `ttyACM1`。**固定的是程序使用的别名，不是内核分配的编号。**这里的别名是教学示例，队内工程已有命名约定时应使用工程约定。

`udevadm info --name=设备 --attribute-walk` 查看设备及其父设备属性；`--query=property` 查看设备属性记录。厂商编号 VID（Vendor ID）、产品编号 PID（Product ID）通常只能识别型号；还应匹配设备的唯一序列号。

示例 1.33 识别目标板并写入规则 先确认目标板对应的设备，再查看完整属性：

```
udevadm info --name=/dev/ttyACM0 --attribute-walk
udevadm info --name=/dev/ttyACM0 --query=property
```

在属性遍历输出中，找到**同一个 USB 父设备块**里的 `idVendor`、`idProduct` 和 `serial`。不要把 USB 集线器的属性当作电控板属性，也不要把不同父设备块的 `ATTRS` 条件拼成一条规则。

用编辑器创建本地规则文件：

```
sudo nano /etc/udev/rules.d/99-rm-control.rules
```

文件内容如下。**先将三个大写占位值替换为刚才查到的真实值**，VID/PID 按输出填写四位十六进制数字，不加 `0x`；它们不是队内板子的已知编号。

```
SUBSYSTEM=="tty", KERNEL=="ttyACM*", \
  ATTRS{idVendor}=="YOUR_VID", ATTRS{idProduct}=="YOUR_PID", \
  ATTRS{serial}=="YOUR_SERIAL", \
  SYMLINK+="rm_control", GROUP="dialout", MODE="0660"
```

这是**规则文件内容，不是 Shell 命令**。行末反斜杠表示续行，反斜杠后不要再留空格；也可合成一整行。== 表示匹配，SYMLINK+= 表示追加链接名称；名称不带 /dev/ 前缀。GROUP 与 MODE 设置目标设备节点的组及读写权限，用户仍需加入该组。

示例 1.34 重载、插拔并验证固定路径 保存规则并退出编辑器，然后重载规则：

```
sudo udevadm control --reload-rules
```

重载只让后续设备事件使用新规则，不会立即重建已有设备链接。先停止串口程序，在允许断开板子的状态下拔插目标 USB 设备，再运行：

```
udevadm settle
ls -l /dev/rm_control
readlink -f /dev/rm_control
ls -lL /dev/rm_control
udevadm info --name=/dev/rm_control --query=property
```

应看到别名指向一个实际的 ttyACM 节点；ls -lL 检查的是目标节点权限，不能把符号链接自身的 lrwxrwxrwx 当作设备开放给所有人。再次插拔、换 USB 端口或改变两块板的插入顺序，核对别名仍指向相同序列号的板。

最后将程序配置中的设备路径改为 /dev/rm_control。程序不会因为创建了别名就自动修改原来的配置。拔出设备时链接会消失；重新插入时 udev 重新创建链接，程序仍需关闭旧句柄并重新打开。

笔记 若两块同型号板没有唯一序列号，只匹配 VID/PID 会让它们争用同一个别名，不能可靠区分。优先由电控固件提供稳定、唯一的 USB 序列号；临时也可使用实际存在的 /dev/serial/by-path/ 路径，但它绑定的是 USB 端口拓扑，换插口或集线器后可能变化。若一块复合设备暴露多个 ACM 端口，还须区分接口，例如核对属性记录中的 ID_USB_INTERFACE_NUM，再用对应的 ENV 条件匹配，不能让同一块板的多个端口抢同一个别名。

规则未生效时，依次检查文件是否以.rules 结尾、占位值是否替换、父设备属性是否匹配，以及是否重载后重新插拔。不要把 ln -s /dev/ttyACM0 /dev/rm_control 当作持久绑定：它仍指向会变化的编号，也没有建立设备身份匹配规则。

练习 1.24 为一块 ACM 板编写固定别名规则，说明每个匹配条件和权限设置的作用。验证换 USB 插口后别名对应的序列号不变。若两块板的 VID/PID 相同，为什么不能只用这两个条件？没有硬件时提交带占位值的规则草稿和验证步骤，不将它记为实机通过。

1.9.4 设置参数与观察收发

stty 用于查看或修改终端设备的通信参数，通用形式为 stty -F 设备 [设置项]。一定指定 -F，否则可能修改当前终端。常用设置如下，其余参数请查阅 man stty。

参数	作用
`-F 设备`、`-a`	选择设备；显示全部当前设置。
`115200`、`cs8`	设置波特率与 8 个数据位。
`-parenb -cstopb`	禁用奇偶校验，采用 1 个停止位。
`raw -echo`	使用原始模式，关闭终端行处理与自动回显。
`-ixon -ixoff -crtscts`	关闭软件流控与 RTS/CTS 硬件流控。
`clocal cread`	忽略调制解调器控制信号，启用接收。

示例 1.35 检查并设置 115200、8N1 确认设备空闲后执行：

```
PORT=/dev/rm_control
stty -F "$PORT" raw -echo 115200 cs8 -parenb -cstopb \
    -ixon -ixoff -crtscts clocal cread
stty -F "$PORT" -a
```

本例需先完成别名绑定；也可临时填写已确认的实际设备路径。输出中应能找到波特率 115200、`cs8`、`-parenb`、`-cstopb` 等。对 ACM 设备，这些是主机请求的配置，固件如何使用须查板端约定。后续程序打开串口时可能重新设置参数，因此在终端设好一次不等于所有程序都会沿用。

使用 pySerial 附带的 miniterm 可以交互收发；通用形式为 `python3 -m serial.tools.miniterm [选项] 设备 波特率`。本节在 Ubuntu 中安装系统包，并显式使用系统 Python，避免 Conda 环境读不到系统包。

示例 1.36 队内 ACM 设备的收发验证 先由电控同学确认测试固件的协议和安全状态。只有**固件明确提供文本回显测试**时，才使用下面的交互实验；若板端只支持二进制帧，应使用对应的协议测试程序，不能随意输入 `hello` 来判定链路好坏。

```
sudo apt update
sudo apt install python3-serial
PORT=/dev/rm_control
/usr/bin/python3 -m serial.tools.miniterm --parity N \
    --eol LF "$PORT" 115200
```

参数按测试固件约定调整。在文本回显固件下，输入约定的测试文字并回车，应收到相同内容；此处未开启本地回显，文字来自板端返回。按 Ctrl+] 退出。没有文本回显固件时，先做下一节软件回环，保留实板协议测试为待完成项。ACM 收发不需要短接板上 UART 引脚。

示例 1.37 扩展：USB 转 TTL 模块的物理回环 将 USB 转 TTL 串口模块从电脑拔下，确认它没有连接开发板，再短接**该模块自身的 TX 与 RX**，重新插入电脑。配置权限后运行：

```
sudo apt update
sudo apt install python3-serial
PORT=/dev/ttyUSB0
/usr/bin/python3 -m serial.tools.miniterm --parity N \
    --eol LF "$PORT" 115200
```

miniterm 默认使用 8 个数据位、1 个停止位，未指定流控选项时关闭流控；这里也未开启本地回显。输入 `hello` 并回车，应看到从 RX 返回的文字。按 **Ctrl+]** 退出；Ctrl+T 后按 Ctrl+H 可查看快捷键帮助。

退出并拔下模块，移除短接线，再插入并重开 miniterm。此时输入同样内容应没有返回，用以区分真正收到数据与本地回显。回环通过只能说明该模块及电脑侧收发基本正常，不能证明开发板协议或双方波特率配置正确。

1.9.5 没有硬件时：用软件回环练习字节收发

pySerial 提供 `loop://` 软件回环：写入的数据会回到同一个程序的接收端。它不创建真实设备，也不检验 Linux 设备权限、驱动或接线。下面的代码只是收发实验工具，不要求先掌握 Python 编程。

示例 1.38 验证原始字节与读取超时 先按上一例安装依赖。将以下代码保存为 `serial_loop.py`：

```
import serial

with serial.serial_for_url(
    "loop://", baudrate=115200, bytesize=8, parity="N",
    stopbits=1, timeout=1, write_timeout=1,
    xonxoff=False, rtscts=False, dsrdtr=False
) as port:
    payload = b"hello\n"
    port.write(payload)
    received = port.read(len(payload))
    print("RX:", repr(received))
    print("HEX:", received.hex(" "))
    assert received == payload
    print("EMPTY:", repr(port.read(1)))
```

运行 `/usr/bin/python3 serial_loop.py`，预期依次输出 `RX: b'hello\n'`、`HEX: 68 65 6c 6c 6f 0a`，最后等待约 1 秒输出 `EMPTY: b''`。这里读取设置了超时；默认无限等待可能让程序看起来“卡住”。最后一次空读取表示超时期间没有读到字节，并不自动等于设备断开。

串口传输的是字节流，不自带消息边界。发送一次 20 字节，接收端可能分几次读到；连续发送两条消息，也可能被一次读到。真实设备上，带超时的 `read(n)` 可能返回少于 n 个字节，必须累计并按协议解析。文本协议可以约定换行结束，二进制协议则需要明确的帧结构；不要把任意二进制数据直接当 UTF-8 文本解码。

✍ **练习 1.25** 将软件回环的发送内容改为 `bytes([0x00, 0x41, 0xFF, 0x0A])`，记录十六进制输出。解释为什么终端未显示四个可见字符，以及为什么真实串口上一次读取不足四字节不能直接判定丢包。

1.9.6 常见问题：按层排查

现象	优先检查
设备不存在	查看插拔日志，检查 USB 数据线、供电与驱动；确认设备名是否变化。虚拟机还需将 USB 设备交给 Ubuntu。
`Permission denied`	对比设备所属组与当前会话的组，确认加组后已完整重新登录。
设备忙或数据残缺	退出其他串口终端、旧程序；避免多个读取者抢走同一条字节流。
收到乱码	先确认是否为二进制协议；UART 链路还需核对波特率等参数，ACM 则按固件约定检查配置。
能打开但没有数据	ACM 检查固件是否发送、协议及 DTR 约定；UART 另查 TX/RX、共地、电平和流控。
插拔后程序失效	关闭失效句柄，重新识别并打开设备；路径仍相同不代表旧句柄恢复可用。

示例 1.39 查找设备占用者 `fuser [选项]` 设备用于查找正在使用该设备的进程，`-v` 显示详细信息。在 Ubuntu 中可通过 `sudo apt install psmisc` 安装，然后执行：

```
PORT=/dev/rm_control
sudo fuser -v "$PORT"
```

根据进程名确认是否为自己的调试工具，再从对应窗口正常退出；不要直接批量杀进程。若怀疑系统服务探测串口，先用日志确认具体服务及其用途，再针对该设备处理。

结论 串口排错依次看接线与电平 → 设备与权限 → 参数与占用 → 字节与协议。能打开不等于能通信，回环通过不等于板端协议通过；同一串口应由一个程序负责读写，再向其他模块分发数据。先用有限超时和十六进制记录看清收到的字节，再进入卷二的协议设计。

笔记 工具细节参考官方文档：Linux ACM 驱动：https://www.kernel.org/doc/html/latest/usb/acm.html；udev 规则：https://www.freedesktop.org/software/systemd/man/latest/udev.html；GNU stty：https://www.gnu.org/software/coreutils/manual/html_node/stty-invocation.html；pySerial 工具与软件回环：https://pyserial.readthedocs.io/en/latest/tools.html、https://pyserial.readthedocs.io/en/latest/url_handlers.html。

1.10 本章小结

本章围绕”装好系统 → 用好系统 → 护好系统”这条主线展开：

- **安装与维护**：双系统安装的四个流程（分区、制作启动盘、安装、设置）需要亲手走一遍；timeshift 快照相当于配环境前的存档位，每次大改环境前都应先存档，环境炸了随时回档。系统扩容、grub 美化、卸载重装属于选修内容，用到时再回来查即可。
- **命令行基础**：目录结构、”一切皆文件”与挂载是必须理解的概念；文件管理（ls/cp/mv/rm）、查找（find/grep）、重定向与管道、vim 基本操作是每天都要用的基本功，要求熟练；各命令的参数不必死记，会用参数表和 man 手册查即可。
- **shell 与环境变量**：会编写并运行简单的 shell 脚本；理解 PATH 的作用，掌握修改环境变量的三种方式及其生效范围，日常配置优先写入 ~/.bashrc。

- **git**:理解工作区、暂存区、本地版本库、远程仓库四个区域的流转关系,熟练掌握 add/commit/push/pull 的日常闭环，会写基本的.gitignore。
- **conda**（选修）：会创建、激活、退出和删除虚拟环境，配环境时先在虚拟环境中试错。
- **图形开发工具**：安装 CLion 与 VS Code，识别菜单、项目树、工具栏、输出面板和状态栏；用同一份 CMake 工程完成配置、构建、运行和断点调试，能区分编辑器与编译工具链。
- **串口通信**：识别队内 USB CDC ACM 设备及所属组，用 udev 为目标板绑定固定别名；区分 USB 通信与 UART 参数的含义，通过约定的测试协议验证收发；按设备、权限、命名、占用和协议逐层排错。

完成本章验收后，就可以揣着新装好的系统进入第 2 章了。配置环境的道路还会不断延伸——不要停下来啊，第 2 章现代 C++ 见。

第一章 练习

1. 说出双系统安装的四个主要流程，并解释安装类型为什么必须选择” 其它选项”。
2. 解释” 挂载” 的含义，并说明 U 盘插入 Ubuntu 后一般被自动挂载到哪个目录下。
3. 写出命令：以易读单位统计当前目录下各子目录的磁盘占用（只统计一层）。
4. （基础）解释 `chmod 755 run.sh` 中三个数字各自的含义，并写出等价的符号写法。串口设备属于 `dialout` 组但当前用户不在该组时，应如何取得访问权限？
5. 写出命令:在 `/etc` 目录下查找所有以`.conf` 结尾的文件,并把结果保存到家目录下的 `conf_list.txt` 中。
6. 说明`./test.sh` 与 `bash test.sh` 两种脚本运行方式的区别，以及前者为什么需要先执行 `chmod +x`。
7. 说明 git 中工作区、暂存区、本地版本库三者的区别，以及 `git add` 与 `git commit` 各自的作用。
8. （基础）分别说明 CLion 的锤子、绿色三角、小虫按钮的作用，以及 VS Code 的 Explorer、Extensions、Terminal、Output 各负责什么。为什么装好 VS Code 的 C/C++ 扩展后仍可能无法编译？
9. （提高）若把 PATH 误写为 `export PATH=/opt/app/bin`（未包含原来的 `$PATH`），会出现什么现象？分别就” 终端内直接修改” 和” 写入.bashrc” 两种情况说明恢复方法。
10. （实践）编写脚本 `backup.sh`：将家目录下指定的某个目录打包为以当天日期命名的 `tar.gz` 文件，保存到 `~/backups` 目录中，并解压验证内容完整。
11. （串口综合）队内两块 USB CDC ACM 板的 VID/PID 相同，序列号分别为 `CTRL-A` 和 `CTRL-B`。解释 ACM 与 USB 转 UART 的区别，说明为什么设置 115200、8N1 不能直接推出原生 USB 的吞吐上限。以 VID/PID 占位值写出将 A 板绑定为 `/dev/rm_control` 的 udev 规则，给出重载和核验步骤，并解释为什么绑定后仍需处理断线重连、一次读取仍不一定得到一条完整消息。

1.11 本章验收

1. 演示在 Ubuntu 中定位目录、检查文件权限、运行自己的 Shell 脚本，并解释一次 Git 暂存与提交的差别；安装与备份部分使用已有操作记录核验。
2. 在 CLion 和 VS Code 中分别打开教学工程，指出菜单、文件树、编辑区、构建入口、运行入口、调试入口、终端和状态栏；两套软件均成功输出 `total = 75`。

3. 任选一套工具现场断点调试：在函数调用处暂停，演示步入、步过、步出、继续；指出当前栈帧与变量值，并解释为什么暂停行的赋值尚未生效。
4. 使用队内 USB CDC ACM 板，演示设备识别、普通用户权限检查和固定别名绑定，改变插入顺序后仍能定位同一块板；在电控确认的安全测试状态下，按约定协议验证收发。由考核者给出一个权限或命名故障，说明定位依据。无硬件时先演示软件回环、十六进制输出和超时，提交 udev 规则草稿，并注明设备绑定与板端协议验收待补。

第二章 现代 C++ 基础

内容提要

- 学习目标：掌握类、继承与模板；编写 lambda；理解引用与移动；使用智能指针与常用 STL；掌握多线程同步与常见模型，并理解进程内存分区与堆栈差异。
- 建议学时：约 1.5–2 周。
- 本卷边界：ROS2 执行器与节点生命周期中的多线程用法见卷二；本节只讲标准 C++ 并发工具、经典问题与基础线程池。

2.1 面向对象编程：类

2.1.1 类的基本概念

类是**面向对象**（OO，Object-Oriented）最基本的概念，也是 C++ 区别于 C 语言的重要特征。面向对象编程的基本思想是抽象出对象，将对象作为程序的基本单元，通过消息传递来进行通信和协作。对象是类的实例，具有状态和行为：状态存储在对象的数据成员中，行为由对象的成员函数实现。面向对象编程的优点是代码的可重用性、可扩展性和可维护性都得到了提高。

在 C++ 中，类是一种抽象数据类型，它定义了对象的行为和状态，并提供了对这些数据的访问接口。类可以包含数据成员、成员函数、构造函数、析构函数等，这些成员构成了类的接口。类可以派生出新的类，从而实现代码的重用和扩展。

示例 2.1 类的基本结构（骨架代码，先混个眼熟）：

```
class ClassName{          //类名
    public:               //修饰限定符public
        ClassName();      //构造函数
        ~ClassName();     //析构函数
        void memberFunction();  //成员函数
    private:                    //修饰限定符private
        int dataMember;         //数据成员
        void privateFunction(); //私有成员函数
};  //注意：类定义结束的花括号后面有一个分号
```

1. **构造函数和析构函数**：负责对象的初始化和释放，其特点是函数名与类名相同，没有返回值。
2. **成员函数**：实现类的行为，它们实现了类的功能。
3. **数据成员**：存储类的数据信息。

修饰限定符	作用
private	只能被类的成员函数使用或调用
public	可以被类外的其他函数或派生类中函数调用
protected	只能被类的成员函数和派生类使用或调用

顺带一提，C++ 是一门**多范式**语言：既可以像 C 语言那样面向过程（数据和函数分离，通过函数调用组织逻辑），也可以面向对象，还可以用模板进行泛型编程。本章后面的小节会逐一见到它们。

笔记 struct 和 class 在 C++ 中几乎完全等价，唯一的区别是默认权限：struct 的成员默认 public、默认公有继承；class 的成员默认 private、默认私有继承。除此之外，struct 一样可以有构造函数、成员函数、继承和多态。习惯上 struct 用于纯数据打包（如坐标点），class 用于需要封装的对象。

番外：指名初始化（Designated Initializers）。对于没有自定义构造函数的简单结构体（聚合类型，Aggregate Type），C++20 允许在初始化时指名道姓地给成员赋值，而不必按照成员的声明顺序来写：

示例 2.2 指名初始化：

```
//类型名 对象名{.成员名1 = 初始值1, .成员名2 = 初始值2, ...};
struct Foo {int a, b};
Foo foo = {.b = 3, .a = 1};
```

练习 2.1 定义一个 Point 类，包含私有数据成员 x、y（double 类型），编写含参构造函数和打印坐标的成员函数，并在 main 中创建两个点验证。

2.1.2 静态和友元

静态成员（static member）属于整个类，而不是某个具体对象。无论创建多少个对象，静态成员在内存中只有一份，被类的所有对象共享，因此可以实现一些全局变量的功能。静态成员在类内用 *static* 关键字**声明**，但必须在类外**定义**（分配内存），且只定义一次。静态成员的生命周期和整个程序相同，程序结束时才释放。

友元函数（friend function）：有时我们希望某个函数可以访问类的私有成员，但又不希望它成为类的成员函数。这时可以在类的声明中用 *friend* 关键字把它声明为友元函数。友元函数不是类的成员，实现时不加类的作用域，但可以访问类的私有成员。

示例 2.3 静态成员与友元函数的用法：

```
#include <iostream>

class ClassName {
    public:
        ClassName() = default;
        ~ClassName() = default;

        static void memberFunction();           //静态成员函数
        static int dataMember;                  //静态数据成员，类内声明
        friend void FriendFunction(ClassName& obj); //友元函数声明
    private:
        int secret = 42;                        //私有数据成员
};

int ClassName::dataMember = 10; //静态数据成员在类外定义，且只定义一次

void FriendFunction(ClassName& obj){ //友元函数实现时，不可以添加类的作用域
    std::cout << "友元函数访问私有成员：" << obj.secret << std::endl;
}

```

```
int main(int argc, char *argv[]) {
    ClassName obj1, obj2;
    std::cout << obj1.dataMember << std::endl;  //输出10
    std::cout << obj2.dataMember << std::endl;  //还是10，两个对象共享同一份
    FriendFunction(obj1);
    return 0;
}
```

笔记 友元函数不属于类，它破坏了类的封装性，因此不建议使用！把私有成员的访问权随意送出去，相当于把家门钥匙配了一把给邻居——邻居是不是好人另说，钥匙给多了总会出事。

练习 2.2 为 Point 类添加一个静态数据成员 count，在构造函数中令其加一、析构函数中令其减一，在 main 中分不同作用域创建若干对象，打印 count 观察对象数量的变化。

2.1.3 函数重载

函数重载（overload）是指在同一个作用域中，存在多个同名函数，但它们的参数个数或参数类型不同。函数重载的作用是为了使程序更加灵活，提高代码的可读性和可维护性。

在 C++ 中，函数重载的规则是：

1. 函数名相同；
2. 参数个数不同，或参数类型不同。

示例 2.4 同名不同参的 CalSum：

```
#include <iostream>

class ClassName {
public:
    ClassName() = default;
    ~ClassName() = default;

    int CalSum(int a,int b){
        return a+b;
    }

    double CalSum(double a,double b){
        return a+b;
    }
};

int main(int argc, char *argv[]) {
    ClassName obj;
    std::cout << obj.CalSum(1,2) << std::endl;
    std::cout << obj.CalSum(1.5,2.5) << std::endl;
    return 0;
}
```

在上例中，CalSum 函数的两个版本参数类型不同，编译器会根据调用时传入的实参，自动选择最匹配的函数版本。

笔记仅返回值不同不能构成重载。调用函数时编译器看不到你打算把结果赋给什么类型，如果只靠返回值区分，编译器将无法确定该调用哪一个版本，直接报二义性错误。

2.1.4 构造函数

构造函数的类型：

1. 默认构造：默认构造函数是指没有参数的构造函数，它会创建一个对象，并对其进行初始化。
2. 含参构造：含参构造函数是指有参数的构造函数，可以根据参数的值对对象进行初始化。
3. 拷贝构造：拷贝构造函数是指将一个对象拷贝到另一个对象时，会调用拷贝构造函数。

示例 2.5 三种构造函数的调用时机，注意每一次的输出：

```
#include<iostream>
#include<string>
class ClassName {
public:
    ClassName():member(1) //初始化列表
    {std::cout << "你调用了默认构造函数" << std::endl;}
    ClassName(std::string& str)
    {std::cout << "你调用了含参构造函数，参数是" << str << std::endl;member=2;}
    ClassName(ClassName &obj) //这里必须是引用传参，思考一下为什么
    {std::cout << "你调用了拷贝构造函数" << std::endl;
    this->member = obj.member;}
    ~ClassName() = default;

    int member;
};

int main(int argc, char *argv[]) {
    ClassName obj1;
    std::string str = "Hello World";
    ClassName obj2(str);
    obj1.member = 10;
    ClassName obj3(obj1);

    std::cout << "obj1:" << obj1.member << std::endl;
    std::cout << "obj2:" << obj2.member << std::endl;
    std::cout << "obj3:" << obj3.member << std::endl;

    return 0;
}
```

有一些 tips 可以快速设置构造函数：

```
ClassName() = default;                //默认构造函数,使用默认的方式
```

```
ClassName(ClassName& obj) = delete; //禁止拷贝构造，单例实现对象
```

2.1.5 运算符重载与 this 指针

在 C++ 中，类提供了一个接口，可以将运算符进行重载，从而实现对类更加灵活的操作。运算符重载的语法如下：

```
[返回值] operator [操作符] ([参数列表]) {
    //操作符的实现
}
```

示例 2.6 重载加法运算符与流插入运算符：

```
#include <iostream>

class ClassName {
public:
    ClassName():a(1),b(2) {}

    ClassName& operator+(ClassName& obj){//重载加法运算符
        this->b += obj.b;   //this指针，指向自己
        this->a += obj.a;
        std::cout << "这里执行过了" << std::endl;
        return *this;   //*this表示当前对象的引用
    }

    //重载流插入运算符，思考一下为什么是这样写？
    friend std::ostream& operator<<(std::ostream& out, ClassName& obj);

    int a;
    int b;
};

std::ostream& operator<<(std::ostream& out, ClassName& obj){
    out << "a:" << obj.a << "␣b:" << obj.b << std::endl;
    return out;
}

int main(int argc, char *argv[]) {
    ClassName obj1;
    ClassName res = obj1 + obj1;
    std::cout << res;
    return 0;
}
```

this 指针是一个隐指针，指向这个类本身。在类的成员函数中，*this* 指针可以用来访问类的成员变量和成员函数。一般情况下，*this* 指针可以不写，编译器会自动生成。但是有些情况，出于项目代码可读性的目的，会写显式的 *this* 指针。

✍ 练习 2.3 为 `Point` 类重载 `+` 运算符（对应坐标相加）和 `<<` 运算符（以 `(x, y)` 格式输出），在 `main` 中验证 `Point(1,2) + Point(3,4)` 的输出。

2.1.6 类的继承

类的继承是面向对象编程的重要特征之一，它允许创建新的类，从而继承基类的成员，并添加新的成员。

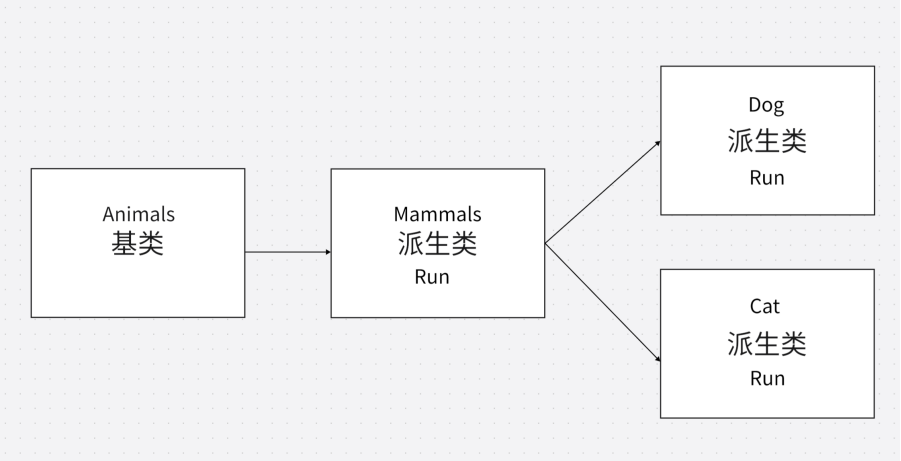


图 2.1: 类的继承

如图 2.1 所示，派生类从基类中继承了数据成员和成员函数，并可以添加新的成员。派生类可以重写基类中的成员虚函数，从而实现新的功能。派生类可以重载基类中的运算符，从而实现新的运算。

基类成员的访问限定符，决定了派生类和类外部能否访问它们。这张表和” 类的基本概念” 中的是同一张，这里从继承的视角再看一次——重点关注” 派生类” 这一行：

访问位置	public 成员	private 成员	protected 成员
同一个类	yes	yes	yes
派生类	yes	no	yes
类外部	yes	no	no

继承可以用下面的写法实现：

示例 2.7 抽象基类与两个派生类：

```
#include <iostream>

class Shape {//这个基类只含有纯虚函数，又叫做抽象类
public:
    Shape() = delete;
    Shape(double A):Area(A)
    {std::cout << "基类的构造函数已经被调用" << std::endl;}
```

```
    virtual double calArea() = 0; //这里是纯虚函数，也可以使用普通函数
    double Area;
};

class Rectangle
        : public Shape{
public:
    Rectangle() = delete;
    Rectangle(double a,double b):Shape(a*b),a(a),b(b)
    {std::cout << "Rectangle类的构造函数已经被调用，面积为" << Area << std::endl;}
    double a;
    double b;
    double calArea() override{  //重写基类的函数
        return a*b;
    }

};

class Circle
        : public Shape{
public:
    Circle() = delete;
    explicit Circle(double r):Shape(3.14*r*r),r(r)
    {std::cout << "Circle类的构造函数已经被调用，面积：" << Area << std::endl;}
    double r;
    double calArea() override{
        return 3.14*r*r;
    }

};

int main(int argc, char *argv[]) {
    Circle circle(1.0);
    Rectangle rect(1.0,2.0);
    return 0;
}
```

笔记继承方式有三种：public、protected、private（不写时默认 private），它决定了基类成员在派生类中的访问权限上限。工程中绝大多数情况都用 public 继承，另外两种仅作了解。含纯虚函数的抽象类不能直接实例化，只能作为基类使用。

另外注意初始化列表的写法：基类构造函数写在成员之前，成员之间按声明顺序初始化，与列表里的书写顺序无关。写错顺序编译器会发出 -Wreorder 警告，如果成员之间存在初始化依赖，还会得到未定义的值。

结论重载（overload）与重写（override）仅一字之差，但完全是两回事：

- 重载 overload：同一作用域内的同名函数，参数列表不同，编译期由编译器选择版本，与继承无关；
- 重写 override：派生类重新实现基类的 `virtual` 函数，函数签名必须相同，运行期根据对象实际类型决定调用哪个版本（多态），建议显式标注 `override` 让编译器帮忙检查；
- 纯虚函数 `= 0`：基类只给接口不给实现，含有纯虚函数的类是抽象类，不能实例化。

练习 2.4 在上例基础上，于 `main` 中创建 `Shape*` 数组分别指向 `Circle` 和 `Rectangle` 对象，循环调用 `calArea()`，观察多态的调用结果。

2.1.7 模版

在开发过程中，我们可能会遇到这样的场景：我们有一个函数或者类，但是它只适用于特定的数据类型，比如 *int* 或者 *double*。为了让这两种类型都可以调用函数，就得把代码复制一遍，但这样做就造成了代码冗余，不利于维护。这时候就需要用到模版（*template*）。

模版是 C++ 中一个重要的特性，它允许我们定义一个类或函数，使其可以接受不同的数据类型或者不同的参数。模版的定义和使用都比较复杂，这里只介绍最基本的用法。**函数模版**用 *template* 加尖括号声明一个类型参数（习惯命名为 T），之后 T 就可以像普通类型一样使用。语法如下：

```
template<class T>
T function(T a){
    return a;
}
```

这里的 *class* T 表示一种数据类型，可以是任意类型。在函数的定义中，T 可以作为函数的参数，也可以作为函数的局部变量。在使用函数模板时，需要首先进行函数的实例化。

示例 2.8 函数模版的实例化与调用：

```
#include <iostream>
#include <functional>

template<class T>
T function(T a){
    return a;
}

int main() {
    //调用的时候实际上分为两步，实例化和调用
    function<int>(10);
    function<double>(20.0);
    //在编译器可以推导出类型的时候可以不加<>
    auto a = function(1.0);
    //在无法推导的时候，要先进行实例化
    std::function<int(int)> ptr = function<int>;
    return 0;
}
```

模版的优点：

1. 代码重用性高：可以定义一个通用的函数，然后根据需要实例化不同的类型，实现代码的重用。
2. 类型安全：模版可以保证类型安全，避免类型错误。
3. 效率高：模版可以提高效率，因为编译器可以对代码进行优化。

类模版：类模板的语法和函数模板类似，只不过在类名而不是函数名前面加上关键字 *template*。在普通类中也可以添加成员函数模版，只需要在函数名前面加上关键字 *template* 即可。

示例 2.9 类模版的基本写法：

```cpp
#include <iostream>

template<class T1,class T2>
class A{
public:
    A() = default;
    ~A() = default;
    T1 val;
    T2 id;
    void print(){
        std::cout << val << "␣" << id << std::endl;
    }
};

int main() {
    A<int, double> a; //实例化时填具体类型
    a.val = 1;
    a.id = 2.5;
    a.print();
    return 0;
}
```

模板参数除了类型，还可以是**编译期常量**（非类型模板参数），常用于指定数组长度这类编译期就要确定的值：

示例 2.10 非类型模板参数：

```cpp
template<int N>
int getConst(){
    return N;
}

int main() {
    int v = getConst<42>(); //v = 42
    return 0;
}
```

在 *template* 后面，可以定义多个类型参数，用逗号分隔。需要注意的是，类中的函数模版需要在头文件中实现，也就是说，模版的声明和实现不可以分离，否则会出现很多问题。对于类模板也可以

实现继承和多态，与普通类相同，这里不再展开，感兴趣的同学可以自行探索。

模板的实例化：模板的实例化指的是将模板定义和具体的模板参数绑定在一起，生成一个具体的类或函数。写 $A < int, double >$ 时，编译器会根据尖括号里的参数生成对应的版本。模板参数可以填两类东西：

1. 具体类型：如 int、$double$ 等基本类型，$struct$、$class$、$union$ 等复合类型，以及指针和引用类型；
2. 编译期可以确定的常量：字面常量、宏、$constexpr$，对应上面的非类型模板参数。

平时我们写的 $function < int >(10)$ 其实叫**隐式实例化**——用到时才生成；也可以在代码里写 `template class A<int, double>;` 主动要求编译器立刻生成，这叫**显式实例化**，常用于模板分离编译的场景，初学阶段了解即可。

除此之外，还有一种变参模版，它可以接受任意数量的类型参数，但是在实例化时，必须指定具体的类型。这种情况比较复杂，这里不再展开。

笔记 对变参模版感兴趣的同学可以自行探讨：

https://blog.csdn.net/fl2011sx/article/details/128077440

练习 2.5 编写函数模版 `myMax`，返回两个同类型参数中较大的一个，分别用 `int`、`double` 测试，并尝试用 `myMax(1, 2.5)` 调用，观察编译器的报错信息，思考为什么。

2.2 Lambda 表达式、引用与移动

随着时代的发展和项目变得越来越复杂，人们创造了一系列新的代码特性，其中一些特性在 C++11 和 C++17 中被引入，在不熟悉这些特性的人看来，这些代码看上去显得十分无厘头，甚至有些难以理解。本节将介绍 C++11 和 C++17 中引入的一些新的代码特性，并通过实例来说明它们的用法。

2.2.1 lambda 表达式

匿名函数（anonymous function）是一种在程序中定义的函数，它没有名字，只能在运行时创建。C++11 引入了 lambda 表达式，它是一种更简洁的语法，可以用来创建匿名函数。lambda 表达式的语法如下：

```
完成版：[捕获列表](参数列表) mutable throw(异常列表) -> 返回值 {函数体}
简化版：[捕获列表](参数列表) {函数体}
```

1. 捕获列表：捕获列表可以用来捕获外部作用域中的变量，使得 lambda 表达式可以访问这些变量。
2. 参数列表：参数列表定义了 lambda 表达式的参数，可以有 0 到多个参数。
3. mutable[可选]：mutable 关键字用来声明 lambda 表达式的捕获列表中的变量是可修改的。
4. throw(异常列表)[可选]：throw 关键字用来指定 lambda 表达式可能抛出的异常。
5. 返回值：返回值定义了 lambda 表达式的返回类型，编译器一般可以自动推导，可以省略。
6. 函数体：函数体定义了 lambda 表达式的主体，可以是一条表达式或多条语句。

示例 2.11 用 lambda 作为比较器，对自定义结构体数组排序：

```
#include<iostream>
#include<functional>

template<class T>
```

```
void sort(T* array,int n,std::function<bool(T&,T&)> compare){
    for(int i=0;i<n;i++){
        for(int j=i;j<n;j++){
            if(!compare(array[i],array[j])){    //为false时，交换（按照指标降序）
                std::swap(array[i],array[j]);
            }
        }
    }
}

typedef struct Block{
    char ch;
    int id;
}Block;

int main(int argc, char *argv[]) {
    auto sum = [](int a,int b){
        return a+b;
    };

    std::cout << sum(1,2) << std::endl;

    Block blocks[5]={{'a',2},{'d',4},{'q',1},{'t',7},{'s',-1}};
    sort<Block>(blocks,5,[](Block& a1,Block& a2){//lambda表达式作为参数传递
        return a1.id > a2.id;
    });
    for(int i=0;i<5;i++){
        std::cout << "ch:" << blocks[i].ch << "␣␣id:" << blocks[i].id << std::endl;
    }
    return 0;
}
```

捕获列表：可以捕获外部作用域中的变量，包括 this 指针。捕获的方式包括两种，即值捕获和引用捕获。

捕获方式	拷贝方式	是否可以修改值	语法示例
值捕获	通常深拷贝	否（配合 mutable 可改副本）	[a,b]，全捕获 [=]
引用捕获	浅拷贝	是	[&a,&b]，全捕获 [&]

如果使用值捕获，这里是否是深拷贝取决于拷贝构造函数，如果拷贝构造函数中是移动或者引用语义，那么遵从拷贝构造的语义。但是这个值会被设置成不可操作的状态，它的值被视为常量，不能被修改。这时，可以使用 mutable 关键字来声明这个值是**可修改**的。

示例 2.12 mutable 与值捕获：

```
int a = 0;
```

```
auto f = [a]() mutable {
    return ++a;//如果不写mutable，编译器会报错
};
```

使用 lambda 表达式可以简化代码，提高可读性。由于捕获列表的存在，使得 lambda 表达式可以访问外部作用域的变量，相比于普通函数，lambda 表达式更加灵活。lambda 表达式的本质是一个**匿名的函数对象**（闭包），可以用 *auto* 或 `std::function` 承接；没有任何捕获的 lambda 还可以隐式转换为函数指针，作为函数参数进行传递。

示例 2.13 用 `std::function` 和 `auto` 承接 lambda：

```
//使用function承接返回值
std::function<int(int,int)> add = [](int a,int b){return a+b;};
int result = add(1,2);//调用lambda表达式
//也可以使用auto关键字自动推导返回类型
auto sub = [](int a,int b){return a-b;};
```

在 C++14 和 C++17 中，lambda 表达式增加了一些新的用法，使用这些新的特性可以简化代码，更加灵活地解决实际问题。

泛型 Lambda 表达式：可以使用 auto 关键字指定形参的类型，从而快速地实现函数**泛用**。这也是**泛化编程**的一种方式，感兴趣的同学可以自行了解。

```
auto f=[](auto a,auto b){return a+b;};
std::cout << f(1,2) << "␣" << f('a',2.0) << std::endl;
```

this 指针捕获：可以使用 [this] 的方式捕获类中的 *this* 指针，*this* 和 $*this$ 都可以。给一个简单的例子，可以自己理解：

```
class A{
private:
    int a;
public:
    A(){
        auto function = [this](){//[]中使用*this作用相同
            return a+1;
        };
    }
};
```

初始化捕获：在 C++14 中，可以使用初始化捕获来简化代码。可以初始化一个变量，并将其绑定到 lambda 表达式中，这样就不需要再声明这个变量了。

```
auto f = [a=1]() mutable {
    return a;
};
```

✍ 练习 2.6 使用 `std::sort`（头文件 `<algorithm>`）配合 lambda 比较器，把一个 `std::vector<int>` 先按升序排序、再按降序排序，分别打印结果。

2.2.2 引用和移动

左值与右值：左值一般为我们自己定义的变量，在定义时开辟了内存，我们可以对这块内存赋值，修改内存中的值，如果有 const 也仅从语法层面上不允许修改，这块内存在其生命周期结束的时候销毁。

右值一般为临时变量，是程序运行时产生的中间产物，它不是我们用户自己定义开辟空间的，是由编译器帮我们开辟空间，并且在用完就立即销毁。右值的生命周期一般只在当前语句，当我们要对右值进行赋值时，它已经释放空间了，此时我们再进行访问就是野访问（与野指针一样造成内存问题），所以我们不能对右值进行修改，编译器强制语法检查，遇到修改操作就报错。

右值中特殊的就是字面常量，它们存储在内存的常量区，内存为只读属性，不可以修改，不可以取地址，它们的生命周期与程序的生命周期一样。**分辨左右值最常用方法就是判断它是否可以取地址。如果不可以就为右值。**

```
int a;
a = 10;//这个赋值过程中，a是左值，10是右值（字面常量）
int b;
b = a;//这个赋值过程中，b是左值，a是左值（左值可以作为右值使用）
//10 = a;//这样的语句显然是错误的，即右值不能被修改
```

显而易见的是，左值可以被隐式地转换为右值，而右值不可以被转换为左值（从这一点上看，右值类似于常量）。

引用：C++ 的引用是一种别名，用于引用已存在的变量。引用一旦初始化，就不能再引用其他变量。在创建引用时，编译器不会重新开辟内存，而是直接引用已存在的内存（这一点引用类似于指针）。引用的主要用途包括函数参数传递和返回值优化。使用引用可以避免复制，提高效率，特别是在处理大型对象时。引用通过 & 符号来声明。例如:

```
int& ref = var;
```

这段代码创建了一个名为 ref 的引用，它引用了变量 var。引用分为两种，左值引用（&）和右值引用（&&）。顾名思义，左值引用只能绑定一个左值，而右值引用只能绑定一个右值。

首先说左值引用，这个十分简单，可以认为是取别名，操作左值引用就相当于操作被绑定的那个左值本身，这一点就算在函数传参过程中也是如此。

示例 2.14 左值引用传参：

```
#include <iostream>

void func(int& A){//可以试一下把引用去掉
    A *= 2;
}

int main(int argc, char *argv[]) {
    int a = 10;
    func(a);
    std::cout << a << std::endl; //输出20
    return 0;
```

```
}
```

在这里 a 的值被修改为 20，因为传入 $func()$ 的 a 是引用，所以操作的是 a 本身，而不是 a 的拷贝。这就是引用在传参和提高函数效率上的用处。

再说右值引用，这个就有点复杂了，右值引用是为了解决左值引用不能绑定右值的限制。

示例 2.15 右值引用绑定右值：

```
#include <iostream>

void func(int&& a){
    std::cout << "a的地址: " << &a << std::endl;
    a *= 2;
}

int main(int argc, char *argv[]) {
    int&& c = 10;
    std::cout << "c的值为: " << c << std::endl;
    func(std::move(c)); //c本身是左值，需要move转换为右值才能传入
    std::cout << "经过函数之后，c的值为: " << c << std::endl;
    std::cout << "c的地址为" << &c << std::endl;
    return 0;
}
```

咦？c 明明绑定了一个右值，为什么能取到地址、还能被修改？

原因在于两点：一是用右值引用绑定右值后，编译器不再在语句结束时自动回收这块临时内存，而是把它的操控权交给了引用；二是**有名字的右值引用本身会被当作左值来使用**——既然有名字，就能取地址、能修改，用起来和普通左值引用没有区别。

那右值引用存在的意义是什么？设想一个超级大的对象：如果只有左值引用，想把它传出函数就只能整份拷贝，效率太低。右值引用让我们可以直接”接管”临时对象的内存，避免这次拷贝——它不创造新数据，只转移资源的所有权。这就是移动构造函数存在的理由，马上就能在下面的例子里看到。

示例 2.16 一个写全了五种函数（默认/含参/拷贝/移动/赋值）的类，请仔细体会每一行的输出：

```
#include <iostream>
class A {
public:
    A() : val("a0") { std::cout << "你调用了默认构造" << std::endl; }

    A(std::string a) : val(a) { std::cout << "你调用了含参构造" << std::endl; }

    A(A &obj) {
        std::cout << "你调用了拷贝构造" << std::endl;
        this->val = obj.val;
    }

    A(A &&obj) noexcept {
```

```
14          std::cout << "你调用了移动构造" << std::endl;
15          this->val = std::move(obj.val);
16      }
17
18      A &operator=(A &obj) {
19          if (this == &obj)
20              return *this;
21          std::cout << "你调用了等号赋值" << std::endl;
22          this->val = obj.val;
23
24          return *this;
25      }
26
27      ~A() { std::cout << "对象被析构，val为： " << val << std::endl; }
28
29      std::string val;
30  };
31
32  int main(int argc, char *argv[]) {
33      std::cout << "a1,";
34      A a1("a1");
35      std::cout << std::endl;
36
37      std::cout << "a2,";
38      A a2 = a1;
39      std::cout << "val值为： " << a2.val << std::endl;
40      std::cout << std::endl;
41
42      a2 = a1;//区别上面的
43      std::cout << std::endl;
44
45      std::cout << "a3,";
46      A a3(a2);
47      std::cout << "val值为： " << a3.val << std::endl;
48      std::cout << std::endl;
49
50      std::cout << "a4,";
51      A &a4 = a1;
52      std::cout << "val值为： " << a4.val << std::endl;
53      std::cout << std::endl;
54
55      std::cout << "a5,";
56      A &&a5 = std::move(a1);
57      std::cout << "val值为： " << a5.val << std::endl;
58      std::cout << std::endl;
```

```
59
60      a2.val = "new␣a2";
61      std::cout << "a1的val值为: " << a1.val << std::endl;
62      a4.val = "new␣a4";
63      std::cout << "a1的val值为: " << a1.val << std::endl;
64      std::cout << "a5的val值为: " << a5.val << std::endl;
65      std::cout << std::endl;
66
67      std::cout << "a6,";
68      A a6(std::move(a1));
69      std::cout << "val值为: " << a6.val << std::endl;
70      std::cout << "a1被传入移动构造后,";
71      std::cout << "val值为: " << a1.val << std::endl;
72      std::cout << std::endl;
73
74      return 0;
75  }
```

请读者仔细体会一下这段代码的输出。

移动语义：首先了解一个 std 中的函数 move，它的作用是将参数类型强制转换为右值，不管参数原来是左值还是右值。

示例 2.17 std::move 的效果：

```
1  #include <iostream>
2  using namespace std;
3  void Fun(int& x) { cout << "左值引用" << endl; }
4  void Fun(const int& x) { cout << "const␣左值引用" << endl; }
5  void Fun(int&& x) { cout << "右值引用" << endl; }
6  void Fun(const int&& x) { cout << "const␣右值引用" << endl; }
7
8  int main() {
9      Fun(10);           // 右值
10     int a = 1;
11     Fun(a);            // 左值
12     Fun(std::move(a)); // 右值
13     const int b = 8;
14     Fun(b);      // const 左值
15     Fun(std::move(b)); // const 右值
16     return 0;
17 }
```

如代码所见，当我们使用 std::move() 时，编译器将左值转换为右值。

再介绍一种利用模版实现完美转发的方式。在模板中，T&& 可以接收左值和右值，要利用到 forward<>() 函数，它会保留引用的类型，并将其转发给另一个函数，否则，传参的引用都会被转换为左值引用。

示例 2.18 完美转发：

```cpp
#include <iostream>
using namespace std;
void Fun(int& x) { cout << "左值引用" << endl; }
void Fun(const int& x) { cout << "const 左值引用" << endl; }
void Fun(int&& x) { cout << "右值引用" << endl; }
void Fun(const int&& x) { cout << "const 右值引用" << endl; }

template<typename T>
void PerfectForward(T&& t)
{
    Fun(t);
    //Fun(std::forward<T>(t));//使用完美转发
}

int main() {
    PerfectForward(10);           // 右值
    int a = 1;
    PerfectForward(a);            // 左值
    PerfectForward(std::move(a)); // 右值
    const int b = 8;
    PerfectForward(b);            // const 左值
    PerfectForward(std::move(b)); // const 右值
    return 0;
}
```

笔记 先把上面注释里不带 forward 的版本跑一遍，再取消注释跑一遍，对比两次输出的差异——这就是”引用塌缩成左值”和”完美转发”的区别。

结论 左值、右值与两种引用的对照：

- 左值：有名字、可取地址的变量；右值：临时变量与字面常量，不可取地址，语句结束即销毁；
- 左值引用 T&：只能绑定左值，是变量的别名，常用于传参避免拷贝；
- 右值引用 T&&：只能绑定右值，接过临时对象内存的操控权，避免深拷贝；有名字的右值引用本身是左值；
- std::move：把左值强制转成右值（本身不移动任何数据），std::forward：在模板中保留参数的左右值属性。

左值引用和右值引用的概念比较容易混淆，要在实践中不断运用，才能熟练掌握。

练习 2.7 编写一个函数 void swapVal(int& a, int& b) 交换两个整数的值，在 main 中调用并验证；再把参数类型改为按值传递（去掉 &），观察交换是否仍然生效，解释原因。

2.3 智能指针

动态内存若使用 *new/delete* 手动管理，易因遗漏释放导致内存泄漏；在对象生命周期复杂、函数嵌套较深时尤甚。C++11 引入智能指针，在对象析构时自动释放所管理的堆内存。常用类型包括

unique_ptr、*shared_ptr* 与 *weak_ptr*。

下文示例中的类型 A、B 定义如下。

```
class A;
class B;
class A{
public:
    A(){std::cout << "你调用了A的默认构造函数" << std::endl;}
    ~A(){std::cout << "A对象已被析构，val值为： " << val << std::endl;}
    int getVal(){return val;};
    void setVal(int val){this->val = val;}
    std::shared_ptr<B> B_ptr;
private:
    int val;
};

class B{
public:
    B(){std::cout << "你调用了B的默认构造函数" << std::endl;}
    ~B(){std::cout << "B对象已被析构，val值为： " << val << std::endl;}
    int getVal(){return val;};
    void setVal(int val){this->val = val;}
    std::weak_ptr<A> A_ptr; //注意：这里是weak_ptr，不是shared_ptr
private:
    int val;
};
```

2.3.1 unique_ptr

unique_ptr 为独占式智能指针：同一时刻仅允许一个指针拥有资源。离开作用域时自动释放资源。拷贝构造与拷贝赋值被禁用，所有权只能通过移动转移。

示例 2.19 unique_ptr 的所有权转移：

```
#include <iostream>
#include <memory>

int main() {
    {
        std::unique_ptr<A> ptr1 = std::make_unique<A>();
        ptr1->setVal(1);
        //std::unique_ptr<A> ptr2 = ptr1;会报错，独占指针不允许拷贝
        std::unique_ptr<A> ptr2 = std::move(ptr1);
        if(ptr1 == nullptr){
            std::cout << "ptr1是一个空指针" << std::endl;
        }
```

```
        std::cout << "ptr2的val值为: " << ptr2->getVal() << std::endl;
        ptr1 = std::make_unique<A>();
        ptr1->setVal(2);
    }//离开作用域，ptr1和ptr2管理的对象被自动析构
    return 0;
}
```

笔记 创建智能指针优先使用 std::make_unique / std::make_shared，而不是先 new 再交给智能指针接管。前者一步完成，更安全也更高效；手动 new 出来的原始指针一旦被两个智能指针同时接管，会被析构两次，直接崩溃。

练习 2.8 编写函数 std::unique_ptr<A> create(int v)，在堆上创建一个 A 对象、设置 val 为 v 并返回；在 main 中接收返回值并打印 val。思考：为什么这里返回 unique_ptr 不需要担心拷贝被禁用？

2.3.2 shared_ptr

当需要多个指针共同管理同一资源时，应使用 *shared_ptr*。其内部维护引用计数：拷贝使计数加一，析构使计数减一；计数归零时释放资源。移动操作转移所有权，不增加引用计数。

示例 2.20 观察引用计数的变化：

```
#include <iostream>
#include <memory>

int main() {
    {
        std::shared_ptr<A> ptr1 = nullptr;
        {
            auto ptr2 = std::make_shared<A>();
            ptr2->setVal(1);
            ptr1 = ptr2;
            //ptr1 = std::move(ptr2);//看一下这两种写法有什么区别?
            std::cout << "ptr1对象的引用次数为: " << ptr1.use_count() << std::endl;
            std::cout << "ptr2对象的引用次数为: " << ptr2.use_count() << std::endl;
        }
        std::cout << "ptr2已经被析构了" << std::endl;
        std::cout << "但A对象还活着，引用次数为: " << ptr1.use_count() << std::endl;
    }//到这里A对象才被析构
    return 0;
}
```

2.3.3 循环引用与 weak_ptr

若两个 *shared_ptr* 互相持有对方，将形成循环引用，引用计数无法归零，对象无法析构，从而造成泄漏。

示例 2.21 循环引用演示（先把 B 类中的 A_ptr 临时改回 shared_ptr<A> 再运行）：

```cpp
int main(int argc, char *argv[]) {
    {
        std::shared_ptr<A> pA(new A);
        std::shared_ptr<B> pB(new B);

        std::cout << pA.use_count() << std::endl;
        std::cout << pB.use_count() << std::endl;
    }
    std::cout << "上面是符合我们预期的" << std::endl << std::endl;
    {
        std::shared_ptr<A> pA(new A);
        std::shared_ptr<B> pB(new B);

        pA->B_ptr = pB;
        pB->A_ptr = pA;

        std::cout << pA.use_count() << std::endl;
        std::cout << pB.use_count() << std::endl;
    }
    std::cout << "咦？这是什么情况呢？为什么没有被析构呢？" << std::endl;
    return 0;
}
```

此时应将一侧改为 *weak_ptr*（正如本节开头 B 类中的写法）。*weak_ptr* 不增加引用计数，也不直接解引用资源；需要访问时通过 *lock*() 升级为 *shared_ptr*，并检查返回值是否为空。

示例 2.22 用 weak_ptr 打破循环引用：

```cpp
int main() {
    {
        std::shared_ptr<A> pA(new A);
        std::shared_ptr<B> pB(new B);

        pA->B_ptr = pB;
        pB->A_ptr = pA; //weak_ptr不增加引用计数

        std::cout << pA.use_count() << std::endl;
        std::cout << pB.use_count() << std::endl;

        std::shared_ptr<A> PtrA = pB->A_ptr.lock(); //升级为shared_ptr再使用
        if(PtrA != nullptr)
            PtrA->setVal(1234);
        PtrA.reset();
    }//这次两个对象都能正常析构了
    return 0;
}
```

在多线程环境下，*lock*() 可用于在访问前确认共享对象仍存活。相关同步问题见多线程章节。

练习 2.9 运行” 循环引用演示” 示例，观察两段的析构输出差异；然后保持 `weak_ptr` 写法再运行一遍，确认两个对象都能正常析构，并用自己的话解释原因。

2.3.4 三种智能指针对照

结论

智能指针	指向数量	析构条件	是否可操作资源
unique_ptr	只能一个	离开作用域	是
shared_ptr	可多个指向同一对象	引用计数为 0	是
weak_ptr	可观察 shared_ptr 管理的对象	不单独决定析构	否，需 lock 升级

选择原则：默认用 `unique_ptr`；确实需要共享所有权时才用 `shared_ptr`；只想” 看一眼” 而不想延长对象寿命时用 `weak_ptr`。这就好比宿舍钥匙：独居一把钥匙（unique），合租人手一把、最后走的人锁门（shared），访客只登记不配钥匙（weak）。

最后补充说明循环引用的两种形态，它们的处理难度完全不同：

- **静态环**：像本节 A、B 两个类这样，互相持有的关系在代码里写得明明白白。这种环一眼就能看出来，设计时把一侧改成 `weak_ptr` 即可解决；
- **动态环**：环是在运行过程中” 长” 出来的——比如用 `shared_ptr` 组织的链表、图结构，随着数据不断增加，节点之间可能意外连成一个环。这种环在代码里看不到，等到内存泄漏时往往已经很难排查。

笔记 对于动态环，标准库没有通用的解决方案：`weak_ptr` 只能破解你预先知道存在的那条边，而运行时才形成的环你根本不知道该在哪一侧用 `weak_ptr`。工程上的应对办法是在设计阶段就规定好所有权方向（比如父子结构中父持有子、子用 `weak_ptr` 回指父），并通过代码评审和内存检测工具（如 AddressSanitizer 的泄漏检查）来兜底。

2.4 标准模版库（STL）

本节介绍常用数据结构对应的 STL 容器，以及迭代器的基本用法与失效问题。

2.4.1 数据结构概览

所谓数据结构，就是指数据的组织、存储、操作和处理方式。数据结构是计算机科学中最基本的概念之一，是计算机程序设计的基础。基本的操作有四：**插入、删除、改动、查找**（增删改查）。

下面我们将简单介绍几种常见的数据结构，以及它们的 STL 容器：

1. **数组（Array）**：相同类型的元素按顺序存储在一块连续内存中的线性结构，可以通过下标在 $O(1)$ 时间内直接访问任意元素。
2. **链表（Linked List）**：由一系列在内存中不连续的节点串成的线性结构，每个节点存储数据和指向下一个节点的指针；已知位置时插入、删除只需改变指针，但查找必须从头节点开始逐个遍历。

3. **队列（Queue）**：先进先出（FIFO，First In First Out）的线性结构，只允许在队尾插入元素、在队头删除元素，先进入的元素先被取出。
4. **优先队列（Priority Queue）**：每个元素都带有一个优先级的队列，每次出队的不是最早进入的元素，而是当前优先级最高的元素。
5. **堆栈（Stack）**：后进先出（LIFO，Last In First Out）的线性结构，插入（压栈）和删除（弹栈）都只能在栈顶一端进行。
6. **树（Tree）**：一种非线性结构，其中有且仅有一个根节点，其余每个节点都恰好有一个父节点；任意两个节点之间有且仅有一条路径相连，即树中不存在环路。
7. **二叉树（Binary Tree）**：每个节点至多有两个子节点的树，两个子节点区分左右，分别称为左子节点和右子节点。
8. **堆（Heap）**：满足”堆性质”的完全二叉树——最大堆中每个节点的值都大于或等于其子节点，最小堆则相反；因此堆顶永远是最大值（或最小值）。

它们的头文件如下：

```
#include <array>     //静态数组array
#include <vector>    //动态数组vector
#include <list>      //链表list
#include <queue>     //队列queue 优先队列priority_queue
#include <stack>     //堆栈stack
#include <map>       //映射map 是一种快速查找关联容器
```

需要注意的是，上面所列举的数据结构类型并不是并列的，三大数据结构分别是：线性表（包括 1、2、3、4、5），树（包括 6、7、8），图（太复杂，暂时不介绍）。

2.4.2 顺序容器

数组与 vector。

数组是最基本的数据结构，它是一组相同类型的数据，按照顺序存储在一块连续的内存中，如图 2.2。



图 2.2: 数组

数组（Array），又称**顺序表**，它的 STL 容器有两种，分别是 *array* 和 *vector*。

array 是固定大小的数组。*vector* 是一种动态数组，可以动态增长，可以根据需要自动分配内存。两者的声明方式如下：

```
std::array<int,5> arr;
std::vector<int> vec;
```

array 并不常用，因为它是固定大小的，如果需要改变大小，只能重新声明一个新的数组。



笔记 想详细了解 array 可以参考这篇 blog：

https://blog.csdn.net/qq_38410730/article/details/102802239

vector 由于其动态性，可以根据需要自动分配内存，因此使用起来比较方便，在工程实践中也更常用。*vector* 的实现原理是当检测到 *vector* 的容量不足时，会自动分配新的内存（原来的大小 $\times 2$），并将原有数据复制到新内存中。这个复制的过程非常缓慢，因此，在使用大型的资源时，*vector* 的效率可能会比较低。

示例 2.23 vector 的常用成员函数：

```
#include <vector>

std::vector<int> vec{1,2,3,4,5,6};
vec[0];          //访问（快速）
vec.at(0);       //访问（安全，越界会抛异常）
vec.front();     //第一个元素
vec.back();      //最后一个元素
vec.empty();     //检查是否为空
vec.size();      //返回容器大小
vec.reserve(10);//预留空间，避免反复扩容
vec.clear();     //清除所有元素
vec.insert(vec.begin(),1);//插入元素
vec.erase(vec.begin());//删除元素
vec.push_back(1); //在最后添加
vec.emplace_back(); //在最后原位构造，省去一次拷贝
vec.pop_back();      //删除最后一个
```

链表。

链表是一种非连续的内存块，它是由一系列节点组成，每个节点都包含数据和指针。链表的实现原理是每个节点都包含一个指针，指向下一个节点。链表的头节点称为头指针，尾节点称为尾指针。链表的插入、删除操作都比较容易，只需要改变指针即可，但是查找操作比较麻烦，需要从头节点开始遍历。



笔记 理论上链表的插入删除是 $O(1)$、数组的中间插入是 $O(n)$，但实际工程中链表经常跑不过 vector：链表的节点分散在内存各处，CPU 缓存（Cache）帮不上忙，每次访问都可能发生缓存未命中；而 vector 的内存是连续的，缓存预取非常友好。所以除非有频繁的中部插入删除，否则默认选 vector。

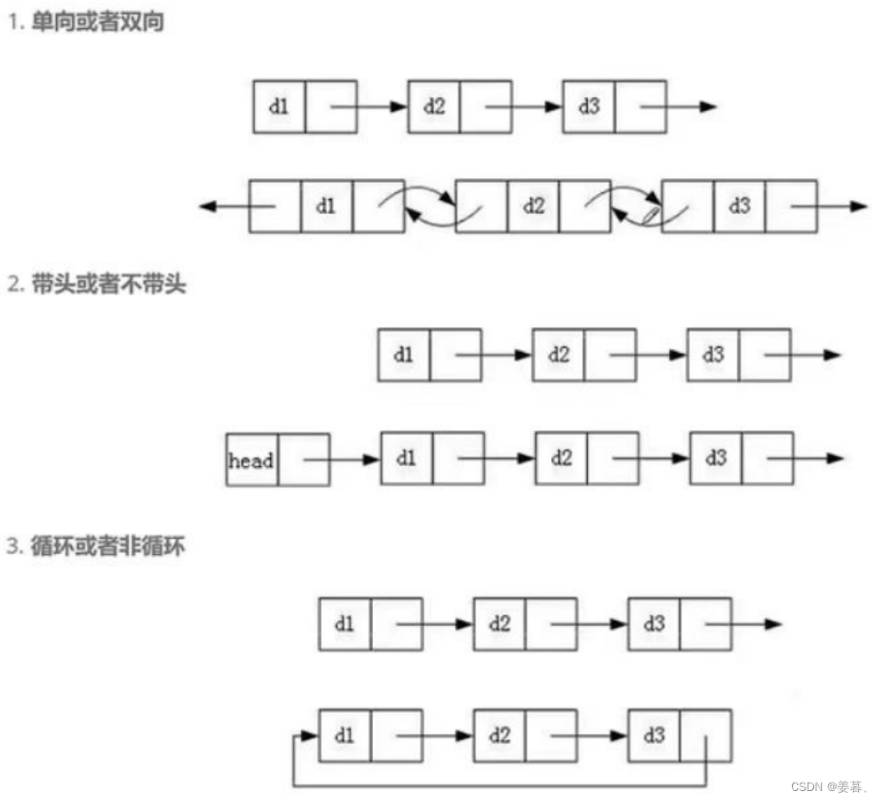


图 2.3: 链表

它的 STL 容器是 *list*（双向链表）和 *forward_list*（单向链表）。比较简单，请自行查阅相关文档：https://zh.cppreference.com/w/cpp/container/list

✍ 练习 2.10 向 `std::vector<int>` 中依次 `push_back` 数字 1 ～ 10，在每次插入后打印 `size()` 和 `capacity()`，观察容量翻倍的时机，验证” 容量不足时扩容为原来 2 倍” 的说法。

2.4.3 队列、优先队列与堆栈

队列。

队列是一种线性数据结构，它按照**先进先出**的原则进行操作。队列有两个基本操作，分别是**入队**和**出队**。入队操作是在队列的末尾添加一个元素，而出队操作是从队列的头部移除一个元素。队列有两个端点，队头和队尾，队头是队列的第一个元素，出队操作发生的地方，队尾是队列的最后一个元素，入队操作发生的地方。通常情况下，**不允许随机访问**队列中的元素，只能访问队头元素。队列在**多任务处理、打印任务管理、缓冲处理**等场景中有广泛应用，它帮助维持操作的顺序和流程控制。（很常用）

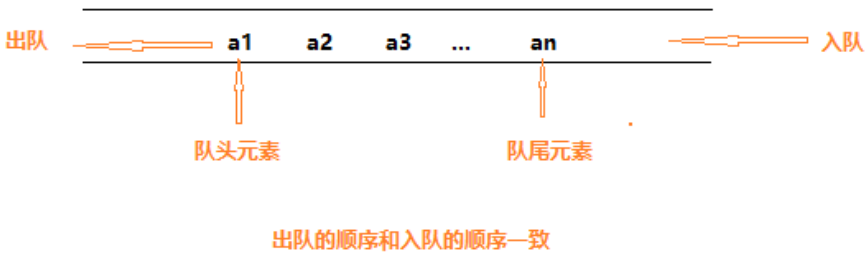


图 2.4: 入队和出队操作

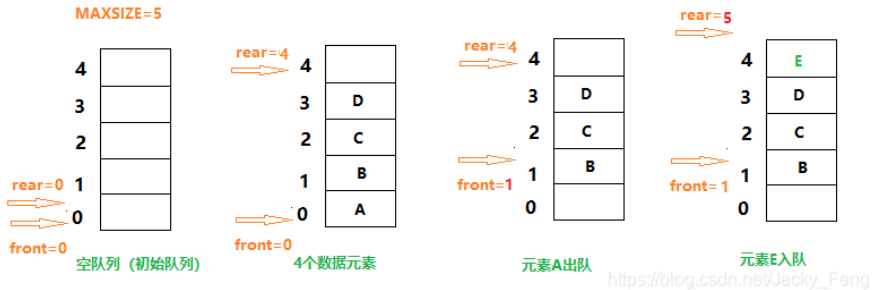


图 2.5: 队列实现方式

它的 STL 容器是 *deque*，基于双向队列，C++ 还实现了 3 种容器适配器：*stack*（堆栈）、*queue*（单向队列）、*priority_queue*（优先队列）。相较于 *deque*，我们更常用 *queue* 进行队列操作。

笔记 具体用法请自行查阅相关文档：

https://zh.cppreference.com/w/cpp/container/deque

https://zh.cppreference.com/w/cpp/container/queue

优先队列。

优先队列是一种特殊的队列，它允许插队的出现，可以根据“重要性”对队中元素进行排序。优先队列的实现原理是每个元素都有一个优先级，优先级越高，元素越容易被取出。优先队列的应用场景有很多，如任务调度、排序、搜索等。

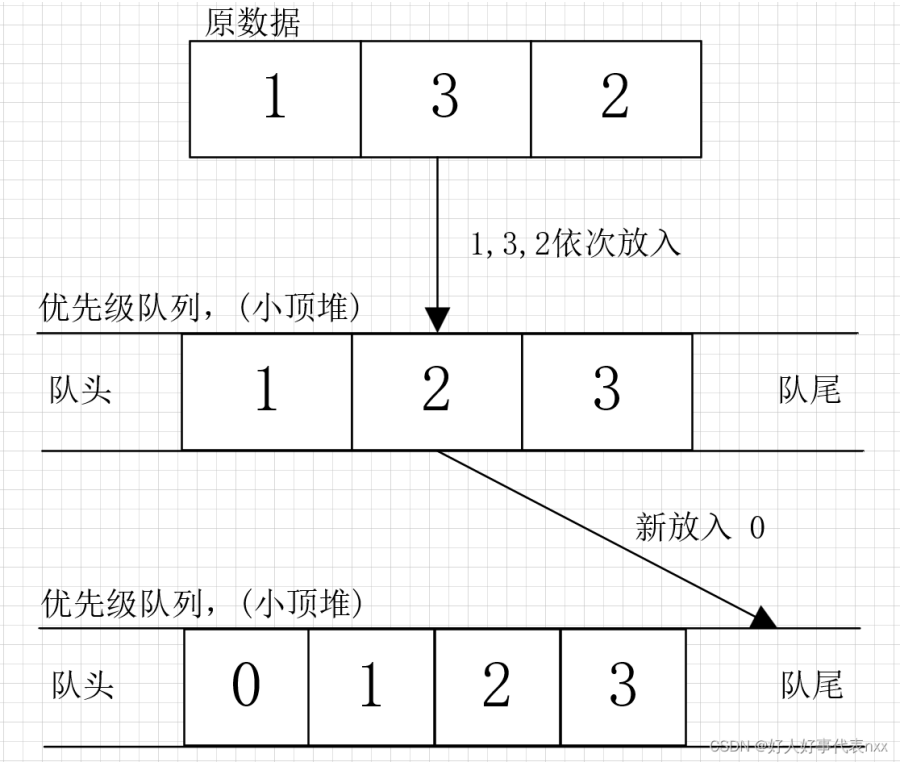


图 2.6: 优先队列

它的实现是容器适配器 *priority_queue*，具体用法请自行查阅相关文档：

https://zh.cppreference.com/w/cpp/container/priority_queue

堆栈。

堆栈是一种线性数据结构，它按照**先进后出**的原则进行操作。堆栈有两个基本操作，分别是**压栈**和**弹栈**。压栈操作是在堆栈的顶部添加一个元素，而弹栈操作是从堆栈的顶部移除一个元素。堆栈有两个端点，栈顶和栈底，栈顶是堆栈的顶部，压栈操作发生的地方，栈底是堆栈的底部，弹栈操作发生的地方。堆栈在**函数调用、表达式求值、回溯、计算器、括号匹配、迷宫寻路**等场景中有广泛应用。

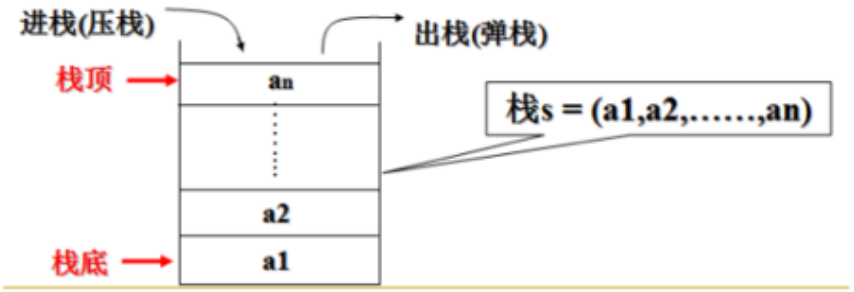


图 2.7: 压栈和弹栈操作

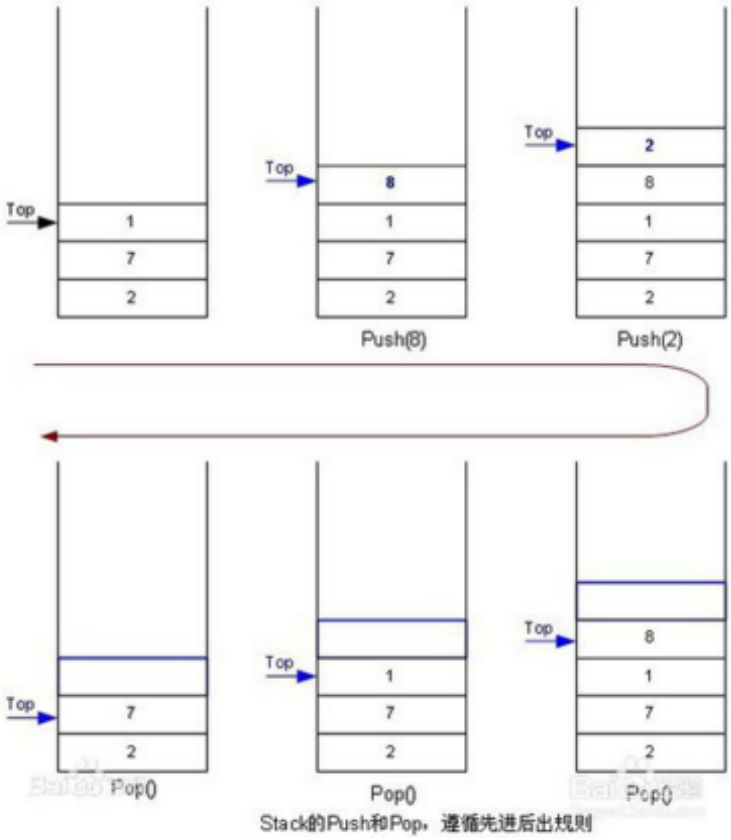


图 2.8: 堆栈实现方式

它的实现是容器适配器 *stack*，具体用法请自行查阅相关文档：

https://zh.cppreference.com/w/cpp/container/stack

✍ 练习 2.11 使用 `std::stack<char>` 编写括号匹配检查：读入一串只含（、）的字符串，判断括号是否配对（提示：遇左括号压栈，遇右括号弹栈，弹空或结束时栈非空则不匹配）。

2.4.4 树与堆

树。

树是一种非线性数据结构，由若干**节点（Node）**和连接节点的**边（Edge）**组成。它有且仅有一个根节点（Root），其余每个节点都恰好有一个父节点，从根节点出发可以到达任意节点，且任意两个节点之间有且仅有一条路径——也就是说，树里不存在环路。

树还有一个更本质的**递归定义**：一棵树要么是空树；要么由一个根节点和若干棵互不相交的子树组成，而每棵子树本身也是一棵树。”树的每个部分还是树”——正是这种自相似的结构，让递归成为处理树最自然的方式（遍历、求高度、搜索都一行递归搞定）。

树的种类很多，如二叉树、二叉搜索树、AVL 树、红黑树、B 树、B+ 树、B* 树等。

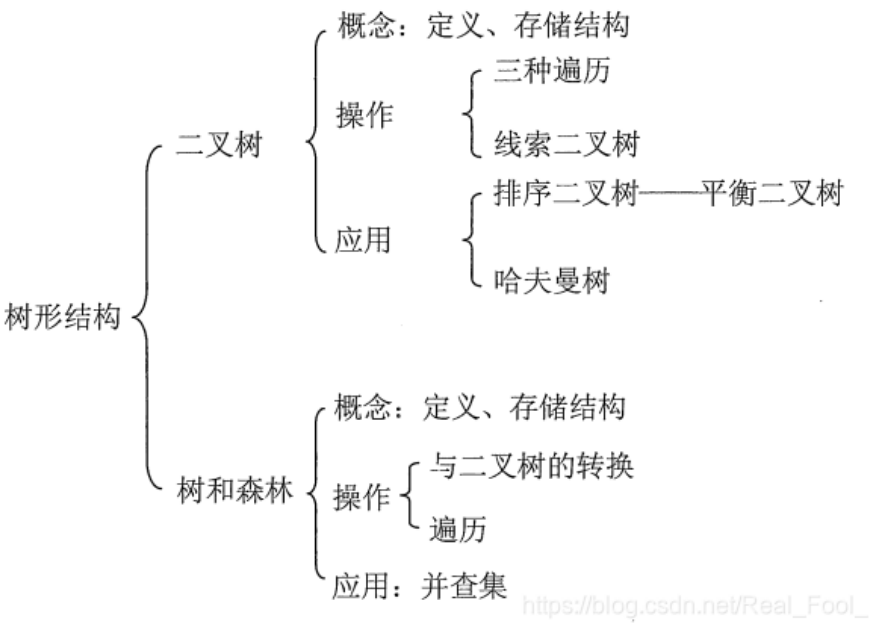


图 2.9: 树的逻辑图（了解即可）

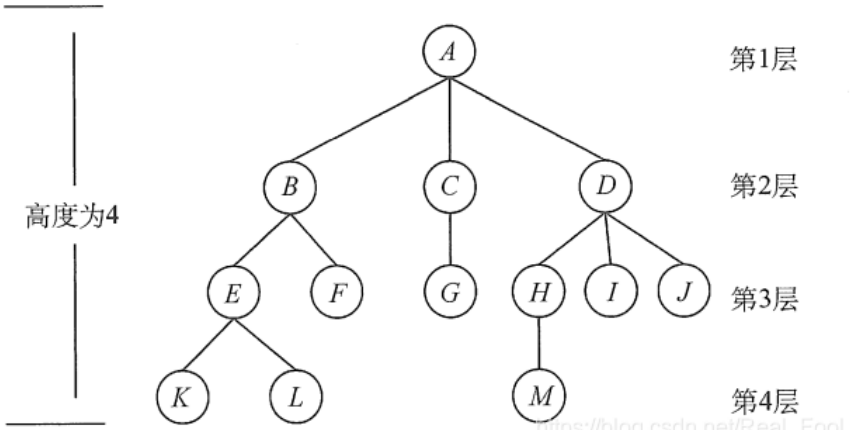


图 2.10: 树的示意图

堆。

堆是一种特殊的树，它是一个**完全二叉树**，即除叶节点外，其他的每个节点都有两个子节点。每个节点都有一个值，并且每个节点**都大于或等于其子节点**（这是最大堆 max-heap，类似的，还有最小堆 min-heap）。堆的应用场景有很多，如排序、优先队列、图算法、堆排序等。

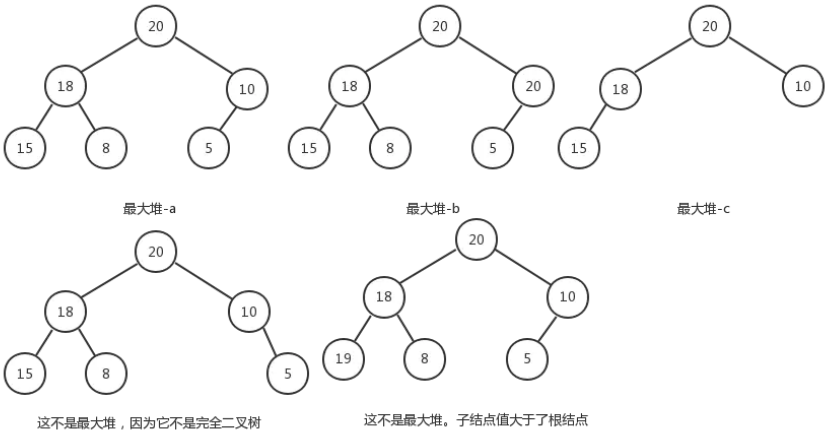


图 2.11: 最大堆的示意图

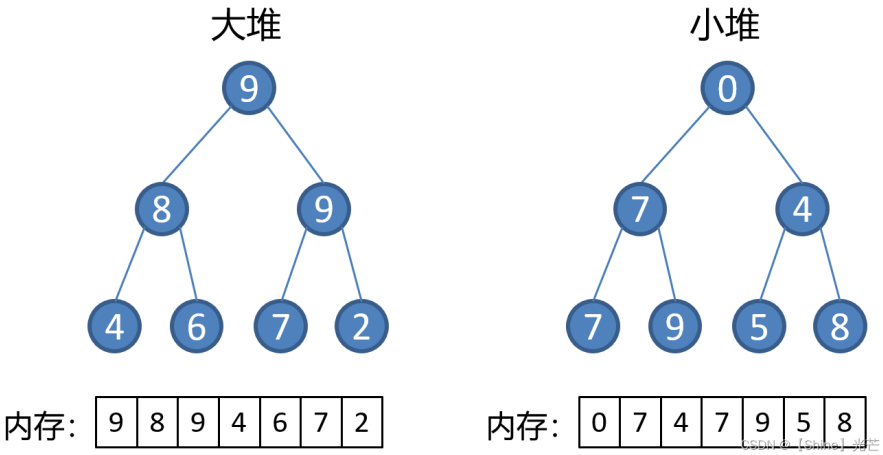


图 2.12: 最大堆和最小堆

上一小节介绍的优先队列正是基于堆实现的：它通过为每个任务的重要程度编号，把编号大的任务放在堆顶，从而快速取出最重要的任务；堆的调整算法很快，因此优先队列被广泛应用于**任务管理**中。优先队列的容器用法已在上一小节给出，这里不再重复。

2.4.5 映射

映射是一种关联容器，它是一个**键值对**的集合，其中每个键都是唯一的，值可以重复。映射有两种主流实现：基于哈希表的实现查找、插入、删除的平均时间复杂度是 $O(1)$；基于红黑树的实现则是 $O(logN)$，但能保持键有序。

红黑树（Red-Black Tree）是一种随机查找速度比较快的**平衡二叉树**[1]，它的查找、插入、删除操作的时间复杂度都是 $O(logN)$[2]。

哈希表（HashTable）是一种**散列表**，它通过**哈希函数**将键映射到值。由于其使用计算来查找，因

[1]平衡二叉树，也叫 AVL 树，它或者是一棵空树，或者具有以下性质的二叉排序树：它的左子树和右子树的高度之差（平衡因子）的绝对值不超过 1，且它的左子树和右子树都是一棵平衡二叉树

[2]O(logN) 表示时间复杂度，N 是问题的规模，O 表示运行消耗时间的上界

此，查找操作的平均时间复杂度是 $O(1)$ [3]。但是哈希表的建立比较困难，需要消耗很多资源。而且存储长度有限，当元素过多时，哈希表的性能会下降。当哈希表满时，需要执行**再散列（Rehashing）**，解决**哈希冲突（Hash Collision）**问题，这个过程需要消耗大量时间。

两者各有优劣，在实际应用中，需要根据具体问题选择。红黑树型映射的 STL 容器是 *map*，哈希表型映射的 STL 容器是 *unordered_map*。

示例 2.24 map 与 unordered_map 的基本用法：

```cpp
#include <iostream>
#include <map>
#include <unordered_map>
#include <string>

int main() {
    std::map<std::string,int> score;              //红黑树实现，按键有序
    score["alice"] = 90;                          //插入或修改
    score.emplace("bob", 85);
    std::cout << score["alice"] << std::endl;     //按键访问
    std::cout << score.count("carol") << std::endl; //判断键是否存在：0
    for(auto& [name, s] : score){                 //遍历，按字母序输出
        std::cout << name << ":␣" << s << std::endl;
    }

    std::unordered_map<std::string,int> fastScore; //哈希表实现，无序但更快
    fastScore["alice"] = 90;
    return 0;
}
```

笔记 具体用法请自行查阅相关文档：

https://zh.cppreference.com/w/cpp/container/map

注意：map 的 operator[] 在键不存在时会自动插入一个默认值，只想查询不想改动时，请用 find() 或 count()。

练习 2.12 使用 std::map<std::string,int> 统计一段文字中每个单词出现的次数，并按键的字母序打印结果。

2.4.6 迭代器与失效

迭代器是一种检查容器内元素并遍历元素的数据类型，通常用于对 C++ 中各种容器内元素的访问，但不同的容器有不同的迭代器，初学者可以将迭代器理解为一个可以**自动向后移动的指针**。

常用的可迭代的容器有：

1. 序列容器：*array*、*vector*、*deque*、*list*、*forward_list* 等
2. 关联容器：*set*、*map*、*multimap* 等

[3] 常数的时间复杂度意味着不管问题的规模如何，对于同一个长度的 HashTable，算法的运行时间都不会随着问题的规模增加而增长。

queue、*stack*、*priority_queue* 是容器适配器，必须满足一定规则，没有迭代器。

对于可迭代容器，一般可以使用.begin() 和.end() 方法获取迭代器，.begin() 方法返回指向第一个元素的迭代器，.end() 方法返回指向最**后一个元素的迭代器的下一位置**。

示例 2.25 用迭代器遍历容器：

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> vec{1,2,3,4,5,6,7,8,9,0};
    for(auto it = vec.begin();it != vec.end();++it){
        std::cout << *it << std::endl;
    }
    return 0;
}
```

也可以简写为范围 for 循环：

```cpp
for (auto it : vec) {
    std::cout << it << std::endl;
}
```

示例 2.26 迭代器的常见修改操作：

```cpp
#include <algorithm> //sort、remove_if、iter_swap 都在这个头文件里

std::vector<int> vec{1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
vec.erase(vec.begin()+5);             //删除第6个元素
vec.erase(vec.begin()+5, vec.end()); //删除第6个元素之后的所有

std::vector<int> vec1{1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
std::iter_swap(vec1.begin(), vec1.begin()+1); //交换第一个和第二个元素

std::vector<int> vec2{1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
std::sort(vec2.begin(), vec2.end(), [](int a, int b){
    return a >= b;                    //降序排序
});

//remove_if 只把不满足条件的元素挪到前面，并不真正删除
//正确姿势是 erase-remove 惯用法：
vec2.erase(std::remove_if(vec2.begin(), vec2.end(), [](int a){
    return a > 5;
}), vec2.end());                      //这才真正删除了大于5的元素
```



笔记 std::remove_if 并不会改变容器大小，它只是把”要保留”的元素搬到前面、返回新的逻辑结尾迭代器，必须配合 erase 才能完成删除——这就是著名的 erase-remove 惯用法。忘记接 erase，容器里会留下一堆”僵尸元素”。更多迭代器内容见 cppreference：

https://zh.cppreference.com/w/cpp/iterator

迭代器失效。

迭代器失效指的是迭代器因为某些操作而不再指向原来的元素，或者不再保持有效的状态，这样的迭代器如果继续使用，可能会导致未定义行为，包括程序崩溃。通俗的理解就是，当你对容器进行了操作时，更改了容器的大小，但是迭代器仍然指向原来的位置，这样就会导致读取错误的内存，这就是迭代器失效。

```
std::vector<int> vec={1,2,3,4,5,6,7,8,9,10};
for(auto it = vec.begin(); it != vec.end(); ++it){
    if(*it % 2 == 0){
        vec.erase(it); //错误示范：erase之后it已经失效
    }
}
```

看上面的代码，当执行到 vec.erase(it) 时，it 已经失效，因为执行了 erase 之后，vec 的大小发生了变化，这导致了迭代器失效。

在实践过程中，下面的几种情况下迭代器会失效，需要**避免**出现：

1. 当执行 erase 方法时，指向删除节点的迭代器失效，指向删除节点之后的迭代器也全部失效
2. 当进行 push_back() 方法时，end 操作返回的迭代器肯定失效
3. 当插入（push_back）一个元素后，capacity 返回值与插入之前相比有改变，则说明容器整体搬了家，此时所有迭代器都会失效
4. 当插入（push_back）一个元素后，如果空间未重新分配，指向插入位置之前的元素的迭代器仍然有效，但指向插入位置之后元素的迭代器全部失效

对于关联容器，删除一个键值对只会使当前这个迭代器失效，别的迭代器不会受到影响，这可能是由于底层使用红黑树实现的原因。但是实际应用中，仍然不应该冒这个风险，因此，必须**坚决避免迭代器失效问题的发生**。

在具体的应用中，我们在使用迭代器遍历时，应该注意不要对容器进行增删操作，更不应该改变容器的大小。正确的删除写法是利用 erase 的返回值——它会返回指向被删元素**下一个**元素的有效迭代器：

示例 2.27 遍历中安全删除元素：

```
std::vector<int> vec={1,2,3,4,5,6,7,8,9,10};
for(auto it = vec.begin(); it != vec.end(); ){ //注意：这里没有++it
    if(*it % 2 == 0){
        it = vec.erase(it); //erase返回下一个有效迭代器
    }else{
        ++it;
    }
}
```

结论迭代器失效自救指南：

- 遍历容器时不要随手增删元素，先想清楚迭代器下一步指向哪；
- 删除元素用 it = vec.erase(it); 接住返回值，而不是 vec.erase(it); 后继续用旧迭代器；
- push_back 导致扩容后，之前保存的所有迭代器、指针、引用全部作废，需要遍历就重新获取；

- 批量按条件删除，优先考虑 erase-remove 惯用法，一行解决，不容易写错。

练习 2.13 把”错误示范”的代码抄下来运行，观察运行结果或崩溃现象；再用”遍历中安全删除元素”的写法修复它，删除所有偶数并打印剩余的奇数。

2.5 C++ 多线程

本节展开多线程同步与常见模型，并将系统内存布局作为相关基础一并说明。

2.5.1 基本概念

多线程（multithreading）是指一个进程中同时运行多个线程（thread）。线程共享进程的地址空间，适合把互不依赖的工作拆开执行，例如同时接收相机数据、处理上一帧图像和保存日志。

多线程不等于程序一定更快。线程创建、调度和同步都有成本；如果多个线程频繁争抢同一份数据，程序反而可能变慢。因此，设计多线程程序时要先回答两个问题：哪些工作可以并行，哪些共享状态必须同步。

线程之间常见的协作方式有两类：

- **阻塞等待**：线程暂时停下来等待事件，例如等待队列中出现任务，不会持续占用 CPU。
- **非阻塞尝试**：线程先尝试获取资源，失败后立即返回或执行其他工作。原子操作和 `try_lock` 常用于这种场景。

如果多个线程同时读写同一对象，且至少有一个线程写入，却没有合适的同步，就会产生数据竞争（data race）。在 C++ 中，数据竞争会导致未定义行为，不能简单理解为“结果偶尔不准”。

结论 并发程序的基本思路可以概括为：独立的工作并行执行，共享的数据明确保护，线程的启动、等待和退出形成完整闭环。后文将依次介绍互斥锁、共享锁、原子变量、条件变量和异步结果传递。

2.5.2 创建与生命周期

C++11 的 `<thread>` 头文件提供了 `std::thread`。它解决的问题是：把一个函数或可调用对象放到新的执行流中运行。

```
std::thread 线程对象(可调用对象, 参数...);
```

参数默认按值复制到新线程。需要在线程中修改原对象时，应显式使用 `std::ref` 传递引用。

示例 2.28 创建线程、传递参数并等待线程结束：

```
#include <iostream>
#include <thread>

void print_range(int begin, int end) {
    for (int i = begin; i <= end; ++i) std::cout << i << '␣';
    std::cout << '\\n';
}

void add_one(int& value) { ++value; }

```

```cpp
int main() {
    std::thread first(print_range, 1, 3);
    int value = 0;
    std::thread second(add_one, std::ref(value));
    first.join();
    second.join();
    std::cout << "value = " << value << '\\n';
    return 0;
}
```

join() 会阻塞当前线程，直到目标线程结束；detach() 会把线程与 std::thread 对象分离。分离后的线程仍可能访问原来的数据，因此初学阶段应优先使用 join()。

笔记 std::thread 对象析构时，如果仍然处于可连接状态，程序会调用 std::terminate()。每个线程都必须明确 join() 或 detach()；还要避免在线程函数中引用已经销毁的局部变量。

练习 2.14 创建两个线程，分别打印 1 ～ 5 和 6 ～ 10，在每次打印之间调用 std::this_thread::sleep_for 暂停一小段时间。观察输出是否交错，并说明为什么输出顺序不能依赖。

2.5.3 互斥锁与 RAII 锁

当多个线程修改同一份共享数据时，需要把“读—改—写”的完整过程保护起来。std::mutex（互斥量，mutex）保证同一时刻只有一个线程进入临界区。

下面的自增操作看起来只有一行，实际上包含读取、计算和写回三个步骤。两个线程可能读到同一个旧值，造成更新丢失。

示例 2.29 使用互斥锁保护共享计数器：

```cpp
#include <iostream>
#include <mutex>
#include <thread>

int counter = 0;
std::mutex counter_mutex;

void add_many_times() {
    for (int i = 0; i < 100000; ++i) {
        std::lock_guard<std::mutex> lock(counter_mutex);
        ++counter;
    }
}

int main() {
    std::thread first(add_many_times);
    std::thread second(add_many_times);
    first.join();
    second.join();
    std::cout << counter << '\\n';
```

```
    return 0;
}
```

std::lock_guard 是 RAII（资源获取即初始化）锁。构造时加锁，离开作用域时自动解锁，即使临界区中途抛出异常也能释放锁。需要手动解锁、延迟加锁或转移锁所有权时，可以使用 std::unique_lock。

```
std::unique_lock<std::mutex> lock(mtx);  // 默认立即加锁
lock.unlock();                           // 可以提前解锁
lock.lock();                             // 也可以再次加锁
```

笔记 不要把共享数据的读操作放在锁外、写操作放在锁内，除非你能证明这样做是安全的。锁保护的不是某一行代码，而是一个必须保持一致性的临界区。

练习 2.15 将示例中的两个线程改成四个线程，每个线程自增 50000 次。分别使用普通 int、互斥锁和原子变量实现，并比较三种写法的结果。

2.5.4 共享锁与相容性矩阵

在读多写少的场景中，多个线程同时读取通常不会互相破坏数据。如果所有读者都使用普通互斥锁，读者之间也会被迫排队。std::shared_mutex 提供两种锁模式：std::shared_lock 是共享读锁，允许多个读者同时持有；std::unique_lock 是独占写锁，写入时阻塞所有读者和其他写者。

两种锁的相容性可以用下面的矩阵记忆。行表示当前已经持有的锁，列表示申请的新锁。

当前持有的锁 / 新申请的锁	共享读锁	独占写锁
共享读锁	相容	冲突
独占写锁	冲突	冲突

多个读者可以并行读取；只要有一个写者进入，其他读者和写者都必须等待。共享锁只能保护“读操作确实不会修改数据”的代码。

笔记 std::shared_mutex 和 std::scoped_lock 需要 C++17 或更新的标准。编译示例时可使用 g++ -std=c++17；如果项目仍使用 C++11，应先用普通 std::mutex 实现等价逻辑。

示例 2.30 用共享锁保护读多写少的配置表：

```
#include <iostream>
#include <mutex>
#include <shared_mutex>
#include <string>
#include <thread>
#include <unordered_map>

std::unordered_map<std::string, int> config{{"exposure", 10}};
std::shared_mutex config_mutex;

int read_config(const std::string& key) {
    std::shared_lock<std::shared_mutex> lock(config_mutex);
    return config.at(key);
```

```
}

void write_config(const std::string& key, int value) {
    std::unique_lock<std::shared_mutex> lock(config_mutex);
    config[key] = value;
}

int main() {
    std::thread reader1([] { std::cout << read_config("exposure") << '\\n'; });
    std::thread reader2([] { std::cout << read_config("exposure") << '\\n'; });
    std::thread writer([] { write_config("exposure", 12); });
    reader1.join();
    reader2.join();
    writer.join();
    return 0;
}
```

结论 普通互斥锁、共享读锁和独占写锁的选择：

- 数据会被修改，或读写逻辑无法清楚区分：使用普通互斥锁。
- 读操作远多于写操作，且读操作不会修改数据：考虑 shared_mutex。
- 读锁之间相容，写锁与任何锁都冲突；写入必须使用独占锁。

练习 2.16 把配置表示例改成三个读者和两个写者，并在读写函数中加入短暂延时。根据相容性矩阵，解释哪些线程可能并行、哪些线程必须等待。

2.5.5 原子变量

原子变量（atomic variable）适合表示一个简单的共享状态，例如计数器、标志位或索引。std::atomic<T> 保证对该对象的单次读写或指定的原子操作不会被其他线程看到“做到一半”的状态。

原子变量不是“万能同步工具”。两个原子变量之间的联合不变量仍可能被其他线程观察到中间状态；一组复杂数据结构也不能仅靠把某个整数改成原子类型来保护。

示例 2.31 用原子变量实现并发计数和停止信号：

```
#include <atomic>
#include <iostream>
#include <thread>

std::atomic<int> counter{0};
std::atomic<bool> stop{false};

void add_many_times() {
    for (int i = 0; i < 100000; ++i) ++counter;
}

void wait_for_stop() {
    while (!stop.load()) std::this_thread::yield();
```

```
    std::cout << "received stop signal\\n";
}

int main() {
    std::thread first(add_many_times);
    std::thread second(add_many_times);
    std::thread observer(wait_for_stop);
    first.join();
    second.join();
    stop.store(true);
    observer.join();
    std::cout << "counter = " << counter.load() << '\\n';
    return 0;
}
```

常用操作包括 load()、store()、fetch_add() 和 exchange()。直接使用自增运算符也可以执行原子自增。若需要多个共享对象保持整体一致，应回到互斥锁或更高层的并发结构。

笔记 不要用普通的 bool 作为多个线程共享的停止标志，也不要用“先读取再判断再写回”的普通变量操作代替原子操作。数据竞争属于未定义行为，不能依靠多运行几次没有出错来证明程序正确。

练习 2.17 使用 std::atomic<int> 实现一个线程安全的任务编号分配器，让四个线程各领取 100 个不重复编号，并验证最终编号数量。

结论 原子变量和条件变量解决的是两类不同的问题：原子变量适合完成一个简单状态的无竞争读写；条件变量适合让线程在某个受锁保护的条件不成立时休眠等待。原子变量不会自动把线程挂起，条件变量也不能代替锁来保护复杂数据。

2.5.6 条件变量

条件变量（condition variable）解决的问题是：线程需要等待某个共享状态成立，但又不应该一直轮询占用 CPU。例如，消费者要等待队列非空，工作线程要等待“停止”或“有新任务”。

条件变量必须和互斥锁、共享状态一起使用。等待流程是：先获得锁，调用带谓词的 wait；被唤醒后重新检查谓词，条件满足才继续执行。

```
std::condition_variable cv;
std::mutex mtx;
bool ready = false;
{
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [] { return ready; });
}
cv.notify_one();        // 唤醒一个等待线程
cv.notify_all();        // 唤醒所有等待线程
```

wait 在等待期间会暂时释放互斥锁，被唤醒后重新加锁再返回。必须使用谓词版本，因为可能出现虚假唤醒，而且其他线程被唤醒后可能已经先一步改变了共享状态。

示例 2.32 一个线程准备数据，另一个线程等待数据准备完成：

```cpp
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <thread>

std::mutex mtx;
std::condition_variable cv;
bool ready = false;

void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [] { return ready; });
    std::cout << "data is ready\\n";
}

void producer() {
    {
        std::lock_guard<std::mutex> lock(mtx);
        ready = true;
    }
    cv.notify_one();
}

int main() {
    std::thread c(consumer);
    std::thread p(producer);
    c.join();
    p.join();
    return 0;
}
```

通知通常放在修改共享状态并释放锁之后。正确性的关键仍然是“状态由锁保护、等待带谓词”。

练习 2.18 给上例增加超时：如果 1 秒内没有准备完成，就输出超时信息并退出。提示：使用 `wait_for` 的带谓词版本。

2.5.7 future、promise 与 async

互斥锁和条件变量适合传递“状态已经发生”的信号；当线程需要返回一个值或传递异常时，`std::future` 和 `std::promise` 更直接。

可以把它们理解为一次性的结果通道：`promise` 是写端，负责设置值或异常；`future` 是读端，负责等待并取得结果。一个 `future` 通常只能成功调用一次 `get()`。

示例 2.33 使用 promise/future 从工作线程取得结果：

```cpp
#include <chrono>
```

```cpp
#include <exception>
#include <future>
#include <iostream>
#include <thread>

void calculate(std::promise<int> result) {
    try {
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
        result.set_value(6 * 7);
    } catch (...) {
        result.set_exception(std::current_exception());
    }
}

int main() {
    std::promise<int> promise;
    std::future<int> future = promise.get_future();
    std::thread worker(calculate, std::move(promise));
    std::cout << "answer␣=␣" << future.get() << '\\n';
    worker.join();
    return 0;
}
```

如果只是想异步调用一个函数，std::async 可以自动建立任务和结果通道。使用 std::launch::async 时，明确要求异步执行；返回的 future 会在调用 get() 时等待任务完成。

示例 2.34 调用 async 函数并取得返回值：

```cpp
#include <future>
#include <iostream>

int square(int value) { return value * value; }

int main() {
    std::future<int> result = std::async(
        std::launch::async, square, 12);
    std::cout << result.get() << '\\n';       // 输出 144
    return 0;
}
```

结论 三种工具的分工：condition_variable 等待共享状态发生变化；promise/future 传递一次性的值或异常；async 用更短的代码启动异步函数，并通过 future 取得结果。

练习 2.19 编写函数 is_armor_valid，使用 std::async 异步判断一组装甲板数据是否满足阈值，主线程通过 future 获取判断结果。

2.5.8 线程池：组合前面的工具

线程池（thread pool）预先创建一组工作线程，用任务队列复用它们。它可以减少频繁创建和销毁线程的开销，也能限制同时运行的任务数量。调用方提交任务，空闲的工作线程从队列中取出任务执行；这正好把互斥锁、条件变量和线程生命周期管理组合到了一起。

实现线程池时至少要处理三件事：任务队列的互斥、空队列时的等待、析构时的停止与唤醒。工作线程取出任务后应先释放队列锁，再执行任务，避免一个慢任务阻塞其他线程提交或取任务。

示例 2.35 一个可正常停止的简易线程池：

```
#include <condition_variable>
#include <functional>
#include <iostream>
#include <mutex>
#include <queue>
#include <stdexcept>
#include <thread>
#include <vector>

class ThreadPool {
private:
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex mtx_;
    std::condition_variable cv_;
    bool stopping_ = false;

public:
    explicit ThreadPool(std::size_t count) {
        for (std::size_t i = 0; i < count; ++i) {
            workers_.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(mtx_);
                        cv_.wait(lock, [this] {
                            return stopping_ || !tasks_.empty();
                        });
                        if (stopping_ && tasks_.empty()) return;
                        task = std::move(tasks_.front());
                        tasks_.pop();
                    }
                    task();
                }
            });
        }
```

```
    }

    ~ThreadPool() {
        {
            std::lock_guard<std::mutex> lock(mtx_);
            stopping_ = true;
        }
        cv_.notify_all();
        for (auto& worker : workers_) worker.join();
    }

    void enqueue(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(mtx_);
            if (stopping_) throw std::runtime_error("thread␣pool␣is␣stopping");
            tasks_.push(std::move(task));
        }
        cv_.notify_one();
    }
};

int main() {
    ThreadPool pool(3);
    for (int i = 0; i < 8; ++i) {
        pool.enqueue([i] {
            std::cout << "task␣" << i << "␣is␣running\\n";
        });
    }
    return 0;        // 析构函数等待任务完成并回收线程
}
```

停止条件是“正在停止且任务队列为空”。析构时会先停止接收新任务，再唤醒所有工作线程；工作线程会把队列中已有的任务处理完，最后退出并被 `join()` 回收。若要让任务返回结果，可以在任务内部使用 `promise`，或设计返回 `future` 的提交接口。

✍ **练习 2.20** 为线程池增加一个返回 `std::future<int>` 的 `enqueue` 重载，使调用方可以提交“计算平方”的任务并取得结果。提示：结合 `std::packaged_task` 或 `std::promise` 实现。

2.5.9 并发进阶：经典问题

前面的互斥锁、条件变量和共享锁是零件；下面四个问题练习的是“看到场景后，知道该拿哪块零件”。先不要急着记解法。每个问题都先回答三件事：extbf 谁在同时做事、extbf 谁在共享什么、extbf 不协调会发生什么。视觉工程里的帧队列、参数表和设备资源竞争，都能在这里找到原型。

生产者—消费者问题。想象相机线程不断产生图像帧，识别线程不断取帧处理。前者是 extbf 生产者，后者是 extbf 消费者，两者之间只有一个容量为 3 的帧队列。它们共享的是队列，而不是“某一帧

图像”。

这个问题要保证三件事：每个放入的任务只取出一次；队列不能超过容量；队列空时消费者不能取一个不存在的任务，队列满时生产者也不能继续塞。若用循环反复检查队列，会白白占用 CPU；因此队列空或满时，对应线程应该睡眠，状态变化后再被唤醒。生产者把数据放入有限容量的缓冲区，消费者从缓冲区取出数据；这正是条件变量最典型的使用场景。

示例 2.36 有限容量的线程安全队列：

```
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>

class BoundedQueue {
private:
    std::queue<int> queue_;
    const std::size_t capacity_;
    std::mutex mtx_;
    std::condition_variable not_empty_;
    std::condition_variable not_full_;
    bool closed_ = false;

public:
    explicit BoundedQueue(std::size_t capacity) : capacity_(capacity) {}

    bool push(int value) {
        std::unique_lock<std::mutex> lock(mtx_);
        not_full_.wait(lock, [this] {
            return queue_.size() < capacity_ || closed_;
        });
        if (closed_) return false;
        queue_.push(value);
        not_empty_.notify_one();
        return true;
    }

    bool pop(int& value) {
        std::unique_lock<std::mutex> lock(mtx_);
        not_empty_.wait(lock, [this] {
            return !queue_.empty() || closed_;
        });
        if (queue_.empty()) return false;
        value = queue_.front();
        queue_.pop();
        not_full_.notify_one();
```

```
        return true;
    }

    void close() {
        {
            std::lock_guard<std::mutex> lock(mtx_);
            closed_ = true;
        }
        not_empty_.notify_all();
        not_full_.notify_all();
    }
};

int main() {
    BoundedQueue queue(3);
    std::thread producer([&] {
        for (int i = 0; i < 10; ++i) queue.push(i);
        queue.close();
    });
    std::thread consumer([&] {
        int value = 0;
        while (queue.pop(value)) std::cout << "consume␣" << value << '\\n';
    });
    producer.join();
    consumer.join();
    return 0;
}
```

这里的 `closed_` 很重要。没有关闭状态，消费者无法区分“暂时没有数据”和“以后永远不会有数据”，线程就可能永久等待。

读者—写者问题。想象多个线程都要读取一份曝光参数表，只有调参线程偶尔会修改它。读参数的线程是 extbf 读者，修改参数的线程是 extbf 写者，共享资源是这份参数表。两个读者同时看到同一份稳定的参数没有问题；但写者写到一半时，任何读者都不能读，两个写者也不能同时写。

这个问题关心的不只是“正确”，还关心“效率”：若所有人都用普通互斥锁，读者之间也只能排队，读多写少时就浪费了并行机会。因此读者可以并行，写者必须独占。还要留意公平性：如果系统总让新读者先进入，写者可能长期得不到机会，这叫写者饥饿。`std::shared_mutex` 能表达基本的读写互斥关系，公平性则要根据具体需求额外设计。

示例 2.37 多个读者共享读取、写者独占更新：

```
#include <iostream>
#include <mutex>
#include <shared_mutex>
#include <thread>
#include <vector>

```

```
int score = 0;
std::shared_mutex score_mutex;

void reader(int id) {
    std::shared_lock<std::shared_mutex> lock(score_mutex);
    std::cout << "reader␣" << id << ":␣" << score << '\\n';
}

void writer(int value) {
    std::unique_lock<std::shared_mutex> lock(score_mutex);
    score = value;
}

int main() {
    std::vector<std::thread> readers;
    for (int i = 0; i < 3; ++i) readers.emplace_back(reader, i);
    std::thread writer_thread(writer, 100);
    for (auto& reader_thread : readers) reader_thread.join();
    writer_thread.join();
    return 0;
}
```

哲学家进餐问题。这是一个把“多个任务同时申请多份资源”放大的小故事。五位哲学家各是一条线程，五把叉子各是一份只能被一个人使用的资源。第 i 位哲学家只有同时拿到左、右两把叉子才能吃饭；而每把叉子又与相邻哲学家共用。

麻烦出在“先拿到一份资源，再等另一份资源”。若每个人都先拿左叉子，就可能五个人各拿一把叉子，再一起等右边那把；谁也不能继续。真实工程中的“同时锁住相机和串口”“同时更新两个关联缓存”也是同一种结构。

一种简单的规避办法是打破对称性；更通用的办法是使用 `std::scoped_lock` 一次性获取两把锁，避免拿到第一把后长期等待第二把。

示例 2.38 使用 scoped_lock 安全获取两把叉子：

```
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>

const int philosopher_count = 5;
std::mutex forks[philosopher_count];

void dine(int id) {
    int left = id;
    int right = (id + 1) % philosopher_count;
    for (int round = 0; round < 2; ++round) {
```

```
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
        std::scoped_lock lock(forks[left], forks[right]);
        std::cout << "philosopher␣" << id << "␣is␣eating\\n";
    }
}

int main() {
    std::vector<std::thread> philosophers;
    for (int i = 0; i < philosopher_count; ++i)
        philosophers.emplace_back(dine, i);
    for (auto& philosopher : philosophers) philosopher.join();
    return 0;
}
```

死锁。死锁（deadlock）不是“程序运行得慢”，而是线程组成了一个谁也无法先让步的等待圈。比如线程 A 拿着资源 1 等资源 2，线程 B 拿着资源 2 等资源 1；资源 2 只有 B 释放后 A 才能得到，资源 1 也只有 A 释放后 B 才能得到。两条线程都不会自动恢复，程序就停在这里。哲学家进餐问题正是死锁可能出现的一种场景。

上面的等待圈并非偶然。死锁发生时会同时满足以下四个必要条件：

1. **互斥**：资源同一时刻只能被一个线程占用；
2. **持有并等待**：线程已经持有资源，同时等待其他资源；
3. **不可抢占**：资源不能被强行从持有者手中夺走；
4. **循环等待**：线程之间形成首尾相接的等待环。

只要破坏其中一个条件，就能避免死锁。工程中最常用的做法是给资源规定全局获取顺序；需要同时获取多把锁时使用 `std::lock` 或 `std::scoped_lock`；还可以设置超时，让等待失败后回滚。

示例 2.39 两个线程以相反顺序获取两把锁，会形成循环等待：

```
#include <mutex>
#include <thread>

std::mutex first;
std::mutex second;

void left_to_right() {
    std::lock_guard<std::mutex> a(first);
    std::lock_guard<std::mutex> b(second);
}

void right_to_left() {
    std::lock_guard<std::mutex> b(second);
    std::lock_guard<std::mutex> a(first);
}

int main() {
```

```
    std::thread one(left_to_right);
    std::thread two(right_to_left);
    one.join();
    two.join();
    return 0;
}
```

上例可能卡死：线程一持有 first 等待 second，线程二持有 second 等待 first。下面的写法使用 std::scoped_lock 同时获取两把锁，由标准库负责避免这种中间状态。

```
void safe_update() {
    std::scoped_lock lock(first, second);
    // 同时安全地访问由 first 和 second 保护的数据
}
```

笔记 “把锁的作用域写得更小”可以降低竞争，但不能自动消除死锁。只要多个线程以不同顺序获取多把锁，死锁风险仍然存在。请在接口设计阶段统一资源顺序，避免让调用者随意嵌套加锁。

结论经典问题对应的解决工具：

问题	共享状态	主要思路
生产者—消费者	有限容量队列	互斥锁保护队列，条件变量等待非空或非满
读者—写者	读多写少的数据	读者共享、写者独占，关注公平性
哲学家进餐	多个线程竞争多把资源	同时加锁、统一顺序或限制并发数量
死锁	线程形成等待环	破坏四条件之一，优先统一加锁顺序

练习 2.21

1. 给线程安全队列增加两个消费者，验证每个数据只被一个消费者取出。
2. 写出哲学家进餐问题中的四个死锁条件，并指出 std::scoped_lock 如何避免中间等待。
3. 构造一个“读者长期占用、写者迟迟不能进入”的场景，并思考如何改善写者饥饿。

2.6 本章小结

本章围绕” 组织数据 → 组织函数 → 组织内存 → 组织计算” 这条主线展开：

- **类与面向对象**：类的三大成员（数据成员、成员函数、构造/析构函数）和三种访问限定符是天天要用的基本功，要求熟练；函数重载、运算符重载、继承与多态是阅读和编写类代码的必备能力；模板只需掌握最基本的函数模板和类模板写法，遇到变参模板等进阶内容会查文档即可。
- **lambda、引用与移动**：lambda 的捕获列表（值捕获 vs 引用捕获）和 std::function 承接要熟练使用，回调和排序比较器里到处都是它；左值/右值、两种引用与 std::move/std::forward 是本

章最容易混淆的部分，记住”能取地址的是左值、有名字的右值引用是左值”这两句就够应付日常，其余在实践中慢慢消化。

- **智能指针**：掌握 `unique_ptr`（独占）、`shared_ptr`（计数共享）、`weak_ptr`（观察不占有）的分工，默认优先 `make_unique`/`make_shared`；循环引用的成因和用 `weak_ptr` 破解的方法必须理解。
- **STL**：`vector` 的常用成员函数和扩容机制要熟练；队列、堆栈、映射等容器知道各自解决什么问题、会去 cppreference 查用法即可；迭代器失效是新手重灾区，删除元素时牢记用 `erase` 的返回值更新迭代器。
- **多线程与内存**：会创建线程，使用 `mutex`/`lock_guard`、共享锁、原子变量和条件变量；能用 `future`、`promise`、`async` 传递异步结果；理解生产者—消费者、读者—写者、哲学家进餐和死锁；进程内存分区（代码/常量/静态/堆/栈）和栈帧机制是理解悬垂指针、栈溢出等经典 bug 的地基，务必建立图景。

本章验收通过后，恭喜你已经掌握了阅读现代 C++ 工程代码的主要语法设施。下一章我们来看看这些代码是如何被编译器组装成可执行文件的——CMake 见。

第二章 练习

1. 说出 `public`、`private`、`protected` 三种访问限定符的区别，并说明基类的 `private` 成员能否被派生类直接访问。
2. 判断并说明理由：`int f(int)` 与 `double f(int)` 能否构成函数重载？
3. 写出 lambda 表达式的简化语法形式，并说明值捕获 `[a]` 与引用捕获 `[&a]` 在”修改原变量”这一点上的区别。
4. 解释左值与右值的区别，写出判断一个表达式是左值还是右值的最常用方法。
5. 说明 `std::move` 的作用，并回答：对一个对象执行 `std::move` 之后，该对象本身的数据被搬走了吗？
6. 比较 `unique_ptr` 与 `shared_ptr` 的所有权语义，并说明为什么前者的拷贝构造被禁用。
7. 解释 `shared_ptr` 循环引用的成因，以及如何用 `weak_ptr` 解决。
8. （提高）写出遍历 `std::vector` 时安全删除满足某条件元素的两种写法（提示：一种利用 `erase` 的返回值，一种是 erase-remove 惯用法）。
9. （提高）解释死锁产生的四个必要条件，并结合”两把锁以相反顺序获取”的例子说明破坏哪一个条件可以修复它。
10. （实践）综合本章内容实现一个小程序：定义 `Armor` 类（含编号、置信度数据成员），用 `std::vector` 管理若干对象，使用 lambda 作为比较器按置信度降序排序，最后以多线程方式（两个线程）分别统计其中置信度高于/低于 0.5 的数量并打印，注意线程安全。

第三章 编译过程与 CMake 基础

内容提要

- 学习目标：分步生成编译产物；用 CMake 构建、链接并安装多文件工程；查找 OpenCV 依赖并定位常见构建错误。
- 建议学时：约 3–5 天（含练习）。
- 本卷边界：只考核标准 C++ 工程与 OpenCV 的基础构建；CUDA、ament 构建细节及图像算法见卷二，章末 ROS2 工程片段为选读。

上一章，我们写了不少 C++ 代码。这一章，一起看看这些代码怎样变成能运行的程序。先手动走一遍编译过程，再交给 CMake 管理。以后终端刷出一屏报错，你就能先判断卡在哪一步，不用一上来就怀疑人生。

下面的命令都在 Ubuntu 22.04 的 Bash 中运行。每个示例都会写清楚：在哪个目录操作、要建哪些文件、跑完应该看到什么。打开终端，跟着做就行。

3.1 C++ 的编译过程

C++ 从源文件到可执行文件，通常经历以下四步：

1. **预处理（Preprocessing）**：展开头文件、宏和条件编译，得到预处理后的源文件。
2. **编译（Compilation）**：检查语法与语义，将预处理结果翻译为汇编代码。
3. **汇编（Assembly）**：将汇编代码转换为目标文件（Object File）。
4. **链接（Linking）**：把目标文件和所需库组合起来，解析符号引用，生成可执行文件等产物。

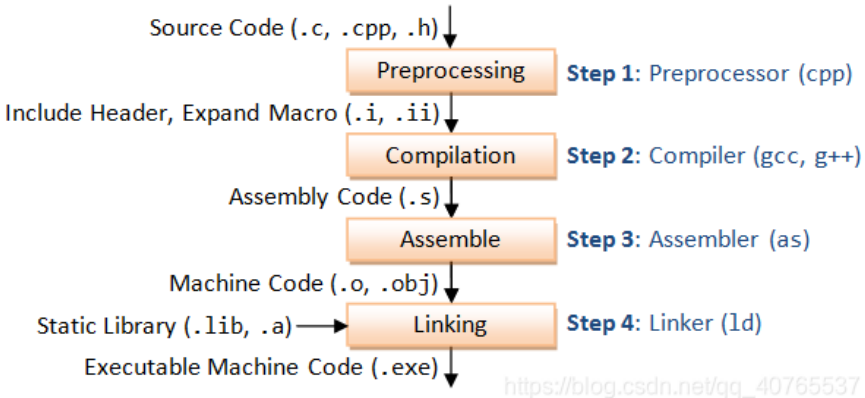


图 3.1: C++ 编译过程

预处理：先把这一份源代码准备好

预处理器先处理那些以 # 开头的指令。`#include` 引入文件内容，`#define` 定义宏，`#if` 等指令决定保留哪部分代码。用到宏时，预处理器会按规则展开它。一个 `.cpp` 文件，加上它包含的头文件，经过预处理后叫作一个**翻译单元（Translation Unit）**。后面编译时，各个翻译单元分别处理。

笔记 同一个头文件，可以在编译不同 `.cpp` 时分别被处理。头文件保护（如 `#ifndef/#define/#endif`）只防止同一翻译单元内重复包含。写入门工程时，头文件里通常放声明，普通非 `inline` 函数的定义放

在一个.cpp 里。这样能避免链接时重复定义。

排查问题时，下面这些名字可以帮助定位代码：

语法	作用
__FILE__	当前源文件名。
__LINE__	当前源文件行号。
__DATE__、*__TIME__*	源文件被翻译时的日期与时间，不是程序运行时间。
__cplusplus	使用的 C++ 标准版本值；C++17 对应 201703L。
__func__	函数体内可用的标准预定义变量，表示函数名；它不是宏。



笔记 __FUNCTION__ 是常见编译器扩展，入门代码优先用 __func__。__STDC_VERSION__ 表示 C 语言标准版本，不是 C++ 编译器版本；查看 GCC 版本可运行 g++ --version。

编译与汇编：检查代码，再生成机器指令

编译器先分析代码中的词、语法和含义，再生成代码，并按选项进行优化。这些检查分别叫词法、语法和语义分析。少写分号、调用未声明的函数，通常会在这里报错。

接着，汇编器把汇编代码变成可重定位目标文件（Relocatable Object File）。在 Linux 中，它常以.o 结尾。目标文件里有机器代码和数据，还有符号表与重定位信息。你可以先把“符号”理解为函数、变量等的名字。其中一些符号由本文件定义，另一些还要去其他文件里找。后续链接时，还需要确定相关地址，所以.o 通常不能直接运行。

链接：让调用找到实现

声明告诉编译器“这个函数怎样调用”，定义给出函数的具体实现。链接器要把函数调用和对应的实现联系起来，再合并所需代码与数据，处理相关地址。这就是解析符号引用和处理重定位。如果程序依赖动态库，还会留下一些信息，等加载器处理。

比如，你声明并调用了一个函数，却漏掉了实现所在的目标文件或库。链接器找不到实现，就可能报 undefined reference（未定义引用）。读到这条错误时，先查函数定义有没有参与链接。

手动控制四个阶段

g++ 是 GCC 的 C++ 编译驱动程序。通用形式为 $g++$ [选项] 输入文件...；不指定停止阶段时，它会继续完成链接。

参数	作用
$-E$	只做预处理。
$-S$	完成编译后停止，生成汇编文件。
$-c$	生成目标文件后停止，不链接。
$-o$ 文件	指定输出文件名。
$-std=$ 标准	选择语言标准，本章使用 c++17。
$-I$ 目录	增加头文件搜索目录。
$-Wall$、$-Wextra$	开启常用编译警告。
$-g$	生成调试信息。

其余参数请查阅 man g++ 手册。

示例 3.1 亲眼看到四种产物 先装好工具，再建一个专门放练习的目录。若你已经用过这个目录名，就换一个，避免覆盖自己的文件。

```
sudo apt update
sudo apt install build-essential cmake
mkdir -p ~/rm-build-lab/stages
cd ~/rm-build-lab/stages
```

创建 hello.cpp，内容如下：

```
#include <iostream>
#define ARMOR_ID 3

int main() {
    std::cout << "armor=" << ARMOR_ID << '\n';
    std::cout << "function=" << __func__ << '\n';
    return 0;
}
```

留在这个目录里，逐条运行下面的命令。每跑完一步，都看看多了哪个文件：

```
g++ -std=c++17 -E hello.cpp -o hello.ii
g++ -std=c++17 -S hello.ii -o hello.s
g++ -c hello.s -o hello.o
g++ hello.o -o hello
./hello
```

预期输出两行：armor=3 和 function=main。再用编辑器打开 hello.ii，搜索 armor=。你会看到这里的 ARMOR_ID 已经变成了 3。文件比原来长很多是正常的，头文件内容也被展开进来了。不用从第一页读到最后一页，我们只找刚才那一行。平时构建时，把这四步交给一条命令就行：

```
g++ -std=c++17 -Wall -Wextra hello.cpp -o hello
./hello
```

静态库与动态库

有些代码会被多个程序用到，我们可以把它们做成库（Library）。库常见有两种：

静态库（Static Library）通常把多个目标文件打包在一起，Linux 中常见文件名为 lib 名称.a。链接时，链接器把所需目标文件中的代码和数据放进最终产物。

动态库或共享库（Shared Library）在 Linux 中常见文件名为 lib 名称.so。普通动态链接会在构建时记下程序依赖哪些库。程序启动或运行时，再由加载器装入这些库。



笔记 “静态库编译一定更快”和“动态库运行一定更慢”都不是可靠结论。速度取决于具体工程和测量结果。刚开始先关心一件事：把程序搬到另一台电脑时，还要带上什么？静态链接某个库后，通常不用再带上这个库的独立文件。动态链接则需要运行环境里有兼容的库。另外，链接了一个静态库，不代表整个程序都采用了静态链接。性能如何，跑一跑再说，别急着给两种库排战力榜。

结论

- .cpp → .ii → .s → .o → 可执行文件对应预处理、编译、汇编、链接。

- 找不到头文件先查包含路径；语法错误先查源代码；未定义引用先查实现及链接依赖。
- 声明用于检查调用，定义提供实现，目标文件承载编译结果，库组织可复用代码。
- 静态库与动态库主要影响链接方式和部署条件，不能直接据此判断性能。

✍ **练习 3.1** 给装甲板换个编号：把 `ARMOR_ID` 改为 7。先直接运行旧的 `hello`，再重新构建并运行。两次输出有什么不同？请结合刚才的四步解释原因。接着故意漏写一个分号，看看哪一步先报错。观察完记得补回来，别给下一节留下伏笔。

3.2 CMake

只有一个源文件时，手写 `g++` 命令很方便。文件一多，你就得记住：谁依赖谁？哪些文件改过？哪些程序要重新构建？这些工作可以交给 Make，它按 `Makefile` 里的规则来做。

CMake 是跨平台的构建系统生成工具。你把工程怎么构建写进 `CMakeLists.txt`，CMake 再生成 Make、Ninja 或 IDE 使用的构建文件。之后，相应的构建工具会调用编译器和链接器。生成器（Generator）决定生成哪种构建文件。CMake 本身不翻译 C++ 代码。

接下来，我们按顺序完成项目配置、添加文件、编译、链接和安装，最后一起读一段实际工程的配置。先分清三个动作：**配置与生成**负责读取构建描述、查找依赖、写出构建文件；**构建**负责真正编译和链接；**安装**负责按规则复制产物。

看到 `Configuring done` 和 `Generating done`，先别开香槟。构建文件准备好了，程序还得接着编译。

3.2.1 项目配置

先在项目根目录建一个 `CMakeLists.txt`，文件名的大小写要写对。里面的命令一般写成*命令名*(*参数...*)。命令名不区分大小写，变量名区分大小写；名称、路径等参数值通常也区分大小写。开头先写最低 CMake 版本，再用 `project` 指定语言，让 CMake 检测编译器。

语法	作用
`cmake_minimum_required(VERSION` 版本`)`	声明所需最低版本。
`project(`名称 `LANGUAGES` 语言`)`	设置项目名，启用指定语言。
`set(`变量 值`...)`	设置普通变量。
`message(STATUS` 内容`...)`	在配置阶段输出信息。
`add_executable(`目标 源文件`...)`	声明一个可执行目标。
`target_compile_features(`目 标 `PRIVATE` 特性`...)`	为目标指定编译特性，如 C++17。

`${`变量名`}` 表示取变量值，不是 C/C++ 的宏替换。`PROJECT_NAME` 是项目名称，`CMAKE_CXX_COMPILER` 是检测到的 C++ 编译器路径。`PRIVATE` 表示这项要求用于当前目标，其他作用域见第 3.2.6 节。

终端命令的通用形式为 *cmake* [选项]。构建目录（Build Tree）保存缓存、目标文件和可执行文件，源码目录（Source Tree）保存源代码和构建描述。

参数	作用
$-S$ 目录	指定源码目录。
$-B$ 目录	指定构建目录，不存在时自动创建。
$-D$变量 $=$ 值	设置缓存变量，如构建类型。
$-G$ 生成器	指定生成器，例如 `Unix Makefiles`。
$--$ *build* 目录	调用该构建目录对应的构建工具。
$--$ *parallel* 数量	限制并行构建任务数，配合 `--build` 使用。
$--$ *verbose*	显示实际构建命令，配合 `--build` 使用。
$--$ *install* 目录	按安装规则复制已构建产物。
$--$ *prefix* 路径	指定安装前缀，配合 `--install` 使用。

其余参数请查阅 `man cmake`；单条 CMake 语言命令可用 `cmake --help-command 命令名` 查询。

示例 3.2 先跑通一个最小工程 在终端新建工程，并复用上节的源文件：

```
mkdir -p ~/rm-build-lab/hello-cmake
cd ~/rm-build-lab/hello-cmake
cp ../stages/hello.cpp .
```

创建 `CMakeLists.txt`，完整内容如下。本章采用 CMake 3.16 及以上版本，不需要追求最新版。

```
cmake_minimum_required(VERSION 3.16)
project(hello_cmake LANGUAGES CXX)

set(ARMOR_NAME "training_armor")
message(STATUS "Armor: ${ARMOR_NAME}")
message(STATUS "Compiler: ${CMAKE_CXX_COMPILER}")

add_executable(hello hello.cpp)
target_compile_features(hello PRIVATE cxx_std_17)
```

配置、构建和运行：

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 2
./build/hello
```

配置时应出现 `Armor: training_armor` 和编译器路径；运行结果与上节一致。项目名是 `hello_cmake`，目标名是 `hello`，两者可以不同。



笔记 `CMakeCache.txt` 会记住编译器、依赖路径等配置。换编译器或生成器时，开一个新的构建目录，别让旧配置继续加戏。想手动选择 GCC，就在第一次配置时传入 `-DCMAKE_CXX_COMPILER=g++`，不要在 `project` 后临时修改编译器。本章使用 Make 等单配置生成器，`CMAKE_BUILD_TYPE=Debug` 用于调试构建；多配置生成器的配置选择用法另查手册。CUDA 的语言启用与工具链配置见卷二 CUDA 章。



练习 3.2 将目标 `hello` 改名为 `armor_hello`，同步修改对应的目标配置，使用新的 `build-rename` 目录构建并运行。说明为什么不需要修改 `project(hello_cmake ...)`。

3.2.2 添加文件

文件准备好后，还得告诉 CMake 怎样使用它们。这里有三件事：列出要编译的源文件、设置头文件搜索目录、查找外部依赖。CMake 把一个可执行程序或一个库称为目标（Target），后面的配置大多围绕目标来写。这三件事要分别完成：设置头文件搜索目录不会自动添加源文件，`find_package` 也不会替你下载和安装依赖。

语法	作用
`set(变量 源文件...)`	保存显式源文件列表。
`target_include_directories(目标 作用域 目录...)`	为目标设置头文件搜索目录。
`find_package(包名 REQUIRED)`	查找已安装的包，缺失时终止配置。
`option(变量" 说明" 默认值)`	声明可由命令行设置的布尔开关。
`if(条件) ... endif()`	根据条件执行 CMake 命令。
`target_compile_definitions(目标 作用域 宏...)`	向目标的编译过程传入宏定义。

示例 3.3 为装甲板程序准备三个文件 来写一个给装甲板打分的小程序。从这里到安装一节，我们一直用这个 `armor-demo` 工程，不用每节重新建一套文件。先建目录：

```
mkdir -p ~/rm-build-lab/armor-demo/include
mkdir -p ~/rm-build-lab/armor-demo/src
cd ~/rm-build-lab/armor-demo
```

创建 `include/armor.hpp`，只声明接口，并加上头文件保护：

```
#ifndef ARMOR_DEMO_ARMOR_HPP
#define ARMOR_DEMO_ARMOR_HPP

int armor_score(int id);

#endif
```

创建 `src/armor.cpp`，提供函数定义：

```
#include "armor.hpp"

int armor_score(int id) {
    return id * 10;
}
```

创建 `src/main.cpp`，使用接口：

```
#include "armor.hpp"
#include <iostream>

int main() {
```

```
    std::cout << "score=" << armor_score(3) << '\n';
#ifdef SAVE_OUTPUT
    std::cout << "save output enabled\n";
#endif
    return 0;
}
```

暂时先用上节学过的编译命令验证文件齐全：

```
g++ -std=c++17 -I include src/main.cpp src/armor.cpp -o armor_manual
./armor_manual
```

预期输出 score=30。其中 -I include 让两个源文件都能找到 armor.hpp。接下来会为它创建完整的 CMake 构建描述。

头文件通常不用单独编译，但编译器必须找得到它。你列出要编译的源文件，CMake 据此安排构建。编译器再按包含规则查找头文件，并记录文件之间的依赖，供后续增量构建使用。所以，源文件列表和头文件路径都要写清楚。

笔记 旧工程可能用 file(GLOB_RECURSE 变量路径模式) 递归收集源文件。我们先把源文件一个个列出来。以后加文件，就顺手改一次列表，检查起来也方便。用通配符时，要小心把其他程序的 main.cpp 也收进来。两个入口争当主角，链接器可不会帮你选。include_directories 设置头文件搜索路径，link_directories 设置库搜索路径；二者都不等于“把实现链接进程序”。

练习 3.3 做两次小型排错练习，每次都从完整命令开始：第一次去掉 -I include，第二次去掉 src/armor.cpp。分别读一读报错，判断哪次找不到头文件、哪次找不到函数实现。最后恢复完整命令，确认程序能运行。

3.2.3 编译

用下面两种形式分别声明可执行目标和库目标：

add_executable(目标 源文件...)
add_library(目标 [类型] 源文件...)

这两条命令先记下“要构建什么”。等你运行构建命令时，编译器才开始工作。

参数	作用
目标	工程中唯一的逻辑名称，供后续 target_* 命令引用。
源文件	要编译的 .cpp 文件，可显式列出或通过变量展开。
STATIC	生成静态库。
SHARED	生成共享库。
省略类型	根据 BUILD_SHARED_LIBS 选择；未设置或为假时默认静态库。

示例 3.4 一次构建两个源文件 在示例 3.3 的 armor-demo 根目录创建 CMakeLists.txt：

```
cmake_minimum_required(VERSION 3.16)
project(armor_demo LANGUAGES CXX)
```

```
3
4  set(ARMOR_SOURCES src/main.cpp src/armor.cpp)
5  add_executable(armor_demo ${ARMOR_SOURCES})
6  target_include_directories(armor_demo PRIVATE
7      "${CMAKE_CURRENT_SOURCE_DIR}/include")
8  target_compile_features(armor_demo PRIVATE cxx_std_17)
9
10 if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
11     target_compile_options(armor_demo PRIVATE -Wall -Wextra)
12 endif()
13
14 option(SAVE_OUTPUT "Enable save-output message" OFF)
15 if(SAVE_OUTPUT)
16     target_compile_definitions(armor_demo PRIVATE SAVE_OUTPUT)
17 endif()
```

CMAKE_CURRENT_SOURCE_DIR 是当前处理的源码目录。警告选项只加给当前目标；开关只演示条件编译，**并不真的保存文件**。

```
1 cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
2 cmake --build build --parallel 2
3 ./build/armor_demo
4 cmake -S . -B build -DSAVE_OUTPUT=ON
5 cmake --build build --parallel 2
6 ./build/armor_demo
```

第一次输出 score=30；打开开关并重新构建后，多一行 save output enabled。

笔记 CMake 变量和 C++ 宏处在不同阶段。option(SAVE_OUTPUT ...) 创建的是配置开关；target_compile_definitions 才把它转换成该目标编译时可见的宏。add_library(name ...) 也不保证生成动态库，需要动态库时明确写 SHARED。

练习 3.4 给 3 号装甲板开一次“双倍积分”：把 armor_score 中的乘数改成 20。只执行构建和运行命令，检查输出是否变成 score=60。再用 -DSAVE_OUTPUT=OFF 重新配置并构建，确认提示消失。练完把乘数改回 10，活动结束，积分恢复正常。

3.2.4 链接

如果几个程序都要用装甲板打分函数，我们可以把实现做成一个库，让这些程序链接它。通用形式为：

target_link_libraries(目标 作用域 依赖...)

第一个参数是已经创建的**目标名**，不是.o 文件名。

参数	作用
目标	需要链接依赖的可执行目标或库目标。
PRIVATE	当前目标使用这项依赖；其接口不要求使用者直接使用它。
PUBLIC	当前目标使用依赖，其使用接口也向下游传播。
INTERFACE	仅作为接口要求提供给下游，当前目标不用于自身链接。
依赖	可为本工程的库目标、包提供的导入目标或库路径等。

链接本工程的库目标时，直接写目标名，CMake 会安排好构建顺序，并找到对应的库文件。注意 `armor` 和 `${armor}` 的区别：前者是目标名，后者在读取同名变量。如果没设置这个变量，读到的值就可能是空的。

示例 3.5 把实现拆成静态库 保持三个 C++ 文件不变，将 `armor-demo/CMakeLists.txt` **完整替换**为下面内容：

```
cmake_minimum_required(VERSION 3.16)
project(armor_demo LANGUAGES CXX)

add_library(armor STATIC src/armor.cpp)
target_include_directories(armor PUBLIC
    "${CMAKE_CURRENT_SOURCE_DIR}/include")
target_compile_features(armor PUBLIC cxx_std_17)

add_executable(armor_demo src/main.cpp)
target_link_libraries(armor_demo PRIVATE armor)

if(CMAKE_CXX_COMPILER_ID MATCHES "GNU|Clang")
    target_compile_options(armor PRIVATE -Wall -Wextra)
    target_compile_options(armor_demo PRIVATE -Wall -Wextra)
endif()

option(SAVE_OUTPUT "Enable save-output message" OFF)
if(SAVE_OUTPUT)
    target_compile_definitions(armor_demo PRIVATE SAVE_OUTPUT)
endif()
```

使用新的构建目录，方便与上一版对照：

```
cmake -S . -B build-lib -DCMAKE_BUILD_TYPE=Debug
cmake --build build-lib --parallel 2
./build-lib/armor_demo
ls build-lib/libarmor.a
```

预期运行输出 `score=30`，并能看到静态库 `libarmor.a`。`armor` 把头文件目录设为 `PUBLIC`，所以链接它的 `armor_demo` 也能找到接口头文件。

结论

- `add_executable/add_library` 创建目标；`target_*` 为已存在的目标补充要求。

- 包含目录解决“头文件在哪”；链接依赖解决“函数实现在哪”；两者各有用途。
- 链接一个库目标会带来它公开的使用要求，例如头文件搜索目录。
- 静态库的 PRIVATE 依赖仍可能成为最终可执行程序必需的链接项，不能理解成“下游链接命令绝不会出现它”。

遇到未定义引用，就沿着这几步查。链接器不接受“我记得写过这个函数”作为证据，得让它真的找到实现。

练习 3.5 在 Ubuntu 中把库类型改为 SHARED，用新目录 build-shared 配置、构建并运行，观察生成的 libarmor.so。记录动态库的文件名，随后将源码中的类型恢复为 STATIC，后续安装示例使用静态库版本。

3.2.5 安装

程序已经能在构建目录里运行了。接下来，我们把它放到指定位置，方便以后使用，这一步叫安装（Install）。声明可执行文件安装规则的通用形式为 install(TARGETS 目标... RUNTIME DESTINATION 相对目录)。TARGETS 后写目标名，RUNTIME 指定运行程序这一类产物，DESTINATION 指定相对安装前缀的目录。

示例 3.6 安装到自己的练习目录 在示例 3.5 的 CMakeLists.txt 末尾追加：

```
install(TARGETS armor_demo RUNTIME DESTINATION bin)
```

在 armor-demo 根目录运行：

```
cmake -S . -B build-lib -DCMAKE_BUILD_TYPE=Debug
cmake --build build-lib --parallel 2
cmake --install build-lib --prefix "$PWD/stage"
./stage/bin/armor_demo
```

预期出现安装路径 stage/bin/armor_demo，运行输出 score=30。该程序已静态链接我们编写的 armor 库，因此这里无需安装独立的 libarmor.a。

笔记 cmake --install 不会自动构建，记得按“配置 → 构建 → 安装”的顺序来。安装可能覆盖目标位置的同名文件。练习时就用自己的 stage 目录，不需要 sudo，先别急着往系统目录里搬。如果换成动态库版本，还要带上所需共享库，并设置好运行时搜索路径。只复制一个可执行文件，到了另一台电脑上可能跑不起来。相关细节以后做部署时再展开。

练习 3.6 不修改安装规则，将同一程序安装到 stage-check 目录，并从那里运行。指出安装前缀和 DESTINATION bin 分别对应最终路径的哪一部分。

3.2.6 补充内容

目录设置与目标设置

读旧工程时，你可能会遇到 include_directories。它会影响当前目录及相关子目录、目标的包含目录设置，不能简单理解成“整个工程都生效”。我们先用 target_include_directories 这类命令：哪个目标需要，就给哪个目标设置。这里的 PRIVATE、PUBLIC、INTERFACE 决定谁会用到这些配置。名字可能眼熟，但它们和 C++ 类的访问权限是两回事。

结论以头文件路径和编译特性为例，三种作用域可这样判断：

- PRIVATE：当前目标自己用，例如库内部实现专用的包含目录。
- PUBLIC：当前目标和使用它的下游都用，例如示例中的 armor.hpp 所在目录。
- INTERFACE：仅下游使用，常见于只提供头文件的接口库。

把调试函数放进单独的文件

配置出了问题，又看不出变量里装着什么，就先把它打印出来。我们把打印操作写成函数，再放进单独的文件，以后就能重复使用。自定义函数写成 function(函数名参数...) ... endfunction()。include(模块名) 用来载入模块，list(APPEND 变量元素...) 用来往列表中追加元素。

示例 3.7 打印编译器与构建类型 先在 armor-demo 根目录创建模块目录：

```
mkdir -p cmake
```

新建 cmake/Function.cmake，写入：

```
function(print_var var)
    if(DEFINED ${var})
        message(STATUS "${var} = [${${var}}]")
    else()
        message(STATUS "${var} is not defined")
    endif()
endfunction()
```

在 CMakeLists.txt 的 project 后加入：

```
list(APPEND CMAKE_MODULE_PATH
    "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
include(Function)
print_var(CMAKE_CXX_COMPILER)
print_var(CMAKE_BUILD_TYPE)
print_var(NOT_DEFINED_YET)
```

运行 cmake -S . -B build-lib -DCMAKE_BUILD_TYPE=Debug，预期打印编译器路径、Debug 和未定义变量提示。var 接收的是变量名。读 ${${var}} 时，从里往外看：先拿到名字，再读取这个名字对应的值。空字符串也可以直接输出，不用特意截取最后一个字符或补换行。

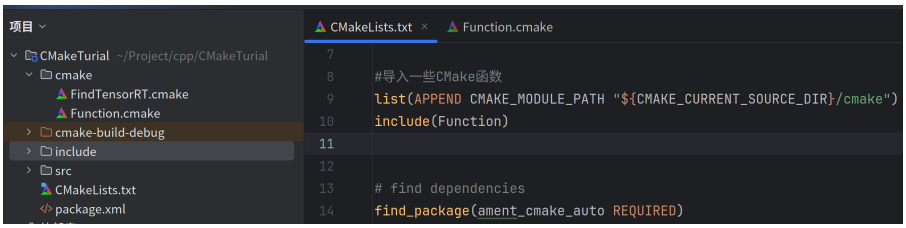


图 3.2: 原工程中拆分 CMake 函数文件的结构示意；练习以正文给出的文件路径为准

按失败阶段排查

先别和满屏红字对线，我们从第一处错误看起。后面的“构建失败”往往只是前面出错后的结果。先判断它发生在哪个阶段，再按下面的方向查，通常比盲目改配置省事。

命令 / 现象	作用 / 排查方向
配置时报找不到包	检查开发包是否安装，以及包配置文件路径。
No such file or directory（头文件）	检查文件名与目标的包含目录。
undefined reference	检查函数定义、签名，以及源文件或库是否参与链接。
multiple definition	检查普通函数是否被重复定义或重复加入目标。
error while loading shared libraries	程序已构建，但运行时找不到所需共享库。
cmake --build build-lib --verbose	查看实际编译与链接命令；无改动时可能没有命令可显示。

练习 3.7 在静态库示例中把 armor 的包含目录作用域从 PUBLIC 改为 PRIVATE，重新构建。指出失败的是库还是可执行目标，解释原因并恢复原配置。

3.2.7 实例

使用外部依赖：OpenCV 图像读取

自己的库已经能用了，再试试别人写好的 OpenCV。步骤还是刚才那几步：查找依赖、创建目标、设置包含目录、链接依赖。find_package(OpenCV REQUIRED) 查找 OpenCV 提供的包信息；OpenCV_INCLUDE_DIRS 是包含目录，OpenCV_LIBS 是需要链接的库列表。

示例 3.8 从零编译并显示图像 在 Ubuntu 桌面环境中安装开发包，建立独立目录：

```
sudo apt install libopencv-dev
mkdir -p ~/rm-build-lab/opencv-demo
cd ~/rm-build-lab/opencv-demo
```

创建 main.cpp：

```
#include <opencv2/opencv.hpp>
#include <iostream>

int main(int argc, char** argv) {
    if (argc != 2) {
        std::cerr << "Usage: image_viewer <image_path>\n";
        return 1;
    }
    const cv::Mat image = cv::imread(argv[1]);
    if (image.empty()) {
        std::cerr << "Cannot read image: " << argv[1] << '\n';
        return 2;
    }
    std::cout << "size=" << image.cols << 'x' << image.rows << '\n';
    cv::imshow("Armor image", image);
    cv::waitKey(0);
    return 0;
```

```
}
```

创建 CMakeLists.txt：

```
cmake_minimum_required(VERSION 3.16)
project(opencv_demo LANGUAGES CXX)

find_package(OpenCV REQUIRED)
add_executable(image_viewer main.cpp)
target_compile_features(image_viewer PRIVATE cxx_std_17)
target_include_directories(image_viewer PRIVATE ${OpenCV_INCLUDE_DIRS})
target_link_libraries(image_viewer PRIVATE ${OpenCV_LIBS})
```

先生成一张纯文本格式的测试图，不需要额外下载素材：

```
printf 'P3\n2␣2\n255\n255␣0␣0␣␣0␣255␣0\n0␣0␣255␣␣255␣255␣255\n' > test.ppm
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 2
./build/image_viewer test.ppm
```

运行后应输出 size=2x2，并打开显示测试图的窗口。在图像窗口中按任意键，程序就会结束。这张图只有四个像素，先委屈一下你的显卡。跑通后，再换成自己拍的装甲板图像。

顺便检查两种出错情况：传入不存在的图片路径，程序应提示读取失败，返回 2；不传参数，程序应打印用法，返回 1。

笔记 显示图像需要图形桌面。如果你在纯命令行服务器上操作，或远程连接没有配置图形转发，窗口可能打不开。先检查运行环境，别急着重写构建文件。如果配置时就找不到 OpenCV，先确认装了开发包 libopencv-dev。Python 的 opencv-python 不能替代它。如果 OpenCV 是你自己安装的，就找到 OpenCVConfig.cmake，再用 -DOpenCV_DIR= 目录指向它所在的目录。这里不能随手填一个头文件目录。

练习 3.8 不传参数运行程序，再传入不存在的 missing.png。每次运行后立刻执行 echo $?，检查退出码分别为 1 和 2。说明为什么这两种错误不需要修改链接配置。

阅读原有 ROS2 工程片段（选读）

最后看一眼实际工程，就当是卷二的预告片。下面这段来自一个包含 Kalman、MLS、LMS 模块的 ROS2 工程。先找一找模块目录、库目标和链接关系，看看能认出多少熟面孔。

这是**工程阅读片段**。要运行它，还需要原工程的源文件、package.xml、自定义函数文件和 ROS2 环境，不能单独复制出来构建。ament 的完整用法放在卷二 ROS2 章，这里先读懂主线就够了。

```
cmake_minimum_required(VERSION 3.16)
project(filter LANGUAGES CXX)

list(APPEND CMAKE_MODULE_PATH "${CMAKE_CURRENT_SOURCE_DIR}/cmake")
include(Function)
find_package(ament_cmake_auto REQUIRED)
ament_auto_find_build_dependencies()
find_package(OpenCV REQUIRED)
find_package(Eigen3 REQUIRED)
```

```
10
11 set(KALMAN_INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include/kalman")
12 set(MLS_INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include/mls")
13 set(LMS_INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include/lms")
14 set(EXEC_INCLUDE_DIR "${PROJECT_SOURCE_DIR}/include/exec")
15
16 # 保留旧工程的源文件收集方式，新工程优先显式列出文件。
17 file(GLOB_RECURSE KALMAN_SRCS "${PROJECT_SOURCE_DIR}/src/kalman/*.cpp")
18 file(GLOB_RECURSE MLS_SRCS "${PROJECT_SOURCE_DIR}/src/mls/*.cpp")
19 file(GLOB_RECURSE LMS_SRCS "${PROJECT_SOURCE_DIR}/src/lms/*.cpp")
20 file(GLOB_RECURSE EXEC_SRCS "${PROJECT_SOURCE_DIR}/src/exec/*.cpp")
21
22 set(ALL_INCLUDE ${KALMAN_INCLUDE_DIR} ${MLS_INCLUDE_DIR}
23     ${LMS_INCLUDE_DIR} ${EXEC_INCLUDE_DIR} ${OpenCV_INCLUDE_DIRS})
24 include_directories(${ALL_INCLUDE})
25 print_var(ALL_INCLUDE)
26
27 add_library(filter_kalman STATIC ${KALMAN_SRCS})
28 add_library(filter_mls STATIC ${MLS_SRCS})
29 add_library(filter_lms STATIC ${LMS_SRCS})
30 set(FILTER_LIBS filter_kalman filter_mls filter_lms)
31 foreach(lib IN LISTS FILTER_LIBS)
32     target_compile_features(${lib} PUBLIC cxx_std_17)
33     target_link_libraries(${lib} PUBLIC Eigen3::Eigen ${OpenCV_LIBS})
34 endforeach()
35
36 ament_auto_add_executable(filter ${EXEC_SRCS})
37 # 保留 ament 宏使用的无作用域签名，避免对同一目标混用签名。
38 target_link_libraries(filter ${FILTER_LIBS})
39
40 if(BUILD_TESTING)
41     find_package(ament_lint_auto REQUIRED)
42     ament_lint_auto_find_test_dependencies()
43 endif()
44 ament_auto_package()
```



笔记 自动依赖查找命令依据 ROS2 包清单工作。ament_auto_add_executable 处理可执行目标的相关约定，ament_auto_package 处理包的安装与导出约定。它们不是普通 CMake 命令的通用替代品。上面的目录设置沿用了原工程的写法。每个库到底需要哪些依赖，还得看对应源码。这些命令先混个脸熟，不用今天就把卷二速通；本章验收不考它们。

结论 读陌生构建文件时，先回答三个问题：

- 依赖什么：找 find_package。
- 生成什么：找 add_executable 和 add_library。
- 谁链接谁：找 target_link_libraries。

找到这三处，再回头看变量和条件分支。第一次读不完很正常，先把程序怎么构建说明白，就已经有收获了。



笔记参数查阅入口：

- GCC 编译阶段与选项：
  https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html
- CMake 构建模型：
  https://cmake.org/cmake/help/latest/manual/cmake-buildsystem.7.html
- OpenCV 的 CMake 示例：
  https://docs.opencv.org/4.x/db/df5/tutorial_linux_gcc_cmake.html

查到当前问题需要的那一条就好，不用把手册从头刷到尾。其他功能，等用到时再回来查。

3.3 本章小结

这一章，我们把代码一步步变成了能运行的程序。回头看，最常用的是这几件事：

- **看懂编译过程**：说清预处理、编译、汇编和链接各做什么，能区分声明与定义、头文件与目标文件、静态库与动态库。四步命令亲手跑过即可，日常交给构建工具。
- **跑通 CMake 工作流**：用 `cmake -S/-B` 配置，用 `cmake --build` 构建，再用 `cmake --install` 安装。这几条要练熟，源码目录和构建目录也要分开。
- **写清目标与依赖**：先用 `add_executable` 或 `add_library` 创建目标，再给它设置头文件路径和链接依赖。这部分要会写，也要能说出哪些配置只给自己用，哪些会传给下游。
- **按阶段排错**：先判断卡在配置、编译、链接还是运行，再查相应的路径、实现或依赖。参数细节、自定义模块和 ROS2 的构建约定，用到时会查就行。

现在拿一份构建文件，试着说出它要生成什么、依赖什么、怎样运行验证。能说清，也能亲手跑通，就可以带着这些基本功去卷二了。以后再遇到满屏报错，至少知道从哪里查起——这已经比对着终端许愿靠谱多了。

第三章 练习

1. （基础）写出从 `demo.cpp` 到可执行文件的四个阶段及其中间文件扩展名，并说明 `g++ -c demo.cpp` 停在哪一步。
2. （基础）你把项目命名为 `training`，把可执行目标命名为 `viewer`。这两个名字分别管什么？终端出现 `Generating done` 时，程序已经编译好了吗？
3. （基础）程序需要 `main.cpp` 和 `score.cpp`，头文件放在 `include` 目录。你已经设置了头文件搜索路径，还需要把 `score.cpp` 加进目标吗？说说理由。
4. （基础）比较 `add_library(score STATIC score.cpp)`、显式写 `SHARED` 和省略类型三种写法的含义。
5. （基础）你用 `add_library(score STATIC score.cpp)` 创建了库，但没定义名为 `score` 的变量。链接时写 `score` 和 `${score}`，结果一样吗？
6. （基础）安装规则的目标目录为 `bin`，安装前缀为 `/home/student/demo-stage`。目标 `viewer` 会安装到哪里？只运行安装命令能否替代构建？
7. （基础）库需要两个头文件目录：一个是库自己和使用这个库的程序都要用的，另一个只供库内部使用。它们分别该设成什么作用域？`INTERFACE` 又适合什么情况？
8. （基础）`find_package(OpenCV REQUIRED)` 在配置阶段失败。它会自动安装 OpenCV 吗？自行安装 OpenCV 后，`OpenCV_DIR` 应指向哪类目录？
9. （提高）你把 `SAVE_OUTPUT` 从 `OFF` 改成 `ON`，配置完成后就运行了原程序，却没看到新提示。请按 CMake 变量、编译宏、可执行文件的顺序，解释中间漏了哪一步。
10. （实践）观察一次“哪些文件需要重新编译”。在静态库工程中，先不改任何文件，连续构建两次；然后只改 `src/armor.cpp` 中的乘数，再构建一次。用 `--verbose` 保存后两次构建的输出，看看各自做了哪些工作。最后把乘数改回来。

卷一小结

本卷从安装 Ubuntu 开始，经过命令行、版本管理和现代 C++，最后走到了多文件工程的构建与运行。这些内容共同回答一个问题：**拿到一份 C++ 工程，怎样在自己的电脑上读懂它、修改它、构建它，并把可验证的修改交给队友？**卷首介绍的视觉组职责，也因此有了具体的落脚点：图像处理、目标识别与后续决策需要算法，而算法要成为可以协作开发、稳定运行的程序，还需要本卷这些基本功。

下面按三章回顾知识，再把它们放回同一条开发流程中。复习时可以一边读，一边打开终端或已有练习工程验证。正文中已经给出的安装步骤、代码示例和参数表，仍是动手时的查阅入口；这里着重说明每部分解决什么问题、与其他内容怎样衔接，以及应该掌握到什么程度。

一、Linux 工作环境：让开发有一个可靠的起点

本章的串口基础把 Linux 的设备文件、用户组权限与实际外设连接串起来：能识别队内 USB CDC ACM 设备、用 udev 绑定固定别名、按固件约定完成收发验证，并保留字节与超时记录，才有依据继续检查板端协议。复习时区分软件回环、物理回环与整条板端链路各自能验证的范围；具体步骤见第 1.9.1 节。

1. 安装系统：先认清磁盘，再建立工作环境。双系统安装依次涉及分区、制作启动盘、从 U 盘安装和安装后的系统设置。分区决定两套系统各自使用哪些存储空间，启动盘提供安装环境，安装时的挂载点决定 Linux 如何使用这些分区。复习时应能说明根目录 `/`、家目录 `/home`、交换空间与 EFI 系统分区各自的用途，并能对照自己的磁盘布局指出对应位置。安装界面上的设备名、容量、文件系统和挂载点，要结合实际机器核对，不能照搬截图中的盘符与数字。

安装完成的判断也要落到操作上：Ubuntu 和 Windows 都能正常启动，网络与常用外设可用，能够打开终端、安装软件并保存自己的文件。系统扩容、GRUB 引导菜单美化和桌面美化属于选修；前者服务于空间调整，后两者改善使用体验。遇到这类需要时，应回到对应小节查具体步骤，并先确认当前系统与磁盘的状态。

2. 备份与恢复：给环境修改留下退路。本章介绍了 Timeshift 系统快照和用 `tar` 打包的系统备份。快照主要用于在系统配置出错后回到先前状态；归档备份是否完整，则取决于实际纳入的目录、排除规则和保存位置。能够创建备份只是第一步，还应知道备份保存在哪里、包含哪些内容，以及恢复时需要什么环境。个人代码、数据和重要文档也要有明确的备份安排，不能默认系统快照已经包含它们。

卸载重装一节进一步涉及系统分区与引导关系：移除 Ubuntu 的分区之后，还要考虑剩余系统能否正常引导以及磁盘空间如何重新使用。这些维护内容按选修复习，但配置环境前先备份的习惯应保留。能恢复的环境，才适合放心地继续实验。

笔记 格式化、删除分区、系统恢复和覆盖文件都可能造成数据丢失。执行前应确认目标设备与目录，确认重要数据已有可用备份。终端中的 `rm` 删除通常不会进入桌面回收站；不要通过反复执行破坏性命令来试探路径是否正确。

3. 文件系统：用目录、路径和挂载组织资源。Linux 以 `/` 为起点组织目录树，绝对路径从根目录写起，相对路径则依赖当前工作目录。`/home` 通常保存用户文件，`/etc` 保存许多系统配置，`/dev` 提供设备文件入口，`/proc` 暴露进程与内核的相关信息。“一切皆文件”帮助我们用相近的接口访问不同资源，但普通文件、目录和设备文件仍有不同用途，不能随意互换操作。挂载则是把某个文件系统接入目录树：找到挂载点，才能沿路径访问其中的文件。

路径概念会贯穿后两章。源码在哪里、程序从哪里读取图片、构建目录建在哪里，都与当前目录和路径写法有关。遇到“文件不存在”，先核对路径、大小写和工作目录，往往比立刻修改代码更有效。

4. 常用命令：从管理文件到观察系统。本章的命令可以按实际任务串起来：

- **用户与权限：**区分普通用户和管理员操作，读懂文件的所有者、所属组与其他用户权限，使用 `chmod` 调整读、写、执行权限。权限不足时，先判断需要哪一项权限。
- **系统信息与软件安装：**用 `uname` 查看系统信息，用 `free` 观察内存，用 `df` 查看文件系统容量，用 `du` 统计目录占用。通过 `apt` 管理软件包时，要分清更新软件包索引、安装软件和升级已安装软件。
- **文件与目录管理：**用 `pwd`、`cd`、`ls` 定位，用 `mkdir`、`cp`、`mv`、`rm` 完成创建、复制、移动和删除，用 `tar`、`zip` 归档或压缩文件。操作前要能解释源路径与目标路径。
- **链接：**硬链接为同一文件提供另一个名字，软链接保存通向目标的路径。复习时应能解释为何目标被移走后软链接可能失效，以及为什么删除一个硬链接不等于立即删除所有文件内容。
- **查找与处理输出：**`find` 按文件名、类型等条件查找文件，`grep` 按模式筛选文本；管道把前一条命令的标准输出交给后一条命令，重定向把输入或输出接到文件等位置。要分清覆盖写入与追加写入。
- **阅读与编辑：**用 `cat`、`head`、`tail` 等查看内容，利用 Vim 的模式切换完成输入、保存与退出。阅读配置文件和构建日志，是这些操作最常见的用途。

AstroNvim 是本章介绍的选修编辑环境。它提供更完整的终端编辑体验，使用它时仍会用到文件定位、Vim 操作和配置管理这些基础。常用命令应练到能够独立完成任务；不常用的参数，通过参数表和手册查阅即可。

5. Shell 与环境变量：把一次操作变成可以重复的流程。终端提供交互界面，Shell 解释并执行命令；Bash 是本卷主要使用的 Shell。脚本把一系列命令保存为文件，第一行的解释器声明用于直接执行脚本时选择解释器。直接运行 `./test.sh` 需要执行权限；写成 `bash test.sh` 则是显式让 Bash 读取脚本。应能把已经手动验证的准备目录、启动程序等操作整理成简单脚本，并解释每一步的效果。

环境变量影响进程如何寻找程序与读取配置。`PATH` 保存命令搜索目录，追加路径时要保留原有值。在终端中设置变量、写入用户的 `.bashrc`、修改系统级配置，生效对象与时机不同。还要分清“启动一个子进程执行脚本”和“用 `source` 在当前 Shell 中读取脚本”：前者对环境的修改不会自动带回父 Shell。Zsh、Oh My Zsh 与提示符美化属于选修，切换 Shell 后也应检查使用的是哪份配置文件。

6. Git：把修改变成有来由、可协作的版本。 Git 的主线是工作区、暂存区、本地版本库与远程仓库之间的流转。编辑文件改变工作区，`git add` 选择本次准备提交的内容，`git commit` 在本地形成版本，`git push` 把提交发送到远程。`git clone` 用于取得仓库，`git pull` 用于获取并整合远程更新。提交前查看状态与差异，能帮助我们确认提交的是预期修改；提交说明则帮助队友和未来的自己读懂修改目的。

提交者信息用于标记版本作者，HTTPS 令牌或 SSH 密钥用于远程访问认证，二者承担不同职责。`.gitignore` 规定哪些未跟踪文件应被忽略，适合排除构建产物等本地文件，但不会自动取消对已跟踪文件的管理。分支用于组织不同开发线路，合并与变基用于整合修改，子模块用于在工程中引用独立仓库。日常提交与同步需要熟练；阅读分支历史、处理冲突和使用子模块时，要先认清当前状态，再查相应操作，避免凭命令名称猜测效果。

7. Conda（选修）：把不同项目的依赖分开。 Conda 一节介绍安装配置，以及环境的创建、激活、

退出和删除。它让不同项目能够使用各自的 Python 与软件包组合，减少依赖互相影响。使用时要先确认当前激活的是哪个环境，再安装或运行软件；环境激活还可能改变 `PATH`，因此它与前面的环境变量知识直接相关。后续进入深度学习等专项时，这种按项目管理环境的习惯会继续发挥作用。

二、现代 C++ 基础：组织数据、行为、资源和并发任务

1. 类与对象：把相关数据和操作放在一起。类描述一类对象具有哪些数据和行为，对象是具体实例。数据成员保存状态，成员函数提供操作，构造函数建立初始状态，析构函数处理对象结束生命周期时的清理。`public`、`private` 与 `protected` 用来划定访问边界：调用者使用公开接口，类内部维护自己的状态。复习时可以拿装甲板对象举例，说明哪些数据需要保存、哪些操作应由成员函数完成，以及哪些状态不应让外部任意修改。

静态成员与类本身相关，友元为指定函数或类提供访问权限，`this` 指向当前对象。函数重载通过不同参数列表提供同名操作，不能只靠返回类型区分；运算符重载则让自定义类型支持相应运算。构造函数的初始化列表、拷贝构造和赋值也要分清：创建一个新对象，与修改一个已经存在的对象，是不同的时机。

2. 继承、多态与模板：让接口和代码能够复用。继承建立基类与派生类的关系；虚函数允许通过基类指针或引用调用实际对象对应的实现。`override` 帮助编译器检查重写关系，纯虚函数用于约定需要派生类完成的接口，抽象类不能直接实例化。这里应能区分“同名函数的重载”和“派生类对虚函数的重写”，并通过调用结果验证自己的判断。

函数模板和类模板把类型等信息作为参数，使同一份代码适用于不同实例。阅读容器、智能指针和通用函数时，都需要这种视角。本卷要求能写简单模板、识别实例化与类型推导；复杂模板技巧留到实际需要时继续学习。

3. Lambda：把一小段行为放到使用它的位置。Lambda 表达式适合排序比较器、回调和线程任务等场景。捕获列表说明如何取得外部状态，参数列表说明调用者需要传入什么，函数体描述具体工作；必要时可用 `std::function` 保存符合指定调用形式的可调用对象。泛型 Lambda、初始化捕获和 `this` 捕获，扩展了这种写法的适用范围。

值捕获保存副本，引用捕获访问原对象。选择捕获方式时，除了考虑是否要修改原变量，更要考虑对象能活多久：若回调在线程或队列中稍后执行，被引用的局部变量可能已经销毁。能够写出 Lambda 之后，还应能说清它依赖哪些外部对象，以及执行时这些对象是否仍然有效。

4. 引用、拷贝与移动：说明函数怎样使用对象。引用为已有对象提供别名。按值传参会形成参数对象，引用传参可以直接访问原对象；`const T&` 常用于只读访问并避免不必要的拷贝。左值与右值描述表达式的类别，左值引用和右值引用的绑定规则则影响重载选择与资源转移。尤其要记住：**有名字的右值引用变量，在表达式中使用这个名字时仍是左值。**

`std::move` 进行类型转换，为选择移动操作创造条件，本身不搬运数据；实际是否转移资源，要看随后调用的构造函数或赋值操作。仅仅把一个引用绑定到转换后的表达式，并不等于原对象的资源已经被取走。`std::forward` 用于模板转发，按照推导出的类型保留实参的值类别；它与无条件表达移动意图的 `std::move` 各有用途。复习正文中的输出示例时，应逐次判断发生的是构造、拷贝、赋值、移动，还是单纯建立别名。

5. 智能指针：把资源释放责任写进类型。资源获取即初始化（RAII）把资源的持有与对象生命周

期联系起来；智能指针用这种方式管理动态对象，减少遗漏释放与重复释放的风险。三种指针的关键区别在于所有权：

- `unique_ptr` 表达独占所有权，不能拷贝，可以移动。一个明确的持有者负责对象生命周期时，优先考虑它。
- `shared_ptr` 表达共享所有权，通过引用计数管理对象；最后一个共享所有者释放后，对象才会销毁。
- `weak_ptr` 观察共享对象而不增加强引用计数，使用前通过 `lock` 尝试取得有效的共享指针，适合打破循环引用。

通常通过 `make_unique`、`make_shared` 创建相应对象。两个对象互相持有 `shared_ptr`，可能让引用计数始终无法归零；把合适的一方改成不拥有资源的观察关系，才能解决所有权环。另外，共享所有权不代表所指向的数据可以被多个线程任意同时修改，数据同步仍需要单独设计。

6. STL：根据访问方式选择容器。标准模板库（STL）把常用数据结构、迭代器与算法组合起来。选择容器之前，先考虑数据如何进入、如何查询、如何删除，以及是否需要顺序：

- **顺序容器：**`array` 适合固定大小数据，`vector` 提供连续存储和按下标访问，`deque` 支持两端操作，`list` 等链式容器体现节点连接的组织方式。`vector` 的大小与容量不同，扩容可能重新分配存储。
- **容器适配器：**`queue` 对应先进先出，`stack` 对应后进先出，`priority_queue` 每次提供当前优先级最高的元素。它们限制访问方式，不提供普通容器那样的迭代器遍历接口。
- **树、堆与映射：**树的层次结构、堆的优先级关系帮助解释相应数据结构的行为。`map` 保持键有序，`unordered_map` 基于哈希组织键值对；查找复杂度、遍历顺序和实际开销都需要结合任务考虑。

按键访问时也要留意操作是否修改容器：例如映射的下标操作可能插入不存在的键，只查询时可使用 `find` 或 `count`。应熟练操作 `vector`，能够为排队、撤销、优先处理和按编号查询等需求选择合适的容器。

7. 迭代器与算法：在修改容器时保持访问有效。迭代器把容器与算法连接起来。常见遍历范围是左闭右开的 `[begin(), end())`，`end()` 表示尾后位置，不能解引用。插入、删除或重新分配存储后，原有迭代器、指针或引用可能失效，具体规则取决于容器与操作。例如 `vector` 重新分配存储后，指向原存储的迭代器都会失效；删除元素时，删除位置及其后的迭代器会受到影响。

遍历删除时可以用 `erase` 的返回值继续前进；批量删除时，要分清 `remove_if` 调整逻辑范围与 `erase` 真正改变容器大小的作用。排序时将 Lambda 用作比较器，就把本章的行为表达与容器操作连在了一起。这部分的掌握标准是能够解释一次修改后哪些访问仍有效，而不只是记住某段循环写法。

8. 多线程：让多个任务有序地共享资源。线程共享所属进程的资源，并发表示多个任务交错推进，并行则要求它们确实同时执行。创建 `std::thread` 之后，应安排好线程的结束与回收；`join` 等待线程完成，`detach` 解除线程对象与执行线程的关联，后者尤其需要谨慎处理被访问对象的生命周期。多个线程访问共享数据，只要涉及冲突的读写，就需要考虑同步。

互斥锁保护共享状态，`lock_guard` 等 RAII 锁把解锁与作用域绑定。共享锁允许多个读者同时进入，写者则需要独占；原子变量适合相应的单次原子操作，但多个原子操作组合起来不自动成为一个不可分割的整体。条件变量用于等待某个状态发生变化，等待时应检查谓词，并由互斥锁保护相关共享状态。

`promise` 提交异步结果，`future` 获取结果或异常，`async` 提供按启动策略执行任务并取得结果的方式。线程池进一步组合工作线程、任务队列、锁和条件变量，让已有线程反复处理任务；关闭时必

须安排停止接收、唤醒等待者和回收线程等步骤。这些工具分别解决互斥访问、等待条件和传递结果的问题，不能把“加一把锁”当成所有并发问题的统一答案。

生产者—消费者说明队列非空、非满时怎样协调，读者—写者说明共享访问与公平性的取舍，哲学家进餐说明多个资源之间可能形成循环等待。死锁的四个必要条件是互斥、持有并等待、不可抢占和循环等待。统一获取顺序或使用合适的多锁获取工具，是本卷介绍的常用处理思路。复习时应能在代码中指出共享状态、保护它的锁、等待条件，以及线程退出的路径。

9. 内存与生命周期：为前面的错误建立解释。结合本章涉及的内存基础，还应建立常见进程内存布局的图景：代码与只读数据区域、静态存储数据、动态分配区域，以及线程调用栈。栈帧保存一次函数调用需要的信息，局部对象的生命周期结束后，指向它的指针或引用可能悬垂；过深递归或过大的栈上对象可能耗尽栈空间。动态对象的释放则取决于相应的资源管理机制。具体布局由实现与系统决定，不能仅凭变量名字判断它位于哪一块内存。把这些知识与引用捕获、智能指针、容器扩容和线程退出联系起来，才能解释为什么有些程序“能编译，运行却不稳定”。

三、编译过程与 CMake：把源码组织成可以交付的程序

1. 编译过程：区分每一步的输入与产物。本章先用分步命令展示源码如何成为可执行文件：预处理处理包含与宏等指令，编译生成汇编代码，汇编生成目标文件，链接组合目标文件与所需库。常见中间文件依次为`.i`（C++ 常用`.ii`）、`.s` 和`.o`；`g++ -c` 停在生成目标文件之后，不进行最终链接。声明告诉编译器接口是什么，定义提供实际实现；头文件被找到，只能说明相关声明等内容可见，并不代表对应实现已经参与链接。

静态库把目标文件组织成归档，链接器按需要取用其中内容；动态库则还涉及程序运行时的装载与查找。因此，编译通过、链接通过和运行成功是不同检查点。理解它们的区别，才能正确解释后面的构建错误。

2. CMake 工作流：配置、构建与安装各有职责。 CMake 根据 `CMakeLists.txt` 描述生成构建系统，再由底层构建工具调用编译器与链接器。`cmake -S/-B` 分别指定源码目录与构建目录，配置成功只表示构建规则已经生成；`cmake --build` 执行构建，`cmake --install` 按安装规则放置产物。源码目录与构建目录分开，便于保留干净源码，也便于使用不同构建配置。

`cmake_minimum_required` 声明最低版本要求，`project` 声明项目及使用语言。项目名与可执行目标名可以不同；Debug、Release 等构建类型影响相应配置，修改配置或源码之后还需要重新构建，才能运行更新后的程序。缓存变量会保存在构建目录中，排错时也要检查正在使用哪一个构建目录和配置。

3. 目标与依赖：明确生成什么、需要什么。 `add_executable` 创建可执行目标，`add_library` 创建库目标；加入目标的源文件决定哪些实现参与构建。`target_include_directories` 提供头文件搜索目录，`target_link_libraries` 描述链接依赖，编译特性与编译定义则分别表达语言要求和宏等配置。目标名代表构建系统中的实体，变量展开代表读取变量内容，二者不能混用。

现代 CMake 把配置尽量附着到具体目标，再通过依赖关系传播使用要求。`PRIVATE` 供当前目标使用，`PUBLIC` 供当前目标与下游使用，`INTERFACE` 供下游使用。这些作用域描述构建配置的使用范围，与 C++ 类的成员访问权限并不是同一套机制。应能对着一个库说明：哪些头文件是公开接口，哪些只供内部实现使用，使用这个库的程序需要继承哪些配置。

4. 配置与安装：让构建描述能够重复使用。变量、列表、条件分支和选项用于表达可变化的构建配置；CMake 变量要通过相应命令传给编译器，才会影响 C++ 中的条件编译。`function`、`include` 与

模块搜索路径帮助拆分并复用 CMake 逻辑，打印变量可以检查配置阶段实际拿到了什么值。这些补充内容的目标是读懂已有构建文件，并在需要时组织自己的配置。

安装规则决定产物的目录布局，安装前缀决定相对安装路径从哪里开始计算。例如相对目的地 `bin` 会放在所选安装前缀下；构建目录中的程序与安装目录中的程序，需要分清正在运行的是哪一个。安装步骤不能代替事先完成构建。

5. 外部依赖与实例：把自己的代码接到现有库上。OpenCV 示例把查找依赖、创建目标、配置包含目录、链接库和运行验证串成了一次完整练习。`find_package` 查找已有包的信息，并不会自动替你安装依赖；自行安装 OpenCV 时，`OpenCV_DIR` 应指向包含包配置文件的目录。Python 中安装的 OpenCV 包不能直接替代这个 C++ 示例所需的开发包。

图像查看器既要验证正常输入，也要检查未传参数、图片不存在等情况；窗口显示还依赖可用的图形环境。章末 ROS2 片段是选读材料，用于识别模块目录、库目标和依赖关系；它依赖原工程与 ROS2 环境，不能作为独立示例复制运行。ament 的具体约定与图像算法分别留到卷二相应章节继续学习。

6. 排错：先定位失败阶段，再检查对应对象。

- **配置失败：**检查工具、包是否可用，以及包配置文件路径与缓存设置。
- **编译失败：**检查首个有效诊断涉及的语法、类型、头文件与包含目录。
- **链接失败：**检查函数是否有定义、签名是否一致、实现文件或库是否参与链接，以及是否存在重复定义。
- **运行失败：**检查输入文件、工作目录、动态库搜索与运行环境；崩溃或结果异常还需要回到对象生命周期、边界访问与并发逻辑。

需要了解工具实际执行了什么时，可查看详细构建输出。增量构建只更新受修改影响的产物，因此“没有重新编译所有文件”通常是正常行为；关键是能说明这次修改影响哪些目标，并运行验证结果。

四、把三章连起来：一次修改怎样成为可验证的成果

假设现在接到一个练习任务：读取一组模拟目标数据，筛选并排序，最后输出结果。可以沿着下面的顺序完成，并逐步增加复杂度：

1. **准备环境与取得代码：**确认系统、工具和工作目录可用，用 Git 取得练习工程，查看说明与依赖要求，确认开始修改前的仓库状态。
2. **设计数据与接口：**用类表达目标数据，用容器管理集合，用函数或 Lambda 表达筛选、排序规则；在接口处说明是读取、修改、复制还是转移对象。
3. **明确生命周期：**判断普通对象与引用是否足够，需要动态资源时再选择合适的智能指针；确认回调引用的对象不会提前销毁。
4. **组织与构建工程：**把声明、实现与入口按职责放入文件，使用 CMake 创建目标、加入源文件并设置依赖，完成配置与构建。
5. **运行与定位问题：**先验证单线程的正常输入和边界情况，保存结果。确有并发需要时，再加入任务分工与同步，并检查线程能否正常退出。
6. **整理与交付：**把重复操作写成脚本，记录依赖与构建运行方式，检查 Git 差异，提交源码、构建文件与必要说明，并按协作约定同步。

Linux 管理文件与执行环境，C++ 决定程序行为，CMake 描述构建关系，Git 保存修改的历史。这四部

分共同支撑可复现的开发：换一个干净的构建目录，甚至换一位队友，仍能按说明得到一致的预期结果。

五、卷末自查：用已有工程证明自己掌握了什么

各章习题用来检查具体知识点，卷末则适合带着已有练习工程做一次综合演示。复习时可以逐项记录“能独立完成”“查阅后能完成”或“仍需重做”，把下一轮练习放在真正薄弱的位置。

1. **环境与文件**：在 Ubuntu 中独立定位工程、创建与复制文件、调整脚本权限，解释常见目录与当前工作路径；能够查看磁盘和内存使用情况。
2. **命令组合**：查找指定文件，从日志中筛选信息，将结果保存到文件，并解释管道与重定向各自处理了什么。
3. **脚本与变量**：运行自己的准备脚本，说明解释器、执行权限和环境变量的生效范围；能判断新终端与当前终端的配置是否一致。
4. **版本管理**：展示一次修改从差异检查、暂存到提交的过程，说明哪些文件应纳入版本库，哪些属于本地构建产物。
5. **C++ 表达**：在已有代码中指出类的接口、构造与析构、模板和 Lambda，解释一次参数传递是否发生拷贝，以及移动操作是否真的被调用。
6. **资源与容器**：解释一处对象由谁持有、何时销毁，说明容器的选择依据，并检查一段删除或插入代码是否继续使用了失效的迭代器。
7. **并发协作**：在多线程练习中指出共享数据、同步方式、等待条件和退出流程，能够解释遗漏同步或改变加锁顺序会带来什么问题。
8. **构建与复现**：在新的构建目录中重新配置、构建并运行多文件工程，按已有规则安装产物；展示一次错误的定位过程和修复后的验证结果。

备份恢复、扩容、重装、桌面与终端美化、AstroNvim、Conda 等选修内容不计入通识考核，遇到实际需求时再回查与练习。函数参数不必全部背下，复杂模板与进阶并发也需要后续积累；应当练熟的是常用操作，并能说清代码、资源与构建配置之间的关系。参考题解放在本卷末尾，建议先独立作答或运行，再对照检查推理与操作遗漏。

六、带着这些基础进入专项学习

完成本卷后，可以进入**卷二 · 专项技术栈**，按任务需要学习 ROS2、OpenCV、深度学习、估计与控制、导航等内容。阅读 ROS2 节点时，会再次见到类、回调、智能指针和线程；处理图像时，会继续面对数据存储、对象共享与计算开销；接入外部库时，会继续使用依赖查找和目标链接。自瞄、雷达与导航的具体任务不同，但都需要能运行、能排错、能协作修改的工程基础。

卷三 · RM 视觉概述再把技术放回比赛场景，讨论系统链路与工程准则。学习专项时，可以随时带着实际问题回查本卷：找不到程序就查环境与路径，数据失效就查生命周期，构建失败就查目标和依赖。当你能独立复现一个小工程、解释关键选择，并留下队友可以继续工作的代码与说明时，本卷的学习目标就落到了实处。

参考题解

先自己试一遍，再来核对。答案写法不止一种：能解释思路、跑出正确结果、处理题目里的边界情况，就算做到了。下面按原题号整理；“随堂练习 1.1”和“第一章章末第 1 题”是两套编号，别串台。操作题以你的实际记录为准，不用把参考容量、用户名或设备名照抄到自己电脑上。

第一章：Linux 工作环境

随堂练习

随堂练习 1.1：规划分区 解先记录磁盘编号、各分区容量、剩余空间和准备压缩出的空间。例如，已有 Windows 的 UEFI/GPT 电脑，可以预留约 200 GiB：给 `/` 分配约 80 GiB，其余给 `/home`。这只是便于讨论的方案，容量要按你的磁盘和项目需求调整。交换空间可以使用交换文件；若单独建交换分区，应选择“交换空间”，它没有 `/swap` 这个挂载点。UEFI 安装还要确认 EFI 系统分区（ESP），通常复用已有分区并挂载到 `/boot/efi`，保留其中的 Windows 引导文件。

笔记这道题只要求记录和规划，不需要真的格式化。`/boot` 和 EFI 系统分区不是同一个概念；GPT 也不按旧式 MBR 的主分区、逻辑分区规则来划分。动手安装前，先备份文件，再核对磁盘、分区和格式化勾选项。分区表不是试错用的草稿纸。

随堂练习 1.2：记录启动顺序 解进入 BIOS/UEFI 设置，找到 Boot 页面，记录 `Windows Boot Manager`、`ubuntu`、USB 启动项等的先后顺序。再记录调整顺序的按键或菜单位置，最后选择不保存退出。不同机器菜单不同，检查标准是：你能找到启动项，能指出如何把 Windows 的启动项放到首位，并且这次没有误改设置。

随堂练习 1.3：桌面插件（选修） 解按正文步骤安装 Extension Manager，在其中查找并启用与当前 GNOME 版本兼容的 Caffeine 和 Clipboard Indicator。验证时分两步：先记下自动熄屏时间，启用 Caffeine，等待超过该时间；再复制三段普通文字，从剪贴板历史中重新选择第一段并粘贴。分别记录插件状态和验证结果。没有兼容版本时，记录版本及提示，不把“能搜到插件”当作“已经启用”。剪贴板测试用普通文字就好，不要用密码。

随堂练习 1.4：创建快照（选修） 解先在 Timeshift 中选好快照类型和保存位置，再创建快照；也可在完成初始配置后执行：

```
sudo timeshift --create --comments "before-training" --tags O
sudo timeshift --list
```

列表中应出现这次快照的时间、名称及注释。记录保存位置，并确认创建过程没有报错。这道题不要求执行恢复；系统快照也不能代替个人文件备份。

随堂练习 1.5：tar 参数（选修） 解 `-c` 创建归档；`-v` 显示处理的文件；`-p` 在解包时尽量恢复记录的权限；`-z` 使用 gzip 压缩或解压；`-f` 指定归档文件。注意，`-p` 不是“压缩得更完整”：归档本身会记录常规权

限信息，能否完整恢复还受当前用户权限和文件系统限制。-f 后面要跟归档文件名，别把要备份的目录误放在这个位置。

随堂练习 1.6：根目录和设备文件 解

```
ls /
ls /dev
ls -l /dev/null /dev/zero
```

/etc 保存许多系统级配置；/home 通常保存普通用户的家目录；/dev 提供设备文件。上面两项是字符设备，权限字符串的第一位为 c；磁盘等块设备通常以 b 开头。设备名要看本机输出，不是每台电脑都有 /dev/sda。

随堂练习 1.7：查看根分区 解

```
df -hT
df -hT /
```

第二条直接查看根目录所在的文件系统。按列记录文件系统、类型、总容量、已用空间、可用空间和挂载点。例如类型可能是 ext4，挂载点为 /；具体数值以你的输出为准。tmpfs、其他分区的容量不要抄进根分区这一行。

随堂练习 1.8：安装和卸载软件 解以 cowsay 为例：

```
apt search cowsay
sudo apt install cowsay
/usr/games/cowsay "Hello,␣RoboMaster!"
sudo apt remove cowsay
```

应先看到字符画输出，再看到卸载成功的提示。使用完整路径可以避开 /usr/games 未加入 PATH 的情况。remove 一般保留配置文件，本题不要求进一步清理所有依赖或配置。

随堂练习 1.9：文件操作 解确认家目录里没有需要保留的同名 practice 目录，再执行：

```
mkdir -p "$HOME/practice/sub"
touch "$HOME/practice/hello.txt"
cp "$HOME/practice/hello.txt" "$HOME/practice/hello_bak.txt"
mv "$HOME/practice/hello.txt" "$HOME/practice/sub/"
ls -R "$HOME/practice"
rm -ri -- "$HOME/practice"
```

删除前应看到根目录里的备份文件，以及 sub 中的原文件。删除后用 test ! -e "$HOME/practice" 检查，退出码应为 0。



笔记最后一条会递归删除练习目录，-i 会逐项确认。先核对路径和内容，再回答确认提示。若同名目录原来就有资料，请换一个练习目录名。

随堂练习 1.10：查找最近修改的 Python 文件 解

```
find "$HOME" -type f -name '*.py' -mtime -7
```

`-type f` 限定普通文件，单引号让 `*.py` 交给 find 匹配，`-mtime -7` 选择修改时间距今不足七个 24 小时的文件。没有输出也可能是没有匹配项。可以先在练习目录里建一个 `check.py`，再运行命令验证。

随堂练习 1.11：管道筛选和计数 解

```
apt list --installed | grep '^[^/]*python[^/]*/'
apt list --installed | grep '^[^/]*python[^/]*/' | wc -l
```

软件包名在第一个斜杠前，这个匹配方式只检查包名是否含有 python。第一条列结果，第二条统计结果行数。apt 可能在标准错误输出提示命令行接口并不稳定；这条提示不会作为软件包行传给管道。脚本中做严格包查询时，可改用 dpkg-query。

随堂练习 1.12：Vim 保存与放弃修改 解运行 `vim note.txt`，按 `i` 进入插入模式，输入三行内容。按 Esc，输入 `:wq` 并回车，再用 `cat note.txt` 检查。第二次打开后，在普通模式输入 `2Gdd` 删除第二行，再输入 `:q!` 放弃修改并退出。再次用 cat 查看，文件仍应有原来的三行。`:q!` 放弃的是本次未保存的编辑，不会撤销上次已经保存的内容。

随堂练习 1.13：AstroNvim 搜索（选修） 解按正文步骤安装，在项目根目录启动 `nvim`，等待插件初始化完成。先按空格、`f`、`f`，输入文件名的一部分，选中文件并打开；再按空格、`f`、`w`，搜索文件中确实存在的一段文字，并跳到结果行。记录打开的文件名、搜索词和跳转行。若快捷键随版本变化，按空格查看当前菜单；文字搜索失败时先检查项目目录和 `rg` 是否可用。外观不是检查重点，能找到文件、找到内容才算跑通。

随堂练习 1.14：运行脚本 解在独立练习目录保存下面的 `prepare.sh`：

```
#!/usr/bin/env bash
set -e
mkdir -p mydir
touch mydir/a.txt
ls mydir
```

在同一目录分别运行：

```
chmod +x prepare.sh
./prepare.sh
bash prepare.sh
```

两次都应列出 `a.txt`。第一种需要执行权限，由首行指定解释器；第二种由你显式调用 Bash，脚本需要可读，但不要求执行权限。脚本中的相对路径以运行时的工作目录为准。

随堂练习 1.15：别名的生效和清理 解在 ~/.bashrc 加入 alias cls='clear'，然后运行：

```
source ~/.bashrc
type cls
cls
```

type cls 应显示该别名。练完删掉配置文件里新增的行，再在当前终端执行 unalias cls。仅删除文件中的那一行再 source，并不会自动删除当前 Shell 已有的别名。新开一个正常读取该配置的终端，确认它也不再定义 cls。

随堂练习 1.16：PATH 的作用范围 解在 Bash 中记录旧值，再追加目录：

```
old_path=$PATH
mkdir -p "$HOME/mybin"
export PATH="$PATH:$HOME/mybin"
printf '%s\n' "$PATH"
```

从桌面终端启动器新开一个独立终端，通常不会有这次追加的目录，因为原终端的修改没有写入启动配置。但如果你从已修改的终端启动子 Shell，例如直接运行 bash，子进程会继承 PATH，修改仍然可见。所以“新开终端一定失效”要看它由谁启动。最后在原终端执行 export PATH="$old_path" 恢复旧值。

随堂练习 1.17：迁移 zsh 配置（选修） 解按正文完成安装，确认 ~/.zshrc 中的主题是 p10k，插件列表包含 zsh-autosuggestions。只迁移自己添加的、兼容 zsh 的配置，例如 alias cls='clear'，不要整份复制 .bashrc。新开 zsh 后，用 type cls 检查别名；执行一次普通命令，再输入相同前缀，应该出现历史建议，按右方向键可接受。提示符图标应能正常显示。若仍是方框，检查终端字体；不要为了一个图标把整个 Shell 重装一遍。

随堂练习 1.18：提交和比较差异 解为了不打扰真实项目，建一个独立仓库：

```
mkdir git-answer-lab
cd git-answer-lab
git init
git config user.name "Training␣Student"
git config user.email "student@example.invalid"
printf 'first\n' > note.txt
git add note.txt
git commit -m "Add␣first␣line"
printf 'second\n' >> note.txt
git add note.txt
git commit -m "Add␣second␣line"
git log --oneline -2
printf 'third\n' >> note.txt
git diff
```

日志应有两次提交，差异中应出现新增的 third。仓库配置只用于这个练习目录。git diff 默认比较工作区与暂存区；若再次 add，这部分差异会转到 git diff --cached 中。

随堂练习 1.19：忽略规则与例外 解 .gitignore 写为：

```
*.log
/build/*
!/build/keep.txt
```

/build/* 忽略根目录 build 内的内容，随后取消对 keep.txt 的忽略。不要先写 /build/ 把整个父目录排除，否则 Git 不会继续遍历它来找这个例外。在练习仓库验证：

```
mkdir -p build
touch run.log build/cache.tmp build/keep.txt
git status --short --untracked-files=all
git check-ignore -v run.log build/cache.tmp
git check-ignore -v --no-index build/keep.txt
```

status 应显示 build/keep.txt，不显示另外两项。最后一条可能打印以 ! 开头的例外规则，这表示取消忽略；不要只凭“有输出”就判断被忽略。已经被 Git 跟踪的文件，不会因为新增规则自动停止跟踪。

随堂练习 1.20：Conda 环境（选修） 解确认没有需要保留的同名环境，再执行：

```
conda create --name rm-answer-py310 python=3.10
conda activate rm-answer-py310
conda install numpy
python --version
conda list numpy
python -c "import␣numpy;␣print(numpy.__version__)"
conda deactivate
conda remove --name rm-answer-py310 --all
conda env list
```

应看到 Python 3.10、numpy 的版本信息；删除完成后，环境列表中不再有该环境。



笔记 --all 会删除指定环境的全部包，确认环境名无误再执行，不要把平时用的环境拿来交作业。

新增随堂练习题解（第1.8节） 解 CLion 练习：将 bonus 改为 20 后，输出应为 total = 80。在加法行暂停时 current 为 60、bonus 为 20，result 尚未完成初始化；执行该行后 result 为 80。恢复 bonus 为 15 后重新构建，输出恢复为 75。

VS Code 练习：把光标放在 main 中 bonus 的声明或引用上，执行 Rename Symbol（默认 F2），输入 reward。该局部变量的声明与调用处引用一起变化；函数形参 bonus 是另一个作用域的符号，不应被同步改名。程序仍输出 75。撤销后保存，再检查构建和输出。

随堂练习1.23：识别串口设备 解实际设备名、所属组与链接以插拔日志及 ls -l 输出为准。队内 USB CDC ACM 设备通常呈现为 ttyACM0 等节点，编号可能因设备发现顺序改变；有唯一标识时可优先使用对应

的 /dev/serial/by-id/ 链接，或按下一题配置 udev 别名。设备不存在先查连接、驱动、虚拟机 USB 直通及路径；权限不足先对比设备权限、所属组与当前会话的组。Ubuntu 中常见的处理是加入 dialout 后完整注销并重新登录，而不是放宽为所有用户可写。

随堂练习1.24：绑定 ACM 固定名称 解先运行 udevadm info --name=/dev/ttyACM0 --attribute-walk，从目标板的同一个 USB 父设备块中记录 VID、PID 和唯一序列号。规则模板如下，三个大写占位值必须用实测值替换：

```
SUBSYSTEM=="tty", KERNEL=="ttyACM*", \
  ATTRS{idVendor}=="YOUR_VID", ATTRS{idProduct}=="YOUR_PID", \
  ATTRS{serial}=="YOUR_SERIAL", \
  SYMLINK+="rm_control", GROUP="dialout", MODE="0660"
```

前两个条件限定 tty 子系统与 ACM 节点；VID/PID 匹配型号，序列号区分同型号的板。SYMLINK+= 创建别名，GROUP 与 MODE 设置目标节点所属组及读写权限。规则保存到 /etc/udev/rules.d/99-rm-control.rules 后，重载规则并重新插拔设备：

```
sudo udevadm control --reload-rules
# 先停止串口程序，再拔插目标设备
udevadm settle
readlink -f /dev/rm_control
ls -lL /dev/rm_control
udevadm info --name=/dev/rm_control --query=property
```

换插口后应仍通过该别名查到同一序列号；底层 ACM 编号允许变化。VID/PID 相同的两块板若不匹配唯一序列号，会争用别名。没有唯一序列号时需完善固件标识，或按实际条件使用绑定端口位置的路径，并说明换插口会影响该路径。仅完成规则草稿不能替代硬件验证。

随堂练习1.25：字节与消息边界 解十六进制输出应为 00 41 ff 0a。其中 00 是空字符，41 对应 ASCII 字母 A，ff 不是合法的单字节 UTF-8 字符，0a 为换行，因此不能期待四个可见字符。串口是字节流，数据分批到达或读取超时都可能导致短读，应累计字节并按约定边界解析，不能仅凭一次短读判定丢包。软件回环成功不检验真实设备权限、接线或波特率匹配。

章末习题

章末第 1 题：双系统安装流程 解四步是：准备空闲分区、制作启动盘、从 U 盘安装、完成系统设置。正文采用“其它选项”手动核对分区和挂载点，这样才能清楚指定 Ubuntu 装在哪里，并保留 Windows 的分区。题目中的“必须”针对本书这套手动流程；其他受支持的安装流程也可能使用“与 Windows 共存”等选项。无论走哪条流程，都要核对安装目标，不能误选“清除整个磁盘”。分区细节见随堂练习 1.1 的说明。

章末第 2 题：挂载 解挂载就是把一个文件系统接到现有目录树的某个目录上。之后访问这个目录，就能访问该文件系统的内容。Ubuntu 桌面自动挂载 U 盘时，常见路径是 /media/用户名/卷标；没有卷标时，最后一段可能是其他标识。实际位置可用 lsblk -f 或 findmnt 查看。

章末第 3 题：统计一层磁盘占用 解

```
du -h --max-depth=1 .
```

-h 使用易读单位，--max-depth=1 限制显示层级。输出通常包含各子目录和当前目录总计；统计时仍会递归计算子目录里的内容。

章末第 4 题：文件权限与串口访问权限 解三位依次对应所有者、所属组、其他用户。7=4+2+1 表示读写执行，5=4+1 表示读和执行。普通脚本可写成：

```
chmod u=rwx,go=rx run.sh
```

这里用 = 明确设置权限，比在未知原权限上用 + 更准确。若文件原先带有特殊权限位，要精确清除它们可再执行 chmod u-s,g-s,o-t run.sh。

对于题设串口，在普通用户终端运行 sudo usermod -aG dialout "$USER"，完整注销并重新登录，再用 id -nG 检查当前会话的组。-aG 表示追加附加组；不要用 chmod 777 或长期以 root 运行程序来代替正确的组权限配置。

章末第 5 题：查找配置文件并保存 解

```
find /etc -type f -name '*.conf' > "$HOME/conf_list.txt"
```

> 保存标准输出，权限不足等错误仍显示在终端。若确实需要读取受限目录，可在确认权限范围后给 find 加 sudo。本写法查找普通文件；若题意也包括同名符号链接，可去掉 -type f。

章末第 6 题：两种脚本运行方式 解 ./test.sh 请求系统执行这个文件，需要执行权限；对带有有效首行解释器声明的脚本，系统按该声明启动解释器。bash test.sh 是先执行 Bash，再让它读取脚本，因此脚本需要可读，不必有执行权限；此时由 Bash 执行，首行写的其他解释器不会替代它。chmod +x 的作用就是补上执行权限。

章末第 7 题：Git 三个区域 解工作区是你正在编辑的文件；暂存区记录准备放进下一次提交的内容；本地版本库保存已经提交的历史。git add 把当时的文件内容放进暂存区；之后继续编辑，不会自动更新暂存区。git commit 把暂存区里的内容保存为一次本地提交，并不会自动推送到远程。

章末第 8 题：开发工具界面与编译器 解 CLion 的锤子构建目标，绿色三角启动普通运行，小虫带调试器启动；普通运行不会因为设置了断点而暂停。VS Code 的 Explorer 管理文件树，Extensions 管理扩展，Terminal 提供可输入命令的 shell，Output 显示扩展日志。C/C++ 扩展提供代码分析和调试接入，本身不包含编译器；必须另外安装并选中 GCC、MSVC 等工具链。本例还需要 CMake 和相应构建工具。

章末第 9 题：修复 PATH 解 PATH 被覆盖后，Shell 找不到原搜索目录中的外部命令，常见现象是 ls 等报“找不到命令”。Shell 内建命令和使用完整路径调用的程序仍可用。如果只是当前终端改错，优先恢复事先保存的值。没保存时，可以先临时恢复一组常见系统路径：

```
export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

这不是你的原始 PATH 全量备份，Conda、用户工具等路径可能仍需补回。如果错误写进了 ~/.bashrc，用完整路径打开编辑器，例如 /usr/bin/nano ~/.bashrc，删掉错误行或改为：

```
export PATH="$PATH:/opt/app/bin"
```

修正文件后，先恢复当前终端的 PATH，再 source，或者从未受影响的桌面会话打开新终端。反复 source 一份仍有错误的配置，只会把错误再执行一遍。

章末第 10 题：备份脚本与解压验证 解下面把要备份的目录作为参数。只接受家目录内的子目录，并拒绝把备份输出目录一起打包，避免归档不断包含自身。保存为 backup.sh：

```
#!/usr/bin/env bash
set -euo pipefail
if [[ $# -ne 1 ]]; then
    printf 'Usage: %s <directory-under-home>\n' "$0" >&2
    exit 1
fi
home_root=$(realpath -e -- "$HOME")
src=$(realpath -e -- "$1")
[[ -d "$src" ]] || { printf 'Not a directory\n' >&2; exit 1; }
case "$src" in
    "$home_root"/*) ;;
    *) printf 'Choose a subdirectory under HOME\n' >&2; exit 1 ;;
esac
mkdir -p "$HOME/backups"
dest=$(realpath -e -- "$HOME/backups")
case "$dest/" in
    "$src/"*) printf 'Backup output is inside source\n' >&2; exit 1 ;;
esac
archive=$(mktemp "$dest/$(date +%F)-XXXXXX.tar.gz")
parent=$(dirname -- "$src")
base=$(basename -- "$src")
tar -czf "$archive" -C "$parent" -- "$base"
verify=$(mktemp -d "$dest/verify.XXXXXX")
tar -xzf "$archive" -C "$verify"
diff -r -- "$src" "$verify/$base"
printf 'Verified archive: %s\nExtracted copy: %s\n' "$archive" "$verify"
```

验证方法：

```
mkdir -p "$HOME/backup-answer-source"
printf 'armor=3\n' > "$HOME/backup-answer-source/config.txt"
bash backup.sh "$HOME/backup-answer-source"
```

成功时会打印备份路径和解压目录，文件名包含当天日期，同一天重复运行也不会覆盖上次归档。diff -r 无输出且退出码为 0，表示本例的目录内容一致；它不验证所有扩展属性和权限。备份期间不要修

改源目录，需要一致的在线系统备份时应采用适合的快照方案。验证目录保留下来供你查看，确认无误后再自行清理。

章末第 11 题：USB CDC ACM 与固定别名 解 ACM 是 USB 通信设备类中的抽象控制模型，队内板端直接通过 USB 与电脑通信，Linux 提供 ttyACM 串口接口。USB 转 UART 则在转换芯片另一侧通过 TX/RX 传输 UART 信号。115200、8N1 表示波特率、8 数据位、无校验、1 停止位；UART 每字节再加一个起始位，才得到每字节 10 位的估算。原生 USB ACM 的线路编码配置是否影响行为由固件决定，它不是 USB 总线速率，所以不能直接套用该吞吐上限。

将 A 板绑定为指定别名的规则为：

```
SUBSYSTEM=="tty", KERNEL=="ttyACM*", \
  ATTRS{idVendor}=="YOUR_VID", ATTRS{idProduct}=="YOUR_PID", \
  ATTRS{serial}=="CTRL-A", \
  SYMLINK+="rm_control", GROUP="dialout", MODE="0660"
```

VID/PID 按题意留为占位值，部署时必须替换为目标板的真实值；CTRL-A 是本题给定序列号，不是队内设备的默认值。重载及核验命令见随堂练习 1.24 的答案。先只接 A，再接 B，并交换插入顺序，核对别名始终对应 A 的序列号和正确权限；仅看 readlink 指向哪个编号还不能证明身份正确。程序配置必须改为该别名。

udev 在设备重新出现时重建链接，但旧的已打开句柄不会恢复，程序需处理断线并重新打开。ACM 提供的仍是字节流接口，一次读取可能只得到半条消息或同时读到多条消息，应累计字节并按协议定界。USB 底层的数据包边界不能当作应用层消息边界。

第二章：现代 C++ 基础

下面给出的完整 C++ 程序各自保存为一个文件，不要把多个 `main` 拼到一起。统一用下面的方式编译；把 `answer.cpp` 换成实际文件名即可：

```
g++ -std=c++17 -Wall -Wextra -pthread answer.cpp -o answer
./answer
```

`-pthread` 用于多线程程序，放在本节统一命令里也方便复制。以下程序按 C++17 编写。

随堂练习

随堂练习 2.1：Point 类 解把坐标放进私有成员，构造时初始化，通过成员函数打印。

```cpp
#include <iostream>
class Point {
    double x_, y_;
public:
    Point(double x, double y) : x_(x), y_(y) {}
    void print() const {
        std::cout << '(' << x_ << ",␣" << y_ << ")\n";
    }
};
int main() {
    Point a(1, 2), b(3.5, 4.5);
    a.print();
    b.print();
}
```

应输出 (1, 2) 和 (3.5, 4.5)。打印不修改对象，所以成员函数标为 `const`。

随堂练习 2.2：统计存活对象 解这里统计的是“当前还活着多少个对象”，不是累计创建过多少个。普通构造和拷贝构造都要计数，析构时减回去。

```cpp
#include <iostream>
class Point {
    double x_, y_;
public:
    inline static int count = 0;
    Point(double x, double y) : x_(x), y_(y) { ++count; }
    Point(const Point& other) : Point(other.x_, other.y_) {}
    ~Point() { --count; }
};
int main() {
    std::cout << Point::count << '\n';
    {
        Point a(1, 2);
```

```
        std::cout << Point::count << '\n';
        {
            Point b(3, 4), c = a;
            std::cout << Point::count << '\n';
        }
        std::cout << Point::count << '\n';
    }
    std::cout << Point::count << '\n';
}
```

输出依次为 0、1、3、1、0。inline static 从 C++17 起允许在类内定义这个静态成员。本题只在单线程里计数；多线程同时创建对象时，还需要同步。

随堂练习 2.3：运算符重载 解 + 返回新点，<< 返回流的引用，方便继续输出。

```
#include <iostream>
class Point {
    double x_, y_;
public:
    Point(double x, double y) : x_(x), y_(y) {}
    Point operator+(const Point& other) const {
        return Point(x_ + other.x_, y_ + other.y_);
    }
    friend std::ostream& operator<<(std::ostream& out, const Point& p) {
        return out << '(' << p.x_ << ",␣" << p.y_ << ')';
    }
};
int main() {
    std::cout << Point(1, 2) + Point(3, 4) << '\n';
}
```

应输出 (4, 6)。友元输出函数可以读取私有坐标，但不会因此成为类的成员函数。

随堂练习 2.4：用基类指针调用派生类 解数组存放基类指针，调用虚函数时仍会选择实际对象的实现。

```
#include <iostream>
class Shape {
public:
    virtual ~Shape() = default;
    virtual double calArea() const = 0;
};
class Circle : public Shape {
    double r_;
public:
    explicit Circle(double r) : r_(r) {}
    double calArea() const override { return 3.14 * r_ * r_; }
```

```
};
class Rectangle : public Shape {
    double a_, b_;
public:
    Rectangle(double a, double b) : a_(a), b_(b) {}
    double calArea() const override { return a_ * b_; }
};
int main() {
    Circle circle(1);
    Rectangle rectangle(1, 2);
    Shape* shapes[] = {&circle, &rectangle};
    for (const Shape* shape : shapes) {
        std::cout << shape->calArea() << '\n';
    }
}
```

输出为 3.14 和 2。对象都在栈上，数组只借用地址，不负责释放。基类提供虚析构函数，也为以后通过基类指针管理对象留好接口。

随堂练习 2.5：函数模板推导　解

```
#include <iostream>
template<class T>
T myMax(T a, T b) {
    return a < b ? b : a;
}
int main() {
    std::cout << myMax(2, 5) << '\n';
    std::cout << myMax(1.5, 2.5) << '\n';
    std::cout << myMax<double>(1, 2.5) << '\n';
}
```

输出为 5、2.5、2.5。把最后一行改成 myMax(1, 2.5) 会推导失败：同一个 T 不能同时推成 int 和 double。显式写 myMax<double> 后，两个参数都按 double 处理。

随堂练习 2.6：lambda 排序　解

```
#include <algorithm>
#include <iostream>
#include <vector>
int main() {
    std::vector<int> values{3, 1, 4, 2};
    auto print = [&values] {
        for (int x : values) std::cout << x << '␣';
        std::cout << '\n';
    };
```

```
    std::sort(values.begin(), values.end(),
              [](int a, int b) { return a < b; });
    print();
    std::sort(values.begin(), values.end(),
              [](int a, int b) { return a > b; });
    print();
}
```

先输出 1 2 3 4，再输出 4 3 2 1。比较器用严格比较，不要把 < 随手改成 <=，否则不满足排序要求。

随堂练习 2.7：引用交换与副本交换 解为了在同一程序中对照，按值版本换一个名字，避免同名重载带来歧义。

```
#include <iostream>
void swapVal(int& a, int& b) {
    int temp = a; a = b; b = temp;
}
void swapCopy(int a, int b) {
    int temp = a; a = b; b = temp;
}
int main() {
    int x = 1, y = 2;
    swapCopy(x, y);
    std::cout << x << '␣' << y << '\n';
    swapVal(x, y);
    std::cout << x << '␣' << y << '\n';
}
```

两行分别是 1 2、2 1。按值版本只交换函数里的副本；引用版本操作的就是 main 中的两个整数。

随堂练习 2.8：返回独占指针 解

```
#include <iostream>
#include <memory>
struct A {
    int val;
    explicit A(int v) : val(v) {}
};
std::unique_ptr<A> create(int v) {
    return std::make_unique<A>(v);
}
int main() {
    auto ptr = create(42);
    std::cout << ptr->val << '\n';
}
```

输出 42。这里直接返回临时结果，在 C++17 下不需要复制它。如果先创建局部 unique_ptr 再返回该局部变量，也可以利用返回值优化，或在适用规则下移动所有权；不需要启用拷贝。

随堂练习 2.9：打破循环引用 解两个对象没有互相持有时，离开作用域后计数归零，都会析构。两侧都用 shared_ptr 并互相赋值后，作用域内两个计数都为 2；外部指针销毁后，内部仍互相持有，计数各剩 1，所以没有析构输出。修复版可单独验证：

```cpp
#include <iostream>
#include <memory>
struct B;
struct A {
    std::shared_ptr<B> peer;
    ~A() { std::cout << "destroy␣A\n"; }
};
struct B {
    std::weak_ptr<A> peer;
    ~B() { std::cout << "destroy␣B\n"; }
};
int main() {
    auto a = std::make_shared<A>();
    auto b = std::make_shared<B>();
    a->peer = b;
    b->peer = a;
    std::cout << a.use_count() << '␣' << b.use_count() << '\n';
    if (auto owner = b->peer.lock()) {
        std::cout << "A␣is␣alive\n";
    }
}
```

先输出 1 2，离开 main 时两个析构函数都会执行。weak 指针不增加强引用计数，观察时用 lock() 临时取得所有权，并检查结果。

随堂练习 2.10：观察 vector 容量 解

```cpp
#include <iostream>
#include <vector>
int main() {
    std::vector<int> values;
    for (int i = 1; i <= 10; ++i) {
        values.push_back(i);
        std::cout << values.size() << '␣' << values.capacity() << '\n';
    }
}
```

size 应依次为 1 到 10，capacity 始终不小于 size，且不会每次都增加。



笔记 题目中的“扩容为 2 倍”是常见实现的现象，不是 C++ 标准保证。应记录你使用的标准库实际输出，不要求每个人的容量序列一样。这道题要验证的是扩容发生的时机，不能用一次实验替所有标准库宣布规则。

随堂练习 2.11：括号匹配 解 右括号出现时，先检查有没有可匹配的左括号；处理完后栈必须为空。

```cpp
#include <iostream>
#include <stack>
#include <string>
bool matched(const std::string& text) {
    std::stack<char> pending;
    for (char c : text) {
        if (c == '(') {
            pending.push(c);
        } else if (c == ')') {
            if (pending.empty()) return false;
            pending.pop();
        } else {
            return false;
        }
    }
    return pending.empty();
}
int main() {
    std::string text;
    std::getline(std::cin, text);
    std::cout << (matched(text) ? "matched" : "not␣matched") << '\n';
}
```

(())() 和空串应匹配；)(、(() 应不匹配。不要对空栈调用 pop()。

随堂练习 2.12：单词计数 解 先约定：这里按空白分词，区分大小写，不自动去掉标点。这样输入和预期结果都明确。

```cpp
#include <iostream>
#include <map>
#include <sstream>
#include <string>
int main() {
    std::string line;
    std::getline(std::cin, line);
    std::istringstream input(line);
    std::map<std::string, int> counts;
    std::string word;
    while (input >> word) ++counts[word];
    for (const auto& [key, count] : counts) {
```

```
        std::cout << key << '␣' << count << '\n';
    }
}
```

输入 armor camera armor，依次输出 armor 2、camera 1。默认比较器按字符串字典序排列键，不按出现先后顺序排列。

随堂练习 2.13：安全删除偶数 解错误版本使用了失效迭代器，属于未定义行为（Undefined Behavior）。它可能崩溃，也可能看起来“没事”，不能规定某个错误输出。修复时接住 erase 返回的下一个有效迭代器：

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> values{1, 2, 3, 4, 5, 6};
    for (auto it = values.begin(); it != values.end(); ) {
        if (*it % 2 == 0) it = values.erase(it);
        else ++it;
    }
    for (int x : values) std::cout << x << '␣';
    std::cout << '\n';
}
```

应输出 1 3 5。删除分支里不再额外自增，否则可能跳过元素。

随堂练习 2.14：两个线程的输出 解用锁保证每次输出完整的一项，延时放在锁外，让另一个线程有机会运行。

```
#include <chrono>
#include <iostream>
#include <mutex>
#include <thread>
std::mutex output_mutex;
void print_range(int first, int last) {
    for (int i = first; i <= last; ++i) {
        {
            std::lock_guard<std::mutex> lock(output_mutex);
            std::cout << i << '\n';
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(10));
    }
}
int main() {
    std::thread a(print_range, 1, 5), b(print_range, 6, 10);
    a.join();
    b.join();
```

```
}
```

1 到 10 应各出现一次。同一个线程内仍按自己的顺序输出，但两组之间的交错由调度决定，sleep 不能保证严格轮流执行。

随堂练习 2.15：四线程计数　解正确目标值为 4×50000=200000。锁版本把整个自增放进临界区；原子版本用一次原子读改写。

```
#include <atomic>
#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
template<class F>
void run_four(F work) {
    std::vector<std::thread> threads;
    for (int i = 0; i < 4; ++i) threads.emplace_back(work);
    for (auto& t : threads) t.join();
}
int main() {
    int locked = 0;
    std::mutex mutex;
    run_four([&] {
        for (int i = 0; i < 50000; ++i) {
            std::lock_guard<std::mutex> lock(mutex);
            ++locked;
        }
    });
    std::atomic<int> atomic_count{0};
    run_four([&] {
        for (int i = 0; i < 50000; ++i) ++atomic_count;
    });
    std::cout << locked << ' ' << atomic_count.load() << '\n';
#ifdef DEMO_DATA_RACE
    // 故意保留数据竞争，仅用于讨论错误写法。
    int plain = 0;
    run_four([&] {
        for (int i = 0; i < 50000; ++i) ++plain;
    });
    std::cout << plain << '\n';
#endif
}
```

默认只运行正确版本，应输出 200000 200000。普通 int 的错误分支需额外加 -DDEMO_DATA_RACE 才会编译进程序。它有数据竞争，属于未定义行为，结果没有保证；即使碰巧得到 200000，也不能证明

正确。volatile 同样不能修复这个问题。

随堂练习 2.16：三个读者、两个写者 解延时要放在持锁期间，才能观察到锁被占用时的等待。实际项目应尽量缩短持锁时间。

```
#include <chrono>
#include <iostream>
#include <mutex>
#include <shared_mutex>
#include <thread>
#include <unordered_map>
#include <string>
std::unordered_map<std::string, int> config{{"exposure", 10}};
std::shared_mutex mutex;
void read_config(int& result) {
    std::shared_lock<std::shared_mutex> lock(mutex);
    std::this_thread::sleep_for(std::chrono::milliseconds(20));
    result = config.at("exposure");
}
void write_config(int value) {
    std::unique_lock<std::shared_mutex> lock(mutex);
    std::this_thread::sleep_for(std::chrono::milliseconds(20));
    config["exposure"] = value;
}
int main() {
    int a = 0, b = 0, c = 0;
    std::thread r1([&] { read_config(a); });
    std::thread r2([&] { read_config(b); });
    std::thread r3([&] { read_config(c); });
    std::thread w1(write_config, 12), w2(write_config, 14);
    r1.join(); r2.join(); r3.join(); w1.join(); w2.join();
    std::cout << a << '␣' << b << '␣' << c << '\n';
    std::cout << "final=" << config.at("exposure") << '\n';
}
```

读者之间可以同时持锁，写者与读者、其他写者都互斥。读到的值可能是 10、12 或 14，最终值是最后完成写入的那个值。这里把结果存进各自变量，在线程全部结束后统一打印，避免输出本身影响观察。

随堂练习 2.17：原子编号分配器 解 fetch_add(1) 返回加一之前的值，每次原子操作取得不同的编号。各线程使用自己的结果容器，编号原子化并不会顺便让共享 vector 也变安全。

```
#include <algorithm>
#include <array>
#include <atomic>
#include <iostream>
#include <thread>
```

```cpp
#include <vector>
int main() {
    std::atomic<int> next{0};
    std::array<std::vector<int>, 4> local;
    std::vector<std::thread> workers;
    for (int i = 0; i < 4; ++i) {
        workers.emplace_back([&, i] {
            for (int j = 0; j < 100; ++j) {
                local[i].push_back(next.fetch_add(1));
            }
        });
    }
    for (auto& worker : workers) worker.join();
    std::vector<int> all;
    for (const auto& ids : local) all.insert(all.end(), ids.begin(), ids.end());
    std::sort(all.begin(), all.end());
    bool valid = all.size() == 400;
    for (std::size_t i = 0; i < all.size(); ++i) valid &= all[i] == int(i);
    std::cout << all.size() << '␣' << next.load() << '␣' << valid << '\n';
}
```

应输出 400 400 1。验证排序后恰好是 0 到 399，同时检查了数量、范围和不重复。

随堂练习 2.18：带超时的等待 解带谓词的 `wait_for` 返回 false，表示超时检查时条件仍未满足。下面让消费者最多等待一秒；超时后通知生产者取消等待，并回收两个线程，避免主线程又在 join 上一直等下去。

```cpp
#include <chrono>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <string>
#include <thread>
int main(int argc, char** argv) {
    const int delay_ms = argc > 1 ? std::stoi(argv[1]) : 2000;
    std::mutex mutex;
    std::condition_variable cv;
    bool ready = false, cancelled = false;
    int result = 0;
    std::thread producer([&] {
        std::unique_lock<std::mutex> lock(mutex);
        if (cv.wait_for(lock, std::chrono::milliseconds(delay_ms),
                        [&] { return cancelled; })) return;
        ready = true;
        lock.unlock();
```

```
        cv.notify_all();
    });
    std::thread consumer([&] {
        std::unique_lock<std::mutex> lock(mutex);
        if (!cv.wait_for(lock, std::chrono::seconds(1), [&] { return ready; })) {
            cancelled = true;
            result = 2;
            lock.unlock();
            cv.notify_all();
            std::cout << "timeout\n";
            return;
        }
        std::cout << "ready\n";
    });
    consumer.join();
    producer.join();
    return result;
}
```

传参数 100 时，正常调度下应输出 ready；传 2000 时，应输出 timeout，返回 2。调度和重新加锁可能让实际返回稍晚于一秒，这不是实时系统的硬截止时间。超时也不会自动强杀后台线程；本例用共享状态主动取消生产者的等待。

随堂练习 2.19：异步检查装甲板 解题目没有规定阈值和判定规则，这里约定：整组数据非空，且每个置信度都大于等于 0.5，才算有效。

```
#include <algorithm>
#include <future>
#include <iostream>
#include <vector>
bool is_armor_valid(const std::vector<double>& scores, double threshold) {
    return !scores.empty() &&
           std::all_of(scores.begin(), scores.end(),
                       [threshold](double score) { return score >= threshold; });
}
int main() {
    auto good = std::async(std::launch::async, is_armor_valid,
                           std::vector<double>{0.8, 0.6}, 0.5);
    auto bad = std::async(std::launch::async, is_armor_valid,
                          std::vector<double>{0.8, 0.3}, 0.5);
    std::cout << good.get() << '␣' << bad.get() << '\n';
}
```

应输出 1 0。显式使用 std::launch::async，要求异步执行；输入按值传给任务，不引用已经销毁的局部变量。每个 future 只调用一次 get。

随堂练习 2.20：线程池返回 future 解只改返回类型不能构成重载。这里保留原来的单参数任务接口，再增加“计算函数加一个整数参数”的双参数重载。packaged_task 本身不能复制，用 shared 指针包住它后，就能放进原来的 std::function<void()> 队列。

```
#include <condition_variable>
#include <functional>
#include <future>
#include <iostream>
#include <memory>
#include <mutex>
#include <queue>
#include <stdexcept>
#include <thread>
#include <utility>
#include <vector>
class ThreadPool {
    std::vector<std::thread> workers_;
    std::queue<std::function<void()>> tasks_;
    std::mutex mutex_;
    std::condition_variable cv_;
    bool stopping_ = false;
public:
    explicit ThreadPool(std::size_t count) {
        if (count == 0) throw std::invalid_argument("empty pool");
        for (std::size_t i = 0; i < count; ++i) {
            workers_.emplace_back([this] {
                while (true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(mutex_);
                        cv_.wait(lock, [this] { return stopping_ || !tasks_.empty(); });
                        if (stopping_ && tasks_.empty()) return;
                        task = std::move(tasks_.front());
                        tasks_.pop();
                    }
                    task();
                }
            });
        }
    }
    ~ThreadPool() {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            stopping_ = true;
        }
```

```
        cv_.notify_all();
        for (auto& worker : workers_) worker.join();
    }
    void enqueue(std::function<void()> task) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            if (stopping_) throw std::runtime_error("pool stopped");
            tasks_.push(std::move(task));
        }
        cv_.notify_one();
    }
    std::future<int> enqueue(std::function<int(int)> function, int value) {
        auto task = std::make_shared<std::packaged_task<int()>>(
            [function = std::move(function), value] { return function(value); });
        auto future = task->get_future();
        enqueue([task] { (*task)(); });
        return future;
    }
};
int main() {
    ThreadPool pool(2);
    auto a = pool.enqueue([](int x) { return x * x; }, 6);
    auto b = pool.enqueue([](int x) { return x * x; }, 7);
    std::cout << a.get() << ' ' << b.get() << '\n';
}
```

应输出 36 49。任务在队列锁外执行；析构时处理完已有任务，再退出线程。计算任务抛出的异常会由 packaged task 存进共享状态，在 get 时重新抛出。原来的 void 接口仍要求任务自行处理异常，不能把教学用线程池直接当成完整生产实现。

随堂练习 2.21：队列、死锁与写者等待 解第一问：多个消费者。这里采用一个生产者、三个消费者。取队首和弹出必须在同一把锁内完成；生产结束后也要让消费者把剩余数据取完。

```
#include <algorithm>
#include <array>
#include <condition_variable>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>
class Queue {
    std::queue<int> data_;
    std::mutex mutex_;
    std::condition_variable cv_;
```

```
    bool closed_ = false;
public:
    void push(int value) {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            data_.push(value);
        }
        cv_.notify_one();
    }
    bool pop(int& value) {
        std::unique_lock<std::mutex> lock(mutex_);
        cv_.wait(lock, [this] { return closed_ || !data_.empty(); });
        if (data_.empty()) return false;
        value = data_.front();
        data_.pop();
        return true;
    }
    void close() {
        {
            std::lock_guard<std::mutex> lock(mutex_);
            closed_ = true;
        }
        cv_.notify_all();
    }
};
int main() {
    Queue queue;
    std::array<std::vector<int>, 3> received;
    std::vector<std::thread> consumers;
    for (int i = 0; i < 3; ++i) {
        consumers.emplace_back([&, i] {
            int value;
            while (queue.pop(value)) received[i].push_back(value);
        });
    }
    for (int i = 0; i < 100; ++i) queue.push(i);
    queue.close();
    for (auto& consumer : consumers) consumer.join();
    std::vector<int> all;
    for (const auto& part : received) all.insert(all.end(), part.begin(), part.end());
    std::sort(all.begin(), all.end());
    bool valid = all.size() == 100;
    for (std::size_t i = 0; i < all.size(); ++i) valid &= all[i] == int(i);
    std::cout << all.size() << '␣' << valid << '\n';
}
```

应输出 100 1，说明没有丢失或重复消费。每个消费者分到多少个由调度决定，甚至可能有一个没取到数据；正确性不要求平均分工。本例约定生产者 close 后不再 push。

第二问：四个死锁条件。筷子互斥使用；拿着一根再等另一根是持有并等待；不能强行抢走别人持有的筷子是不可抢占；每个人等下一人的筷子形成循环等待。对同一组互斥锁使用 std::scoped_lock，会采用避免死锁的多锁获取算法，避免各线程永久持有部分锁并互相等待。它不承诺一次硬件原子操作拿齐锁，也不自动解决程序中所有其他锁和资源形成的死锁。

第三问：读者占用导致写者等待。先用一个可控实验，确保写者等到读锁释放：

```
#include <atomic>
#include <chrono>
#include <future>
#include <iostream>
#include <mutex>
#include <shared_mutex>
#include <thread>
int main() {
    std::shared_mutex mutex;
    std::shared_lock<std::shared_mutex> reader(mutex);
    std::atomic<bool> entered{false};
    std::promise<void> attempting;
    auto signal = attempting.get_future();
    std::thread writer([&] {
        attempting.set_value();
        std::unique_lock<std::shared_mutex> lock(mutex);
        entered = true;
    });
    signal.get();
    std::this_thread::sleep_for(std::chrono::milliseconds(200));
    std::cout << "before=" << entered.load() << '\n';
    reader.unlock();
    writer.join();
    std::cout << "after=" << entered.load() << '\n';
}
```

应输出 before=0、after=1。把这个思路扩展为多个读者反复持锁，就可以观察写者在持续读负载下的等待时间。shared_mutex 不保证公平性，但某些实现会优先照顾写者，因此不能要求每台机器都稳定复现无限饥饿。改进思路是缩短读锁时间、复制快照后在锁外处理，或使用明确提供公平策略的读写锁。设计写者优先策略时，可在有写者排队后暂停接纳新读者，也要考虑反过来饿住读者的问题。

章末习题

章末第 1 题：访问权限 解 public 允许类外代码访问；private 只允许该类的成员和友元访问；protected 还允许派生类在相应访问规则下使用。派生类不能直接访问基类的 private 成员，但可以调用基类提供

的 public 或 protected 接口间接使用它们。“不能直接访问”不等于“派生类对象里不存在这部分数据”。

章末第 2 题：返回类型与重载 解不能。`int f(int)` 和 `double f(int)` 参数列表相同，不能仅靠返回类型区分重载。对这样的两个声明，编译器应报冲突。可以改变参数类型或参数个数来设计重载。

章末第 3 题：lambda 捕获 解简化形式为 `[捕获列表](参数列表) -> 返回类型 {函数体}`，能推导时可以省略返回类型。对普通整数 a，`[a]` 保存副本，默认不能修改这个副本；加 mutable 后可以改副本，但原整数不变。`[&a]` 引用原变量，若 a 本身可修改，就能改变原整数。验证如下：

```
#include <iostream>
int main() {
    int a = 1;
    auto by_value = [a]() mutable { return ++a; };
    std::cout << by_value() << '␣' << a << '\n';
    auto by_reference = [&a] { ++a; };
    by_reference();
    std::cout << a << '\n';
}
```

输出为 2 1，再输出 2。引用捕获还要求被引用对象在调用 lambda 时仍然活着。

章末第 4 题：左值与右值 解值类别描述的是表达式。普通变量名表达式（如 x）是左值；字面量和许多计算结果是纯右值；`std::move(x)` 是将亡值。纯右值和将亡值统称右值。先根据表达式的形式、运算符规则和函数返回类型判断，比只背“放在等号哪一边”准确。正文的“能否取地址”可以辅助判断普通表达式，但不是通用定义：位域是左值却不能取地址，自定义取地址运算符也会干扰这个方法。对本题这些普通表达式，可以用额外括号中的 decltype 核对：

```
#include <type_traits>
#include <utility>
int main() {
    int x = 1;
    int&& ref = 2;
    static_assert(std::is_lvalue_reference_v<decltype((x))>);
    static_assert(std::is_lvalue_reference_v<decltype((ref))>);
    static_assert(std::is_rvalue_reference_v<decltype((std::move(x)))>);
    static_assert(!std::is_reference_v<decltype((x + 1))>);
}
```

编译通过即可。注意有名字的右值引用 ref，在普通使用它的表达式中仍是左值。

章末第 5 题：std::move 做了什么 解 `std::move` 本身是类型转换，使表达式能够参与移动相关的重载选择，不会直接搬走资源。真正的移动发生在之后被调用的移动构造、移动赋值等操作中；有些类型或 const 对象还可能最终走拷贝。例如，对一个字符串仅执行 `(void)std::move(s);`，字符串内容不会因此改变。执行 `std::string target = std::move(s);` 后才可能转移资源。此后的 s 仍可析构、可重新赋值，但不要把“它一定为空”当作通用规则。

章末第 6 题：独占与共享所有权 解 unique_ptr 表示独占所有权，允许移动转交，不允许复制出第二个同样拥有资源的指针。shared_ptr 通过控制块记录共享所有权，复制会增加强引用计数，最后一个强引用释放时销毁对象。禁用 unique 指针的拷贝，是为了保持“只有一个所有者”的约束。shared 指针的计数管理也不代表所指向对象的读写天然线程安全。

章末第 7 题：循环引用 解两个对象各用 shared 指针持有对方，外部所有者退出后，内部引用仍让两边计数无法归零。把其中一条不应拥有资源的反向关系改成 weak 指针，就不会增加强引用计数；访问前用 lock 获取临时所有权。完整实现与计数变化见本题解的随堂练习 2.9。选择哪一侧改成 weak，要根据对象之间实际的所有权关系决定。

章末第 8 题：两种安全删除写法 解第一种边遍历边 erase，并更新迭代器；第二种先 remove_if 整理保留元素，再 erase 删除尾部。

```
#include <algorithm>
#include <iostream>
#include <vector>
int main() {
    std::vector<int> first{1, 2, 3, 4, 5, 6};
    auto second = first;
    auto is_even = [](int x) { return x % 2 == 0; };
    for (auto it = first.begin(); it != first.end(); ) {
        if (is_even(*it)) it = first.erase(it);
        else ++it;
    }
    second.erase(std::remove_if(second.begin(), second.end(), is_even),
                 second.end());
    for (int x : first) std::cout << x << '␣';
    std::cout << '\n';
    for (int x : second) std::cout << x << '␣';
    std::cout << '\n';
}
```

两行都应为 1 3 5。remove_if 自己不会改变 vector 的 size，最后的 erase 才真正缩小容器。

章末第 9 题：死锁条件与统一顺序 解四个必要条件是互斥、持有并等待、不可抢占、循环等待。例如线程 A 先拿锁 1，再等锁 2；线程 B 先拿锁 2，再等锁 1，就可能形成环。让所有相关线程统一按锁 1、锁 2 的顺序获取，可以破坏循环等待。也可用 std::scoped_lock 一起管理两把锁，采用避免死锁的算法。不要把 sleep 当成修复方案：它可能改变复现概率，却没有消除死锁条件。

章末第 10 题：装甲板排序与并行计数 解先排序，再启动两个只读线程。两个线程分别写自己的计数，主线程在 join 后读取结果。题目要求“高于”和“低于”，所以恰好等于 0.5 的对象两边都不计入。

```
#include <algorithm>
#include <iostream>
```

```cpp
#include <thread>
#include <vector>
class Armor {
    int id_;
    double confidence_;
public:
    Armor(int id, double confidence) : id_(id), confidence_(confidence) {}
    int id() const { return id_; }
    double confidence() const { return confidence_; }
};
int main() {
    std::vector<Armor> armors{{1, 0.3}, {2, 0.9}, {3, 0.5}, {4, 0.7}};
    std::sort(armors.begin(), armors.end(), [](const Armor& a, const Armor& b) {
        return a.confidence() > b.confidence();
    });
    for (const auto& armor : armors) {
        std::cout << armor.id() << ':' << armor.confidence() << '\n';
    }
    int high = 0, low = 0;
    std::thread above([&] {
        for (const auto& armor : armors) {
            if (armor.confidence() > 0.5) ++high;
        }
    });
    std::thread below([&] {
        for (const auto& armor : armors) {
            if (armor.confidence() < 0.5) ++low;
        }
    });
    above.join();
    below.join();
    std::cout << "high=" << high << "␣low=" << low << '\n';
}
```

排序后的编号为 2、4、3、1，最后输出 high=2 low=1。工作期间不能再让其他线程修改容器或对象。这里没有并发写同一个计数，所以不需要给这两个计数额外加锁。

第三章：编译过程与 CMake 基础

随堂练习

第三章的操作题沿用正文里的目录和源文件。下面只列出要改的配置、要执行的命令和结果，完整工程见对应示例。

随堂练习 3.1：改源文件与改程序 解只把源文件中的编号改为 7，直接运行旧程序时仍输出 armor=3。重新执行四步构建，或执行正文的一步构建命令后，才会输出 armor=7。第二行始终为 function=main。少写普通语句末尾的分号时，预处理通常仍能完成，翻译成汇编的编译阶段会首先报错。修复后要重新构建，不要把上次残留的可执行文件当成这次成功的证据。

随堂练习 3.2：修改目标名 解把原配置中创建和设置 hello 目标的两行同步改成：

```
add_executable(armor_hello hello.cpp)
target_compile_features(armor_hello PRIVATE cxx_std_17)
```

保留其他项目配置，在 hello-cmake 根目录运行：

```
cmake -S . -B build-rename -DCMAKE_BUILD_TYPE=Debug
cmake --build build-rename --parallel 2
./build-rename/armor_hello
```

程序输出与改名前相同。项目名标识整个项目，目标名标识产物，二者可以不同，所以不用改 project。

随堂练习 3.3：缺路径与缺实现 解去掉 -I include 后，编译器找不到 armor.hpp，通常在预处理头文件时报告缺失。保留包含路径但去掉 src/armor.cpp 后，声明仍可见，源文件可以编译，但链接器找不到 armor_score(int) 的定义，会报告未定义引用。恢复原来的完整命令后，应重新得到 score=30。

随堂练习 3.4：修改源码和配置开关 解在多文件版本的 armor-demo 中把乘数改为 20，然后执行：

```
cmake --build build --parallel 2
./build/armor_demo
cmake -S . -B build -DSAVE_OUTPUT=OFF
cmake --build build --parallel 2
./build/armor_demo
```

分数应为 60；关闭开关并重新构建后，不再输出 save output enabled。普通源文件修改只需重新构建；配置开关通过重新配置写入构建要求后，也需要构建。最后把乘数改回 10，再构建一次，确认恢复为 score=30。

随堂练习 3.5：共享库版本 解把库目标改为 add_library(armor SHARED src/armor.cpp)，然后在 Ubuntu 中运行：

```
cmake -S . -B build-shared -DCMAKE_BUILD_TYPE=Debug
cmake --build build-shared --parallel 2
```

```
ls build-shared/libarmor.so
./build-shared/armor_demo
```

应出现共享库文件，程序仍输出 30 分。此时是在构建目录里运行，CMake 通常会安排构建目录的运行时库搜索路径；不能由此推出复制到任意电脑也能运行。最后改回 STATIC，回到正文的静态库版本继续安装练习。

随堂练习 3.6：更换安装前缀 解在静态库版本已配置、已构建的前提下，运行：

```
cmake --install build-lib --prefix "$PWD/stage-check"
./stage-check/bin/armor_demo
```

前缀是 $PWD/stage-check，安装规则中的相对目录是 bin，二者组合成最终安装位置。程序应输出 score=30。若你上一步还没恢复静态库版本，先恢复并重新配置、构建。

随堂练习 3.7：PUBLIC 改成 PRIVATE 解 armor 库自己仍能使用 include 目录，所以其源码能编译。但这个目录不再作为使用要求传给链接它的 armor_demo，后者在编译 src/main.cpp 时找不到 armor.hpp。恢复 PUBLIC 后重新配置并构建，应该通过。要是只改了文件却看到旧程序还能运行，先确认你检查的是本次构建结果。

随堂练习 3.8：检查退出码 解在 opencv-demo 目录执行：

```
./build/image_viewer
echo $?
./build/image_viewer missing.png
echo $?
```

两次退出码分别为 1、2。前者参数数量不对，后者读图失败；两种情况下都已成功启动程序，因此不应靠改链接配置解决。echo $? 必须紧跟待检查的命令，其他命令会覆盖这个状态值。

章末习题

章末第 1 题：编译阶段 解预处理得到.ii，编译得到.s，汇编得到.o，链接得到可执行文件。-c 在生成目标文件后停止，不执行链接。

章末第 2 题：项目名与目标名 解项目名是整个项目的名字，目标名是要构建的产物的名字。此时只准备好了构建文件，还要执行 cmake --build 构建目录。

章末第 3 题：头文件路径与源文件 解需要。头文件搜索路径只帮助编译器找到声明。score.cpp 里的实现仍要参与编译，也可以先做成库，再链接给程序。

章末第 4 题：库类型 解 STATIC 创建静态库，SHARED 创建共享库。省略类型时取决于 BUILD_SHARED_LIBS；该变量未设置或为假时默认静态库。

章末第 5 题：目标名与变量 解不一样。score 指向已创建的库目标；${score} 读取同名变量。题目中没设置这个变量，展开后是空的，程序就没有通过这项配置链接该库。

章末第 6 题：安装位置 解可执行文件路径为 /home/student/demo-stage/bin/viewer。安装不会自动构建，应先配置、构建，再安装。

章末第 7 题：配置作用域 解第一个设为 PUBLIC，库自己和下游都能用；第二个设为 PRIVATE，只给库自己用。INTERFACE 只把要求传给下游，常用于只提供头文件的接口库。

章末第 8 题：OpenCV 查找 解不会自动安装。先检查 C++ 开发包是否安装。找到包配置文件 OpenCVConfig.cmake 后，把它所在的目录赋给 OpenCV_DIR，不能随意指定一个头文件目录。

章末第 9 题：配置开关与重新构建 解漏了重新构建。开关决定是否调用 target_compile_definitions，这条命令会把宏加入目标的编译要求。重新构建后，条件编译的结果才会出现在新程序里；原程序不会自动变化。

章末第 10 题：增量构建 解按本章的构建规则，文件没变时，不需要重新编译或链接。改了 armor.cpp 后，要重新编译它、更新静态库，再重新链接程序。main.cpp 和它的依赖都没变，所以不用重新编译。不同生成器的日志长得不一样，看实际执行了哪些命令就好。



笔记几处易混淆规则可查阅原始文档：

- C++ 表达式值类别：https://eel.is/c++draft/basic.lval
- vector 容量要求：https://eel.is/c++draft/vector.capacity
- 条件变量等待：https://eel.is/c++draft/thread.condition.condvar

理解当前题目需要的部分就够了，文档可以慢慢查。